K1 Concierge Module Specification

K1 Team

2026-02-16

K1 Concierge Module Specification

● ACTIVE DOCUMENT

Status: SKELETON – fill as implementation proceeds Layer: L1 (User-Facing Intelligence) Owner: K1 Team Last Updated: 2026-02-16

Table of Contents

1. Identity & Position in K1

Concierge is Layer 1 – the primary user-facing intelligence in K1, serving as the "brain" that governs ALL conversation flow.

1.1 Position in K1 Layer Hierarchy

LAYER	COMPONENT	DESCRIPTION
L0	External Interfaces	UI/API clients (Web, Mobile, Voice, REST, WebSocket, SSE) – detached/TBD
L0.5	Module Loader	Discovers modules, hot-reload, registers tools/prompts/agents/schemas
L1	**CONCIERGE**	FSM-driven conversation conductor, SOLE WRITER to SessionState
L1.5	Rhythm Controller	Conversational pacing (200-500ms quick, 1-2s thoughtful)
L2	Orchestrator	Pure deterministic actor, Blind DAG Executor, NO LLM calls
L2.5	Capability Fabric	Intelligent Retrieval + Resolution + Execution
L3	Planner	LLM-powered 4-stage pipeline (Sketch->Expand->Validate->Commit)
L4	Sub-Agents & Tools	Spawned agents, MCP runners, WASM sandbox, model inference
L5	SessionState	Central working memory (HOT 48KB, WARM 48KB, LOCAL COLD, K0 Sync)
L6	K0 Durable Storage	Long-term memory via Cross-Kernel Bridge

1.2 Identity Summary

ASPECT	VALUE
Layer	L1 (above L0 External Interfaces, below L2 Orchestrator)
Role	FSM-driven conversation conductor, “brain” of K1
Cardinality	One Concierge instance per session
Writer Model	ONLY writer to SessionState (Single Writer Pattern, ADR-0017g)
LLM Host	Hosts Concierge LLM with 13 cognitive tool calls
Delta Aggregator	Receives and batches all sub-agent deltas (500ms window)
User Gateway	ALL output flows through OUTPUT_CHANNEL

1.3 Core Responsibilities

#	RESPONSIBILITY	DESCRIPTION
1	**User Input Reception**	All user messages flow through Concierge via IInputPort
2	**Two-Phase Turn Processing**	Phase 1: UltraBERT (22ms deterministic), Phase 2: LLM with cognitive tools
3	**Complexity-Based Routing**	LOW/MEDIUM/HIGH/CRISIS tier dispatch based on 5-factor scoring
4	**Single Writer to SessionState**	All mutations pass through MutationGuard, sub-agents publish via Delta Bus
5	**Delta Aggregation**	500ms batching of sub-agent deltas with LWW merge
6	**Output Management**	All responses flow through OUTPUT_CHANNEL (9 event types, 3 priorities)
7	**Safety Gate Enforcement**	CRISIS band = immediate protocol, bypasses all FSM states
8	**HIL Routing**	Routes Planner clarification/approval requests to user

1.4 Single Writer Pattern (ADR-0017g)

1.5 Access Control Matrix

AGENT TYPE	READ ACCESS	WRITE ACCESS	MUTATION REQUEST
Concierge	Full (HOT + WARM)	Full (via MutationGuard)	N/A (is writer)
Sub-Agent	Snapshot only	NEVER	Via Delta Bus
Dynamic Agent	Scoped snapshot	NEVER	Via Delta Bus
Orchestrator	Snapshot (ORCH-01)	NEVER	Via Delta Bus
Planner	Snapshot (PLAN-01)	NEVER	N/A
Tool	None	NEVER	None
External	None	NEVER	None

1.5.1 Sub-Agent vs Dynamic Agent – Detailed Distinction

The Access Control Matrix above lists **Sub-Agent** and **Dynamic Agent** as separate rows. The distinction is not about write behavior (both NEVER write SessionState; both mutate via Delta Bus). The distinction is about **lifetime**, **registry presence**, **context scope**, and **creation path**.

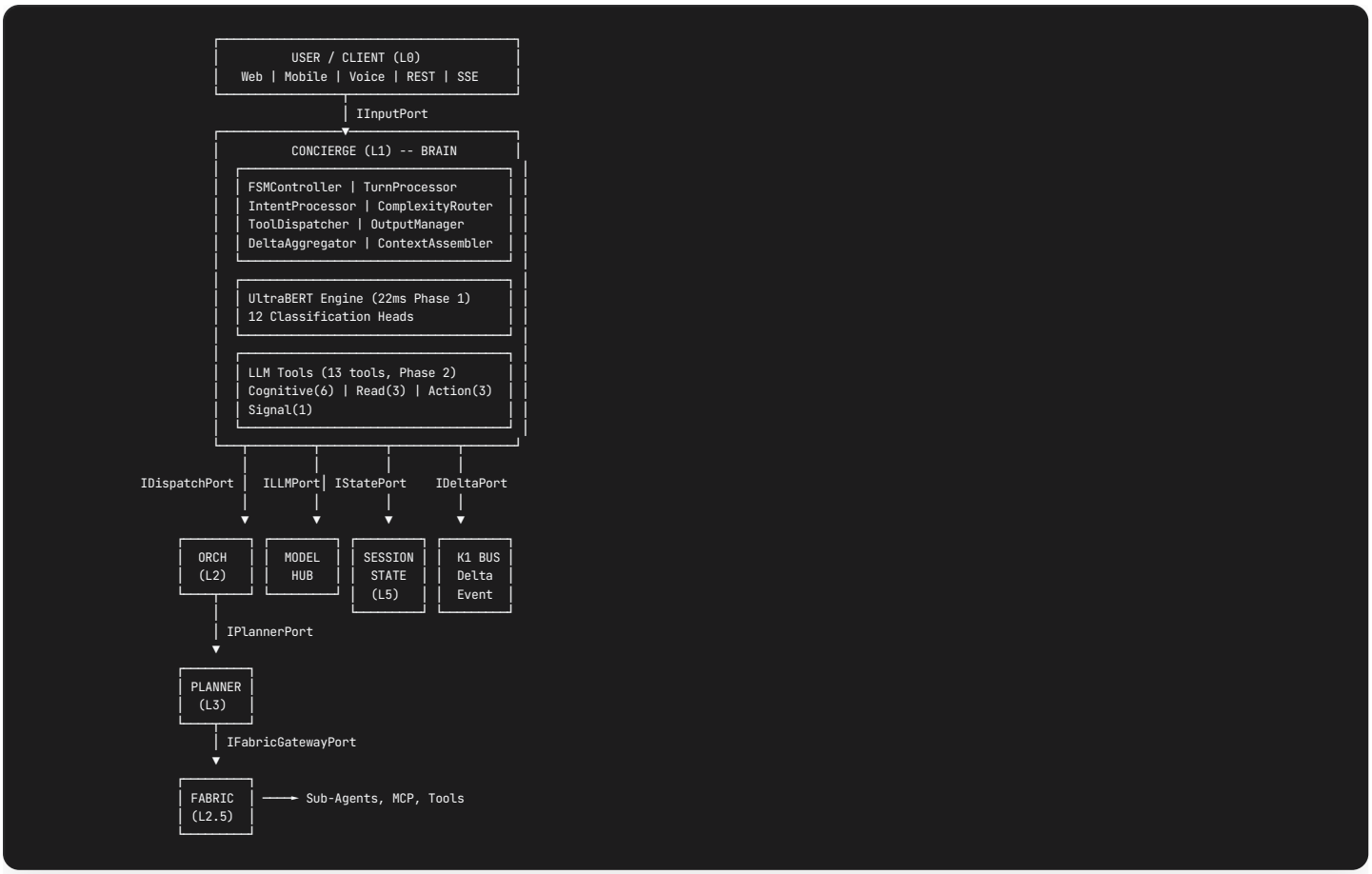
DIMENSION	SUB-AGENT	DYNAMIC AGENT
Lifetime	Session-scoped (lives as long as the session)	Ephemeral (TTL-based, `ttl_turns` from YAML template)
Registry	Pre-registered in Capability Registry at boot	Created at runtime, NOT in Registry until spawned
SessionState Read	Full HOT snapshot (all 8 sections)	Scoped snapshot (`required_context` sections only, per ADR-0086)
Created By	Bootstrap / `init()` phase	`spawn_via_fabric()` (LOW tier) or Planner's `build_agent` DAG step (HIGH tier, ADR-0086)
Tool Access	Full allowlist for its registered capability	Scoped `tools_granted` (FAB-07), subset of creator's band
Safety Band	Inherits session safety band	Inherits creator's max safety band (never escalates)
Example	ProactiveAgent, CuriosityAgent, ReminderAgent	health_agent (spawned for "find me a doctor"), finance_agent

Key rule: A Dynamic Agent's scoped snapshot means it receives ONLY the SessionState sections declared in its YAML template's `required_context` list. This prevents information leakage and reduces token cost. A Sub-Agent receives the full snapshot because it is a known, trusted component.

Both agent types emit deltas to the K1 Bus (delta lane) which the Concierge's DeltaAggregator collects, batches (500ms window), and writes to SessionState as the Single Writer.

> **Cross-reference**: ADR-0086 (Dynamic Agent Creation), Section 34 (Fabric Agent Factory), `concierge.mmd` TP_SUBAGENTS subgraph.

1.6 Architectural Boundaries



1.7 Key ADRs

ADR	TITLE	RELEVANCE
ADR-0017g	SessionState Single Writer	Concierge ownership of SessionState
ADR-0093	ConciergeAgent Pattern	Master coordinator design
ADR-0078	Tool Call Batching Pipeline	Parallel tool execution
ADR-0006f	3-Phase Orchestration	Task routing pattern
ADR-0007	4-Stage Planning Pipeline	HIGH tier coordination
ADR-0086	Dynamic Agent Creation	Agent spawning via Fabric

2. Hard Invariants

22 invariants grouped into 6 categories. **Violation = turn failure or system halt.**

> **Enforcement Philosophy**: Every invariant has a designated enforcing service. The service MUST reject operations that would violate the invariant. No exceptions, no overrides.

2.1 OWNERSHIP (4 invariants)

ID	INVARIANT	LIMIT	ENFORCEMENT	ON VIOLATION
CONC-01	Concierge is ONLY writer to SessionState	N/A	MutationGuard.preflight() rejects writes from other components	Reject + LOG_ERROR
CONC-02	All output flows through OUTPUT_CHANNEL	N/A	OutputManager enforces single egress point	Reject + LOG_ERROR
CONC-03	Single FSM instance per session	1	Kernel bootstrap guarantees via session registry	Bootstrap fails
CONC-04	Turn lock held for entire turn duration	N/A	TurnProcessor acquires at turn_start, releases at turn_end	Deadlock detection

MutationGuard Implementation:

```python
class MutationGuard:
    NEVER_EVICT_SECTIONS = {"control"}

    def preflight(self, request: MutationRequest) -> MutationApproval:
        """All SessionState writes MUST pass through preflight."""
        # Check 1: Verify caller is Concierge
        if request.caller_id != "concierge":
            raise UnauthorizedWriteError(f"Only Concierge may write. Got: {request.caller_id}")
        # Check 2: Verify section not locked
        # Check 3: Verify size limits (HOT ≤ 48KB, WARM ≤ 48KB)
        # Check 4: Verify NEVER_EVICT sections not violated
        return MutationApproval(approved=True, mutation_id=uuid4())
```

2.2 SAFETY (3 invariants)

ID	INVARIANT	LIMIT	ENFORCEMENT	ON VIOLATION
CONC-05	CRISIS safety band bypasses ALL routing	N/A	SafetyGate checks BEFORE any FSM transition	Immediate CRISIS_STATIC response
CONC-06	Safety classification runs BEFORE any LLM call	N/A	UltraBERT Phase 1 gates Phase 2	Turn rejected
CONC-07	PII masking applied before ALL logging	N/A	StructuredLogger PIIMaskingFilter	Log rejected

CRISIS_STATIC Response (hardcoded, zero dependencies):

```
"If you or someone you know is in crisis, please contact emergency services (911)
or the 988 Suicide and Crisis Lifeline."
```

Safety Band Levels:

BAND	DESCRIPTION	ACTION
GREEN	Normal operation	Standard routing
AMBER	Caution required	Enhanced monitoring, proceed carefully
RED	Sensitive content	Restricted operations, elevated logging
CRISIS	Emergency	IMMEDIATE protocol, bypass ALL FSM states

2.3 RATE LIMITS (5 invariants)

ID	INVARIANT	LIMIT	ENFORCEMENT	ON VIOLATION
CONC-08	Max clarification rounds per intent	3	ClarificationTracker.increment()	Force proceed with best hypothesis
CONC-09	Max tool calls per turn (hard cap)	20	ToolDispatcher.call_count >= 20	Reject additional calls
CONC-10	Delta batch window exactly	500ms	DeltaAggregator timer	Flush at window end
CONC-11	Concurrent active turns per session	1	TurnProcessor lock	Queue subsequent messages
CONC-12	Output queue depth	50	OutputManager.queue.maxsize	Backpressure to upstream

Tool Call Budget by Tier:

TIER	TOOL CALLS	LLM CALLS	RATIONALE
LOW	≤ 6	≤ 2	Simple queries, direct response
MEDIUM	≤ 12	≤ 3	Multi-step with Orchestrator
HIGH	≤ 20	≤ 5	Complex planning with Planner

2.4 TIMING (4 invariants)

ID	INVARIANT	TARGET	P99	ENFORCEMENT	ON VIOLATION
CONC-13	Phase 1 ACKING completion	≤ 22ms	25ms	UltraBERT single forward pass	Degrade to heuristic
CONC-14	FSM state transition (no I/O)	≤ 1ms	1ms	FSMController pure computation	LOG_WARN, continue
CONC-15	Turn timeout maximum	120s	N/A	TurnProcessor watchdog	Cancel turn, apologize
CONC-16	Single LLM call timeout	30s	N/A	ILLMPort adapter timeout	CB_MODEL_HUB trigger

Performance Baselines (System-Owned P99):

OPERATION	P99 LATENCY
UltraBERT classification	25ms
Intent ack SSE delivery	50ms
FSM state transition	1ms
MutationGuard preflight	0.2ms
Phase 1 write	0.02ms
Phase 2 write	0.1ms
Complexity routing decision	0.2ms
Output queue to SSE delivery	5ms
Orchestrator envelope dispatch	2ms
Fabric direct dispatch	2ms
Total system overhead/turn	**~32ms**

2.5 STRUCTURAL (3 invariants)

ID	INVARIANT	ENFORCEMENT	ON VIOLATION
CONC-17	FSM transitions are deterministic	FSMController state table lookup	Invalid transition rejected
CONC-18	All ports are protocol-typed	Python Protocol enforcement at boot	Adapter registration fails
CONC-19	No circular imports in concierge/	import_linter CI check	CI fails, PR blocked

FSM Determinism Guarantee:

```
class FSMController:
    # All transitions are pure lookups - NO I/O, NO external calls
    TRANSITION_TABLE: Dict[Tuple[State, Event], Tuple[State, List[Action]]] = {
        (State.LISTENING, Event.USER_MESSAGE): (State.ACKING, [Action.ACQUIRE_LOCK, Action.RUN_PHASE1]),
        (State.ACKING, Event.CLASSIFICATION_COMPLETE): (State.DISPATCHING, [Action.ROUTE_BY_TIER]),
        # ... all transitions enumerated
    }
```

2.6 TOKEN BUDGET (3 invariants)

ID	INVARIANT	LIMIT	ENFORCEMENT	ON VIOLATION
CONC-20	HOT tier memory ceiling	48KB	SessionState partition check	Evict to WARM
CONC-21	WARM tier memory ceiling	48KB	SessionState partition check	Evict to LOCAL COLD
CONC-22	LLM context window per call	128K tokens	ContextAssembler truncation	Summarize oldest history

Token Budget Envelopes by Output Type:

OUTPUT TYPE	TOKEN LIMIT	USE CASE
Intent ack (`acknowledge_request`)	150 tokens	Immediate confirmation (~50ms)
Preliminary ack (MEDIUM/HIGH)	200 tokens	“Working on it...” message
Clarification question	300 tokens	Entropy-minimizing question
LOW tier response	500 tokens	Simple direct answer
MEDIUM tier response	2K tokens	Multi-step result
HIGH tier response	8K tokens	Complex plan execution result

2.7 Invariant Enforcement Summary

3. Core Architecture Components

3.1 Component Hierarchy

```
CONCIERGE MODULE
│
├── FSM Core ──────────────────────────────────────────────────────
│   ├── FSMController (~80 tests)
│   │   - State machine execution
│   │   - Transition validation against state table
│   │   - Experience layer triggers (every 20-30 turns)
│   │   - Interrupt handling coordination
│   │
│   └── StateTransitionTable
│       - 8 states, 15 transitions
│       - Pure lookup, no I/O (CONC-14: ≤1ms)
│
├── Turn Processing ───────────────────────────────────────────────
│   ├── TurnProcessor (~90 tests)
│   │   - Per-turn lifecycle: turn_start → process → turn_end
│   │   - Phase 1 + Phase 2 orchestration
│   │   - Turn lock management (CONC-04)
│   │   - Watchdog timer (CONC-15: 120s max)
│   │
│   ├── IntentProcessor (~70 tests)
│   │   - SafetyGate (CONC-05,06: FIRST, before routing)
│   │   - Hypothesis generation from UltraBERT output
│   │   - Temporal resolution ("next Thursday" → absolute datetime)
│   │   - Spatial resolution (GPS/BLE → semantic place)
│   │   - Context inference from SessionState
│   │
│   └── ComplexityRouter (~40 tests)
│       - 5-factor weighted scoring → tier assignment
│       - Pure computation, no I/O
│       - LOW (<0.3) | MEDIUM (0.3-0.7) | HIGH (>0.7)
│
├── UltraBERT Engine (Phase 1) ────────────────────────────────────
│   ├── UltraBERT Model (22ms target, 25ms P99)
│   │   - 12 classification heads in single forward pass
│   │   - Multi-task learning architecture
```

```
│   │   - Optimized for CPU inference
│   │
│   ├── Classification Heads (12)
│   │   ├── Intent Head (8 classes, multi-label)
│   │   ├── Ingress Head (12 domains, multi-label)
│   │   ├── Safety Head (4 bands: GREEN/AMBER/RED/CRISIS)
│   │   ├── Emotions Head (44 classes, Plutchik wheel + nuanced)
│   │   ├── Sentiment Head (5 levels)
│   │   ├── NER-General Head (GlobalPointer: PER, ORG, LOC, MISC)
│   │   ├── NER-Family Head (GlobalPointer: 10 types incl. PERSON, KINSHIP, PET, HOME_LOC...)
│   │   ├── Temporal Head (GlobalPointer: DATE_ABS, DATE_REL, TIME, DURATION, FREQUENCY, AGE)
│   │   └── Relations Head (15 types: parent_of, child_of, spouse_of, sibling_of, pet_of...)
│   │
│   ├── Temporal Resolution Engine
│   │   - NER TIME entities → parsed → resolved absolute time
│   │   - "next Thursday at 3pm" → 2026-02-19T15:00:00Z
│   │
│   └── Spatial Resolution Engine
│       - GPS/BLE/Wi-Fi → semantic place
│       - GPS + BLE(home_beacon) → Place("home", room="living_room")
│
├── LLM Tools Layer (Phase 2) ──────────────────────────────────────────
│   ├── ToolDispatcher (~75 tests)
│   │   - Tool allowlist by state/tier/safety
│   │   - Schema validation before execution
│   │   - Call budget enforcement (CONC-09: ≤20)
│   │   - Workflow depth tracking (max 3)
│   │
│   ├── Cognitive Tools (6) - LLM detects what to update
│   │   ├── update_scoreboard() → referents, QUD, salience, topic shift
│   │   ├── update_beliefs() → new facts, corrections, invalidations
│   │   ├── update_clarifications() → semantic gaps, resolved gaps
│   │   ├── update_narrative() → thread switch/resume/new/continue
│   │   ├── refine_affect() → override emotion (sarcasm/nuance detection)
│   │   └── promote_belief() → WARM to HOT fact promotion
│   │
│   ├── Read Tools (3) - Context Retrieval
│   │   ├── recall_memory() → K0 long-term memory query
│   │   ├── discover_capabilities() → query Capability Registry via Fabric
│   │   └── summarize_context() → token budget manager
│   │
│   ├── Action Tools (3) - Execution & Delegation
│   │   ├── invoke_capability() → direct Fabric call (LOW tier)
│   │   ├── spawn_via_fabric() → capability-based agent creation
│   │   └── execute_workflow() → workflow capability invocation
│   │
│   └── Signal Tools (1) - Emit-only, no state mutation
│       └── acknowledge() → immediate intent confirmation (~50ms)
│
├── Experience Layer (Heuristic, every 20-30 turns) ────────────────────
│   ├── Emotional Processing (turn % 25)
│   │   - Analyze emotional trajectory over last 25 turns
│   │   - Detect trend: IMPROVING | STABLE | DECLINING
│   │   - Output: EmotionalTrajectory
│   │
│   ├── Emotional Mirroring (after EMOTIONAL_PROCESSING)
│   │   - Adjust conversational tone to match user
│   │   - Formality, verbosity, empathy level
│   │
│   ├── Narrative Weaving (turn % 20)
│   │   - Cluster conversation threads
│   │   - Track QUD (Questions Under Discussion)
│   │   - Output: NarrativeCluster[]
│   │
│   └── Anticipatory Response (turn % 30)
│       - Predict user needs based on patterns
│       - Prefetch context from K0
│       - Output: Anticipation{predicted_intent, confidence, prefetched_context}
│
├── Output Management ──────────────────────────────────────────────────
│   ├── OutputManager (~65 tests)
│   │   - 9 output event types
│   │   - 3 priority levels (REALTIME, INTERACTIVE, BACKGROUND)
│   │   - Queue depth enforcement (CONC-12: ≤50)
│   │   - SSE delivery coordination
│   │
│   ├── DeltaAggregator (~50 tests)
│   │   - 500ms batching window (CONC-10)
│   │   - LWW (Last-Writer-Wins) merge for conflicts
│   │   - 4 delta types: progress, affective, scoreboard, narrative
│   │
│   └── DeliveryChannel
│       - SSE stream management
│       - Reconnection handling
│       - Backpressure signaling
│
└── Support Services ───────────────────────────────────────────────────
    ├── ClarificationTracker (~45 tests)
    │   - Max 3 rounds per intent (CONC-08)
    │   - Entropy-minimizing question generation
    │   - Sub-agent HIL tracking
    │
    └── ContextAssembler (~70 tests)
        - Token budget management (128K window, CONC-22)
        - Summarization trigger at >80% of budget
        - HOT tier prioritization for context building
```

3.2 Service Summary Matrix

SERVICE	TESTS	RESPONSIBILITY	KEY INVARIANTS
FSMController	~80	State transitions, experience triggers	CONC-14, CONC-17
TurnProcessor	~90	Phase 1+2 orchestration, lifecycle	CONC-04, CONC-11, CONC-15
IntentProcessor	~70	SafetyGate, Hypothesis, Resolution	CONC-05, CONC-06, CONC-13
ComplexityRouter	~40	5-factor tier selection	N/A (pure computation)
ToolDispatcher	~75	Tool allowlist, schema, execution	CONC-09
OutputManager	~65	Event types, priorities, SSE	CONC-02, CONC-12
DeltaAggregator	~50	500ms batching, LWW merge	CONC-10
ClarificationTracker	~45	Max rounds, question gen	CONC-08
ContextAssembler	~70	Token budgets, summarization	CONC-22
Total	**~585**		

3.3 8 Hexagonal Ports (Boundary Contracts)

PORT	DIRECTION	PURPOSE	ADAPTER
`IInputPort`	Inbound	`receive()` → UserMessage	`InputWebSocketAdapter`
`IOutputPort`	Outbound	`send(OutputEvent)` → DeliveryReceipt	`OutputSSEAdapter`
`IClassificationPort`	Outbound	`classify(text)` → ClassificationResult	`UltraBERTAdapter`
`ILLMPort`	Outbound	`execute(HubRequest)` → HubResponse	`LLMGatewayAdapter`
`IStatePort`	Both	`read()/write()` → SessionState	`SessionStateAdapter`
`IDispatchPort`	Outbound	`dispatch_direct()/dispatch_envelope()`	`DispatchAdapter`
`IDeltaPort`	Both	`subscribe()/publish()` → delta stream	`DeltaBusAdapter`
`IMemoryPort`	Outbound	`recall()` → K0 memory	`BridgeAdapter`

3.4 Port-to-Service Assignment Matrix

SERVICE	PORTS USED
FSMController	IInputPort, IOutputPort, IStatePort
TurnProcessor	IStatePort, IDeltaPort
IntentProcessor	IClassificationPort
ComplexityRouter	(none - pure computation)
ToolDispatcher	ILLMPort, IDispatchPort, IMemoryPort
OutputManager	IOutputPort, IDeltaPort
DeltaAggregator	IDeltaPort
ClarificationTracker	(none - state only)
ContextAssembler	IStatePort, IMemoryPort

3.5 7 Circuit Breakers

CIRCUIT BREAKER	TIMEOUT	RESET	FALLBACK	OWNER
CB_MODEL_HUB	Per provider	Per provider	Cached/template response	Concierge
CB_ORCHESTRATOR	60s	60s	Degrade to LOW tier	Concierge
CB_PLANNER	45s	60s (3 failures)	Skip planning, direct exec	Orchestrator
CB_FABRIC	30s	30s	Capability unavailable	Concierge
CB_MCP	10s	30s	Tool offline	Concierge
CB_SESSIONSTATE	100ms	10s	Stale read	Concierge
CB_SSE	5s reconnect	-	Polling mode	Concierge

3.6 Two-Phase Turn Model

4. Interface with Orchestrator

4.1 Overview

Orchestrator is L2 – a PURE DETERMINISTIC actor. No LLM calls, no tool execution, no state writes.

ASPECT	ORCHESTRATOR (L2)
Architecture	Blind DAG Executor
LLM Calls	NONE (ORCH-02)
Tool Calls	NONE (ORCH-03)
State Writes	NONE (ORCH-01) – read-only via IStateReadPort
Role	Coordinate Planner + Fabric execution

4.2 IDispatchPort (Concierge-side Protocol)

```python
class IDispatchPort(Protocol):
    """Concierge's outbound port for dispatching to Orchestrator/Fabric."""

    async def dispatch_direct(self, capability_id: str, params: Dict) -> ExecutionResult:
        """LOW tier: Direct to Fabric (bypasses Orchestrator entirely).

        Args:
            capability_id: Resolved capability from discover_capabilities()
            params: Parameters for capability execution

        Returns:
            ExecutionResult with output or error
        """
        ...

    async def dispatch_envelope(self, envelope: TaskEnvelope) -> DispatchReceipt:
        """MEDIUM/HIGH tier: To Orchestrator mailbox.

        Args:
            envelope: TaskEnvelope with tier=MEDIUM or tier=HIGH

        Returns:
            DispatchReceipt with envelope_id and queued status

        Raises:
            MailboxFullError: If queue depth > 100
            InvalidTierError: If tier is LOW (should use dispatch_direct)
        """
        ...

    async def cancel_dispatch(self, envelope_id: str) -> CancelResult:
        """Cancel in-flight dispatch.

        Returns:
            CancelResult with status (CANCELLED | NOT_FOUND | ALREADY_COMPLETE)
        """
        ...
```

4.3 TaskEnvelope Protocol (Exact Fields)

```python
from dataclasses import dataclass, field
from uuid import uuid4
from typing import Dict, List, Any, Literal

@dataclass(frozen=True)
class TaskEnvelope:
    """Immutable task envelope for Orchestrator dispatch.

    Location: k1/orchestrator/types.py
    """

    # Required fields
    intent: str                          # Classified intent from UltraBERT + LLM
    cognitive_trace_id: str              # FAB-09 trace propagation (MUST be non-empty)

    # Optional fields with defaults
    caller_id: str = "concierge"         # Who sent the envelope
    envelope_id: str = field(default_factory=lambda: str(uuid4()))
    context: Dict[str, Any] = field(default_factory=dict)  # beliefs_snapshot, entities, temporal
    tier: Literal["MEDIUM", "HIGH"] = "MEDIUM"  # LOW/CRISIS never reach Orchestrator
    capabilities: List[str] = field(default_factory=list)  # For MEDIUM: specific caps (max 2)
    params: Dict[str, Dict[str, Any]] = field(default_factory=dict)  # Keyed by capability_name
    constraints: Dict[str, Any] = field(default_factory=dict)  # Timeout, budget, permissions
    timeout_ms: int = 30_000             # Per-task timeout

    # Validation constants
    _VALID_TIERS = frozenset({"MEDIUM", "HIGH"})  # LOW bypasses Orchestrator

    def __post_init__(self):
        """Validate envelope on creation."""
        if self.tier not in self._VALID_TIERS:
            raise ValueError(f"tier must be MEDIUM or HIGH, got: {self.tier}")
        if not self.intent:
            raise ValueError("intent must be non-empty")
        if not self.cognitive_trace_id:
            raise ValueError("cognitive_trace_id must be non-empty")
        if self.tier == "MEDIUM" and not self.capabilities:
            raise ValueError("MEDIUM tier requires non-empty capabilities list")
        if self.tier == "MEDIUM" and len(self.capabilities) > 2:
            raise ValueError("MEDIUM tier allows max 2 capabilities (ORCH-10)")
```

Context Snapshot Contents:

```python
context = {
    "beliefs_snapshot": {...},    # Current beliefs from SessionState
    "entities": [...],            # Extracted entities from NER
    "temporal": {...},            # Resolved time references
    "spatial": {...},             # Resolved location references
    "history_summary": "...",     # Summarized recent history
    "affective_state": {...},     # Current emotional context
}
```

4.4 Tier-Based Routing

TIER	COMPLEXITY SCORE	ROUTE	ORCHESTRATOR ACTION	LATENCY
LOW	< 0.3	`dispatch_direct()` → Fabric	**N/A - Bypassed**	< 2s
MEDIUM	0.3 - 0.7	`dispatch_envelope()` → Orchestrator	Execute 1-2 Fabric calls directly (ORCH-10)	2-10s
HIGH	> 0.7	`dispatch_envelope()` → Orchestrator	Request plan via Planner, execute DAG (ORCH-11)	10-60s
CRISIS	Any + RED/CRISIS	SafetyGate override	**N/A - Bypassed**	< 5s

Key Constraints:

- **ORCH-10**: MEDIUM tier allows max 2 Fabric calls (no Planner involvement)
- **ORCH-11**: HIGH tier REQUIRES CommittedPlan from Planner before execution
- **LOW**: Concierge calls Fabric directly, Orchestrator never sees these
- **CRISIS**: SafetyGate intercepts, hardcoded CRISIS_STATIC response

4.5 Orchestrator Ports (9 Total)

PORT	PURPOSE	DIRECTION
`IMailboxPort`	Inbound message queue (WFQ, 100 capacity)	Inbound
`IFabricGatewayPort`	Capability execution via Fabric	Outbound
`IPlannerPort`	Plan creation/cancellation	Outbound
`IStateReadPort`	Read-only SessionState access (ORCH-01)	Outbound
`IDeltaEmitPort`	Fire-and-forget events + deltas	Outbound
`IBridgeWritePort`	K0 WAL writes, audit logging	Outbound
`IEventSubscriptionPort`	Event Bus subscriptions	Inbound
`IWorkflowStoragePort`	Workflow persistence (SQLite)	Both
`IAdminPort`	HTTP admin endpoints (port 8081)	Inbound

4.6 Mailbox Pattern (WFQ Priority Queue)

```
MailboxMessage = Union[
    TaskEnvelope,        # From Concierge (MEDIUM/HIGH tier)
    CommittedPlan,       # From Planner (plan ready)
    WorkflowRunRequest,  # From Scheduler (triggered workflow)
    WorkflowSaveRequest, # From Concierge (save workflow)
    InterruptRequest,    # From external (cancel/pause)
]
```

Weighted Fair Queuing (WFQ) Configuration:

PRIORITY	WEIGHT	USE CASE
REALTIME	60%	Interrupts, safety band changes
INTERACTIVE	30%	User tasks (TaskEnvelope), plan results
BACKGROUND	10%	Workflow triggers, gap scans

Mailbox Limits:

- `max_depth`: 100 messages
- On full: `MailboxFullError` → Concierge receives REJECTED_FULL status

4.7 Delta Reception

```python
class IDeltaPort(Protocol):
    """Bidirectional delta streaming."""

    async def subscribe(self, topics: List[str]) -> AsyncIterator[Delta]:
        """Subscribe to delta stream from Orchestrator/Planner.

        Concierge subscribes at init and processes throughout session.
        """
        ...

    async def publish(self, delta: Delta) -> None:
        """Publish delta to K1 Bus (fire-and-forget)."""
        ...
```

Subscribed Topics (Concierge → from Orchestrator):

TOPIC	DESCRIPTION	PAYLOAD
`k1.orchestration.task.accepted.v1`	TaskEnvelope accepted	`{envelope_id, tier}`
`k1.orchestration.step.started.v1`	DAG step begins	`{step_id, capability_id}`
`k1.orchestration.step.completed.v1`	DAG step succeeded	`{step_id, result, duration_ms}`
`k1.orchestration.step.failed.v1`	DAG step failed	`{step_id, error, retryable}`
`k1.orchestration.dag.completed.v1`	Full DAG finished	`AggregatedResult`
`k1.orchestration.delta.v1`	User-facing progress	`{progress_pct, message}`
`k1.planner.stage.completed.v1`	Planner stage done	`{stage, duration_ms}`

4.8 Circuit Breaker: CB_ORCHESTRATOR

SETTING	VALUE
Timeout	60s
Reset Timeout	60s
Fallback	Degrade to LOW tier

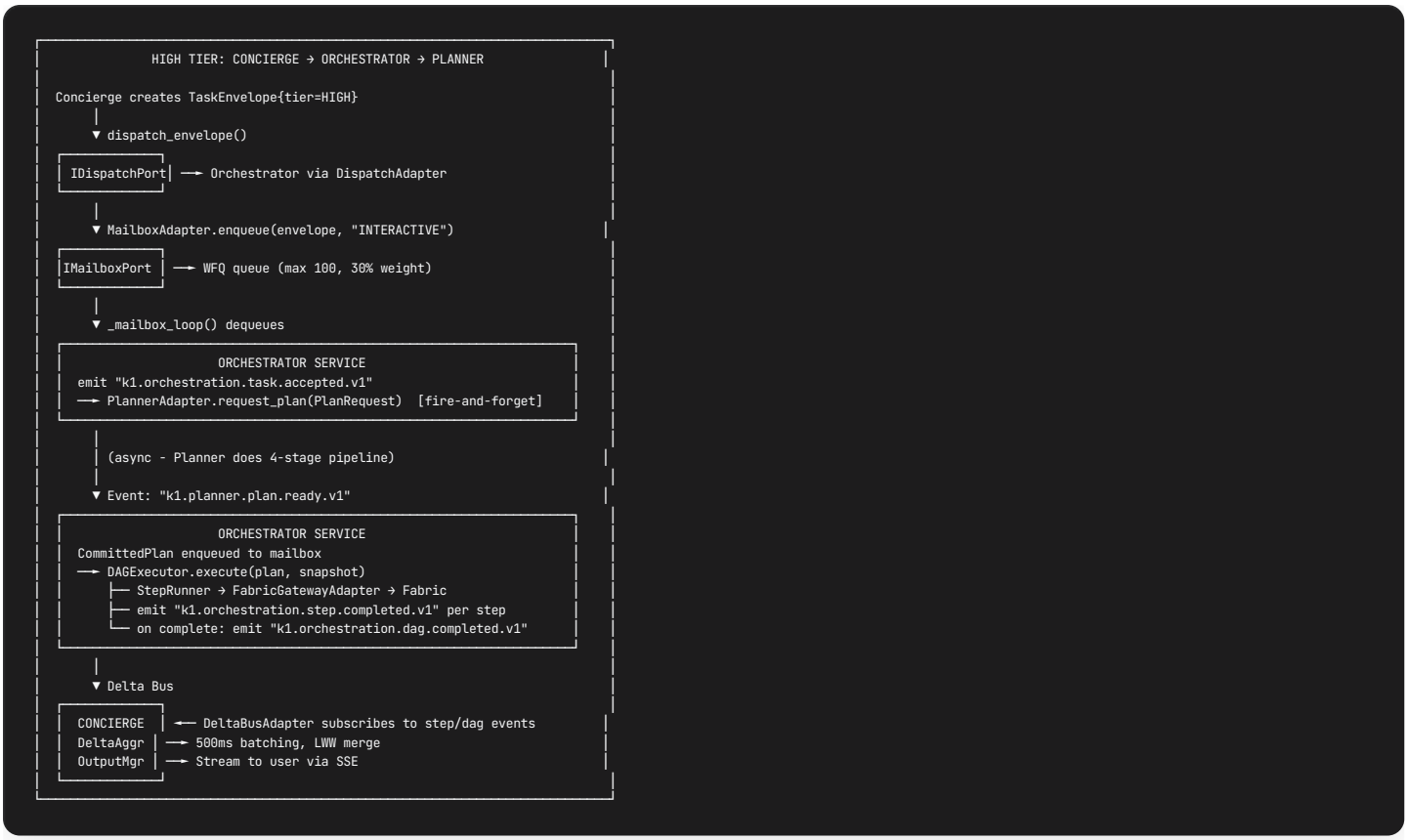
State Machine:

```
CLOSED ──[timeout or error]──► OPEN ──[60s]──► HALF_OPEN ──[probe success]──► CLOSED
                                                    │
                                               [probe fail]
                                                    ▼
                                                  OPEN
```

Behavior when CB_ORCHESTRATOR is OPEN:

1. MEDIUM/HIGH tier requests → degrade to LOW tier (direct Fabric call)
2. Emit `k1.concierge.degraded.v1` event
3. User sees: "I'll handle this directly for now."
4. Track degraded turns for observability

4.9 End-to-End Flow: HIGH Tier TaskEnvelope



4.10 Error Handling

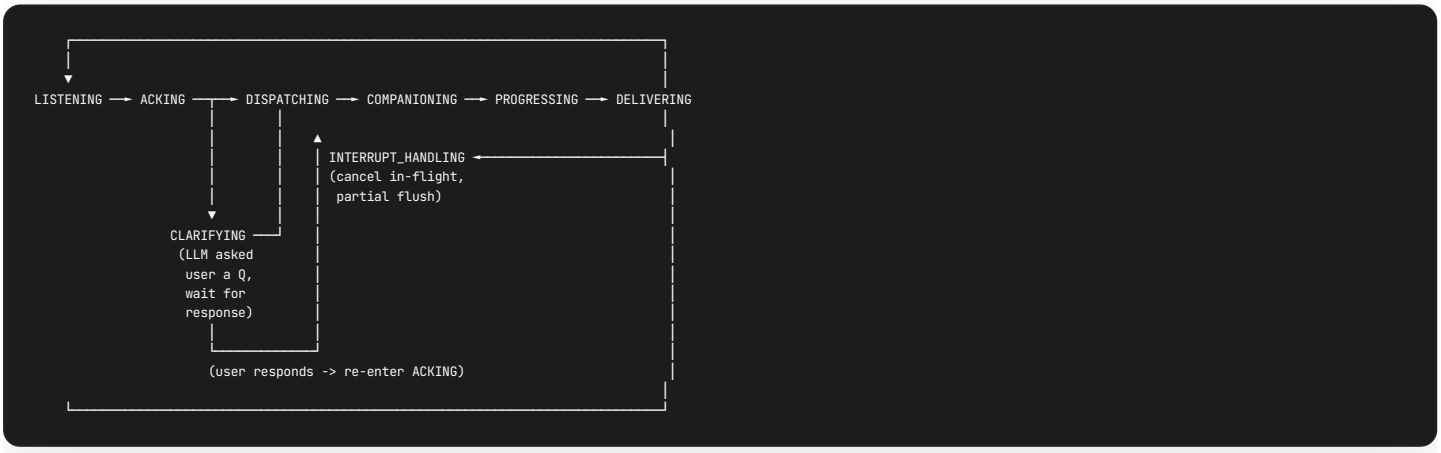
ERROR	CONCIERGE RESPONSE
`MailboxFullError` (queue > 100)	Retry after 100ms, then degrade to LOW
`CB_ORCHESTRATOR` OPEN	Degrade to LOW tier
Orchestrator timeout (60s)	Cancel envelope, apologize, offer retry
Partial DAG failure	Stream completed steps, report failure on rest
Planner failure	Orchestrator handles via CB_PLANNER, may fall back

5. FSM Core Loop Flow

5.1 State Catalog (8 States)

STATE	PURPOSE	ENTRY CONDITION	LATENCY BUDGET	VALID TRANSITIONS
LISTENING	Awaiting user input. FSM is idle.	Session start or DELIVERING complete	Unbounded	ACKING
ACKING	Phase 1: UltraBERT 22ms classification (12 heads). Safety Gate. Complexity routing. Write Elision Gate. No LLM.	`user_message` received	25ms (P95)	CLARIFYING, DISPATCHING
CLARIFYING	LLM decided it needs information from the user before proceeding (HITL). Concierge asks the question, then waits.	LLM emits `clarification_needed` during Phase 2 tool loop	User-dependent	ACKING (on user response)
DISPATCHING	Route by complexity tier. LOW: LLM tool loop (action tools). MED/HIGH: build `TaskEnvelope`, submit to Orchestrator via `IDispatchPort`.	`classification_complete` from ACKING	5ms	COMPANIONING
COMPANIONING	Keep conversation moving while backend works. LOW: emit `acknowledge()` message to user (~50ms). MED/HIGH: Proactive Agent generates warmth + gap-filling questions from K0 (>30s tasks).	Dispatch committed	Tier-dependent	PROGRESSING
PROGRESSING	Stream incremental deltas to the user as backend completes steps. Progress narration. SSE events.	First `delta_received` from Orchestrator/Fabric	Bounded by tier timeout	DELIVERING
DELIVERING	Final results + next steps. LLM call with `tool_results[]` + cognitive tools to produce polished response. Phase 2 SessionState writes (scoreboard, beliefs, narrative).	`task_complete` or all deltas received	2s (LLM call)	LISTENING
INTERRUPT_HANDLING	User changes topic mid-turn. Cancel in-flight work, partial flush, re-enter ACKING with new message.	User sends new message while in DISPATCHING, COMPANIONING, or PROGRESSING	10ms	ACKING

5.2 State Diagram



5.3 Two-Phase Turn Model

Every non-trivial turn passes through two phases within the FSM:

5.4 Transition Table (Full)

FROM	EVENT	GUARD	TO	ACTIONS	INVARIANT
LISTENING	`user_message`	always	ACKING	`turn_start()`: acquire SessionState lock, read HOT snapshot, reset turn buffers, increment `meta.turn_count`	CONC-01
ACKING	`classification_complete`	`safety_band != CRISIS`	DISPATCHING	Route by complexity tier. Emit `Phase1Result` to Phase 2 pipeline.	CONC-05
ACKING	`classification_complete`	`safety_band == CRISIS`	DELIVERING	Return `CRISIS_STATIC` response immediately. No Phase 2.	CONC-06
ACKING	`safety_red`	`safety_band == RED`	DISPATCHING	Elevated safety protocol. Override tier to MEDIUM minimum.	CONC-05
DISPATCHING	`dispatch_committed`	always	COMPANIONING	LOW: start LLM tool loop. MED/HIGH: submit `TaskEnvelope` to Orchestrator.	CONC-10
COMPANIONING	`acknowledge_sent`	LOW tier	PROGRESSING	`acknowledge()` message displayed to user. Proceed to tool execution.	–
COMPANIONING	`proactive_active`	HIGH tier, task >30s	PROGRESSING	Proactive Agent generates warmth messages + KO-originated gap-filling questions for past conversations.	–
COMPANIONING	`delta_received`	MED tier	PROGRESSING	First progress update from Orchestrator. Stream to user.	–
PROGRESSING	`delta_received`	more deltas pending	PROGRESSING	Stream incremental updates to user via SSE.	–
PROGRESSING	`task_complete`	all work done	DELIVERING	Assemble `tool_results[]`, invoke LLM with cognitive tools for final response.	–
DELIVERING	`delivery_complete`	always	LISTENING	`turn_end()`: flush outputs, append to `history_active`, Phase 2 SessionState writes, checkpoint, emit `turn.complete.v1`.	CONC-02
DISPATCHING, COMPANIONING, PROGRESSING	`interrupt`	user sends new message	INTERRUPT_HANDLING	`turn_end_abbreviated()`: cancel in-flight tasks, partial flush, save partial state.	CONC-11
INTERRUPT_HANDLING	`interrupt_processed`	always	ACKING	Re-enter ACKING with the NEW user message. Previous turn's partial results discarded.	–
Phase 2 (any LLM state)	`clarification_needed`	LLM decides info is missing	CLARIFYING	LLM formulates natural question, send to user, pause Phase 2.	CONC-08
CLARIFYING	`user_response`	always	ACKING	Merge user's clarification response into context, re-classify with enriched input.	CONC-09

5.5 Per-State Error Handling

The FSM uses three error severity levels (from concierge.mmd):

SEVERITY	BEHAVIOR	EXAMPLE
RECOVERABLE	Retry/fallback in same phase. No state change.	UltraBERT timeout -> heuristic fallback (Section 11.5)
DEGRADED	Continue to DELIVERING with partial results.	Orchestrator returns partial tool results.
TERMINAL	Jump to DELIVERING with error response.	LLM provider unreachable after retries.

```python
class ErrorSeverity(Enum):
    RECOVERABLE = "recoverable"  # Retry in current state
    DEGRADED = "degraded"        # Proceed with partial results
    TERMINAL = "terminal"        # Emergency delivery

async def handle_error(error: Exception, current_state: ConciergeState) -> ErrorSeverity:
    """Classify error and determine recovery strategy."""
    if isinstance(error, UltraBERTTimeout):
        return ErrorSeverity.RECOVERABLE  # Heuristic fallback
    if isinstance(error, PartialToolResult):
        return ErrorSeverity.DEGRADED     # Deliver what we have
    if isinstance(error, LLMProviderDown):
        return ErrorSeverity.TERMINAL     # Error response
    return ErrorSeverity.DEGRADED         # Default: deliver partial
```

5.6 Lifecycle Hooks

```python
# Per-turn lifecycle hooks

async def turn_start(message: UserMessage) -> None:
    """
    LISTENING -> ACKING entry hook.
    1. Acquire SessionState exclusive write lock (CONC-01)
    2. Read HOT tier snapshot (48KB, 8 sections)
    3. Reset turn buffers (tool_calls, tool_results, response)
    4. Increment meta.turn_count
    5. Record turn_start_ts in meta section
    6. Emit telemetry: turn.start.v1
    """

async def turn_end() -> None:
    """
    DELIVERING -> LISTENING exit hook.
    1. Append user message + assistant response to history_active (8KB ring)
    2. Execute Phase 2 SessionState writes (scoreboard, beliefs, narrative)
    3. Checkpoint SessionState to durable storage
    4. Release SessionState write lock
    5. Emit telemetry: turn.complete.v1 with latency_ms, tier, tool_count
    6. Check Experience Layer triggers (turn_count % 20/25/30)
    """

async def turn_end_abbreviated() -> None:
    """
    Interrupt path. Any interruptible state -> INTERRUPT_HANDLING.
    1. Cancel all in-flight tasks (Orchestrator abort, Fabric cancel)
    2. Partial flush: save any tool results received so far
    3. Do NOT append to history (turn was interrupted)
    4. Release SessionState write lock
    5. Emit telemetry: turn.interrupted.v1
    """
```

5.7 Tier-Dependent Flow Walkthrough

LOW Tier (simple request, <2s total):

```
LISTENING -> ACKING (22ms UltraBERT)
  -> DISPATCHING (route: LLM tool loop)
  -> COMPANIONING (acknowledge() ~50ms to user)
  -> PROGRESSING (tool execution, single iteration)
  -> DELIVERING (LLM final response + cognitive writes)
  -> LISTENING
Total: ~1-2s, 1-3 tool calls
```

MEDIUM Tier (multi-step reasoning, 2-10s):

```
LISTENING -> ACKING (22ms UltraBERT)
  -> DISPATCHING (build TaskEnvelope, submit to Orchestrator)
  -> COMPANIONING (acknowledge() to user, Orchestrator starts DAG)
  -> PROGRESSING (stream 2-5 deltas from Orchestrator steps)
  -> DELIVERING (LLM integrates results, cognitive writes)
  -> LISTENING
Total: ~5-10s, 3-8 tool calls
```

HIGH Tier (complex planning, 10-60s):

```
LISTENING -> ACKING (22ms UltraBERT)
  -> DISPATCHING (TaskEnvelope to Orchestrator, who may invoke Planner)
  -> COMPANIONING (acknowledge() + Proactive Agent fills wait time:
       "While I'm working on that, I had a quick question about
        our last conversation -- you mentioned...")
  -> PROGRESSING (stream deltas over 30-60s, progress narration)
  -> DELIVERING (LLM integrates full results, update scoreboard/narrative)
  -> LISTENING
Total: ~30-60s, 5-15 tool calls, Proactive fills idle time
```

CRISIS Tier (safety override):

```
LISTENING -> ACKING (22ms UltraBERT, CRISIS detected)
  -> DELIVERING (CRISIS_STATIC response, zero LLM, zero tools)
  -> LISTENING
Total: ~25ms, bypasses all Phase 2
```

5.8 FSM Context Object

```python
@dataclass
class FSMContext:
    """Mutable context maintained across FSM states within a single turn."""

    # Turn tracking
    turn_count: int = 0
    turn_start_ts: float = 0.0

    # Phase 1 outputs (populated in ACKING)
    classification: Optional[ClassificationResult] = None
    phase1_result: Optional[Phase1Result] = None

    # Routing (populated in DISPATCHING)
    routing: Optional[RoutingDecision] = None
    complexity_tier: Optional[ComplexityTier] = None

    # Clarification state (persists across turns)
    pending_clarification: bool = False
    clarification_count: int = 0
    max_clarifications: int = 3  # CONC-08

    # Execution state (populated in PROGRESSING)
    tool_calls: List[Dict[str, Any]] = field(default_factory=list)
    tool_results: List[Dict[str, Any]] = field(default_factory=list)
    deltas_received: int = 0

    # Response (populated in DELIVERING)
    response: str = ""
    acknowledge_message: str = ""  # From acknowledge() tool

    def reset_turn(self) -> None:
        """Reset per-turn state. Clarification state persists."""
        self.classification = None
        self.phase1_result = None
        self.routing = None
        self.complexity_tier = None
        self.tool_calls = []
        self.tool_results = []
        self.deltas_received = 0
        self.response = ""
        self.acknowledge_message = ""
```

5.9 State History and Observability

Every turn records the full state transition history for debugging and telemetry:

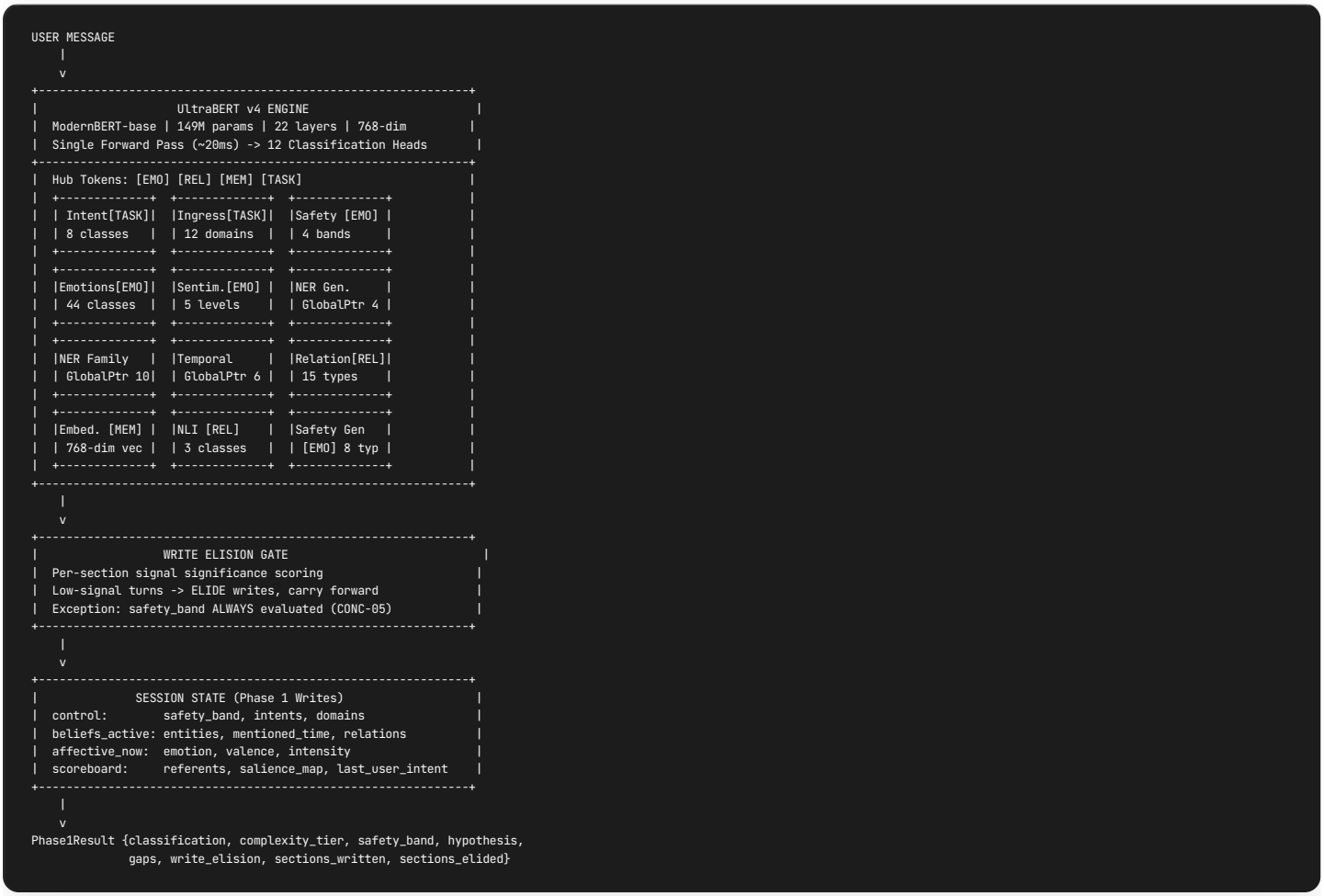
```python
@dataclass
class TurnResult:
    """Immutable result of a single turn through the FSM."""
    final_state: ConciergeState
    response: str
    needs_clarification: bool
    classification: Optional[ClassificationResult]
    tier: ComplexityTier
    tool_calls: List[Dict[str, Any]]
    tools_executed: int
    tools_blocked: int           # By ToolBundleValidator
    state_history: List[ConciergeState]  # Full path taken
    total_ms: int
    classification_ms: int       # Phase 1 latency
    execution_ms: int            # Phase 2 latency
    sections_written: int        # SessionState sections modified
    sections_elided: int         # SessionState sections carried forward
```

6. Phase 1 ACKING Deep Dive

6.1 Purpose

22ms deterministic classification – no LLM calls. Single forward pass through UltraBERT v4 produces structured understanding from 12 task heads. Populates up to 6 of 8 HOT CORE SessionState sections via the Write Elision Gate. Feeds Phase 2 with pre-grounded context so the LLM already knows safety band, intent, entities, emotions, and temporal anchors before generating a single token.

6.2 UltraBERT v4 Pipeline



6.3 Classification Heads (12 Heads, Exact Labels)

HEAD	HUB	TYPE	CLASSES	OUTPUT
`intent`	[TASK]	LabelDescriptionHead	8: `log_memory`, `query_memory`, `set_reminder`, `express_feeling`, `seek_advice`, `share_news`, `reflect`, `other`	Primary + all[] + scores{}
`ingress`	[TASK]	LabelDescriptionHead	12: `DIARY`, `TASK`, `HEALTH`, `FINANCE`, `RELATIONSHIP`, `WORK`, `META`, `MEMORY`, `PLANNING`, `CELEBRATION`, `CONCERN`, `GRATITUDE`	Domains[] above threshold
`safety_familyos`	[EMO]	Hierarchical	4 bands: `GREEN`, `AMBER`, `RED`, `CRISIS` (12 subcategories)	Band + subcategory
`emotions`	[EMO]	Multi-label (ASL)	44: 8 core + 12 positive + 10 negative + 14 family-specific	Top-K emotions + intensities
`sentiment`	[EMO]	Sequence	5: `very_negative`, `negative`, `neutral`, `positive`, `very_positive`	Valence mapped to -1.0..+1.0
`ner_general`	–	GlobalPointer	4: `PER`, `ORG`, `LOC`, `MISC`	Span[] with scores
`ner_family`	–	GlobalPointer	10: `PERSON`, `KINSHIP`, `NICKNAME`, `PET`, `HOME_LOC`, `FAMILY_EVENT`, `ROUTINE`, `TRADITION`, `MILESTONE`, `HEIRLOOM`	Span[] with scores
`temporal`	–	GlobalPointer	6: `DATE_ABS`, `DATE_REL`, `TIME`, `DURATION`, `FREQUENCY`, `AGE`	Span[] with scores
`relation`	[REL]	Pair	15: `no_relation`, `parent_of`, `child_of`, `spouse_of`, `sibling_of`, `grandparent_of`, `grandchild_of`, `aunt_uncle_of`, `niece_nephew_of`, `cousin_of`, `pet_of`, `friend_of`, `colleague_of`, `lives_at`, `owns`	Relation type + entity pair
`nli`	[REL]	Pair	3: `entailment`, `neutral`, `contradiction`	Class + confidence
`embedding`	[MEM]	Vector	768-dim dense	For memory recall queries (K0 P01)
`safety_generic`	[EMO]	Multi-label	8 Jigsaw toxicity types	Audit/logging only

6.4 Phase 1 -> SessionState Write Mapping

Which UltraBERT head outputs write to which SessionState HOT CORE sections:

SESSIONSTATE SECTION	FIELDS WRITTEN	SOURCE HEAD(S)	WHEN WRITTEN
control (8KB)	`safety_band`	`safety_familyos`	ALWAYS (CONC-05)
control (8KB)	`intents`	`intent`	If intent != `other` OR confidence > 0.5
control (8KB)	`domains[]`	`ingress`	If any domain above threshold
beliefs_active (8KB)	`mentioned_entities[]`	`ner_general` + `ner_family`	If any entities detected
beliefs_active (8KB)	`mentioned_time`	`temporal`	If any time expressions detected
beliefs_active (8KB)	entity graph enrichment	`relation`	If any relations detected
affective_now (4KB)	`current_emotion`, `intensity`	`emotions`	If non-neutral emotion detected
affective_now (4KB)	`valence`	`sentiment`	If sentiment changed from previous
scoreboard (6KB)	`referents[]`, `salience_map`	`ner_general` + `ner_family`	If any entities detected
scoreboard (6KB)	`last_user_intent`	`intent`	If intent != `other`
scoreboard (6KB)	`topic_stack`	`ingress`	If domains detected

Sections NOT touched by Phase 1 (written elsewhere):

SECTION	WHY NOT PHASE 1	WRITTEN BY
history_active (8KB)	Needs full assistant response	`turn_end()` (TurnProcessor)
clarifications (4KB)	Generated by sub-agents during Phase 2	Delta aggregation -> Concierge
narrative_active (4KB)	Requires Planner/Orchestrator thread analysis	Phase 2 cognitive tool `update_narrative()`
meta (2KB)	Only timestamps and counters	`turn_start()` / `turn_end()`

Heads with indirect use (not SessionState writes):

HEAD	USE IN PHASE 1	DESTINATION
`embedding` (768-dim)	Memory recall query vector	K0 via `IMemoryPort` (P01 pipeline)
`nli` (3 classes)	Contradiction detection in beliefs	Flags stale facts, does not write dedicated field
`safety_generic` (8 types)	Jigsaw toxicity logging	Telemetry/audit trail only

6.5 Write Elision Gate (Signal Significance)

Core problem: Not every user message carries new information. A message like “ummm what more?” produces empty/neutral outputs from all 12 heads. Writing these to SessionState is **wasteful** (unnecessary I/O) and **destructive** (overwrites meaningful emotion, belief, and entity state from the previous turn with empty/neutral values).

Design: The Write Elision Gate computes a per-section significance score from UltraBERT outputs. Sections with no new information are ELIDED – the previous turn’s values carry forward unchanged.

```python
@dataclass
class WriteElisionDecision:
    """Per-section write/elide decision from Phase 1."""
    control_safety: WriteAction     # ALWAYS_WRITE (CONC-05, invariant)
    control_intents: WriteAction    # WRITE if intent != 'other' or confidence > 0.5
    control_domains: WriteAction    # WRITE if any domain above threshold
    beliefs_active: WriteAction     # WRITE if entities or temporal detected
    affective_now: WriteAction      # WRITE if non-neutral emotion
    scoreboard: WriteAction         # WRITE if new referents detected


class WriteAction(Enum):
    WRITE = "write"       # New significant data detected
    ELIDE = "elide"       # No new data -- carry forward previous turn's state
    ALWAYS = "always"     # Invariant requires evaluation regardless


def compute_write_elision(
    classification: ClassificationResult,
    previous_state: HotTierSnapshot,
) -> WriteElisionDecision:
    """Determine which SessionState sections need Phase 1 writes."""

    return WriteElisionDecision(
        # CONC-05: safety_band ALWAYS evaluated before routing
        control_safety=WriteAction.ALWAYS,

        # Intent: only write if meaningful (not 'other' with low confidence)
        control_intents=(
            WriteAction.WRITE
            if classification.intent.primary != "other"
            or classification.intent.confidence > 0.5
            else WriteAction.ELIDE
        ),

        # Domains: only write if any domain crosses threshold
        control_domains=(
            WriteAction.WRITE
            if len(classification.ingress.domains) > 0
            else WriteAction.ELIDE
        ),

        # Beliefs: only write if entities or temporal spans detected
        beliefs_active=(
            WriteAction.WRITE
            if (len(classification.entities) > 0
                or len(classification.temporal) > 0
                or len(classification.relations) > 0)
            else WriteAction.ELIDE
        ),

        # Affect: only write if non-neutral emotion or sentiment changed
        affective_now=(
            WriteAction.WRITE
            if (classification.emotions.primary != "neutral"
                or classification.sentiment.label
                != previous_state.affective_now.sentiment_label)
            else WriteAction.ELIDE
        ),

        # Scoreboard: only write if new referents to track
        scoreboard=(
            WriteAction.WRITE
            if len(classification.entities) > 0
            else WriteAction.ELIDE
        ),
    )
```

Significance categories:

CATEGORY	SIGNAL PROFILE	EXAMPLE MESSAGES	PHASE 1 WRITES
Full Signal	All heads produce data	"Remind me to call Mom next Thursday at 3pm"	4 sections
Partial Signal	Some heads produce data	"I'm feeling anxious about work"	2 sections (affect + control)
Continuation	Intent only (low confidence)	"ummm what more?", "go on", "and then?"	1 section (safety, idempotent)
Backchannel	Zero signal	"ok", "yeah", "mmhmm", "sure", "got it"	1 section (safety, idempotent)
Filler	Zero signal	"hmm", "umm", "let me think"	1 section (safety, idempotent)

Estimated impact: Write elision reduces Phase 1 writes from 4 sections to 0-1 on ~30-40% of turns (backchannels, continuations, fillers), preserving previous turn's context without cost.

6.6 Walkthrough: "ummm what more?"

Step-by-step of a low-signal message through Phase 1:

```
1. User: "ummm what more?"
   |
2. UltraBERT v4 fires (22ms, all 12 heads):
   | intent:        other (confidence: 0.38)     -- below 0.5 threshold
   | ingress:       [] (no domains)              -- empty
   | safety:        GREEN (confidence: 0.99)     -- no concern
   | emotions:      [neutral: 0.82]              -- no meaningful emotion
   | sentiment:     neutral (confidence: 0.91)   -- unchanged from previous
   | ner_general:   []                           -- no entities
   | ner_family:    []                           -- no entities
   | temporal:      []                           -- no time expressions
   | relation:      []                           -- no relations
   | embedding:     [768-dim vector]             -- exists but generic
   | nli:           N/A (no pair input)
   | safety_generic: clean
   |
3. Write Elision Gate:
   | control_safety:  ALWAYS  -> write GREEN (same value, idempotent)
   | control_intents: ELIDE   -> keep previous turn's intent
   | control_domains: ELIDE   -> keep previous turn's domains
   | beliefs_active:  ELIDE   -> keep previous turn's entities/time
   | affective_now:   ELIDE   -> keep previous turn's emotion (e.g. "excited")
   | scoreboard:      ELIDE   -> keep previous turn's referents/salience
   |
   | Result: 1 write (safety_band, idempotent), 5 elisions
   |
4. Phase1Result:
   | complexity_tier: LOW (minimal signal -> simple continuation)
   | safety_band: GREEN
   | hypothesis: {primary: "continuation_request", confidence: 0.6,
   |              is_continuation: true}
   | gaps: []
   | sections_written: 1, sections_elided: 5
   | latency_ms: 22
   |
5. Phase 2 LLM:
   | Context includes FULL previous SessionState (emotions, beliefs, entities
   | from previous turn still intact -- not overwritten with neutral/empty)
   | LLM sees: user is continuing the conversation, respond with more detail
   | acknowledge() SKIPPED (trivial turn, per ACKING_CORE design)
```

Key benefit: Previous turn's `affective_now.current_emotion = "excited"` survives the backchannel. The LLM still knows the user's emotional state. Without write elision, `affective_now` would be overwritten with `neutral`, losing context.

6.7 Safety Gate

```python
# CONC-05: Safety Gate evaluates FIRST, before any routing
# CONC-06: CRISIS band = immediate protocol, bypasses all FSM states
if classification.safety_band == SafetyBand.CRISIS:
    # CRISIS_STATIC: hardcoded string, zero dependencies
    # Works even if every other component is down
    return CrisisOverride(response=CRISIS_STATIC_RESPONSE)

if classification.safety_band == SafetyBand.RED:
    # Elevated protocol -- bypass normal routing
    return await safety_protocol(classification)
```

Safety subcategories (12 types):

BAND	SUBCATEGORIES
GREEN	`none`
AMBER	`stress`, `mild_sadness`, `frustration`, `health_mention`
RED	`persistent_sadness`, `isolation`, `hopelessness`, `substance`
CRISIS	`self_harm_ideation`, `suicide_ideation`, `harm_to_others`, `abuse_disclosure`

6.8 Complexity Scoring

5-factor weighted scoring -> tier assignment:

FACTOR	WEIGHT	SOURCE	LOW SIGNAL BEHAVIOR
Intent complexity	0.25	`intent` head probabilities	`other` with low confidence -> 0
Domain count	0.20	`ingress` head	0 domains -> 0
Entity count	0.20	`ner_general` + `ner_family`	0 entities -> 0
Temporal complexity	0.15	`temporal` head	0 spans -> 0
Historical patterns	0.20	SessionState `history_active`	Previous turn context

Complexity -> Tier mapping:

COMPLEXITY SCORE	TIER	LATENCY BUDGET	TOKEN BUDGET
0.0 - 0.3	LOW	2s	500 tokens
0.3 - 0.6	MEDIUM	10s	2K tokens
0.6 - 1.0	HIGH	45s	8K tokens
CRISIS band	CRISIS	5s	Static response

6.9 Outputs

```python
@dataclass
class Phase1Result:
    classification: ClassificationResult       # All 12 head outputs
    complexity_tier: Literal["LOW", "MEDIUM", "HIGH"]
    safety_band: SafetyBand
    hypothesis: IntentHypothesis
    gaps: List[InformationGap]
    resolved_time: Optional[ResolvedTime]
    resolved_location: Optional[ResolvedLocation]
    write_elision: WriteElisionDecision        # Per-section write/elide decisions
    sections_written: int                       # Count of sections actually written
    sections_elided: int                        # Count of sections elided (carry-forward)
    latency_ms: float                           # Must be <= 22ms (P95: 25ms)
```

7. Phase 2 LLM Processing Deep Dive

7.1 Purpose

LLM-powered response generation with cognitive and action tools. Phase 2 starts in DISPATCHING and spans through COMPANIONING, PROGRESSING, and DELIVERING. The LLM operates in a **ReAct-style agentic tool loop**: call LLM -> execute tool calls -> feed results back -> repeat until LLM produces a final text response (no more tools). Max 10 iterations per turn (safety limit).

Phase 2 receives `Phase1Result` from ACKING, which provides pre-grounded context: safety band, classified intent, entities, emotions, temporal anchors, and complexity tier. The LLM never starts cold – it already knows the user's emotional state, what entities were mentioned, and what safety band applies.

7.2 acknowledge() Tool – First Signal to User

Core insight: When the LLM starts working, the user should see immediate feedback. The `acknowledge()` tool is emitted as the **FIRST tool call** in the LLM's response bundle. Its message is displayed to the user immediately (~50ms after Phase 2 starts) while the rest of the tool bundle executes.

Tool Schema:

```python
@tool(name="acknowledge")
def acknowledge(
    ack_type: Literal["commit", "progress", "closure"],
    message: str,
    next_tool: str,  # Enum of ALL registered tool names, or "none"
) -> AcknowledgeResult:
    """
    REQUIRED FIRST: Acknowledge user input before taking action.

    RULES:
    1. Must be called BEFORE any effectful tool (booking, sending, creating)
    2. Must be bundled with the next tool in the SAME response
    3. next_tool must match the actual tool called next
    4. Message must be SPECIFIC -- no 'got it', 'understood', 'noted'

    Example:
      acknowledge(ack_type='progress', message='Searching Sonoma hotels for Feb 7',
                  next_tool='search_accommodations')
      -> Then IMMEDIATELY call: search_accommodations(...)
    """
```

ack_type semantics:

ACK_TYPE	MEANING	WHEN USED	EXAMPLE
`commit`	State change happening	Writing beliefs, booking, updating persona	"Noting that Mike is allergic to peanuts"
`progress`	Work starting	Searches, lookups, multi-step tasks	"Searching Sonoma hotels for Feb 7"
`closure`	Branch complete	Task finished, wrapping up a thread	"Anniversary trip is fully booked"

ACK-first enforcement (Tier-1 ToolBundleValidator):

Before any tool bundle executes, the `ToolBundleValidator` enforces three rules:

RULE	CHECK	ON VIOLATION
ACK-first	Effectful tools require `acknowledge()` as first call in bundle	Reject bundle, re-prompt LLM
ACK-tool pairing	`next_tool` param must match actual next tool in bundle	Reject bundle, re-prompt LLM
Message quality	ACK message must not contain banned phrases ("got it", "understood", "noted", "I see", "let me")	Warning (non-blocking)

```
# Validation flow (before execution)
validation = validate_tool_bundle(tool_calls, strict=True)
if not validation.passed:
    # Reject and re-prompt LLM with error feedback
    messages.append({"role": "system", "content": f"Tool validation failed: {validation.errors}"})
    continue  # Next iteration of agentic loop
```

When acknowledge() is SKIPPED:

TURN TYPE	ACKNOWLEDGE()	WHY
Backchannel ("ok", "yeah")	SKIP	No action to preview, response is short text only
Continuation ("ummm what more?")	SKIP	Trivial turn, LLM responds with `next_tool="none"` or pure text
Greeting ("hi!")	SKIP	Simple response, no tools needed
Any turn with effectful tools	REQUIRED	User must see what action is about to happen

7.3 Tool Categories

CATEGORY	COUNT	TOOLS	PHASE 2 WRITE TARGET
Signal	1	`acknowledge(ack_type, message, next_tool)`	None (display only)
Cognitive	6	`update_scoreboard()`, `update_beliefs()`, `update_clarifications()`, `update_narrative()`, `refine_affect()`, `promote_belief()`	SessionState HOT sections
Read	3	`recall_memory()`, `discover_capabilities()`, `summarize_context()`	None (read-only)
Action	3	`invoke_capability()`, `spawn_via_fabric()`, `execute_workflow()`	External (Fabric/Orchestrator)

Total: 13 tools. The LLM sees only tools allowed for the current state + tier (see Section 7.6 Allowlist).

7.4 Dynamic Prompt Assembly

The system prompt is rebuilt every turn by `DynamicPromptBuilder`, injecting live SessionState context. This ensures the LLM never uses stale information.

```
[IDENTITY]
  You are the FamilyOS Concierge for {family_name}.
  You have READ/WRITE access to the family's SessionState via tools.
  You LEARN information through conversation -- never assume or pre-fill.

[PLAN CONTEXT] (from PlanController, if active)
  AUTHORITATIVE PLAN STATE:
  - Plan: {plan_name}
  - Current step: {step_n} of {total}
  - Completed: {completed_steps}
  - Pending: {remaining_steps}

[SESSION CONTEXT] (from SessionState sections)
  KNOWN FACTS:
    {beliefs_active -- learned preferences, allergies, family members}

  USER PREFERENCES:
    {persona -- communication style, interests, traits}

  CURRENT FOCUS:
    {scoreboard -- topic, QUD (question under discussion), active tasks}

  EMOTIONAL STATE:
    {affective_now -- emotion, intensity, valence}

  PENDING CLARIFICATIONS:
    {clarifications -- unresolved gaps from previous turns}

[ALREADY RESOLVED] (from ContextResolver)
  DO NOT ASK ABOUT THESE:
  - "next Saturday" = 2026-02-08 (LOCKED)
  - "Mom" = Sarah Chen (LOCKED)
  {resolved_refs -- temporal/entity references already disambiguated}

[CLASSIFICATION FROM PHASE 1]
  Intent: {phase1.classification.intent.primary} ({confidence})
  Domains: {phase1.classification.ingress.domains[]}
  Safety: {phase1.safety_band}
  Entities: {phase1.classification.entities[]}
  Emotions: {phase1.classification.emotions.primary} ({intensity})

[TOOLS AVAILABLE]
  {tool_schemas_for_current_state_and_tier}

[TOOL CALLING RULES]
  1. acknowledge() MUST be called FIRST before ANY effectful tool
  2. If acknowledge(next_tool="X"), you MUST call X immediately after
  3. Effectful tools without acknowledge -> REJECTED by validator
  4. For text-only: acknowledge(ack_type="progress", message="...", next_tool="none")

[RESPONSE STYLE]
  - SHORT when appropriate, DETAILED when needed
  - Greetings: 1-2 sentences max
  - NEVER repeat back everything the user said
  - Reference stored facts naturally ("I remember you mentioned...")
  - ALWAYS ask about dietary restrictions before booking food
  - Core principle: ASK, DON'T ASSUME

[CONSTRAINTS]
  - Max {budget.max_tool_calls} tool calls
  - Response within {budget.timeout_ms}ms
  - Max output: {budget.max_tokens} tokens
```

7.5 Agentic Tool Loop (ReAct Pattern)

The LLM does not execute a single call-response cycle. It runs a loop where each iteration can produce tool calls, which are executed and fed back, until the LLM produces a final text response.

```python
MAX_TOOL_ITERATIONS = 10  # Safety limit

iteration = 0
all_tool_calls = []
final_content = ""

while iteration < MAX_TOOL_ITERATIONS:
    iteration += 1

    # 1. Call LLM with current messages + tools
    response = await llm.complete_with_tools(
        system_prompt=dynamic_prompt,
        messages=messages,
        tools=allowed_tools_for_tier,
    )

    tool_calls = response.get("tool_calls", [])
    content = response.get("content", "")

    # 2. No tool calls? LLM gave final response. Done.
    if not tool_calls:
        final_content = content
        break

    # 3. Validate tool bundle (ACK-first rule, pairing, quality)
    if iteration == 1:
        validation = validate_tool_bundle(tool_calls, strict=True)
        if validation.errors:
            # Re-prompt with error feedback
            messages.append({"role": "system", "content": f"Errors: {validation.errors}"})
            continue

    # 4. Execute tools and collect results
    tool_results = []
    for call in tool_calls:
        result = tool_executor.execute(call["name"], call["args"])
        tool_results.append(result)

        # If acknowledge(), display message immediately to user
        if call["name"] == "acknowledge":
            await stream_to_user(result.data["formatted_message"])

    all_tool_calls.extend(tool_calls)

    # 5. Feed results back to LLM for next iteration
    messages.append({"role": "assistant", "content": f"[Executing: {tool_names}]"})
    messages.append({"role": "user", "content": f"Tool results:\n{results_text}\n\nContinue."})

# 6. Final response (after loop ends)
return LLMPhase2Response(
    response_text=final_content,
    tool_calls=all_tool_calls,
    iterations=iteration,
)
```

Typical iteration counts by tier:

TIER	ITERATIONS	TOOL CALLS	ACKNOWLEDGE	EXAMPLE
LOW	1-2	1-3	Yes (if effectful)	Greeting: 1 iter, 0 tools. Belief update: 2 iter, ack + add_belief.
MEDIUM	2-4	3-8	Yes	Search + book: ack -> search -> results -> ack -> book.
HIGH	3-6	5-15	Yes	Multi-step plan: ack -> search -> search -> compare -> ack -> book -> confirm.

7.5.1 Parallel Tool Execution Strategy (ADR-0078)

The default ReAct loop above executes tool calls **sequentially** (`for call in tool_calls`). ADR-0078 defines a **tiered parallelism** strategy that allows safe concurrent execution of independent tool calls within a single LLM iteration, without violating Single Writer.

Tool Dependency Graph (per iteration):

```
acknowledge_request()  ← MUST be first (user-facing, no deps)
      │
      ▼
┌─────────────────────────────────────────────┐
│  Group A: Read Tools (SAFE to parallelize)  │
│  recall_memory(), discover_capabilities(),  │
│  summarize_context()                        │
│  No SessionState writes, no side-effects.   │
│  → asyncio.gather() all reads in one batch. │
└─────────────────────────────────────────────┘
      │
      ▼
┌─────────────────────────────────────────────┐
│  Group B: Action Tools (parallelizable with A)  │
│  invoke_capability(), execute_workflow(),       │
│  spawn_via_fabric()                         │
│  External side-effects, but no SS writes.   │
│  → asyncio.gather() with reads when no deps.│
└─────────────────────────────────────────────┘
      │
      ▼
┌─────────────────────────────────────────────┐
│  Group C: Cognitive Tools (SEQUENTIAL — always)  │
│  update_scoreboard(), update_beliefs(),     │
│  update_clarifications(), update_narrative(),    │
│  refine_affect(), promote_belief()          │
│  Write SessionState via MutationGuard.      │
│  Order matters (belief → scoreboard deps).  │
│  → Execute one-by-one, preserve write ordering.  │
└─────────────────────────────────────────────┘
```

Parallel dispatch pseudocode (replaces sequential loop in Section 7.5 step 4):

```python
# 4. Execute tools with tiered parallelism (ADR-0078)
ack_calls = [c for c in tool_calls if c["name"] == "acknowledge"]
read_calls = [c for c in tool_calls if c["name"] in READ_TOOL_NAMES]
action_calls = [c for c in tool_calls if c["name"] in ACTION_TOOL_NAMES]
cognitive_calls = [c for c in tool_calls if c["name"] in COGNITIVE_TOOL_NAMES]

tool_results = []

# Phase A: Acknowledge first (user-facing, immediate)
for call in ack_calls:
    result = await tool_executor.execute(call["name"], call["args"])
    tool_results.append(result)
    await stream_to_user(result.data["formatted_message"])

# Phase B: Parallel reads + actions (no SS writes, safe to gather)
parallel_calls = read_calls + action_calls
if parallel_calls:
    parallel_results = await asyncio.gather(
        *(tool_executor.execute(c["name"], c["args"]) for c in parallel_calls)
    )
    tool_results.extend(parallel_results)

# Phase C: Sequential cognitive writes (order-dependent SS mutations)
for call in cognitive_calls:
    result = await tool_executor.execute(call["name"], call["args"])
    tool_results.append(result)
```

Estimated latency improvements:

SCENARIO	SEQUENTIAL	PARALLEL	IMPROVEMENT
3 read tools (recall + discover + summarize)	~600ms	~220ms	~63%
2 reads + 1 action	~800ms	~300ms	~62%
MEDIUM tier (ack + 2 reads + 2 cognitive)	~1200ms	~700ms	~42%
HIGH tier (ack + 3 reads + 1 action + 3 cognitive)	~2400ms	~1500ms	~37%

Invariant compliance: Cognitive tools remain sequential, preserving CONC-01 (Single Writer) and CONC-02 (MutationGuard preflight). Read and action tools have no write conflicts.

> **Cross-reference**: ADR-0078 (Tool Call Batching Pipeline), Section 23.2 (Async Task Topology).

7.6 Tool Allowlist by Tier

Tools available to the LLM are filtered by complexity tier to prevent over-execution on simple turns:

TIER	SIGNAL	COGNITIVE	READ	ACTION	MAX TOOLS/TURN
LOW	`acknowledge`	All 6	`recall_memory`, `summarize_context`	`invoke_capability`	6
MEDIUM	`acknowledge`	All 6	All 3	`invoke_capability`, `execute_workflow`	10
HIGH	`acknowledge`	All 6	All 3	All 3	15
CRISIS	None	None	None	None	0 (static response)

7.7 Context Resolution (Pre-LLM)

Before the LLM sees the prompt, the `ContextResolver` disambiguates references using Phase 1 NER + SessionState:

REFERENCE TYPE	SOURCE	RESOLUTION	EFFECT
Temporal ("next Saturday")	`temporal` head spans + user timezone	Absolute date (2026-02-08)	Injected as LOCKED in prompt
Entity ("Mom")	`ner_family` spans + `beliefs_active`	Resolved person (Sarah Chen)	Injected as LOCKED in prompt
Pronoun ("she")	`ner_general` + `scoreboard.referents`	Most salient referent	Injected as LOCKED in prompt
Ambiguous time ("at 9")	`temporal` head + context	09:00 vs 21:00 based on domain	If unresolvable, LLM asks (HITL)

Resolved references prevent the LLM from re-asking already-answered questions. They appear in the `[ALREADY RESOLVED]` section of the prompt with a `(LOCKED)` tag.

7.8 Walkthrough: "Remind me to call Mom next Thursday at 3pm"

```
Phase 1 (ACKING, 22ms):
  intent:     set_reminder (confidence: 0.92)
  ingress:    [TASK]
  safety:     GREEN
  ner_family: ["Mom" -> PERSON]
  temporal:   ["next Thursday" -> DATE_REL, "3pm" -> TIME]
  complexity: LOW (single intent, 1 entity, 1 time)

Context Resolution (pre-LLM):
  "Mom" -> Sarah Chen (from beliefs_active)    -> LOCKED
  "next Thursday" -> 2026-02-19                -> LOCKED
  "3pm" -> 15:00 (user timezone: America/Denver) -> LOCKED

Phase 2 (LLM, iteration 1):
  Prompt includes:
    ALREADY RESOLVED: Mom = Sarah Chen, next Thursday = Jan 30, 3pm = 15:00
    TOOLS: acknowledge, update_scoreboard, invoke_capability, ...

  LLM response (tool calls):
    1. acknowledge(ack_type="progress",
                   message="Setting reminder to call Sarah Chen on Jan 30 at 3pm",
                   next_tool="invoke_capability")
    2. invoke_capability(capability="reminder.create",
                         params={person: "Sarah Chen", date: "2026-02-19", time: "15:00"})

  -> acknowledge() displayed immediately to user
  -> invoke_capability() executes via Fabric

Phase 2 (LLM, iteration 2):
  Tool results fed back:
    [invoke_capability] SUCCESS: Reminder created, id=rem-4821

  LLM final response (no more tools):
    "Done! I'll remind you to call Sarah Chen next Thursday at 3pm."

Phase 2 writes (turn_end):
  scoreboard: task "reminder" -> completed
  beliefs_active: no new beliefs
  narrative_active: thread "reminder" closed

Total: ~1.2s, 2 tool calls, 2 LLM iterations
```

7.9 Structured Output Types

```python
@dataclass
class LLMPhase2Response:
    """Result of Phase 2 agentic tool loop."""
    response_text: str                          # Final text to user
    tool_calls: List[ToolCall]                  # All tools called across iterations
    iterations: int                             # Agentic loop iterations used
    acknowledge_message: Optional[str]          # The acknowledge() message (if any)
    scoreboard_updates: Optional[ScoreboardDelta]
    belief_updates: Optional[BeliefDelta]
    narrative_updates: Optional[NarrativeDelta]
    latency_ms: int                             # Total Phase 2 time
    tokens_used: int                            # Total tokens across iterations


@dataclass
class ToolCall:
    """A single tool call from the LLM."""
    name: str
    args: Dict[str, Any]
    iteration: int                # Which loop iteration produced this
    result: Optional[ToolResult]  # Populated after execution
    latency_ms: int


@dataclass
class ToolResult:
    """Result of executing a tool."""
    tool_name: str
    success: bool
    message: str
    data: Dict[str, Any]
    display_immediately: bool = False  # True for acknowledge()
```

8. Clarification System

8.1 Design Philosophy

The LLM handles clarification naturally, through human-in-the-loop (HITL) conversation – just as a real concierge would. There is no pre-LLM heuristic gap detection engine. No rule-based system decides what questions to ask. The LLM itself, with full SessionState context and Phase 1 classification, decides when it needs more information and asks the user directly.

This is the same pattern as a human conversation: if you don't have enough information to proceed, you ask. The CLARIFYING FSM state exists to track that the LLM has asked and is waiting for a user response.

8.2 How Clarification Works

```
Phase 2 (LLM agentic loop):
  LLM sees: intent = "plan something", entities = [], time = []
  LLM decides: "I need more info to proceed"
  LLM responds with text: "That sounds great! What kind of activity
    were you thinking, and when would work best?"

  -> FSM transitions: DISPATCHING -> CLARIFYING
  -> Response displayed to user
  -> FSM waits for user response

User responds: "A dinner this Saturday for our anniversary"

  -> FSM transitions: CLARIFYING -> ACKING
  -> Full Phase 1 re-classification on enriched input
  -> Phase 2 now has: intent = plan_event, entities = [dinner, anniversary],
     time = [this Saturday], plus previous context
  -> LLM proceeds with enough info
```

8.3 CLARIFYING State Behavior

The CLARIFYING state is a pause point in the FSM. It means: "the LLM asked the user something, we are waiting for their answer."

```python
# CLARIFYING state entry
async def enter_clarifying(llm_question: str) -> None:
    """
    LLM decided it needs more info. Pause Phase 2, wait for user.
    1. Display LLM's question to user
    2. Record clarification round in FSMContext
    3. Transition FSM to CLARIFYING
    4. Wait for user_response event
    """
    await stream_to_user(llm_question)
    fsm_context.pending_clarification = True
    fsm_context.clarification_count += 1

# CLARIFYING -> ACKING (on user response)
async def on_clarification_response(user_response: str) -> None:
    """
    User answered. Return to ACKING with enriched context.
    1. Merge user response into turn context
    2. If previous entities/time were resolved, keep them (LOCKED)
    3. Transition to ACKING for re-classification
    4. Phase 1 runs again on the new message
    5. Phase 2 restarts with richer context
    """
    fsm_context.pending_clarification = False
    # Re-enter ACKING with the new user message
    await process_input(user_response)
```

8.4 Clarification Guard Rails (CONC-08, CONC-09)

While the LLM handles clarification naturally, the FSM enforces hard limits:

GUARD	RULE	BEHAVIOR
Max rounds (CONC-08)	3 clarification rounds per intent	After 3 rounds, force proceed with best hypothesis. Never infinite loop.
Max questions (CONC-09)	2 questions per clarification	LLM should batch related questions, not ask one at a time.
No re-asking resolved	ContextResolver LOCKED refs	If “Mom” is already resolved to Sarah Chen, LLM cannot re-ask “which family member?”
Clarification timeout	5 minutes	If user doesn’t respond within timeout, transition to LISTENING (abandon turn).

```python
# Guard enforcement
if fsm_context.clarification_count >= 3:
    # CONC-08: Max 3 rounds. Force proceed with best hypothesis.
    logger.warning("Max clarification rounds reached, proceeding with best guess")
    classification = fsm_context.classification  # Use what we have
    # Skip CLARIFYING, go straight to DISPATCHING
    transition(ConciergeState.DISPATCHING)
```

8.5 What the LLM Sees (Prompt Context for Clarification)

When pending clarifications exist from previous turns, the prompt includes them:

```
[PENDING CLARIFICATIONS]
  - Missing: budget range - "What's your budget for the trip?"
  - Missing: party size - "How many people are going?"
```

This prevents the LLM from forgetting what it already asked. If the user’s response resolves one gap but not another, the remaining gap stays in the prompt for the next turn.

8.6 Clarification vs. Normal Conversation

Not every question the LLM asks is a “clarification.” The FSM distinguishes:

SCENARIO	FSM STATE	EXAMPLE
Clarification (missing info prevents action)	CLARIFYING	“Which restaurant did you want to book?” (can’t proceed without this)
Follow-up (gathering nice-to-have info)	Normal Phase 2	“Any dietary restrictions I should know about?” (can proceed without)
Proactive question (filling wait time)	COMPANIONING	“While I work on that – you mentioned Sarah’s birthday is coming up, any plans?” (K0-originated)

The CLARIFYING state is only used when the LLM **cannot proceed** without the answer. Follow-up questions happen inline during the normal agentic tool loop and don’t change FSM state.

8.7 Walkthrough: “Plan something nice”

```
Turn 1:
  Phase 1: intent = other (low confidence), entities = [], safety = GREEN
  Phase 2: LLM sees vague request, insufficient to act
  LLM: "I'd love to help with that! Are you thinking of a dinner,
        an outing, or maybe a weekend trip? And who would this be for?"
  FSM: LISTENING -> ACKING -> CLARIFYING (waiting for user)

Turn 2:
  User: "Anniversary dinner for me and Sarah, this Saturday"
  Phase 1 re-classification:
    intent = plan_event (0.88), entities = [Sarah:PERSON, dinner:EVENT],
    temporal = [this Saturday:DATE_REL -> 2026-02-21]
  Phase 2: LLM now has enough to proceed
  LLM: acknowledge(ack_type="progress",
         message="Searching anniversary dinner spots for Saturday",
         next_tool="invoke_capability")
       invoke_capability(capability="restaurant.search", ...)
  FSM: ACKING -> DISPATCHING -> COMPANIONING -> PROGRESSING -> DELIVERING
```

8.8 ClarificationTracker Interface

```python
@dataclass
class ClarificationTracker:
    """Tracks clarification state across turns within a single conversation."""

    # Per-intent tracking
    rounds: Dict[str, int] = field(default_factory=dict)
    questions_asked: Dict[str, List[str]] = field(default_factory=dict)

    # Resolved gaps (from ContextResolver LOCKED refs)
    resolved: Dict[str, Any] = field(default_factory=dict)

    def start_round(self, intent_id: str) -> None:
        """Record start of a clarification round for an intent."""
        self.rounds[intent_id] = self.rounds.get(intent_id, 0) + 1

    def should_force_proceed(self, intent_id: str) -> bool:
        """Check if max rounds exceeded (CONC-08: 3 rounds)."""
        return self.rounds.get(intent_id, 0) >= 3

    def record_question(self, intent_id: str, question: str) -> None:
        """Record a clarification question for dedup."""
        if intent_id not in self.questions_asked:
            self.questions_asked[intent_id] = []
        self.questions_asked[intent_id].append(question)

    def was_already_asked(self, intent_id: str, question: str) -> bool:
        """Prevent asking the same question twice."""
        return question in self.questions_asked.get(intent_id, [])

    def mark_resolved(self, key: str, value: Any) -> None:
        """Mark a gap as resolved (from user response)."""
        self.resolved[key] = value
```

9. Experience Layer Deep Dive

9.1 Purpose

Periodic heuristic enhancements that run every 20-30 turns. The Experience Layer is NOT per-turn processing – it is a background analysis layer that fires on modular intervals during `turn_end()`, using pure heuristics (no LLM calls). It updates SessionState sections that inform future turns, creating a sense of "the Concierge remembers and adapts."

The Experience Layer lives in the FSM as a separate subgraph (`FSM_EXPERIENCE_LAYER` in concierge.mmd) with four components: EMOTIONAL_PROCESSING, EMOTIONAL_MIRRORING, NARRATIVE_WEAVING, and ANTICIPATORY_RESPONSE.

9.2 Experience Triggers and Scheduling

Triggers fire at the end of `turn_end()`, checked against `meta.turn_count`:

COMPONENT	TRIGGER	INTERVAL	INPUT	OUTPUT	SESSIONSTATE WRITE
EMOTIONAL_PROCESSING	`turn_count % 25 == 0`	Every 25 turns	`affective_now` from last 25 turns	`EmotionalTrajectory`	`affective_now.trajectory`
EMOTIONAL_MIRRORING	After EMOTIONAL_PROCESSING	Same cycle	`EmotionalTrajectory` + `persona`	Tone adjustment hints	`persona.tone_hints`
NARRATIVE_WEAVING	`turn_count % 20 == 0`	Every 20 turns	`history_active` + `scoreboard`	`NarrativeCluster[]`	`narrative_active.clusters`
ANTICIPATORY_RESPONSE	`turn_count % 30 == 0`	Every 30 turns	`history_active` + `beliefs_active` + `scoreboard`	`Anticipation`	`scoreboard.anticipated_needs`

```python
# Experience Layer trigger check (called in turn_end)
def check_experience_triggers(turn_count: int) -> List[ExperienceTask]:
    """Check which experience components should fire this turn."""
    tasks = []

    if turn_count % 25 == 0:
        tasks.append(ExperienceTask.EMOTIONAL_PROCESSING)
        tasks.append(ExperienceTask.EMOTIONAL_MIRRORING)  # Always follows

    if turn_count % 20 == 0:
        tasks.append(ExperienceTask.NARRATIVE_WEAVING)

    if turn_count % 30 == 0:
        tasks.append(ExperienceTask.ANTICIPATORY_RESPONSE)

    return tasks
```

9.3 Emotional Processing

Analyzes the emotional trajectory over a window of 25 turns. Detects trends (improving, declining, volatile) and identifies dominant emotion patterns. Pure heuristic – no LLM.

```python
@dataclass
class EmotionalTrajectory:
    """Computed from last 25 turns' affective_now snapshots."""
    emotion_sequence: List[EmotionState]  # Ordered by turn
    trend: Literal["IMPROVING", "STABLE", "DECLINING", "VOLATILE"]
    dominant_emotion: str                 # Most frequent primary emotion
    intensity_avg: float                  # Mean intensity over window
    intensity_variance: float             # Stability measure
    transitions: int                      # Number of emotion shifts
    notable_shifts: List[EmotionShift]    # Significant changes (delta > 0.3)


@dataclass
class EmotionShift:
    """A notable emotional transition detected in the window."""
    from_emotion: str
    to_emotion: str
    turn_number: int
    intensity_delta: float


def compute_emotional_trajectory(
    snapshots: List[AffectiveSnapshot],  # Last 25 turns
) -> EmotionalTrajectory:
    """
    Heuristic emotional trajectory computation.

    Trend classification:
    - IMPROVING: intensity_avg declining AND dominant is positive
    - DECLINING: intensity_avg increasing AND dominant is negative
    - VOLATILE: transitions > 5 in 25-turn window
    - STABLE: otherwise
    """
    # Count emotion frequencies
    emotion_counts = Counter(s.primary_emotion for s in snapshots)
    dominant = emotion_counts.most_common(1)[0][0]

    # Compute intensity stats
    intensities = [s.intensity for s in snapshots]
    avg = sum(intensities) / len(intensities)
    variance = sum((i - avg) ** 2 for i in intensities) / len(intensities)

    # Detect shifts
    transitions = 0
    notable_shifts = []
    for i in range(1, len(snapshots)):
        if snapshots[i].primary_emotion != snapshots[i-1].primary_emotion:
            transitions += 1
            delta = abs(snapshots[i].intensity - snapshots[i-1].intensity)
            if delta > 0.3:
                notable_shifts.append(EmotionShift(
                    from_emotion=snapshots[i-1].primary_emotion,
                    to_emotion=snapshots[i].primary_emotion,
                    turn_number=snapshots[i].turn_number,
                    intensity_delta=delta,
                ))

    # Classify trend
    if transitions > 5:
        trend = "VOLATILE"
    elif avg < 0.4 and dominant in POSITIVE_EMOTIONS:
        trend = "IMPROVING"
    elif avg > 0.6 and dominant in NEGATIVE_EMOTIONS:
        trend = "DECLINING"
    else:
        trend = "STABLE"

    return EmotionalTrajectory(
        emotion_sequence=[s.primary_emotion for s in snapshots],
        trend=trend,
        dominant_emotion=dominant,
        intensity_avg=avg,
        intensity_variance=variance,
        transitions=transitions,
        notable_shifts=notable_shifts,
    )
```

9.4 Emotional Mirroring

Always fires immediately after EMOTIONAL_PROCESSING. Adjusts conversational tone hints in `persona` section based on the trajectory. These hints are injected into the Phase 2 prompt on subsequent turns.

```python
@dataclass
class ToneAdjustment:
    """Tone hint for future LLM prompts."""
    warmth_level: Literal["LOW", "MEDIUM", "HIGH"]  # How warm/comforting
    formality: Literal["CASUAL", "NEUTRAL", "FORMAL"]  # Register
    encouragement: bool  # Include encouraging phrases
    caution_topics: List[str]  # Topics to handle gently
    mirroring_note: str  # Free-text hint for LLM


def compute_tone_adjustment(
    trajectory: EmotionalTrajectory,
    current_persona: PersonaSection,
) -> ToneAdjustment:
    """
    Map emotional trajectory to conversational tone.

    Rules:
    - DECLINING trend: HIGH warmth, encouragement ON, avoid triggers
    - VOLATILE: MEDIUM warmth, gentle topic handling
    - IMPROVING: Match positivity, celebrate wins
    - STABLE: Maintain current tone
    """
    if trajectory.trend == "DECLINING":
        return ToneAdjustment(
            warmth_level="HIGH",
            formality="CASUAL",
            encouragement=True,
            caution_topics=[s.to_emotion for s in trajectory.notable_shifts
                            if s.to_emotion in NEGATIVE_EMOTIONS],
            mirroring_note="User has been feeling down recently. Be extra warm and supportive.",
        )
    elif trajectory.trend == "VOLATILE":
        return ToneAdjustment(
            warmth_level="MEDIUM",
            formality="NEUTRAL",
            encouragement=False,
            caution_topics=["stress", "pressure", "deadlines"],
            mirroring_note="User emotions are fluctuating. Be steady and grounding.",
        )
    elif trajectory.trend == "IMPROVING":
        return ToneAdjustment(
            warmth_level="MEDIUM",
            formality="CASUAL",
            encouragement=True,
            caution_topics=[],
            mirroring_note="User is in a good trajectory. Match their positive energy.",
        )
    else:  # STABLE
        return ToneAdjustment(
            warmth_level="MEDIUM",
            formality=current_persona.preferred_formality,
            encouragement=False,
            caution_topics=[],
            mirroring_note="Maintain current conversational style.",
        )
```

9.5 Narrative Weaving

Clusters conversation threads from `history_active` into coherent narratives. Identifies open topics (QUD – Question Under Discussion), completed threads, and recurring themes. Updates `narrative_active` section.

```python
@dataclass
class NarrativeCluster:
    """A conversation thread identified by Narrative Weaving."""
    thread_id: str                    # Unique identifier
    topic: str                        # Primary topic label
    domains: List[str]                # From ingress classification
    turn_range: Tuple[int, int]       # First and last turn in this thread
    turns: List[int]                  # All participating turns
    summary: str                      # Heuristic summary (turn-topic pairs)
    open_questions: List[str]         # QUD - unresolved from scoreboard
    status: Literal["ACTIVE", "DORMANT", "RESOLVED"]
    last_active_turn: int


@dataclass
class NarrativeWeaveResult:
    """Output of the Narrative Weaving component."""
    clusters: List[NarrativeCluster]
    active_threads: int               # Currently ACTIVE clusters
    dormant_threads: int              # No activity in last 10 turns
    resolved_threads: int             # Explicitly closed
    theme_frequency: Dict[str, int]   # Domain -> occurrence count


def weave_narratives(
    history: List[HistoryEntry],      # Last 20 turns from history_active
    scoreboard: ScoreboardSection,    # Current focus, QUD
) -> NarrativeWeaveResult:
    """
    Cluster turns into narrative threads.

    Algorithm (heuristic, no LLM):
    1. Group consecutive turns by dominant domain (from ingress classification)
    2. Merge groups with same domain within 3-turn gap
    3. Extract QUD from scoreboard for each active thread
    4. Mark threads as DORMANT if no activity in 10 turns
    5. Mark threads as RESOLVED if scoreboard shows task completion
    """
    ...
```

9.6 Anticipatory Response

Predicts what the user might need next based on patterns in conversation history and beliefs. Prefetches context via Fabric so that if the prediction is correct, the next turn is faster.

```python
@dataclass
class Anticipation:
    """A prediction of what the user might do next."""
    predicted_intent: str             # e.g., "query_memory", "set_reminder"
    predicted_domain: str             # e.g., "HEALTH", "PLANNING"
    confidence: float                 # 0-1, only act if > 0.6
    reasoning: str                    # Why this prediction (for telemetry)
    prefetched_context: Dict[str, Any] # Pre-loaded data from Fabric/K0
    suggested_response_seed: str      # Optional: opening sentence hint


def compute_anticipation(
    history: List[HistoryEntry],      # Last 30 turns
    beliefs: BeliefsSection,          # Active beliefs
    scoreboard: ScoreboardSection,    # Current tasks and focus
) -> Optional[Anticipation]:
    """
    Predict next user need based on patterns.

    Heuristics:
    - If active task in scoreboard, predict continuation
    - If recurring domain pattern (e.g., HEALTH every morning), predict repeat
    - If time-sensitive belief (appointment tomorrow), predict reminder
    - If emotional trend DECLINING, predict need for companionship
    """
    # Check scoreboard for active tasks
    if scoreboard.active_tasks:
        task = scoreboard.active_tasks[0]
        return Anticipation(
            predicted_intent="continuation",
            predicted_domain=task.domain,
            confidence=0.7,
            reasoning=f"Active task '{task.name}' likely to continue",
            prefetched_context=prefetch_for_task(task),
            suggested_response_seed=f"Ready to continue with {task.name}",
        )

    # Check time-sensitive beliefs
    upcoming = [b for b in beliefs.items if b.is_time_sensitive
                and b.deadline_within_hours(24)]
    if upcoming:
        return Anticipation(
            predicted_intent="set_reminder",
            predicted_domain="TASK",
            confidence=0.65,
            reasoning=f"Time-sensitive belief: {upcoming[0].subject}",
            prefetched_context={"upcoming_item": upcoming[0].to_dict()},
            suggested_response_seed=None,
        )

    return None  # No confident prediction
```

9.7 Proactive Agent (HIGH Tier Gap-Filling)

During HIGH tier flows (>30s processing), the Proactive Agent fills idle time by asking questions **originating from K0 kernel** about **past conversations** – not the current task. This serves two purposes: (1) keeps the user engaged during long waits, (2) fills information gaps from previous sessions that K0 has identified.

```python
@dataclass
class ProactiveQuestion:
    """A gap-filling question generated by K0 kernel."""
    source: Literal["K0_MEMORY_GAP", "K0_BELIEF_STALE", "K0_RELATIONSHIP_UPDATE"]
    question: str
    context: str           # Why K0 is asking this
    priority: float        # 0-1, higher = more relevant to fill now
    related_belief_id: Optional[str]  # If updating a specific belief


class ProactiveAgent:
    """
    Fills conversation gaps during HIGH tier wait times.

    Design: Questions originate from K0 kernel (long-term memory analysis),
    not from the current task. The Proactive Agent wraps them in natural
    conversation:

    "While I'm working on that trip plan, I had a quick question --
     you mentioned last week that Emma started piano lessons.
     How's that going?"
    """

    async def generate_fill_message(
        self,
        wait_elapsed_ms: int,
        k0_questions: List[ProactiveQuestion],
    ) -> Optional[str]:
        """
        Generate a gap-filling message if enough time has passed.

        Rules:
        - Only fire after 5s of wait time (don't interrupt quick tasks)
        - Max 1 proactive question per HIGH-tier turn
        - Frame as natural conversation, not interrogation
        - Yield immediately when backend results arrive
        """
        if wait_elapsed_ms < 5000:
            return None  # Too early

        if not k0_questions:
            # No K0 gaps to fill -- generate warmth instead
            return "Still working on that for you -- just pulling together the details..."

        # Pick highest priority question
        question = sorted(k0_questions, key=lambda q: q.priority, reverse=True)[0]

        # Frame naturally
        return (
            f"While I'm working on that -- {question.context}. "
            f"{question.question}"
        )
```

Proactive Agent flow (HIGH tier only):

```
COMPANIONING state (HIGH tier, task submitted to Orchestrator):
  t=0ms:   acknowledge() displayed to user
  t=5000ms: Proactive Agent checks K0 for gap-filling questions
  t=5100ms: "While I'm working on that trip plan, I had a quick
             question -- you mentioned Emma started piano lessons
             last week. How's that going?"
  t=5200ms: User responds: "She loves it! Practicing every day."
            -> K0 belief updated: Emma -> piano -> enjoying
  t=35000ms: Orchestrator returns results
  t=35100ms: Proactive Agent yields, PROGRESSING takes over
  t=36000ms: Final response with trip plan results
```

9.8 Experience Layer Latency Budget

COMPONENT	LATENCY	WHEN	BLOCKING?
Emotional Processing	<5ms	turn_end(), every 25 turns	Non-blocking (background)
Emotional Mirroring	<2ms	Immediately after Emotional Processing	Non-blocking
Narrative Weaving	<10ms	turn_end(), every 20 turns	Non-blocking (background)
Anticipatory Response	<15ms	turn_end(), every 30 turns	Non-blocking (prefetch async)
Proactive Agent	LLM-dependent	COMPANIONING, HIGH tier only	Non-blocking (streams to user)

All heuristic components run asynchronously during `turn_end()` and do not block the next turn. Their results are written to SessionState and picked up by the next turn's Phase 2 prompt assembly.

10. LLM Tools Deep Dive

10.1 Tool Taxonomy (13 Tools, 4 Categories)

Every tool the Concierge LLM can call during Phase 2 belongs to exactly one of four categories. Category determines validation rules, write behavior, and tier availability.

CATEGORY	COUNT	BAND	WRITE TARGET	VALIDATION
Signal	1	GREEN	None (display only)	ACK-first rule (Section 7.2)
Cognitive	6	GREEN	SessionState HOT via MutationGuard	Preflight + size check (CONC-01, CONC-02)
Read	3	GREEN	None (read-only, no state mutation)	Input schema validation
Action	3	AMBER+	External (Fabric / Orchestrator)	Tier allowlist + schema + CB

Naming Convention: Tool names follow the Fabric canonical pattern `tool.{band}.{verb}_{noun}` for Read/Action tools. Cognitive and Signal tools use short names because they are internal to Concierge (never registered in Fabric Registry).

10.2 Signal Tools (1)

10.2.1 `acknowledge()`

Immediate intent confirmation displayed to user before any effectful work begins. This is the only tool that produces a user-visible message mid-loop. See Section 7.2 for ACK-first rule and ToolBundleValidator enforcement.

```
@tool(name="acknowledge")
async def acknowledge(
    ack_type: Literal["commit", "progress", "closure"],
    message: str,
    next_tool: str,
) -> AcknowledgeResult:
    """
    Emit intent confirmation to user (~50ms display).

    Args:
        ack_type: Signal category (see Section 7.2 for full semantics).
            - "commit": State change happening (writing beliefs, booking, updating)
            - "progress": Work starting (searches, lookups, multi-step tasks)
            - "closure": Branch complete (task finished, wrapping up a thread)
        message: User-facing text (max 150 tokens, CONC-22 envelope).
            Banned phrases: "I cannot", "I don't have access", "I'm sorry but".
        next_tool: Name of the effectful tool to be called immediately after,
            or "none" for text-only responses (greeting, continuation).

    Returns:
        AcknowledgeResult:
            displayed: bool       # True if SSE delivery confirmed
            formatted_message: str  # As displayed to user
            display_latency_ms: int # Time to SSE delivery

    Enforcement:
        - ToolBundleValidator checks acknowledge() is FIRST in iteration 1
        - If next_tool != "none", the named tool MUST appear in same iteration
        - Effectful tools without preceding acknowledge -> REJECTED
    """
```

Execution path: `ToolDispatcher` -> `OutputManager.send_immediate()` -> `IOutputPort` (SSE) -> user display. Zero SessionState writes. Zero Fabric calls.

10.3 Cognitive Tools (6)

All cognitive tools write to SessionState HOT sections via `IStatePort`. Every write passes through `MutationGuard.preflight()` (CONC-01) using the `MutationRequest` / `MutationResponse` protocol from `k1.sessionstate.adapters.direct_writer`:

```
# Shared write path for all cognitive tools
async def _cognitive_write(section: str, operation: str, data: Dict, estimated_bytes: int) -> MutationResponse:
    """
    Common write path. Called by every cognitive tool after computing delta.

    Flow:
    1. Build MutationRequest(caller_id="concierge", section, operation, data, estimated_bytes)
    2. IStatePort.write() -> DirectWriterAdapter.request_mutation()
       a. Validate writer authorization (writer_id in authorized_writers)
       b. MutationGuard.preflight(section, operation, estimated_bytes)
          - Verify caller is concierge
          - Verify section not locked
          - Verify HOT tier <= 48KB (CONC-20)
       c. Apply mutation via SessionStateManager.mutate()
    3. Return MutationResponse (APPLIED | REJECTED | FAILED)
    """
```

Rejection categories (from `RejectionCategory` enum): `AUTHORIZATION`, `CAPACITY`, `LOCKED`, `VALIDATION`, `EMERGENCY`, `INTERNAL`.

10.3.1 `update_scoreboard()`

Task tracking, referent resolution, QUD stack, salience map. Primary Phase 2 write target.

```python
@tool(name="update_scoreboard")
async def update_scoreboard(
    operation: Literal["upsert_task", "resolve_referent", "push_qud", "pop_qud",
                       "update_salience", "shift_topic"],
    task_id: Optional[str] = None,
    status: Optional[Literal["pending", "in_progress", "completed", "failed"]] = None,
    progress_pct: Optional[float] = None,
    referent: Optional[str] = None,
    resolved_to: Optional[str] = None,
    qud: Optional[str] = None,
    topic: Optional[str] = None,
    salience_scores: Optional[Dict[str, float]] = None,
) -> ScoreboardUpdateResult:
    """
    Update task tracking and discourse state in scoreboard section (6KB max).

    Operations:
        upsert_task:       Create or update task entry. Requires task_id + status.
        resolve_referent:  Map referent to resolved entity. Requires referent + resolved_to.
        push_qud:          Push Question Under Discussion. Requires qud.
        pop_qud:           Pop top QUD (no extra args).
        update_salience:   Update salience map. Requires salience_scores.
        shift_topic:       Record topic change. Requires topic.

    Returns:
        ScoreboardUpdateResult:
            success: bool
            section_bytes: int      # Current scoreboard size post-write
            available_bytes: int    # Remaining within 6KB budget
    """
```

Write target: `scoreboard` (6KB budget within HOT 48KB).

10.3.2 `update_beliefs()`

Maintain family knowledge graph. Facts, preferences, corrections, invalidations.

```python
@tool(name="update_beliefs")
async def update_beliefs(
    operation: Literal["add_fact", "correct_fact", "invalidate_fact", "add_preference"],
    subject: str,
    predicate: str,
    object_value: str,
    confidence: float = 0.8,
    source: Literal["user_stated", "inferred", "tool_result"] = "user_stated",
    evidence: Optional[str] = None,
) -> BeliefUpdateResult:
    """
    Maintain family beliefs in beliefs_active section (8KB max).

    Operations:
        add_fact:         Add new belief triple. ("Mom", "allergic_to", "peanuts")
        correct_fact:     Override existing belief. ("Mom", "phone_number", "555-0199")
        invalidate_fact:  Mark belief as no longer valid. ("Dad", "works_at", "Acme")
        add_preference:   Record user preference. ("Sarah", "prefers", "Italian food")

    Args:
        subject: Entity name (must match ner_family or ner_general span, or existing belief).
        predicate: Relationship type (from relation head types or free text).
        object_value: Target value.
        confidence: Belief confidence 0.0-1.0 (user_stated defaults to 0.95).
        source: How this belief was learned.
        evidence: Supporting text from conversation.

    Returns:
        BeliefUpdateResult:
            success: bool
            belief_id: str          # UUID assigned to this belief
            is_correction: bool     # True if overrode existing
            section_bytes: int
    """
```

Write target: `beliefs_active` (8KB budget). Beliefs that persist across sessions are promoted to K0 via `promote_belief()`.

10.3.3 `update_clarifications()`

Record semantic gaps and their resolutions. Feeds the `[PENDING CLARIFICATIONS]` section of future prompts.

```python
@tool(name="update_clarifications")
async def update_clarifications(
    operation: Literal["record_gap", "resolve_gap", "expire_gap"],
    gap_type: Optional[Literal["ENTITY_MISSING", "TIME_AMBIGUOUS",
                               "REFERENCE_UNRESOLVED", "INTENT_AMBIGUOUS",
                               "CONSTRAINT_UNCLEAR"]] = None,
    gap_id: Optional[str] = None,
    description: Optional[str] = None,
    resolution: Optional[str] = None,
    intent_id: Optional[str] = None,
) -> ClarificationUpdateResult:
    """
    Track clarification state in clarifications section (4KB max).

    Operations:
        record_gap:   Register a new information gap. Requires gap_type + description.
        resolve_gap:  Mark gap as resolved. Requires gap_id + resolution.
        expire_gap:   Remove stale gap (timeout or superseded). Requires gap_id.

    Returns:
        ClarificationUpdateResult:
            success: bool
            gap_id: str             # Assigned or provided
            open_gaps_count: int    # Remaining unresolved gaps
    """
```

Write target: `clarifications` (4KB budget). Integrates with `ClarificationTracker` (Section 8.8).

10.3.4 `update_narrative()`

Thread management, conversation structure, discourse state.

```python
@tool(name="update_narrative")
async def update_narrative(
    operation: Literal["new_thread", "switch_thread", "resume_thread",
                       "continue_thread", "close_thread"],
    thread_id: Optional[str] = None,
    topic: Optional[str] = None,
    domains: Optional[List[str]] = None,
    reason: Optional[str] = None,
) -> NarrativeUpdateResult:
    """
    Manage conversation threads in narrative_active section (4KB max).

    Operations:
        new_thread:      Start a new conversation thread. Requires topic + domains.
        switch_thread:   Pause current, activate different thread. Requires thread_id.
        resume_thread:   Return to a DORMANT thread. Requires thread_id.
        continue_thread: Mark current thread as still active (for long-running topics).
        close_thread:    Mark thread as RESOLVED. Requires thread_id.

    Returns:
        NarrativeUpdateResult:
            success: bool
            thread_id: str
            active_threads: int     # Currently ACTIVE thread count
            thread_status: str      # New status of touched thread
    """
```

Write target: `narrative_active` (4KB budget). Clusters feed back into `NarrativeWeaving` experience component (Section 9.5).

10.3.5 `refine_affect()`

Override UltraBERT Phase 1 emotion classification when the LLM detects nuance (sarcasm, irony, mixed signals) that the classifier missed.

```python
@tool(name="refine_affect")
async def refine_affect(
    override_emotion: str,
    override_intensity: float,
    override_valence: Literal["positive", "negative", "neutral", "mixed"],
    reasoning: str,
) -> AffectRefinementResult:
    """
    Override Phase 1 emotion in affective_now section (4KB max).

    This tool is RARE -- only called when the LLM detects emotional nuance
    that UltraBERT missed (sarcasm, understatement, cultural context).
    Phase 1 writes are generally correct and should not be overridden casually.

    Args:
        override_emotion: Corrected primary emotion (from 44-class Plutchik taxonomy).
        override_intensity: Corrected intensity 0.0-1.0.
        override_valence: Corrected valence direction.
        reasoning: Why the LLM disagrees with UltraBERT (for observability).

    Returns:
        AffectRefinementResult:
            success: bool
            previous_emotion: str      # What UltraBERT said
            previous_intensity: float
            override_applied: bool     # True if write succeeded
    """
```

Write target: `affective_now` (4KB budget). This tool mutates a section already written by Phase 1 – `MutationGuard` allows because caller is still `concierge`.

10.3.6 `promote_belief()`

Move a belief from WARM tier (`beliefs_history`) to HOT tier (`beliefs_active`), or persist a HOT belief to K0 long-term memory via Bridge.

```python
@tool(name="promote_belief")
async def promote_belief(
    belief_id: str,
    direction: Literal["warm_to_hot", "hot_to_k0"],
    reason: Optional[str] = None,
) -> PromotionResult:
    """
    Promote belief across storage tiers.

    Directions:
        warm_to_hot: Copy belief from beliefs_history (WARM 48KB) to beliefs_active
                     (HOT 8KB). Used when user references old context ("remember when...").
        hot_to_k0:   Persist belief to K0 long-term memory via IMemoryPort.store().
                     Used for durable facts that should survive session rotation.

    Args:
        belief_id: UUID of the belief to promote.
        direction: Which promotion path.
        reason: Why this belief is being promoted (observability).

    Returns:
        PromotionResult:
            success: bool
            belief_id: str
            destination: str        # "beliefs_active" or "k0"
            k0_store_receipt: Optional[str]  # K0 acknowledgment ID (hot_to_k0 only)
    """
```

Write targets:

- `warm_to_hot`: reads `beliefs_history` (WARM), writes `beliefs_active` (HOT) via `IStatePort`
- `hot_to_k0`: reads `beliefs_active` (HOT), writes K0 via `IMemoryPort.store()`

10.4 Read Tools (3)

Read tools perform no state mutation. They query external systems and return data for the LLM to reason over. All operate in GREEN safety band.

10.4.1 `recall_memory()`

Query K0 long-term memory via Bridge. Returns facts, beliefs, conversation summaries from cross-device synchronized storage.

```python
@tool(name="recall_memory")
async def recall_memory(
    query: str,
    selectors: Optional[List[Literal["beliefs", "conversations", "events",
                                     "relationships", "preferences"]]] = None,
    time_range: Optional[Dict[str, str]] = None,
    max_results: int = 5,
) -> MemoryRecallResult:
    """
    Query K0 long-term memory via IMemoryPort.recall().

    Args:
        query: Natural language query for semantic search.
        selectors: Memory categories to search (default: all).
        time_range: Optional {"start": ISO8601, "end": ISO8601} filter.
        max_results: Maximum results to return (default: 5, max: 20).

    Returns:
        MemoryRecallResult:
            results: List[MemoryItem]  # Ranked by relevance
            total_matched: int
            query_latency_ms: int
            source: str              # "k0_bridge" or "local_cold_fallback"

    Execution path:
        ToolDispatcher -> IMemoryPort.recall(MemoryQuery) -> BridgeRecallAdapter
           -> Bridge Client -> K0 Cloud
        Fallback (CB_BRIDGE OPEN): query LOCAL COLD SQLite archive instead

    Latency: 50-200ms (K0), <5ms (local fallback)
    """
```

Port: `IMemoryPort` -> `BridgeRecallAdapter`. Circuit breaker: `CB_BRIDGE` (Concierge-owned).

10.4.2 `discover_capabilities()`

Query Fabric Capability Registry for available tools, agents, and workflows. Maps to the Fabric `tool.read.discover_capabilities` handler (`DiscoverCapabilitiesHandler` in `k1.fabric.core.discovery_tools`).

```python
@tool(name="discover_capabilities")
async def discover_capabilities(
    intent: str,
    domain: List[str],
    safety_band: Optional[str] = None,
    top_k: int = 10,
) -> DiscoveryResult:
    """
    Discover available capabilities from Fabric Registry.

    This tool wraps the Fabric tool.read.discover_capabilities handler,
    which performs semantic search across the registry:
      1. Embed query (< 5ms)
      2. Hard filter (safety, availability, inputs) (< 2ms)
      3. Soft rank (semantic x 0.4 + domain x 0.3 + success x 0.15 + cost/latency x 0.15)
      4. Top-K selection (< 1ms)

    Args:
        intent: Natural language description of what the user wants to do.
        domain: Domain tags to filter by (e.g., ["HEALTH", "FINANCE"]).
        safety_band: Minimum safety band filter (default: current session band).
        top_k: Number of top results (default: 10, max: 50).

    Returns:
        DiscoveryResult:
            capabilities: List[ScoredCapability]  # Ranked by relevance
            total_matched: int
            query_latency_ms: int

    Execution path:
        ToolDispatcher -> IDispatchPort.dispatch_direct(
            CapabilityRequest(name="tool.read.discover_capabilities", params={...})
        ) -> Fabric -> DiscoverCapabilitiesHandler -> RetrievalEngine

    Latency: <20ms (10K caps), <50ms (100K caps)
    Tier availability: MEDIUM, HIGH (not available at LOW -- LOW already has resolved capability)
    """
```

Port: `IDispatchPort.dispatch_direct()` -> Fabric `CapabilityRequest`. This is a Fabric execution that hits `DiscoverCapabilitiesHandler` which uses `RetrievalLike.discover_capabilities()`.

10.4.3 `summarize_context()`

Generate a compressed summary of current SessionState for token budget management. Used when context exceeds 80% of the 128K token window (CONC-22).

```python
@tool(name="summarize_context")
async def summarize_context(
    sections: Optional[List[str]] = None,
    strategy: Literal["extractive", "abstractive", "hybrid"] = "hybrid",
    target_tokens: Optional[int] = None,
) -> ContextSummary:
    """
    Generate context summary from SessionState sections.

    Uses ContextAssembler's token budget management. Not a Fabric call --
    this is internal to Concierge, reading from IStatePort.

    Args:
        sections: Sections to summarize (default: all HOT). Options:
            control, beliefs_active, scoreboard, history_active,
            clarifications, affective_now, narrative_active, meta
        strategy: Summarization approach.
            extractive: Key sentence extraction (fastest, <5ms).
            abstractive: LLM-powered rewrite (slower, uses ILLMPort).
            hybrid: Extractive first, abstractive if still over budget.
        target_tokens: Target output token count (default: 20% of context window).

    Returns:
        ContextSummary:
            summary: str                  # Compressed context text
            original_tokens: int          # Tokens before summarization
            summary_tokens: int           # Tokens after
            compression_ratio: float      # summary_tokens / original_tokens
            sections_included: List[str]  # Which sections were summarized
            sections_dropped: List[str]   # Sections excluded for budget
            strategy_used: str            # Actual strategy applied

    Execution path:
        ToolDispatcher -> ContextAssembler.summarize(sections, strategy, target)
            -> IStatePort.read_all_hot() for section data
            -> ContextBudget.apply() for token counting + truncation
            -> Optional: ILLMPort.execute() for abstractive pass

    Latency: <5ms extractive, 500ms-2s abstractive
    """
```

Ports: `IStatePort` (read), optionally `ILLMPort` (abstractive summarization). No external system interaction.

10.5 Action Tools (3)

Action tools interact with external systems via Fabric or Orchestrator. They are effectful – they cause real-world side effects (sending messages, booking restaurants, creating agents). The ACK-first rule (Section 7.2) **requires** `acknowledge()` before any action tool.

10.5.1 `invoke_capability()`

Execute a single Fabric capability by name. Used at LOW tier for direct execution and at MEDIUM/HIGH for LLM-initiated inline calls during the Phase 2 loop.

```python
@tool(name="invoke_capability")
async def invoke_capability(
    capability: str,
    params: Dict[str, Any],
    timeout_ms: Optional[int] = None,
) -> CapabilityInvocationResult:
    """
    Execute a single capability via Fabric Role 2 (Resolution & Execution).

    Fabric 9-step execution pipeline:
      1. Emit capability.invoked event
      2. Resolve: Registry.lookup(name) -> Contract
      3. Select: PolicyEngine -> ProviderMatcher -> best provider
      4. Build Context: ContextBuilder reads SessionState + resolves prompts
      5. Execute: Provider.execute() via CircuitBreaker
      6. Validate: 3-tier output validation (structural -> schema -> semantic)
      7. Emit capability.completed event
      8. Update metrics
      9. Return result

    Args:
        capability: Canonical capability name (e.g., "tool.execute.restaurant_booking").
            Must be registered in Fabric Registry.
        params: Parameters matching the capability's required_inputs schema.
        timeout_ms: Per-call timeout (default: 30s from CONC-16).

    Returns:
        CapabilityInvocationResult:
            success: bool
            data: Dict[str, Any]       # Capability output
            error: Optional[str]
            provider_id: str           # Which provider handled it
            duration_ms: int
            trace_id: str

    Execution path:
        ToolDispatcher -> IDispatchPort.dispatch_direct(
            CapabilityRequest(name=capability, params=params, session_id, trace_id)
        ) -> Fabric.execute() -> 9-step pipeline -> CapabilityResult

    Circuit breakers: CB_FABRIC (Concierge-owned), CB_MCP (if MCP provider)
    Tier availability: LOW, MEDIUM, HIGH
    """
```

Port: `IDispatchPort.dispatch_direct()` -> Fabric `execute()`.

10.5.2 `spawn_via_fabric()`

Request dynamic agent creation via Fabric's `tool.write.build_agent` handler. This is a DAG-level operation: the Concierge typically does NOT call this directly. Instead, it appears in Planner-committed DAGs executed by Orchestrator. However, for MEDIUM tier tasks where the LLM identifies a need for a specialist agent, the Concierge can request it directly.

```python
@tool(name="spawn_via_fabric")
async def spawn_via_fabric(
    agent_name: str,
    description: str,
    domain: List[str],
    tools_granted: List[str],
    prompt_template: Optional[str] = None,
    required_context: Optional[List[str]] = None,
    llm_budget_tokens: int = 4096,
    safety_band_min: str = "GREEN",
    ephemeral: bool = True,
) -> SpawnResult:
    """
    Create a dynamic agent via Fabric tool.write.build_agent.

    Fabric 5-step build flow:
      1. AgentSpecValidator validates spec (7 rules: name pattern, tools exist,
         context sections valid, prompt resolves, domains non-empty, safety valid, budgets)
      2. AgentComposer.build() creates AgentContract
      3. Registry.register(contract) adds to capability catalog
      4. Emit k1.fabric.agent.created.v1 event
      5. Return {agent_name, status}

    Safety rules (Issue 4.5.5):
      - AMBER band minimum for build_agent
      - Created agents inherit creator's max safety band (no escalation)
      - No recursive creation (tools_granted cannot include tool.write.build_agent)
      - Depth=1 enforcement (agents cannot invoke other agents)

    Args:
        agent_name: Must match pattern agent.execute.<identifier>.
        description: Agent purpose description.
        domain: Domain tags (non-empty, e.g., ["HEALTH", "MEDICAL"]).
        tools_granted: Tools the agent can use (must all exist in Registry).
        prompt_template: Prompt template name (must resolve via IPromptSystemPort).
        required_context: SessionState sections the agent can read.
            Allowed: beliefs_active, interaction_history, task_context,
                     rhythm_state, active_plans, pending_clarifications.
        llm_budget_tokens: Token budget for agent (512-32768).
        safety_band_min: Minimum safety band (default: GREEN).
        ephemeral: True = one-shot, False = session-scoped reusable.

    Returns:
        SpawnResult:
            success: bool
            agent_name: str
            status: str            # "registered" or "validation_failed"
            errors: List[str]      # Validation errors (if any)

    Execution path:
        ToolDispatcher -> IDispatchPort.dispatch_direct(
            CapabilityRequest(name="tool.write.build_agent", params={...})
        ) -> Fabric -> BuildAgentHandler -> AgentSpecValidator -> AgentComposer -> Registry

    Tier availability: MEDIUM, HIGH (never LOW -- LOW tasks don't need agents)
    """
```

Port: `IDispatchPort.dispatch_direct()` -> Fabric `BuildAgentHandler`.

10.5.3 `execute_workflow()`

Invoke a predefined or dynamically created workflow via Orchestrator. This dispatches a `TaskEnvelope` with the workflow as the intent, letting the Orchestrator's `WorkflowEngine` handle compilation, scheduling, and DAG execution.

```python
@tool(name="execute_workflow")
async def execute_workflow(
    workflow_id: str,
    params: Dict[str, Any],
    timeout_ms: Optional[int] = None,
) -> WorkflowExecutionResult:
    """
    Execute a workflow via Orchestrator WorkflowEngine.

    Orchestrator flow:
      1. Receive TaskEnvelope with capabilities=[workflow_id]
      2. WorkflowRegistry.get(workflow_id) -> WorkflowDefinition
      3. WorkflowCompiler.compile(definition) -> DAG
      4. DAGExecutor.execute(dag, snapshot) -> AggregatedResult
      5. Emit k1.orchestration.workflow.completed.v1

    Args:
        workflow_id: Workflow identifier (e.g., "workflow.birthday_party_planning").
        params: Workflow input parameters.
        timeout_ms: Overall workflow timeout
            (default: 60s for MEDIUM, 120s for HIGH from CONC-15).

    Returns:
        WorkflowExecutionResult:
            success: bool
            envelope_id: str           # Tracking ID for delta subscription
            status: str                # "queued" | "rejected"
            rejection_reason: Optional[str]

    Execution path:
        ToolDispatcher -> build TaskEnvelope(tier=MEDIUM, capabilities=[workflow_id])
        -> IDispatchPort.dispatch_envelope(envelope) -> Orchestrator Mailbox
        -> Results arrive via IDeltaPort subscription (async)

    Note: This is async -- the tool returns immediately with "queued" status.
    Actual results stream back via Delta Bus -> DeltaAggregator -> OutputManager.
    The COMPANIONING -> PROGRESSING -> DELIVERING FSM states handle result delivery.

    Tier availability: MEDIUM, HIGH (never LOW)
    """
```

Port: `IDispatchPort.dispatch_envelope()` -> Orchestrator Mailbox (WFQ INTERACTIVE priority).

10.6 Tool Allowlist by Tier (Canonical Reference)

Tools available to the LLM are filtered by complexity tier. `ToolDispatcher` enforces this at call time – any tool not in the allowlist for the current tier is rejected before execution.

TOOL	LOW	MEDIUM	HIGH	CRISIS
`acknowledge()`	Y	Y	Y	–
`update_scoreboard()`	Y	Y	Y	–
`update_beliefs()`	Y	Y	Y	–
`update_clarifications()`	Y	Y	Y	–
`update_narrative()`	Y	Y	Y	–
`refine_affect()`	Y	Y	Y	–
`promote_belief()`	–	Y	Y	–
`recall_memory()`	Y	Y	Y	–
`discover_capabilities()`	–	Y	Y	–
`summarize_context()`	Y	Y	Y	–
`invoke_capability()`	Y	Y	Y	–
`spawn_via_fabric()`	–	Y	Y	–
`execute_workflow()`	–	Y	Y	–
Total available	**8**	**13**	**13**	**0**

Rationale:

- **LOW** excludes `discover_capabilities`, `spawn_via_fabric`, `execute_workflow`, `promote_belief` – simple tasks already have resolved capability, don't need discovery/spawn/workflow/cross-tier promotion.
- **CRISIS** has zero tools – static CRISIS_STATIC response only (CONC-05).

10.7 Tool Call Budget Enforcement

`ToolDispatcher` tracks calls per turn and enforces hard limits (CONC-09):

TIER	MAX TOOL CALLS	MAX LLM CALLS	MAX ITERATIONS	RATIONALE
LOW	6	2	3	Simple queries, minimal back-and-forth
MEDIUM	12	3	5	Multi-step with Orchestrator
HIGH	20	5	10	Complex planning with Planner

```python
class ToolCallBudget:
    """Enforced by ToolDispatcher per turn."""

    def __init__(self, tier: ComplexityTier):
        self.max_calls = {ComplexityTier.LOW: 6, ComplexityTier.MEDIUM: 12, ComplexityTier.HIGH: 20}[tier]
        self.max_llm = {ComplexityTier.LOW: 2, ComplexityTier.MEDIUM: 3, ComplexityTier.HIGH: 5}[tier]
        self.calls_used = 0
        self.llm_calls_used = 0

    def can_call(self) -> bool:
        return self.calls_used < self.max_calls

    def record_call(self) -> None:
        self.calls_used += 1
        if self.calls_used >= self.max_calls:
            raise ToolBudgetExhaustedError(f"Tool call budget exhausted: {self.calls_used}/{self.max_calls}")
```

10.8 Tool Execution Flow (ToolDispatcher Pipeline)

Every tool call from the LLM passes through `ToolDispatcher`, which enforces all invariants before delegation to the appropriate port:

```
LLM tool_call{name, args}
    |
    v
[1. ALLOWLIST CHECK] -- Is tool in tier allowlist? (Section 10.6)
    |                    NO -> ToolNotAllowedError, feed back to LLM
    v
[2. BUDGET CHECK]    -- Calls remaining in budget? (CONC-09)
    |                    NO -> ToolBudgetExhaustedError, force final response
    v
[3. SCHEMA VALIDATE] -- Args match tool input schema? (JSON Schema Draft 2020-12)
    |                    NO -> ValidationError, feed back to LLM with schema hint
    v
[4. ACK-FIRST CHECK] -- If effectful + iteration 1: was acknowledge() first?
    |                    NO -> ACKFirstViolation, feed back to LLM
    v
[5. SAFETY CHECK]    -- Tool allowed at current safety band?
    |                    NO -> SafetyBandViolation (AMBER+ tools blocked at GREEN for action tools)
    v
[6. DISPATCH]        -- Route to appropriate port:
    |  Signal:     OutputManager.send_immediate()
    |  Cognitive:  IStatePort.write() via MutationGuard
    |  Read:       IMemoryPort / IDispatchPort / IStatePort (read)
    |  Action:     IDispatchPort.dispatch_direct() / dispatch_envelope()
    v
[7. RECORD]          -- Increment budget counter, emit telemetry
    |
    v
ToolResult{success, data, display_immediately}
```

10.9 Tool-to-Port Mapping (Definitive)

TOOL	PORT(S) USED	ADAPTER CHAIN
`acknowledge()`	`IOutputPort`	OutputManager -> OutputSSEAdapter -> SSE
`update_scoreboard()`	`IStatePort`	SessionStateAdapter -> MutationGuard -> DirectWriterAdapter
`update_beliefs()`	`IStatePort`	SessionStateAdapter -> MutationGuard -> DirectWriterAdapter
`update_clarifications()`	`IStatePort`	SessionStateAdapter -> MutationGuard -> DirectWriterAdapter
`update_narrative()`	`IStatePort`	SessionStateAdapter -> MutationGuard -> DirectWriterAdapter
`refine_affect()`	`IStatePort`	SessionStateAdapter -> MutationGuard -> DirectWriterAdapter
`promote_belief()`	`IStatePort` + `IMemoryPort`	SessionStateAdapter (read) + BridgeRecallAdapter (write to K0)
`recall_memory()`	`IMemoryPort`	BridgeRecallAdapter -> Bridge Client -> K0
`discover_capabilities()`	`IDispatchPort`	DispatchAdapter -> Fabric -> DiscoverCapabilitiesHandler
`summarize_context()`	`IStatePort` + `ILLMPort` (optional)	SessionStateAdapter (read) + ContextAssembler
`invoke_capability()`	`IDispatchPort`	DispatchAdapter -> Fabric.execute() (9-step pipeline)
`spawn_via_fabric()`	`IDispatchPort`	DispatchAdapter -> Fabric -> BuildAgentHandler
`execute_workflow()`	`IDispatchPort`	DispatchAdapter -> Orchestrator Mailbox (async)

10.10 Error Handling and Recovery

ERROR CATEGORY	EXAMPLE	TOOLDISPATCHER RESPONSE
Allowlist violation	LOW tier calls `spawn_via_fabric`	Reject, feed error to LLM, LLM picks different tool
Budget exhausted	21st tool call on HIGH turn	Force LLM to emit final response text
Schema validation	Missing required `capability` arg	Reject, feed schema hint to LLM
ACK-first violation	`invoke_capability` without `acknowledge`	Reject, remind LLM of ACK rule
MutationGuard rejection	HOT tier over 48KB	Reject write, suggest eviction to WARM
Fabric CB open	`CB_FABRIC` tripped	Degrade: skip Fabric call, inform user
Bridge CB open	`CB_BRIDGE` tripped	Fallback to LOCAL COLD archive for recall
Timeout	`invoke_capability` > 30s	Cancel, feed timeout to LLM, LLM apologizes
Provider failure	MCP server crashed	Fabric retries (2x), then error to LLM

All errors are fed back to the LLM as structured tool results: `ToolResult(success=False, data={"error_code": "...", "error_message": "..."})`. The LLM can then decide to retry with different parameters, try an alternative tool, or inform the user.

11. UltraBERT Engine Deep Dive

11.1 Architecture

COMPONENT	VALUE
Model	UltraBERT v4 (ModernBERT-base, answerdotai/ModernBERT-base)
Parameters	149M (22 transformer layers, 768-dim hidden, 12 attention heads)
Attention	Flash Attention 2 (2x speedup on A100/H100), RoPE positional encoding
Heads	12 task-specific classification heads across 4 hub tokens
Hub Tokens	[EMO], [REL], [MEM], [TASK] – virtual tokens collecting task-specific info via cross-attention
Inference	Single forward pass, batch size 1
Latency	P95: ~20ms, Average: 10.7ms
Throughput	93.6 inferences/sec
Checkpoint	`checkpoint-18000` (Step 18K, 90.58% weighted score)
Training	Stage A (generic multi-task: CoNLL, SST-2, GoEmotions, MNLI) + Stage B (FamilyOS domain adaptation: unified shards + 15% replay)
EMA	Exponential Moving Average checkpointing (+0.8-1.5pt consistent improvement)

11.2 Head Architecture Types

UltraBERT v4 uses four distinct head architectures:

ARCHITECTURE	HEADS	MECHANISM
GlobalPointer	`ner_general`, `ner_family`, `temporal`	Span-based scoring over token pairs. Eliminates garbage entities at source – no post-processing needed. V2 required 10-step pipeline with 15+ filters; V4 produces 100% clean spans natively.
LabelDescriptionHead	`intent`, `ingress`	Zero-shot expandable via label description matching. Labels can be added at runtime without retraining.
Multi-label (ASL)	`emotions`, `safety_generic`	Independent sigmoid per class. Asymmetric Loss (ICCV 2021 SOTA) for `emotions`: gamma_neg=4.0, gamma_pos=1.0, clip=0.05.
Hierarchical/Sequence	`safety_familyos`, `sentiment`, `nli`, `relation`	Softmax classification with hub token pooling ([EMO] or [REL]).
Vector Projection	`embedding`	Projects [MEM] hub token to 768-dim dense vector for semantic retrieval.

11.3 Hub Token System

Four virtual hub tokens are prepended to input, collecting task-specific information through cross-attention across all 22 layers:

```
Input:  [CLS] [EMO] [REL] [MEM] [TASK] My grandmother called yesterday ...

Hub Token Assignments:
  [EMO]  -> emotions (44), sentiment (5), safety_familyos (4), safety_generic (8)
  [REL]  -> relation (15 types), nli (3 classes)
  [MEM]  -> embedding (768-dim dense vector for K0 recall)
  [TASK] -> intent (8 classes), ingress (12 domains)
```

Hub tokens are initialized via semantic centroid initialization and trained with head-wise learning rates (encoder 2e-5, heads 1e-4, token heads 5e-5). Uncertainty weighting auto-balances multi-task loss across all 12 heads.

11.4 Temporal Resolution Engine

```
NER temporal entities -> Temporal Parser -> Absolute Time Resolution

Example:
  "next Thursday at 3pm"
  -> temporal head: [DATE_REL("next Thursday", 0.88), TIME("3pm", 0.92)]
  -> Parser: relative_day("Thursday", offset=1), time(15, 0)
  -> Timezone: user_tz from device/profile
  -> Resolved: 2026-02-19T15:00:00+05:30

Ambiguity Example:
  "meet at 9"
  -> temporal head: [TIME("9", 0.71)]
  -> Parser: ambiguous (09:00 or 21:00?)
  -> Resolution: check SessionState.history_active for context
  -> If morning context: 09:00. If evening: 21:00. If unclear: GAP_SIGNAL.
```

Supported temporal types:

TYPE	EXAMPLES	RESOLUTION
`DATE_ABS`	“February 20th”, “2026-03-01”	Direct calendar mapping
`DATE_REL`	“next Thursday”, “tomorrow”, “day after”	Anchor to current date
`TIME`	“3pm”, “at 9”, “noon”	AM/PM disambiguation via context
`DURATION`	“for 2 hours”, “30 minutes”	Direct numeric extraction
`FREQUENCY`	“every Tuesday”, “twice a week”	Pattern + next occurrence
`AGE`	“when I was 12”, “5 years old”	Birth year calculation

11.5 Spatial Resolution Engine

```
GPS/BLE/Wi-Fi signals -> Semantic Place Resolution

Example:
  GPS(lat, lon) + BLE(home_beacon)
  -> Resolved: Place("home", room="living_room")

Motion Classification:
  accelerometer + GPS delta -> STATIONARY | WALKING | IN_VEHICLE
```

11.6 Write Elision Gate

Sits between UltraBERT output and `IStatePort`. Prevents destructive overwrites on low-signal messages (see Section 6.5 for full logic).

Decision matrix:

HEAD OUTPUT	SIGNIFICANCE TEST	ELIDE WHEN
`safety_familyos`	Band evaluated	NEVER (CONC-05: always evaluate)
`intent`	Primary != `other` AND confidence > 0.5	`other` OR confidence <= 0.5
`ingress`	Any domain above threshold	No domains qualify
`emotions`	Primary != `neutral`	Only `neutral` detected
`sentiment`	Changed from `previous_state.affective_now`	Same as previous value
`ner_general`	Any span detected	Empty spans
`ner_family`	Any span detected	Empty spans
`temporal`	Any span detected	Empty spans
`relation`	Any relation detected	No relations

Carry-forward semantics: When a section is elided, the previous turn’s value remains in SessionState HOT tier – no read, no write, zero cost. This preserves emotional context, entity references, and discourse state across backchannel turns.

11.7 Hypothesis Generation

```python
@dataclass
class IntentHypothesis:
    primary_intent: str                         # Best guess from intent head
    confidence: float                           # 0.0-1.0
    alternative_intents: List[Tuple[str, float]] # Ranked alternatives
    evidence: List[str]                         # Supporting tokens/phrases
    is_continuation: bool                       # True for low-signal "tell me more"
    is_backchannel: bool                        # True for "ok", "yeah", "mmhmm"
```

11.8 Production Benchmarks (v4.0.1)

HEAD	METRIC	SCORE	LATENCY P95
`ner_general`	F1	95.2%	19.4ms
`ner_family`	F1	80.0%	23.4ms
`temporal`	F1	100.0%	20.5ms
`intent`	Accuracy	90.0%	20.1ms
`ingress`	Accuracy	100.0%	19.1ms
`emotions`	Hit Rate	95.3%	~7ms
`sentiment`	Direction Acc	100.0%	18.5ms
`safety_familyos`	Band Accuracy	87.5%	18.7ms
`safety_familyos`	CRISIS Recall	**100%**	–
`embedding`	Recall@1 (10d)	84.5%	–
`embedding`	Recall@10 (100d)	100%	–

Holistic coherence (FCCS Overall): 89.12% – measures cross-head consistency:

COHERENCE METRIC	SCORE	WHAT IT MEASURES
Head Agreement (HAS)	81.56%	Sentiment-emotion valence alignment
Entity Grounding (EGS)	85.96%	Relations grounded in detected entities
Safety-Emotion (SEC)	99.36%	Distress emotions trigger elevated safety
Temporal Completeness (TCS)	99.72%	Reminder intents have temporal info
Intent-Ingress (IIC)	86.00%	Intent aligns with ingress domain

11.9 Heuristic Fallback (CB_MODEL OPEN)

When UltraBERT circuit breaker is OPEN, the `UltraBERTv4Adapter` falls back to heuristic classification (< 1ms):

1. **CRISIS keywords**: Hardcoded scan (non-negotiable safety invariant)
2. **Question + < 20 words**: `conversational` / LOW
3. **remind/schedule/set**: `task` / MED
4. **plan/help me with**: `planning` / HIGH
5. **Default**: `conversational` / LOW

All Phase 1 SessionState writes are ELIDED during heuristic fallback (no UltraBERT head data to write). Only `safety_band` is set from keyword scan.

12. Hypothesis & Gap Detection Pipeline

12.1 Purpose

Convert raw UltraBERT classification into actionable intent hypotheses and identify information gaps BEFORE routing. This pipeline sits between Phase 1 (UltraBERT) and Phase 2 (LLM), inside `IntentProcessor`. It does NOT decide what questions to ask – that is the LLM's job (Section 8.1). It produces a structured `HypothesisBundle` that feeds the complexity router and seeds the Phase 2 prompt.

```
UltraBERT (22ms)
    |
    v
+-------------------------------------------------+
|            IntentProcessor Pipeline             |
|                                                 |
|  [1] HypothesisGenerator  -- form hypotheses    |
|  [2] GapDetector          -- find missing slots |
|  [3] ReferenceResolver    -- resolve pronouns/refs |
|  [4] ConfidenceScorer     -- compute uncertainty |
|  [5] ComplexityRouter     -- assign tier        |
+-------------------------------------------------+
    |
    v
Phase1Result{hypothesis_bundle, tier, gaps[], safety_band}
```

Key design decision: The pipeline is pure computation with one `IStatePort.read()` call (for reference resolution against `scoreboard.referents` and `beliefs_active`). No LLM calls. No external I/O. Target: <3ms total (CONC-14 compatible, no I/O).

12.2 IntentHypothesis (from UltraBERT Output)

```python
@dataclass
class IntentHypothesis:
    """Primary intent hypothesis from UltraBERT classification heads."""
    primary_intent: str                          # Best guess from intent head (8 classes)
    confidence: float                            # 0.0-1.0 from softmax
    alternative_intents: List[Tuple[str, float]] # Ranked alternatives (up to 3)
    evidence: List[str]                          # Supporting tokens/phrases from NER
    domains: List[str]                           # From ingress head (up to 12 domains)
    safety_band: str                             # From safety_familyos head
    is_continuation: bool                        # True for low-signal "tell me more"
    is_backchannel: bool                         # True for "ok", "yeah", "mmhmm"
    requires_action: bool                        # True if intent implies an effectful operation
```

Intent classes (from intent head, 8 classes – canonical list per Section 6.3):

INTENT	EXAMPLE	TYPICAL TIER	ACTION TOOL
`log_memory`	"Today we went to the park"	LOW	`update_beliefs`
`query_memory`	"When did Dad go to the doctor?"	LOW	`recall_memory`
`set_reminder`	"Remind me to call Mom"	LOW	`invoke_capability`
`express_feeling`	"I'm really stressed about work"	LOW	`refine_affect`
`seek_advice`	"What should I do about..."	LOW-MED	Context-dependent
`share_news`	"Sarah got into college!"	LOW	`update_beliefs`
`reflect`	"I've been thinking about..."	LOW	`update_narrative`
`other`	Unclassifiable	LOW	Context-dependent

12.3 Gap Types

The `GapDetector` identifies 5 categories of missing information. Each gap has a detection mechanism (heuristic, no LLM) and a suggested resolution strategy that the LLM can use during Phase 2.

GAP TYPE	DETECTION MECHANISM	RESOLUTION PATH	EXAMPLE
ENTITY_MISSING	Intent requires entity slot but NER spans empty	LLM asks user	"Remind me to call" (WHO?)
TIME_AMBIGUOUS	Temporal head produces >1 parse with confidence <0.7	LLM disambiguation or ContextResolver auto-resolve	"Meet at 9" (AM or PM?)
REFERENCE_UNRESOLVED	Pronoun detected by NER but no antecedent in `scoreboard.referents` or `beliefs_active`	Check `history_active` for referent chain, else LLM asks	"She said yes" (who is "she"?)
INTENT_AMBIGUOUS	Primary intent confidence <0.6 OR top-2 delta <0.15	LLM decides: either proceed with best guess or clarify	"Can you help?" (help with what?)
CONSTRAINT_UNCLEAR	Action intent without required constraints (budget, dietary, location)	LLM asks before executing action tool	"Book dinner" (where? when? how many?)

12.4 Pipeline Components

12.4.1 HypothesisGenerator

Converts raw UltraBERT `ClassificationResult` into structured `IntentHypothesis`. Pure mapping, no computation beyond threshold checks.

```python
class HypothesisGenerator:
    """Transform classification output into ranked hypotheses."""

    CONTINUATION_INTENTS = frozenset({"other"})
    BACKCHANNEL_TOKENS = frozenset({"ok", "yeah", "mmhmm", "uh-huh", "sure", "right", "yep"})

    def generate(self, classification: ClassificationResult, user_text: str) -> IntentHypothesis:
        """
        Build IntentHypothesis from classification heads.

        Steps:
        1. Extract top-K intents from intent head (K=3)
        2. Check continuation: primary == "other" AND confidence > 0.5
        3. Check backchannel: normalized user_text in BACKCHANNEL_TOKENS
        4. Check requires_action: intent in ACTION_INTENTS (set_reminder, plan_event, configure, manage_task)
        5. Extract evidence tokens from NER spans
        6. Extract domains from ingress head (threshold > 0.3)
        """
        intent_scores = classification.intent.ranked  # List[(intent, score)]
        primary, confidence = intent_scores[0]
        alternatives = intent_scores[1:4]

        return IntentHypothesis(
            primary_intent=primary,
            confidence=confidence,
            alternative_intents=alternatives,
            evidence=[s.text for s in classification.ner_general.spans + classification.ner_family.spans],
            domains=[d for d, s in classification.ingress.scored if s > 0.3],
            safety_band=classification.safety_familyos.band,
            is_continuation=primary in self.CONTINUATION_INTENTS and confidence > 0.5,
            is_backchannel=user_text.strip().lower() in self.BACKCHANNEL_TOKENS,
            requires_action=primary in ACTION_INTENTS,
        )
```

12.4.2 GapDetector

Identifies missing information slots based on intent type and available entities/temporal/constraint data. Uses a per-intent slot schema that defines what information is required vs optional.

```python
class GapDetector:
    """Detect missing information for intent execution."""

    # Required slots per intent (minimum info needed to proceed)
    INTENT_SLOTS: Dict[str, List[SlotRequirement]] = {
        "set_reminder": [
            SlotRequirement("entity", "WHO or WHAT to remind about", required=True),
            SlotRequirement("temporal", "WHEN to remind", required=True),
        ],
        "plan_event": [
            SlotRequirement("entity", "WHAT kind of event", required=True),
            SlotRequirement("temporal", "WHEN", required=False),
            SlotRequirement("constraint", "WHO is attending", required=False),
        ],
        "query_memory": [
            SlotRequirement("entity", "WHAT to query about", required=True),
        ],
        "manage_task": [
            SlotRequirement("entity", "WHICH task", required=True),
        ],
        "configure": [
            SlotRequirement("entity", "WHAT setting", required=True),
            SlotRequirement("constraint", "NEW value", required=True),
        ],
    }

    def detect(
        self,
        hypothesis: IntentHypothesis,
        classification: ClassificationResult,
    ) -> List[Gap]:
        """
        Scan for information gaps.

        Steps:
        1. Look up required slots for hypothesis.primary_intent
        2. For each slot, check if classification provides it:
           - entity slots: check ner_general + ner_family spans
           - temporal slots: check temporal spans
           - constraint slots: hard to detect heuristically -- flag if action intent
        3. Check intent_ambiguous: confidence < 0.6 or delta to #2 < 0.15
        4. Check reference resolution: pronouns without antecedents
        """
        gaps = []
        slots = self.INTENT_SLOTS.get(hypothesis.primary_intent, [])

        for slot in slots:
            if slot.slot_type == "entity" and not classification.has_entities():
                gaps.append(Gap(type=GapType.ENTITY_MISSING, slot=slot.name, description=slot.description))
            elif slot.slot_type == "temporal" and not classification.has_temporal():
                gaps.append(Gap(type=GapType.TIME_AMBIGUOUS, slot=slot.name, description=slot.description))

        # Intent ambiguity check
        if hypothesis.confidence < 0.6:
            gaps.append(Gap(type=GapType.INTENT_AMBIGUOUS, slot="intent", description="Low primary intent confidence"))
        elif len(hypothesis.alternative_intents) > 0:
            alt_conf = hypothesis.alternative_intents[0][1]
            if hypothesis.confidence - alt_conf < 0.15:
                gaps.append(Gap(type=GapType.INTENT_AMBIGUOUS, slot="intent", description="Close alternative intent"))

        return gaps
```

12.4.3 ReferenceResolver

Resolves pronouns and entity references using SessionState context. This is the one component that reads from `IStatePort` (single read, <1ms).

```
class ReferenceResolver:
    """Resolve pronouns and references against SessionState."""

    async def resolve(
        self,
        classification: ClassificationResult,
        state_port: IStatePort,
    ) -> List[ResolvedReference]:
        """
        Resolve references from NER output against scoreboard and beliefs.

        Steps:
        1. Extract pronouns from NER spans (he/she/they/it/that/this)
        2. Read scoreboard.referents for most-salient entity chain
        3. Read beliefs_active for known family members
        4. Match pronouns to antecedents by:
           a. Recency: most recent referent in scoreboard wins
           b. Gender: "she" matches female family members
           c. Topic: "it" matches most-salient non-person entity
        5. Resolve temporal references ("next Thursday") against current date + user timezone
        6. Return LOCKED references for prompt injection (Section 7.7)
        """
        resolved = []

        # Read scoreboard for referent chain (single batch read)
        snapshot = await state_port.read_sections(["scoreboard", "beliefs_active"])
        referents = snapshot.get("scoreboard", {}).get("referents", [])
        beliefs = snapshot.get("beliefs_active", {}).get("entities", [])

        for span in classification.ner_general.spans:
            if span.label == "PRONOUN":
                antecedent = self._find_antecedent(span.text, referents, beliefs)
                if antecedent:
                    resolved.append(ResolvedReference(
                        original=span.text,
                        resolved_to=antecedent.name,
                        resolution_type="pronoun",
                        confidence=antecedent.salience,
                        locked=True,
                    ))
                else:
                    # Unresolved pronoun -- will appear as GAP
                    pass

        return resolved
```

12.4.4 ConfidenceScorer

Computes overall uncertainty score combining intent confidence, gap severity, and reference resolution quality.

```
class ConfidenceScorer:
    """Compute overall hypothesis confidence for complexity routing."""

    def score(
        self,
        hypothesis: IntentHypothesis,
        gaps: List[Gap],
        resolved_refs: List[ResolvedReference],
    ) -> float:
        """
        Compute uncertainty score [0.0 = certain, 1.0 = totally uncertain].

        Formula:
            base = 1.0 - hypothesis.confidence
            gap_penalty = sum(gap.severity_weight for gap in gaps)
            ref_bonus = sum(ref.confidence * 0.05 for ref in resolved_refs)
            score = clamp(base + gap_penalty - ref_bonus, 0.0, 1.0)

        Gap severity weights:
            ENTITY_MISSING:       0.15 (required for action)
            TIME_AMBIGUOUS:       0.10 (resolvable by context)
            REFERENCE_UNRESOLVED: 0.05 (minor, LLM can infer)
            INTENT_AMBIGUOUS:     0.20 (fundamental uncertainty)
            CONSTRAINT_UNCLEAR:   0.10 (nice-to-have detail)
        """
        GAP_WEIGHTS = {
            GapType.ENTITY_MISSING: 0.15,
            GapType.TIME_AMBIGUOUS: 0.10,
            GapType.REFERENCE_UNRESOLVED: 0.05,
            GapType.INTENT_AMBIGUOUS: 0.20,
            GapType.CONSTRAINT_UNCLEAR: 0.10,
        }

        base = 1.0 - hypothesis.confidence
        gap_penalty = sum(GAP_WEIGHTS.get(g.type, 0.05) for g in gaps)
        ref_bonus = sum(r.confidence * 0.05 for r in resolved_refs)

        return max(0.0, min(1.0, base + gap_penalty - ref_bonus))
```

12.5 HypothesisBundle (Pipeline Output)

The complete output of the hypothesis pipeline, consumed by `ComplexityRouter` and injected into the Phase 2 prompt.

```python
@dataclass
class HypothesisBundle:
    """Complete output of the Hypothesis & Gap Detection Pipeline."""

    # Core hypothesis
    hypothesis: IntentHypothesis

    # Detected gaps
    gaps: List[Gap]

    # Resolved references (injected as LOCKED into prompt)
    resolved_references: List[ResolvedReference]

    # Computed scores
    uncertainty_score: float            # 0.0 = certain, 1.0 = uncertain
    complexity_score: float             # Combined score for tier routing

    # Metadata
    pipeline_latency_ms: float          # Total pipeline time (target: <3ms)

    @property
    def has_critical_gaps(self) -> bool:
        """True if any required-slot gap exists (forces CLARIFYING if LLM agrees)."""
        return any(g.type in (GapType.ENTITY_MISSING, GapType.INTENT_AMBIGUOUS) for g in self.gaps)

    @property
    def all_resolved(self) -> bool:
        """True if no gaps remain (LLM can proceed directly)."""
        return len(self.gaps) == 0
```

12.6 Integration with Complexity Router

The `ComplexityRouter` uses the `HypothesisBundle` to assign a complexity tier. The 5-factor weighted scoring:

```python
class ComplexityRouter:
    """5-factor weighted scoring -> tier assignment. Pure computation, no I/O."""

    WEIGHTS = {
        "intent_complexity": 0.30,    # Multi-step vs single-step intent
        "entity_count": 0.15,         # More entities = more coordination
        "gap_count": 0.15,            # More gaps = more clarification cycles
        "uncertainty": 0.25,          # From ConfidenceScorer
        "domain_breadth": 0.15,       # Cross-domain = harder (HEALTH + FINANCE)
    }

    THRESHOLDS = {"LOW": 0.3, "HIGH": 0.7}  # LOW < 0.3, MEDIUM 0.3-0.7, HIGH > 0.7

    def route(self, bundle: HypothesisBundle) -> ComplexityTier:
        """
        Compute complexity score and assign tier.

        Factors:
        1. intent_complexity: map intent to base complexity
           - conversational/emotional/other: 0.1
           - query_memory/set_reminder/configure: 0.3
           - manage_task: 0.5
           - plan_event: 0.8
        2. entity_count: len(evidence) / 5 (normalized, capped at 1.0)
        3. gap_count: len(gaps) / 3 (normalized, capped at 1.0)
        4. uncertainty: uncertainty_score (already 0-1)
        5. domain_breadth: len(domains) / 3 (normalized, capped at 1.0)
        """
        f1 = INTENT_COMPLEXITY_MAP.get(bundle.hypothesis.primary_intent, 0.3)
        f2 = min(len(bundle.hypothesis.evidence) / 5, 1.0)
        f3 = min(len(bundle.gaps) / 3, 1.0)
        f4 = bundle.uncertainty_score
        f5 = min(len(bundle.hypothesis.domains) / 3, 1.0)

        score = (f1 * 0.30 + f2 * 0.15 + f3 * 0.15 + f4 * 0.25 + f5 * 0.15)

        if score < 0.3:
            return ComplexityTier.LOW
        elif score > 0.7:
            return ComplexityTier.HIGH
        else:
            return ComplexityTier.MEDIUM
```

12.7 Integration with Phase 2 Prompt

Gaps are injected into the LLM prompt as structured context so the LLM can decide what to ask:

```
[DETECTED GAPS]
  - ENTITY_MISSING: WHO to remind about (required for set_reminder)
  - TIME_AMBIGUOUS: "at 9" -- AM or PM? (context suggests evening based on history_active)

[RESOLVED REFERENCES]
  - "Mom" = Sarah Chen (from beliefs_active) (LOCKED)
  - "next Thursday" = 2026-02-19 (from temporal head + user timezone) (LOCKED)
```

The LLM uses this information to decide whether to:

1. **Proceed with best guess** (minor gaps, high overall confidence)
2. **Ask a clarifying question** (critical gaps, transition to CLARIFYING state)
3. **Auto-resolve** from context (e.g., “at 9” in evening domain = 21:00)

12.8 Walkthrough: “Can you help with that thing for Mom?”

```
Phase 1 (ACKING, 22ms):
  intent:     other (0.45 -- low confidence)
  ingress:    [] (no clear domain)
  ner_family: ["Mom" -> PERSON]
  ner_general: ["that thing" -> unclear]

HypothesisGenerator:
  primary_intent: "other" (0.45)
  alternatives: [("manage_task", 0.25), ("query_memory", 0.18)]
  evidence: ["Mom"]
  is_continuation: False (confidence < 0.5)
  is_backchannel: False

GapDetector:
  gaps:
    - INTENT_AMBIGUOUS (confidence 0.45 < 0.6)
    - REFERENCE_UNRESOLVED ("that thing" has no antecedent)

ReferenceResolver:
  "Mom" -> Sarah Chen (from beliefs_active) -> LOCKED
  "that thing" -> check scoreboard.referents:
    -> scoreboard has: {referent: "birthday party planning", salience: 0.8, turn: 42}
    -> RESOLVED: "that thing" = "birthday party planning" -> LOCKED

ConfidenceScorer:
  base = 1.0 - 0.45 = 0.55
  gap_penalty = 0.20 (INTENT_AMBIGUOUS) + 0.0 (ref now resolved) = 0.20
  ref_bonus = 0.8 * 0.05 + 0.95 * 0.05 = 0.088
  uncertainty = clamp(0.55 + 0.20 - 0.088) = 0.662

ComplexityRouter:
  score = 0.1*0.30 + 0.2*0.15 + 0.33*0.15 + 0.662*0.25 + 0.0*0.15 = 0.275
  tier = LOW (barely -- but "that thing" resolved to an active task)

Phase 2 Prompt includes:
  [DETECTED GAPS]
    - INTENT_AMBIGUOUS: Low confidence on primary intent (0.45)
  [RESOLVED REFERENCES]
    - "Mom" = Sarah Chen (LOCKED)
    - "that thing" = birthday party planning (from scoreboard, LOCKED)

LLM reasoning:
  "The user is referring to the birthday party planning for Sarah Chen.
   I should continue with that context."
  -> No clarification needed (gap was resolved by scoreboard context)
  -> Proceeds with birthday party planning continuation
```

12.9 Data Types

```python
@dataclass
class SlotRequirement:
    """A required or optional information slot for an intent."""
    slot_type: Literal["entity", "temporal", "constraint"]
    description: str
    required: bool = True


@dataclass
class Gap:
    """A detected information gap."""
    type: GapType
    slot: str                       # Which slot is missing
    description: str                # Human-readable description
    severity_weight: float = 0.1    # Contribution to uncertainty score
    resolvable_from_context: bool = False  # True if ReferenceResolver might fix it


class GapType(str, Enum):
    ENTITY_MISSING = "ENTITY_MISSING"
    TIME_AMBIGUOUS = "TIME_AMBIGUOUS"
    REFERENCE_UNRESOLVED = "REFERENCE_UNRESOLVED"
    INTENT_AMBIGUOUS = "INTENT_AMBIGUOUS"
    CONSTRAINT_UNCLEAR = "CONSTRAINT_UNCLEAR"


@dataclass
class ResolvedReference:
    """A pronoun or reference resolved against SessionState."""
    original: str                   # The reference text ("Mom", "she", "that thing")
    resolved_to: str                # The resolution ("Sarah Chen", "birthday planning")
    resolution_type: Literal["pronoun", "entity", "temporal", "scoreboard"]
    confidence: float               # How confident the resolution is
    locked: bool = True             # Injected as LOCKED in prompt (no re-asking)
    source: str = ""                # Where resolution came from (beliefs_active, scoreboard, etc.)
```

13. SessionState Access Pattern

13.1 Single Writer Pattern (CONC-01)

Only Concierge writes to SessionState. All other components (Orchestrator, Planner, sub-agents) read via lock-free snapshots (< 1ms). Sub-agents emit deltas to the K1 Bus delta lane, which the Concierge aggregates (500ms window) and writes as the single writer.

```
Sub-Agents --> K1 Bus (delta lane) --> AggregationWindow --> Concierge --> WRITE
                                       (500ms batch)      (SINGLE WRITER)
                                                             |
                                                             v
                                                          SessionState

Multi-Reader Access (Lock-Free, < 1ms):
  Concierge, Orchestrator, Planner, Sub-Agents --> SessionState (read-only snapshot)
```

13.2 Two-Phase Write Model

Concierge writes to SessionState in two distinct phases per turn, plus a system append at turn boundary:

PHASE	TIMING	WRITER	SECTIONS TOUCHED	GATED BY
Phase 1 (Deterministic)	~22ms	UltraBERT heads via TurnProcessor	control, beliefs_active, affective_now, scoreboard	Write Elision Gate (Section 6.5)
Phase 2 (LLM Cognitive)	Variable (2-45s)	LLM cognitive tools via ToolDispatcher	scoreboard, beliefs_active, clarifications, narrative_active, affective_now	MutationGuard.preflight()
Turn Boundary	End of turn	TurnProcessor.turn_end()	history_active, meta, telemetry	Always (system append)

13.3 Phase 1 Writes: UltraBERT -> SessionState (via Write Elision Gate)

Direct mapping from 12 UltraBERT heads to HOT CORE sections:

```
UltraBERT v4 (12 heads, 22ms)
    |
    v
+-------------------------------------------------+
|           WRITE ELISION GATE                    |
|  Compute per-section signal significance        |
|  Low-signal? ELIDE (carry forward previous)     |
|  safety_band: ALWAYS evaluate (CONC-05)         |
+-------------------------------------------------+
    |
    +--> control.safety_band     <-- safety_familyos (ALWAYS)
    +--> control.intents         <-- intent (if significant)
    +--> control.domains[]       <-- ingress (if significant)
    +--> beliefs_active.entities <-- ner_general + ner_family (if detected)
    +--> beliefs_active.time     <-- temporal (if detected)
    +--> beliefs_active.relations<-- relation (if detected)
    +--> affective_now.emotion   <-- emotions (if non-neutral)
    +--> affective_now.valence   <-- sentiment (if changed)
    +--> scoreboard.referents    <-- ner_general + ner_family (if detected)
    +--> scoreboard.salience_map <-- combined head scores (if entities exist)
    +--> scoreboard.last_intent  <-- intent (if significant)
    +--> scoreboard.topic_stack  <-- ingress (if domains detected)
```

13.4 Phase 2 Writes: LLM Cognitive Tools -> SessionState (via MutationGuard)

During Phase 2, the Concierge LLM uses 6 cognitive tools that write to SessionState. Every write passes through `MutationGuard.preflight()` (CONC-02).

COGNITIVE TOOL	SESSIONSTATE SECTION	WHAT IT WRITES
`update_scoreboard()`	scoreboard	Referent resolution, QUD stack, salience updates, topic shifts
`update_beliefs()`	beliefs_active	New facts, corrections, invalidations
`update_clarifications()`	clarifications	Semantic gaps, resolved gaps
`update_narrative()`	narrative_active	Thread switch/resume/new/continue
`refine_affect()`	affective_now	Override UltraBERT emotion (sarcasm, nuance detection)
`promote_belief()`	beliefs_history -> beliefs_active	Promote WARM fact to HOT (old context referenced)

13.5 Turn Boundary Writes (System, Non-Cognitive)

Always executed at `turn_end()`, independent of write elision:

SECTION	OPERATION	DATA
`history_active`	Append turn	`Turn{user_msg, assistant_response, tool_calls, timestamp}`
`meta`	Update	`turn_count++` , `last_activity_ms`
`telemetry`	Update	`input_tokens` , `output_tokens` , `llm_calls` , `latency`

13.6 Full Turn Lifecycle: Which Phases Write Which Sections

HOT SECTION	PHASE 1 (ULTRABERT)	PHASE 2 (LLM TOOLS)	TURN BOUNDARY	ELIDE-SAFE?
`control`	safety_band (ALWAYS), intents, domains	–	–	Partial (safety never elided)
`beliefs_active`	entities, time, relations	facts, corrections	–	Yes (carry-forward safe)
`scoreboard`	referents, salience, last_intent, topics	QUD, deeper referents	–	Yes (carry-forward safe)
`history_active`	–	–	Append turn	Never elided (system append)
`clarifications`	–	gaps, resolutions	–	N/A (Phase 2 only)
`affective_now`	emotion, valence, intensity	refine_affect() override	–	Yes (carry-forward safe)
`narrative_active`	–	thread state	–	N/A (Phase 2 only)
`meta`	–	–	turn_count, timestamps	Never elided (system)

13.7 Tier Structure

TIER	SIZE	SECTIONS	ACCESS PATTERN
HOT	48KB	8 sections (control 8KB, beliefs_active 8KB, scoreboard 6KB, history_active 8KB, clarifications 4KB, affective_now 4KB, narrative_active 4KB, meta 2KB)	Read every turn; write per Phase 1/2
WARM	48KB	4 sections (beliefs_history 12KB, history_recent 20KB, persona 8KB, telemetry 8KB)	Read occasionally; write on migration
LOCAL COLD	Unbounded	K1 SQLite archive	Write on rotation; read on recovery
K0 Cloud	Unbounded	Cross-device sync via Bridge	Async sync via Bridge

13.8 Write Cost by Message Type

SCENARIO	PHASE 1 WRITES	PHASE 2 WRITES	TURN BOUNDARY	TOTAL WRITES
Full signal (“Remind Mom Thursday 3pm”)	4 sections	2-4 tool calls	3 sections	9-11
Partial signal (“I’m anxious about work”)	2 sections (affect + control)	1-2 tool calls	3 sections	6-7
Continuation (“ummm what more?”)	1 section (safety, idempotent)	0-1 tool calls	3 sections	4-5
Backchannel (“ok”)	1 section (safety, idempotent)	0 tool calls	3 sections	4

Write elision reduces Phase 1 writes from 4 sections to 0-1 on ~30-40% of turns (backchannels, continuations, fillers), preserving previous turn’s context.

13.9 IStatePort Interface

```python
class IStatePort(Protocol):
    """Bidirectional SessionState access. Single Writer: only Concierge writes."""

    async def read(self, section: str) -> SectionSnapshot: ...
    async def read_all_hot(self) -> HotTierSnapshot: ...
    async def read_sections(self, sections: List[str]) -> Dict[str, SectionSnapshot]: ...
    async def write(self, section: str, delta: SectionDelta) -> WriteResult: ...
    async def write_batch(self, deltas: Dict[str, SectionDelta]) -> BatchWriteResult: ...
    async def checkpoint(self) -> CheckpointResult: ...
    async def restore(self, checkpoint_id: str) -> RestoreResult: ...
```

14. Port Architecture

14.1 Port Overview (8 Hexagonal Ports)

All external dependencies are accessed exclusively through these 8 ports (CONC-16: no service bypasses ports). Each port has exactly one production adapter, one test adapter, and zero or one circuit breaker.

PORT	DIRECTION	PURPOSE	PRODUCTION ADAPTER	CIRCUIT BREAKER	LATENCY BUDGET
`IInputPort`	Inbound	Receive user messages	`WebSocketInputAdapter` / `RESTInputAdapter`	None (passive)	Unbounded (waiting)
`IOutputPort`	Outbound	Send responses to user	`SSEOutputAdapter` / `WebSocketOutputAdapter`	`CB_SSE`	50ms P95
`IClassificationPort`	Outbound	UltraBERT classification	`UltraBERTAdapter`	`CB_MODEL_HUB` (shared)	25ms P95
`ILLMPort`	Outbound	LLM calls via Model Hub	`ModelGatewayAdapter`	`CB_MODEL_HUB` (shared)	2000ms P95
`IStatePort`	Both	SessionState read/write	`SessionKernelAdapter`	`CB_SESSIONSTATE`	1ms read / 5ms write
`IDispatchPort`	Outbound	Dispatch to Orchestrator/Fabric	`FabricOrchestratorAdapter`	`CB_ORCHESTRATOR` + `CB_FABRIC` + `CB_MCP`	Tier-dependent
`IDeltaPort`	Both	Delta stream subscribe/publish	`DeltaBusAdapter`	None (best-effort)	<1ms publish
`IMemoryPort`	Outbound	K0 memory queries via Bridge	`BridgeRecallAdapter`	`CB_BRIDGE` (implicit)	500ms P95

14.2 IInputPort

Direction: Inbound. **Adapter**: `WebSocketInputAdapter` (prod), `TestInputAdapter` (test). **CB**: None.

The entry point for all user messages. Called by `FSMController` to receive the next user message when in LISTENING state. Also used by CLARIFYING state to receive the user's clarification response.

```python
class IInputPort(Protocol):
    """Receive user input from transport layer."""

    async def receive(self) -> UserMessage:
        """Block until the next user message arrives.

        Returns:
            UserMessage with text, timestamp, session_id, trace_id.

        Raises:
            SessionClosedError: WebSocket/connection closed by client.
            TransportError: Network-level failure.
        """
        ...

    async def receive_with_timeout(self, timeout_ms: int) -> Optional[UserMessage]:
        """Wait for a message up to timeout_ms.

        Used by:
        - CLARIFYING state: wait for user response (timeout -> re-prompt or give up)
        - INTERRUPT_HANDLING: check for pending message in buffer

        Returns:
            UserMessage if received within timeout, None otherwise.
        """
        ...

    def has_buffered(self) -> bool:
        """Check if there's already a message waiting (interrupt detection).

        Used by FSMController to detect user interrupts during DISPATCHING,
        COMPANIONING, and PROGRESSING states.
        """
        ...
```

UserMessage dataclass:

```python
@dataclass(frozen=True)
class UserMessage:
    """Immutable user input message."""
    text: str                    # Raw user text (UTF-8, max 4096 chars)
    session_id: str              # Session identifier
    trace_id: str                # Distributed trace ID (FAB-09)
    timestamp_ms: int            # Unix epoch milliseconds
    channel: Literal["websocket", "rest", "test"]  # Transport origin
    metadata: Dict[str, Any] = field(default_factory=dict)  # Optional: device, locale
```

Error handling:

ERROR	ADAPTER BEHAVIOR
`SessionClosedError`	FSMController transitions to cleanup, ends session
`TransportError`	Retry once (100ms), then propagate to FSMController
Message > 4096 chars	Truncate at adapter level, log warning
Invalid UTF-8	Replace invalid sequences at adapter level

14.3 IOutputPort

Direction: Outbound. **Adapter**: `SSEOutputAdapter` (prod), `TestOutputAdapter` (test). **CB**: `CB_SSE`.

All user-facing output flows through this port. Used by `OutputManager` for final responses, `acknowledge()` for immediate signals, and progress streaming.

```python
class IOutputPort(Protocol):
    """Send output events to the user transport layer."""

    async def send(self, event: OutputEvent) -> DeliveryReceipt:
        """Send a single output event.

        Priority routing:
        - REALTIME: 3 retries, 100ms exponential backoff, dead-letter on failure
        - INTERACTIVE: 1 retry, dead-letter on failure
        - BACKGROUND: 0 retries, drop on failure

        Returns:
            DeliveryReceipt with event_id, delivered_at, retry_count.

        Raises:
            DeliveryFailedError: All retries exhausted (REALTIME/INTERACTIVE).
            CBOpenError: CB_SSE is OPEN (adapter buffers up to 5 messages).
        """
        ...

    async def send_stream(self, events: AsyncIterator[OutputEvent]) -> StreamReceipt:
        """Stream multiple events (progress narration, chunked responses).

        Used during PROGRESSING state for Orchestrator delta relay.
        Events are sent as individual SSE messages with same stream_id.
        Backpressure: if client slow, buffer up to 10 events, then drop oldest.

        Returns:
            StreamReceipt with event_count, dropped_count, duration_ms.
        """
        ...

    async def send_typing_indicator(self, active: bool) -> None:
        """Show/hide typing indicator in client UI.

        Sent at:
        - Phase 2 start (active=True)
        - LLM response ready (active=False)
        - Tool execution start (active=True for long tools)
        """
        ...
```

OutputEvent types (9 event types, 3 priority levels):

```python
@dataclass
class OutputEvent:
    """User-facing output event."""
    event_id: str = field(default_factory=lambda: str(uuid4()))
    event_type: Literal[
        "acknowledge",           # Signal tool output (REALTIME)
        "response_text",         # Final LLM response (INTERACTIVE)
        "response_chunk",        # Streaming token chunk (INTERACTIVE)
        "progress_update",       # Backend progress (BACKGROUND)
        "clarification_request", # HITL question (INTERACTIVE)
        "error_message",         # User-visible error (INTERACTIVE)
        "typing_indicator",      # Typing status (REALTIME)
        "delta_summary",         # Aggregated delta batch (BACKGROUND)
        "system_notice",         # Session warnings (BACKGROUND)
    ] = "response_text"
    priority: Literal["REALTIME", "INTERACTIVE", "BACKGROUND"] = "INTERACTIVE"
    payload: Dict[str, Any] = field(default_factory=dict)
    trace_id: str = ""
    timestamp_ms: int = 0
```

CB_SSE behavior:

CB STATE	BEHAVIOR
CLOSED	Normal delivery via SSE
OPEN	Buffer up to 5 messages; flush on reconnect; drop BACKGROUND priority
HALF_OPEN	Probe with `typing_indicator` (lightweight, non-critical)

14.4 IClassificationPort

Direction: Outbound. **Adapter**: `UltraBERTAdapter` (prod), `MockClassificationAdapter` (test). **CB**: `CB_MODEL_HUB` (shared).

Called once per turn by `IntentProcessor` during Phase 1 (ACKING state). Returns the complete 12-head classification result from UltraBERT v4.

```python
class IClassificationPort(Protocol):
    """Classify user input via UltraBERT v4."""

    async def classify(self, text: str, context: Optional[ClassificationContext] = None) -> ClassificationResult:
        """Run UltraBERT v4 inference (12 heads, single forward pass).

        Args:
            text: User message text (tokenized by adapter).
            context: Optional prior context for temporal/reference resolution.

        Returns:
            ClassificationResult with all 12 head outputs:
            - intent: RankedList[str, float]       (8 classes)
            - ingress: RankedList[str, float]       (12 domains)
            - safety_familyos: SafetyBand           (GREEN/AMBER/RED/CRISIS)
            - safety_generic: SafetyBand            (generic model safety)
            - emotions: MultiLabel[str, float]      (44 Plutchik classes)
            - sentiment: Literal[1,2,3,4,5]         (5-level)
            - ner_general: SpanList[NERSpan]        (PER, ORG, LOC, MISC)
            - ner_family: SpanList[NERSpan]         (PERSON, KINSHIP, NICKNAME, PET, HOME_LOC, FAMILY_EVENT, ROUTINE, TRADITION, MILESTONE, HEIRLOOM)
            - temporal: SpanList[TemporalSpan]      (absolute/relative times)
            - relation: List[RelationTriple]        (15 types: parent_of, child_of, spouse_of, sibling_of, grandparent_of, grandchild_of, aunt_uncle_of, niece_nephew_of, cousin_of, pet_

            - nli: Literal["entailment","contradiction","neutral"]
            - embedding: ndarray[768]               (semantic vector)

        Raises:
            ClassificationTimeoutError: Inference > 25ms P99.
            ModelNotLoadedError: UltraBERT checkpoint not loaded.
            CBOpenError: CB_MODEL_HUB is OPEN.
        """
        ...
```

Heuristic fallback (when `CB_MODEL_HUB` is OPEN, <1ms):

```python
class HeuristicFallbackClassifier:
    """Deterministic fallback when UltraBERT is unavailable.

    Activated by UltraBERTAdapter when CB_MODEL_HUB is OPEN.
    5-rule chain, evaluated in order:

    1. CRISIS keywords: hardcoded scan (non-negotiable, CONC-05/06)
       - Matches: "suicide", "kill", "hurt myself", "emergency" -> CRISIS band
    2. Question + <20 words: conversational/LOW
    3. remind/schedule/set: task/MEDIUM
    4. plan/help me with: planning/HIGH
    5. Default: conversational/LOW

    Returns synthetic ClassificationResult with all non-determinable heads
    set to neutral/empty defaults.
    """
```

14.5 ILLMPort

Direction: Outbound. **Adapter**: `ModelGatewayAdapter` (prod), `MockLLMAdapter` (test). **CB**: `CB_MODEL_HUB` (shared with IClassificationPort).

The primary LLM access port. Supports 5 capabilities routed through Model Hub. Called by `ToolDispatcher` during Phase 2 for the ReAct agentic loop.

```python
class ILLMPort(Protocol):
    """LLM execution via Model Hub."""

    async def execute(self, request: HubRequest) -> HubResponse:
        """Execute a single LLM request (non-streaming).

        Used for:
        - Tool-calling turns (TOOL_CALL capability)
        - Structured output extraction (STRUCTURED capability)
        - Final response generation (CHAT capability)

        Returns:
            HubResponse with response_text, tool_calls[], usage stats.

        Raises:
            LLMTimeoutError: Request exceeded per-capability timeout.
            ContextOverflowError: Prompt exceeds model context window.
            RateLimitError: Provider rate limit hit, retry after backoff.
            CBOpenError: CB_MODEL_HUB is OPEN.
        """
        ...

    async def stream_execute(self, request: HubRequest) -> AsyncIterator[HubChunk]:
        """Execute with streaming token output.

        Used for:
        - DELIVERING state: stream final response to user via IOutputPort
        - Long responses: progressive rendering

        Yields:
            HubChunk with token fragment, finish_reason (None until done).

        Raises:
            Same as execute() plus StreamInterruptedError.
        """
        ...

    def available_capabilities(self) -> FrozenSet[str]:
        """Return currently available LLM capabilities.

        Returns subset of: {"CHAT", "TOOL_CALL", "STRUCTURED", "VISION", "REASON"}
        based on Model Hub provider manifest and CB state.
        """
        ...
```

HubRequest and HubResponse:

```python
@dataclass
class HubRequest:
    """LLM request routed through Model Hub."""
    capability: Literal["CHAT", "TOOL_CALL", "STRUCTURED", "VISION", "REASON"]
    payload: Dict[str, Any]             # messages[], tools[], response_format, etc.
    constraints: RequestConstraints      # max_tokens, temperature, timeout_ms
    trace_id: str = ""                   # Propagated trace ID

@dataclass
class RequestConstraints:
    """Per-request resource constraints."""
    max_tokens: int = 4096              # Output token limit
    temperature: float = 0.7            # Sampling temperature
    timeout_ms: int = 30_000            # Per-request timeout
    max_tool_calls: int = 5             # Tool calls per single LLM response

@dataclass
class HubResponse:
    """LLM response from Model Hub."""
    response_text: str                   # Generated text (may be empty if tool_calls)
    tool_calls: List[ToolCall]           # Requested tool invocations
    finish_reason: Literal["stop", "tool_calls", "length", "content_filter"]
    usage: TokenUsage                    # input_tokens, output_tokens, total_tokens
    model_id: str                        # Which model actually served the request
    latency_ms: float                    # Round-trip to Model Hub
```

Capability routing:

CAPABILITY	USE CASE	TYPICAL MODEL	TIMEOUT
`CHAT`	Final response generation	GPT-4o / Claude	10s
`TOOL_CALL`	ReAct agentic loop	GPT-4o / Claude	15s
`STRUCTURED`	JSON schema extraction	GPT-4o-mini	5s
`VISION`	Image understanding	GPT-4o	20s
`REASON`	Complex multi-step reasoning	o1 / Claude	30s

LLM cascade on failure (managed by `ModelGatewayAdapter`):

```
L1: RETRY (timeout / 5xx) -> retry once with same provider
L2: CB HALF-OPEN probe (30s window) -> wait for probe result
L3: CANNED_RESPONSE (CB OPEN) -> return pre-written template
    Template: "I'm having a moment -- let me think about that. Can you try again?"
```

14.6 IStatePort (Reference: Section 13.9)

Direction: Both. **Adapter**: `SessionKernelAdapter` (prod), `InMemoryStateAdapter` (test). **CB**: `CB_SESSIONSTATE`.

Fully documented in **Section 13.9**. Summary of the protocol:

```python
class IStatePort(Protocol):
    """Bidirectional SessionState access. Single Writer: only Concierge writes (CONC-01)."""

    # Read operations (lock-free snapshots, <1ms)
    async def read(self, section: str) -> SectionSnapshot: ...
    async def read_all_hot(self) -> HotTierSnapshot: ...
    async def read_sections(self, sections: List[str]) -> Dict[str, SectionSnapshot]: ...

    # Write operations (MutationGuard-gated, <5ms)
    async def write(self, section: str, delta: SectionDelta) -> WriteResult: ...
    async def write_batch(self, deltas: Dict[str, SectionDelta]) -> BatchWriteResult: ...

    # Checkpoint/restore (session management)
    async def checkpoint(self) -> CheckpointResult: ...
    async def restore(self, checkpoint_id: str) -> RestoreResult: ...
```

Write path: `SessionKernelAdapter` wraps `DirectWriterAdapter` (Section 35) which enforces:

1. Writer authorization check (`validate_writer()`): only `concierge` in `authorized_writers`
2. `MutationGuard.preflight(section, op, bytes)` : capacity limits, section locks, emergency mode (>=95KB, 50ms GC wait)
3. `SessionStateManager.mutate()` : apply delta, update version counter
4. Return `MutationResponse` with status (APPLIED, REJECTED, FAILED) and `RejectionCategory` if rejected

CB_SESSIONSTATE: Timeout 100ms, reset 10s. Fallback: stale cached read (last successful snapshot).

14.7 IDispatchPort (Reference: Section 4.2)

Direction: Outbound. **Adapter**: `FabricOrchestratorAdapter` (prod), `MockDispatchAdapter` (test). **CBs**: `CB_ORCHESTRATOR`, `CB_FABRIC`, `CB_MCP`.

Fully documented in **Section 4.2**. Summary of the protocol:

```python
class IDispatchPort(Protocol):
    """Dispatch to Orchestrator (MEDIUM/HIGH) or Fabric (LOW)."""

    # LOW tier: direct Fabric call, bypass Orchestrator
    async def dispatch_direct(self, capability_id: str, params: Dict) -> ExecutionResult: ...

    # MEDIUM/HIGH tier: enqueue TaskEnvelope to Orchestrator mailbox
    async def dispatch_envelope(self, envelope: TaskEnvelope) -> DispatchReceipt: ...

    # Cancel in-flight dispatch
    async def cancel_dispatch(self, envelope_id: str) -> CancelResult: ...
```

Tier routing (from Section 4.4):

TIER	METHOD	DESTINATION	CBS INVOLVED
LOW	`dispatch_direct()`	Fabric directly	`CB_FABRIC`, `CB_MCP`
MEDIUM	`dispatch_envelope()`	Orchestrator mailbox	`CB_ORCHESTRATOR`, `CB_FABRIC`
HIGH	`dispatch_envelope()`	Orchestrator -> Planner -> Fabric	`CB_ORCHESTRATOR`, `CB_PLANNER`, `CB_FABRIC`

Tier degradation cascade: HIGH fails -> MEDIUM (skip planning) -> LOW (no tools) -> canned response.

14.8 IDeltaPort

Direction: Both. **Adapter**: `DeltaBusAdapter` (prod), `TestDeltaAdapter` (test). **CB**: None (best-effort, fire-and-forget).

Bidirectional delta streaming over K1 Bus. Used by `DeltaAggregator` (subscribe to backend deltas), `OutputManager` (publish user-facing progress), and `TurnProcessor` (emit lifecycle events).

```python
class IDeltaPort(Protocol):
    """Bidirectional delta streaming via K1 Bus."""

    async def subscribe(self, topics: List[str]) -> AsyncIterator[Delta]:
        """Subscribe to delta stream from Orchestrator/Planner/Sub-agents.

        Concierge subscribes at session init and processes throughout lifecycle.
        DeltaAggregator batches incoming deltas in 500ms windows (CONC-10).

        Args:
            topics: List of K1 Bus topic patterns to subscribe to.

        Yields:
            Delta events as they arrive from bus.
        """
        ...

    async def publish(self, delta: Delta) -> None:
        """Publish delta event to K1 Bus (fire-and-forget).

        Used for:
        - Lifecycle events (session.started, session.ended)
        - Turn boundary events (turn.started, turn.completed)
        - Concierge-originated deltas (affective, scoreboard changes)

        Raises:
            Nothing -- fire-and-forget. Failures logged, never propagated (CONC-19).
        """
        ...

    async def errors(self) -> AsyncIterator[BusError]:
        """Stream of bus-level errors for observability.

        Adapter logs these; Concierge never blocks on bus errors.
        """
        ...
```

Subscribed topics (Concierge consumes):

TOPIC PATTERN	SOURCE	HANDLER
`k1.orchestration.task.accepted.v1`	Orchestrator	Update DISPATCHING -> COMPANIONING
`k1.orchestration.step.started.v1`	Orchestrator	Progress narration seed
`k1.orchestration.step.completed.v1`	Orchestrator	DeltaAggregator batch
`k1.orchestration.step.failed.v1`	Orchestrator	Error routing
`k1.orchestration.dag.completed.v1`	Orchestrator	Trigger DELIVERING state
`k1.orchestration.delta.v1`	Orchestrator	User-facing progress stream
`k1.planner.stage.completed.v1`	Planner	Progress update (HIGH tier)
`k1.agent.{id}.delta.v1`	Sub-agents	DeltaAggregator batch (HIL, results)
`k1.hil.request.v1`	Sub-agents	ClarificationTracker (pending_clarifications)

Published topics (Concierge emits):

TOPIC	WHEN	PAYLOAD
`k1.session.started.v1`	Session init	`{session_id, user_id, timestamp}`
`k1.session.ended.v1`	Session teardown	`{session_id, duration_ms, turn_count}`
`k1.concierge.turn.started.v1`	Turn begins	`{turn_id, trace_id, timestamp}`
`k1.concierge.turn.completed.v1`	Turn ends	`{turn_id, tier, latency_ms, tool_count}`
`k1.concierge.degraded.v1`	CB triggers degradation	`{from_tier, to_tier, reason}`
`k1.concierge.safety.escalated.v1`	RED/CRISIS detected	`{safety_band, intent, action}`

Delta dataclass:

```python
@dataclass
class Delta:
    """A single delta event from K1 Bus."""
    topic: str                          # K1 Bus topic
    source: str                         # Originator component
    payload: Dict[str, Any]             # Event-specific data
    trace_id: str                       # Distributed trace ID
    timestamp_ms: int                   # Unix epoch ms
    sequence: int                       # Per-topic sequence number (ordering)
```

14.9 IMemoryPort

Direction: Outbound. **Adapter**: `BridgeRecallAdapter` (prod), `MockMemoryAdapter` (test). **CB**: Implicit via Bridge client.

Access to K0 long-term memory via Bridge. Used by `recall_memory()` tool (read), `promote_belief()` tool (write), and `ContextAssembler` (prefetch).

```python
class IMemoryPort(Protocol):
    """K0 long-term memory access via Bridge."""

    async def recall(
        self,
        query: MemoryQuery,
        selectors: Optional[List[str]] = None,
    ) -> MemoryResult:
        """Query K0 long-term memory.

        Args:
            query: Search query with text, time range, entity filters.
            selectors: Optional memory layer selectors (episodic, semantic, procedural).

        Returns:
            MemoryResult with ranked hits, each containing content, timestamp,
            relevance_score, source_layer.

        Raises:
            BridgeTimeoutError: K0 Bridge did not respond within 500ms.
            BridgeUnavailableError: Bridge connection lost.
        """
        ...

    async def store(self, item: MemoryItem) -> StoreResult:
        """Write to K0 long-term memory via Bridge WAL.

        Used only by promote_belief() tool -- promotes WARM facts to K0 Cloud.
        Async: write goes to Bridge WAL, sync happens in background.

        Args:
            item: MemoryItem with content, metadata, layer classification.

        Returns:
            StoreResult with wal_id, queued status.

        Raises:
            BridgeWriteError: WAL write failed.
        """
        ...

    async def prefetch(self, hints: List[str]) -> None:
        """Prefetch memory based on anticipated queries (fire-and-forget).

        Called by Anticipatory Response (Experience Layer, Section 9.4).
        Results cached locally for 60s.
        """
        ...
```

MemoryQuery and MemoryResult:

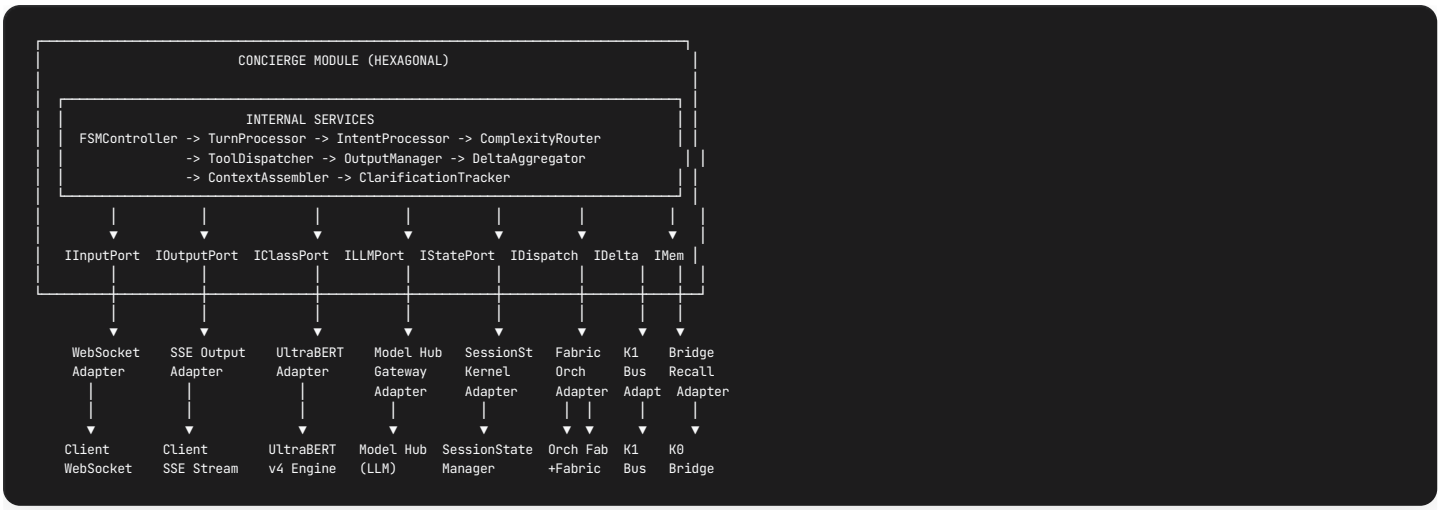
```python
@dataclass
class MemoryQuery:
    """Query for K0 long-term memory."""
    text: str                           # Natural language query
    time_range: Optional[Tuple[int, int]] = None  # Unix ms range
    entity_filter: Optional[List[str]] = None     # Filter by entity names
    layer: Optional[Literal["episodic", "semantic", "procedural"]] = None
    top_k: int = 5                      # Max results

@dataclass
class MemoryResult:
    """Results from K0 memory recall."""
    hits: List[MemoryHit]               # Ranked by relevance
    query_latency_ms: float             # Round-trip to K0
    source: Literal["bridge", "local_cold"]  # Where results came from

@dataclass
class MemoryHit:
    """Single memory recall result."""
    content: str                        # Memory content text
    timestamp_ms: int                   # When the memory was formed
    relevance_score: float              # 0.0-1.0
    source_layer: str                   # episodic, semantic, procedural
    metadata: Dict[str, Any]            # Entity tags, context
```

Offline fallback (when Bridge unavailable):

```
BridgeRecallAdapter:
  1. Try Bridge client (QueryPort) -> K0 Cloud
  2. On timeout/error: fall back to LOCAL COLD (K1 SQLite archive)
  3. Tag result source="local_cold" so LLM knows freshness is uncertain
  4. Never TERMINAL -- Concierge responds without memory if both fail (Edge-First)
```

14.10 Port Wiring Diagram

Complete port-to-adapter-to-touchpoint chain for all 8 ports:



14.11 Port-to-Service Assignment Matrix (Complete)

SERVICE	IINPUT	IOUTPUT	ICLASS	ILLM	ISTATE	IDISPATCH	IDELTA	IMEM
FSMController	R	W	-	-	R	-	-	-
TurnProcessor	-	-	-	-	RW	-	R	-
IntentProcessor	-	-	R	-	-	-	-	-
ComplexityRouter	-	-	-	-	-	-	-	-
ToolDispatcher	-	-	-	RW	RW	RW	-	R
OutputManager	-	W	-	-	-	-	W	-
DeltaAggregator	-	-	-	-	-	-	RW	-
ClarificationTracker	-	-	-	-	-	-	-	-
ContextAssembler	-	-	-	-	R	-	-	R

Legend: R = read, W = write, RW = both, `-` = not used.

14.12 Port Lifecycle

Ports are initialized during Concierge module bootstrap and torn down on session end:

```
Session Init:
  1. Construct all 8 port adapters (prod or test based on config)
  2. Wire adapters to ports via dependency injection
  3. Open IInputPort connection (WebSocket accept)
  4. Initialize IDeltaPort subscriptions (all 9 consumed topics)
  5. Warm IClassificationPort (load UltraBERT checkpoint if not cached)
  6. Verify IStatePort connectivity (read meta section, <1ms)
  7. Emit k1.session.started.v1 via IDeltaPort

Session Teardown (reverse order):
  1. Cancel all in-flight IDispatchPort operations
  2. Flush IDeltaPort publish queue
  3. Send final response via IOutputPort (if pending)
  4. Checkpoint IStatePort (save session snapshot)
  5. Unsubscribe IDeltaPort from all topics
  6. Close IInputPort connection
  7. Emit k1.session.ended.v1 via IDeltaPort
  8. Release all adapter resources
```

14.13 Cross-Reference: Port Appearances

PORT	DEFINED IN	DEEPLY SPECIFIED IN	ALSO REFERENCED IN
IInputPort	14.2	–	5.1 (FSM states), 3.3
IOutputPort	14.3	–	7.3 (acknowledge), 5.1, 3.3, 10.1
IClassificationPort	14.4	11 (UltraBERT)	6 (Phase 1), 12.4
ILLMPort	14.5	7 (Phase 2 LLM)	10 (all tool calls)
IStatePort	14.6	**13 (full spec)**	6.5 (Write Elision), 10.2-10.7 (cognitive tools), 12.4.3
IDispatchPort	14.7	**4.2 (full spec)**	10.8 (action tools), 4.4 (tier routing)
IDeltaPort	14.8	–	4.7 (delta reception), 3.5 (no CB)
IMemoryPort	14.9	–	10.4 (recall_memory), 10.7 (promote_belief)

15. Adapter Specifications

15.1 Adapter Architecture Pattern

All Concierge adapters follow the established K1 adapter pattern observed in Orchestrator and Planner modules:

```python
class ConciergeAdapter:
    """Pattern: K1 adapter conventions."""

    __slots__ = ("_wrapped", "_component_id")   # Minimal memory footprint

    def __init__(self, wrapped: WrappedProtocol, component_id: str = "concierge") -> None:
        self._wrapped = wrapped                  # Injected external dependency
        self._component_id = component_id        # Pre-stamped for tracing/cost

    # Adapter contract:
    # 1. Pure pass-through: translate types, never add business logic
    # 2. Error mapping: catch raw exceptions -> AdapterError(severity, adapter_name, ...)
    # 3. Trace propagation: stamp trace_id on all outbound calls (FAB-09)
    # 4. Thread safety: document concurrency guarantees
    # 5. CB ownership: document which circuit breaker(s) this adapter owns
```

Error severity model (aligned with Orchestrator `ErrorSeverity` enum):

SEVERITY	MEANING	ADAPTER BEHAVIOR
`RECOVERABLE`	Transient failure, retry-safe	Retry once internally, then propagate
`DEGRADED`	Partial failure, fallback available	Return fallback value, log warning
`TERMINAL`	Unrecoverable	Propagate immediately, no retry

15.2 Production Adapters (8)

15.2.1 WebSocketInputAdapter (IInputPort)

```python
class WebSocketInputAdapter:
    """Production IInputPort: WebSocket -> UserMessage.

    Responsibilities:
    - Accept WebSocket connections from client SDK
    - Deserialize JSON frames into UserMessage
    - Buffer incoming messages (max 3, for interrupt detection)
    - Validate: UTF-8, max 4096 chars, inject trace_id if missing
    - No CB (passive listener -- failures are connection-level)

    Constructor Args:
        ws_connection: Active WebSocket connection
        session_id: Pre-bound session identifier
        max_buffer: Message buffer depth (default: 3)

    Thread Safety:
        receive() is called from FSMController's single event loop.
        has_buffered() may be polled from interrupt-check task.
        Internal deque is thread-safe via asyncio single-thread model.
    """

    __slots__ = ("_ws", "_session_id", "_buffer", "_max_buffer", "_closed")

    async def receive(self) -> UserMessage:
        """Block until next message. Maps ws.recv() -> UserMessage.

        Error mapping:
            ConnectionClosed -> SessionClosedError
            InvalidJSON      -> skip frame, wait for next
            Oversized (>4KB) -> truncate, log warning
        """
        ...

    async def receive_with_timeout(self, timeout_ms: int) -> Optional[UserMessage]:
        """Wait up to timeout_ms. Returns None on timeout.

        Uses asyncio.wait_for(self._ws.recv(), timeout=timeout_ms/1000).
        """
        ...

    def has_buffered(self) -> bool:
        """Check internal buffer for pending messages (interrupt detection)."""
        return len(self._buffer) > 0
```

Alternate: `RESTInputAdapter` for HTTP polling mode (degraded experience, no streaming).

15.2.2 SSEOutputAdapter (IOutputPort)

```python
class SSEOutputAdapter:
    """Production IOutputPort: OutputEvent -> SSE stream.

    Responsibilities:
    - Serialize OutputEvent to SSE text/event-stream format
    - Priority-based retry logic (REALTIME: 3x, INTERACTIVE: 1x, BACKGROUND: 0x)
    - Dead-letter queue for failed REALTIME/INTERACTIVE events
    - CB_SSE ownership: buffer up to 5 messages when OPEN, flush on reconnect
    - Backpressure: drop oldest BACKGROUND events when client slow
    - Typing indicator management (start/stop)

    Constructor Args:
        sse_connection: Active SSE response stream
        dead_letter_queue: DeadLetterQueue for failed critical events
        cb_sse: CircuitBreaker instance (timeout=5s reconnect)

    Thread Safety:
        send() called from OutputManager (single-threaded per session).
        send_stream() may run concurrently with send() for progress overlay.
        Internal lock protects write ordering.

    SSE Format:
        event: {event_type}
        id: {event_id}
        data: {json_payload}
        retry: 3000
    """

    __slots__ = ("_sse", "_dlq", "_cb", "_lock", "_buffer", "_stats")

    async def send(self, event: OutputEvent) -> DeliveryReceipt:
        """Send single event with priority-based retry.

        Error mapping:
            ConnectionError + REALTIME  -> retry 3x (100ms exp backoff), then dead-letter
            ConnectionError + INTERACT  -> retry 1x, then dead-letter
            ConnectionError + BACKGROUND-> drop silently
            CB_SSE OPEN                 -> buffer (max 5), return queued receipt
        """
        ...

    async def send_stream(self, events: AsyncIterator[OutputEvent]) -> StreamReceipt:
        """Stream multiple events with shared stream_id.

        Backpressure: buffer up to 10 pending events, drop oldest on overflow.
        Each event sent as separate SSE message for progressive rendering.
        """
        ...

    async def send_typing_indicator(self, active: bool) -> None:
        """Lightweight typing indicator (used as CB_SSE HALF_OPEN probe)."""
        ...

    async def flush_buffer(self) -> int:
        """Flush buffered messages after CB_SSE recovery. Returns count flushed."""
        ...
```

Alternate: `WebSocketOutputAdapter` for duplex WebSocket sessions.

15.2.3 UltraBERTAdapter (IClassificationPort)

```python
class UltraBERTAdapter:
    """Production IClassificationPort: text -> ClassificationResult.

    Responsibilities:
    - Tokenize input text (ModernBERT tokenizer, max 512 tokens)
    - Run UltraBERT v4 forward pass (12 heads, single inference)
    - Map raw tensor outputs to ClassificationResult dataclass
    - CB_MODEL_HUB integration: on OPEN, delegate to HeuristicFallbackClassifier
    - Checkpoint management: load/swap model checkpoints

    Constructor Args:
        model: Loaded UltraBERT v4 model instance
        tokenizer: ModernBERT tokenizer
        fallback: HeuristicFallbackClassifier instance
        cb_model: CircuitBreaker instance (shared with ILLMPort)
        device: Inference device ("cpu" default, "cuda:0" for GPU)

    Thread Safety:
        Single inference at a time (model is not thread-safe).
        Protected by asyncio.Lock for concurrent access prevention.

    Performance:
        P95: 22ms, P99: 25ms (CPU)
        P95: 5ms (GPU with Flash Attention 2)
    """

    __slots__ = ("_model", "_tokenizer", "_fallback", "_cb", "_device", "_lock")

    async def classify(
        self, text: str, context: Optional[ClassificationContext] = None
    ) -> ClassificationResult:
        """Run UltraBERT v4 inference.

        Pipeline:
        1. Check CB_MODEL_HUB state -> if OPEN, delegate to _fallback.classify()
        2. Tokenize (max 512 tokens, truncate with warning)
        3. Forward pass (12 heads, single batch)
        4. Post-process: softmax/sigmoid per head, extract spans (GlobalPointer)
        5. Assemble ClassificationResult

        Error mapping:
            InferenceTimeout (>25ms) -> ClassificationTimeoutError
            ModelNotLoaded           -> ModelNotLoadedError
            CB OPEN                  -> HeuristicFallbackClassifier (<1ms)
            CUDA OOM (GPU mode)      -> fall back to CPU, log warning
        """
        ...
```

HeuristicFallbackClassifier (activated when CB_MODEL_HUB OPEN):

RULE	PATTERN	RESULT
1 (CRISIS)	Hardcoded keyword scan	safety=CRISIS, intent=other, tier=CRISIS
2 (Question)	`?` + <20 words	intent=conversational, tier=LOW
3 (Task)	remind/schedule/set	intent=set_reminder, tier=MEDIUM
4 (Plan)	plan/help me with	intent=plan_event, tier=HIGH
5 (Default)	Everything else	intent=conversational, tier=LOW

15.2.4 ModelGatewayAdapter (ILLMPort)

```python
class ModelGatewayAdapter:
    """Production ILLMPort: HubRequest -> Model Hub -> HubResponse.

    Responsibilities:
    - Route HubRequest through ILLMRequestBus to Model Hub
    - Stamp consumer_id="concierge" for cost attribution (MH-11)
    - Capability routing: CHAT, TOOL_CALL, STRUCTURED, VISION, REASON
    - CB_MODEL_HUB integration: 3-level cascade (retry -> probe -> canned)
    - Stream support via stream_execute() for progressive token delivery

    Constructor Args:
        llm_request_bus: ILLMRequestBus protocol (async request-reply to Model Hub)
        consumer_id: Cost attribution tag (default: "concierge")
        cb_model: CircuitBreaker instance (shared with IClassificationPort)
        canned_responses: Dict[str, str] for CB OPEN fallback

    Thread Safety:
        Safe for concurrent execute() calls from ReAct loop iterations.
        Bus handles request isolation internally.

    Pattern reference: k1/planner/adapters/llm_gateway_adapter.py (same bus protocol)
    """

    __slots__ = ("_bus", "_consumer_id", "_cb", "_canned")

    async def execute(self, request: HubRequest) -> HubResponse:
        """Execute single LLM request via Model Hub.

        Pre-processing:
        1. Stamp consumer_id on request.constraints
        2. Check CB_MODEL_HUB state

        Error mapping:
            asyncio.TimeoutError -> LLMTimeoutError (with stage extraction)
            "budget" in error    -> BudgetExceededError
            Network/bus failure  -> AdapterError(DEGRADED)
            CB OPEN              -> canned response template

        LLM Cascade:
            L1: RETRY (timeout/5xx) -> retry once with same provider
            L2: CB HALF-OPEN (30s window) -> wait for probe result
            L3: CANNED_RESPONSE (CB OPEN) -> pre-written template
        """
        ...

    async def stream_execute(self, request: HubRequest) -> AsyncIterator[HubChunk]:
        """Streaming execution for progressive token delivery.

        Used during DELIVERING state for real-time response streaming.
        Each yielded HubChunk contains a token fragment.

        Error mapping:
            Same as execute() plus StreamInterruptedError on mid-stream failure.
            On stream failure: yield partial result + error marker.
        """
        ...

    def available_capabilities(self) -> FrozenSet[str]:
        """Return capabilities based on Model Hub manifest and CB state.

        CB OPEN: returns {"CHAT"} only (canned responses only).
        CB CLOSED: returns full set from provider manifest.
        """
        ...
```

15.2.5 SessionKernelAdapter (IStatePort)

```python
class SessionKernelAdapter:
    """Production IStatePort: SessionState read/write with MutationGuard.

    Responsibilities:
    - Read: delegate to SessionStateManager read methods (lock-free snapshots)
    - Write: enforce writer authorization + MutationGuard.preflight() before mutate()
    - CB_SESSIONSTATE integration: on OPEN, return stale cached snapshot for reads
    - Emergency mode: >=95KB occupancy triggers 50ms GC wait + retry
    - Batch writes: process ordered list, optional stop-on-rejection

    Constructor Args:
        manager: SessionStateManager instance
        writer_adapter: DirectWriterAdapter (wraps IWriterPort)
        session_id: Pre-bound session identifier
        cb_session: CircuitBreaker instance (timeout=100ms, reset=10s)
        cache: LRU cache for stale-read fallback (max 8 sections)

    Thread Safety:
        Reads: concurrent-safe (lock-free snapshots from SessionStateManager)
        Writes: serialized via DirectWriterAdapter's internal RLock
        Cache: thread-safe LRU with TTL (10s staleness window)

    Pattern reference: k1/sessionstate/adapters/direct_writer.py (MutationGuard flow)
    """

    __slots__ = ("_manager", "_writer", "_session_id", "_cb", "_cache", "_stats")

    async def read(self, section: str) -> SectionSnapshot:
        """Read single section (lock-free snapshot, <1ms).

        On CB_SESSIONSTATE OPEN: return cached version (stale, tagged).
        On cache miss + CB OPEN: return empty SectionSnapshot.
        """
        ...

    async def read_all_hot(self) -> HotTierSnapshot:
        """Read all 8 HOT sections atomically. Used at turn_start()."""
        ...

    async def read_sections(self, sections: List[str]) -> Dict[str, SectionSnapshot]:
        """Batch read of specified sections."""
        ...

    async def write(self, section: str, delta: SectionDelta) -> WriteResult:
        """Single section write via MutationGuard.

        Pipeline:
        1. Check CB_SESSIONSTATE -> if OPEN, reject (DEGRADED)
        2. DirectWriterAdapter.request_mutation():
           a. Check session expiry
           b. validate_writer("concierge") -- CONC-01 enforcement
           c. MutationGuard.preflight(section, op, bytes) -- capacity/lock check
           d. SessionStateManager.mutate() -- apply delta
        3. Update cache with post-write snapshot
        4. Return WriteResult(status, section, version)

        Error mapping:
            RejectionCategory.AUTHORIZATION -> AdapterError(TERMINAL)
            RejectionCategory.CAPACITY      -> AdapterError(DEGRADED, suggest eviction)
            RejectionCategory.LOCKED        -> AdapterError(RECOVERABLE, retry after 10ms)
            RejectionCategory.VALIDATION    -> AdapterError(TERMINAL, schema violation)
            RejectionCategory.EMERGENCY     -> 50ms GC wait, retry once
        """
        ...

    async def write_batch(self, deltas: Dict[str, SectionDelta]) -> BatchWriteResult:
        """Ordered batch write. Stops on first TERMINAL rejection."""
        ...

    async def checkpoint(self) -> CheckpointResult:
        """Snapshot to LOCAL COLD (SQLite). Called at turn_end()."""
        ...

    async def restore(self, checkpoint_id: str) -> RestoreResult:
        """Restore from LOCAL COLD checkpoint. Called during crash_recovery()."""
        ...
```

15.2.6 FabricOrchestratorAdapter (IDispatchPort)

```python
class FabricOrchestratorAdapter:
    """Production IDispatchPort: tier-based routing to Fabric or Orchestrator.

    Responsibilities:
    - LOW tier: dispatch_direct() -> Fabric.execute() (bypass Orchestrator)
    - MEDIUM/HIGH: dispatch_envelope() -> Orchestrator WFQ mailbox
    - Cancel: cancel_dispatch() -> Orchestrator cancel or Fabric abort
    - CB ownership: CB_ORCHESTRATOR, CB_FABRIC, CB_MCP (3 CBs)
    - Tier degradation cascade: HIGH -> MEDIUM -> LOW -> canned

    Constructor Args:
        fabric: Fabric facade instance
        orchestrator_mailbox: IMailboxPort enqueue interface
        cb_orchestrator: CircuitBreaker (timeout=60s, reset=60s)
        cb_fabric: CircuitBreaker (timeout=30s, reset=30s)
        cb_mcp: CircuitBreaker (timeout=10s, reset=30s)

    Thread Safety:
        dispatch_direct() safe for concurrent calls (Fabric is thread-safe).
        dispatch_envelope() serialized by mailbox lock.
        cancel_dispatch() safe (idempotent cancel semantics).

    Pattern reference: k1/orchestrator/adapters/fabric_gateway_adapter.py
    """

    __slots__ = ("_fabric", "_mailbox", "_cb_orch", "_cb_fabric", "_cb_mcp", "_inflight")

    async def dispatch_direct(self, capability_id: str, params: Dict) -> ExecutionResult:
        """LOW tier: direct Fabric execution.

        Pipeline:
        1. Check CB_FABRIC -> if OPEN, return degraded result
        2. Build CapabilityRequest(capability_id, params, trace_id)
        3. Fabric.execute(request) -> CapabilityResult
        4. Map to ExecutionResult

        Error mapping:
            FabricError/timeout  -> AdapterError(DEGRADED, "Fabric timeout")
            CB_MCP OPEN          -> mark specific tool unavailable
            ConnectionError      -> AdapterError(DEGRADED, "Fabric unreachable")
        """
        ...

    async def dispatch_envelope(self, envelope: TaskEnvelope) -> DispatchReceipt:
        """MEDIUM/HIGH: enqueue to Orchestrator.

        Pipeline:
        1. Check CB_ORCHESTRATOR -> if OPEN, trigger tier degradation
        2. mailbox.enqueue(envelope, priority="INTERACTIVE")
        3. Track in _inflight map for cancel support

        Error mapping:
            MailboxFullError (>100) -> retry 100ms, then degrade to LOW
            CB_ORCHESTRATOR OPEN    -> degrade tier, re-route
            CB_PLANNER OPEN (HIGH)  -> degrade HIGH to MEDIUM
        """
        ...

    async def cancel_dispatch(self, envelope_id: str) -> CancelResult:
        """Cancel in-flight dispatch (idempotent).

        Returns: CANCELLED | NOT_FOUND | ALREADY_COMPLETE
        """
        ...
```

Tier degradation cascade (managed internally):

```
HIGH fails: CB_PLANNER OPEN or Planner timeout
  -> strip tier to MEDIUM, re-dispatch (max 2 Fabric calls)
  -> emit k1.concierge.degraded.v1 {from: HIGH, to: MEDIUM}

MEDIUM fails: CB_ORCHESTRATOR OPEN or mailbox full
  -> degrade to LOW, dispatch_direct() to Fabric
  -> emit k1.concierge.degraded.v1 {from: MEDIUM, to: LOW}

LOW fails: CB_FABRIC OPEN or all tools unavailable
  -> return CRISIS_STATIC canned response
  -> emit k1.concierge.degraded.v1 {from: LOW, to: CANNED}
```

15.2.7 DeltaBusAdapter (IDeltaPort)

```python
class DeltaBusAdapter:
    """Production IDeltaPort: K1 Bus subscribe/publish for delta streaming.

    Responsibilities:
    - Subscribe to 9 consumed topics at session init
    - Publish lifecycle events (fire-and-forget, NEVER raises)
    - Error stream for observability (bus-level failures)
    - Bulk unsubscribe at shutdown

    Constructor Args:
        bus: K1 Bus (LocalBus or distributed bus)
        component_id: Pre-stamped source identifier (default: "concierge")

    Thread Safety:
        publish() is fire-and-forget, never blocks (like Planner DeltaBusAdapter).
        subscribe() returns AsyncIterator consumed by DeltaAggregator.
        errors() consumed by observability logger.

    Pattern reference: k1/planner/adapters/delta_bus_adapter.py (fire-and-forget)
    Pattern reference: k1/bus/adapters/session_adapter.py (topic mapping)
    """

    __slots__ = ("_bus", "_component_id", "_handles")

    async def subscribe(self, topics: List[str]) -> AsyncIterator[Delta]:
        """Subscribe to delta topics. Tracks handles for shutdown cleanup.

        Topic format: "k1.orchestration.*", "k1.planner.*", "k1.agent.{id}.*"
        Wildcard matching delegated to bus implementation.
        """
        ...

    async def publish(self, delta: Delta) -> None:
        """Fire-and-forget publish. Catches ALL exceptions, logs warning.

        Pre-stamps: component_id, trace_id, timestamp_ms.
        NEVER raises (CONC-19: delta loss is acceptable).
        """
        ...

    async def errors(self) -> AsyncIterator[BusError]:
        """Stream bus-level errors for observability logging."""
        ...

    def shutdown(self) -> None:
        """Bulk unsubscribe all tracked handles. Called during session teardown."""
        ...
```

15.2.8 BridgeRecallAdapter (IMemoryPort)

```python
class BridgeRecallAdapter:
    """Production IMemoryPort: K0 Bridge for long-term memory access.

    Responsibilities:
    - recall(): query K0 via Bridge QueryPort, fall back to LOCAL COLD
    - store(): write to Bridge WAL (fire-and-forget for promote_belief)
    - prefetch(): warm local cache with anticipated queries
    - Offline detection via bridge.is_available()

    Constructor Args:
        bridge_port: Fabric IBridgePort instance (query, send_command, is_available)
        local_cold_store: LOCAL COLD SQLite archive for offline fallback
        cache_ttl_s: Prefetch cache TTL (default: 60s)

    Thread Safety:
        recall() safe for concurrent calls (Bridge client is thread-safe).
        store() fire-and-forget (WAL append is atomic).
        prefetch() populates shared cache (LRU with TTL, thread-safe).

    Pattern reference: k1/planner/adapters/bridge_adapter.py (offline handling)
    """

    __slots__ = ("_bridge", "_cold_store", "_cache", "_cache_ttl")

    async def recall(
        self, query: MemoryQuery, selectors: Optional[List[str]] = None
    ) -> MemoryResult:
        """Query K0 long-term memory with offline fallback.

        Pipeline:
        1. Check _bridge.is_available()
        2. If online: bridge.query("memory.recall", {query, selectors, trace_id})
           -> Map BridgeCommandResult.data -> MemoryResult(source="bridge")
        3. If offline or error: query LOCAL COLD archive
           -> MemoryResult(source="local_cold")
        4. If both fail: return empty MemoryResult (Edge-First: respond without memory)

        Error mapping:
            BridgeTimeout (>500ms) -> fall to LOCAL COLD
            BridgeUnavailable      -> fall to LOCAL COLD
            LOCAL COLD error       -> empty MemoryResult
            NEVER TERMINAL         -> always returns something
        """
        ...

    async def store(self, item: MemoryItem) -> StoreResult:
        """Write to K0 via Bridge WAL. Fire-and-forget.

        On offline: drop silently (WAL write is supplementary).
        """
        ...

    async def prefetch(self, hints: List[str]) -> None:
        """Warm cache with anticipated queries. Fire-and-forget.

        Called by Anticipatory Response (Experience Layer, Section 9.4).
        Results cached for cache_ttl_s seconds.
        """
        ...
```

15.3 Test Adapters (8)

All test adapters are in-memory, deterministic, and designed for integration testing without external dependencies. They follow the pattern: inject scripted inputs, capture outputs for assertion.

TEST ADAPTER	PORT	KEY CAPABILITIES
`TestInputAdapter`	IInputPort	`inject(msg)`, `inject_sequence([msgs])`, `set_interrupt(msg)`
`TestOutputAdapter`	IOutputPort	`captured: List[OutputEvent]`, `assert_event_type(type, count)`, `assert_order([types])`
`MockClassificationAdapter`	IClassificationPort	`script(ClassificationResult)`, `script_sequence([results])`, `fallback_result`
`MockLLMAdapter`	ILLMPort	`script_response(HubResponse)`, `script_tool_calls([ToolCall])`, `simulate_timeout()`, `call_log: List[HubRequest]`
`InMemoryStateAdapter`	IStatePort	`Dict[str, Any]` backing store, `write_log: List[MutationRequest]`, `simulate_rejection(category)`, `snapshot()`
`MockDispatchAdapter`	IDispatchPort	`envelopes: List[TaskEnvelope]`, `direct_calls: List[(cap_id, params)]`, `simulate_mailbox_full()`, `simulate_cb_open()`
`TestDeltaAdapter`	IDeltaPort	`inject_delta(Delta)`, `published: List[Delta]`, `simulate_bus_error()`
`MockMemoryAdapter`	IMemoryPort	`canned_results: Dict[str, MemoryResult]`, `recall_log: List[MemoryQuery]`, `store_log: List[MemoryItem]`

```python
class TestInputAdapter:
    """In-memory IInputPort for integration tests.

    Usage:
        adapter = TestInputAdapter()
        adapter.inject(UserMessage(text="Remind Mom Thursday"))
        adapter.inject(UserMessage(text="ok"))  # queued

        msg = await adapter.receive()           # returns first
        msg2 = await adapter.receive()          # returns second

        # Interrupt simulation
        adapter.set_interrupt(UserMessage(text="Never mind"))
        assert adapter.has_buffered() is True
    """

    __slots__ = ("_queue", "_interrupt")

    def __init__(self) -> None:
        self._queue: asyncio.Queue[UserMessage] = asyncio.Queue()
        self._interrupt: Optional[UserMessage] = None

    def inject(self, msg: UserMessage) -> None: ...
    def inject_sequence(self, msgs: List[UserMessage]) -> None: ...
    def set_interrupt(self, msg: UserMessage) -> None: ...
    async def receive(self) -> UserMessage: ...
    async def receive_with_timeout(self, timeout_ms: int) -> Optional[UserMessage]: ...
    def has_buffered(self) -> bool: ...


class TestOutputAdapter:
    """In-memory IOutputPort for assertion-based testing.

    Usage:
        adapter = TestOutputAdapter()
        await service.process(msg)              # service uses adapter internally

        assert len(adapter.captured) == 3
        adapter.assert_event_type("acknowledge", count=1)
        adapter.assert_event_type("response_text", count=1)
        adapter.assert_order(["acknowledge", "response_text", "delta_summary"])
    """

    __slots__ = ("captured", "_stream_log", "_typing_state")

    def __init__(self) -> None:
        self.captured: List[OutputEvent] = []
        self._stream_log: List[List[OutputEvent]] = []
        self._typing_state: bool = False

    async def send(self, event: OutputEvent) -> DeliveryReceipt: ...
    async def send_stream(self, events: AsyncIterator[OutputEvent]) -> StreamReceipt: ...
    async def send_typing_indicator(self, active: bool) -> None: ...

    def assert_event_type(self, event_type: str, count: int) -> None: ...
    def assert_order(self, expected_types: List[str]) -> None: ...
    def assert_no_event_type(self, event_type: str) -> None: ...


class MockLLMAdapter:
    """Scripted ILLMPort for deterministic LLM behavior in tests.

    Usage:
        adapter = MockLLMAdapter()
        adapter.script_response(HubResponse(
            response_text="I'll remind Mom!",
            tool_calls=[ToolCall(name="update_scoreboard", args={...})],
            finish_reason="tool_calls",
        ))

        response = await adapter.execute(any_request)  # returns scripted
        assert len(adapter.call_log) == 1              # records all calls
    """

    __slots__ = ("_responses", "_call_log", "_timeout_next")

    async def execute(self, request: HubRequest) -> HubResponse: ...
    async def stream_execute(self, request: HubRequest) -> AsyncIterator[HubChunk]: ...
    def available_capabilities(self) -> FrozenSet[str]: ...

    def script_response(self, response: HubResponse) -> None: ...
    def script_tool_calls(self, calls: List[ToolCall]) -> None: ...
    def simulate_timeout(self) -> None: ...

    @property
    def call_log(self) -> List[HubRequest]: ...
```

15.4 Adapter Wiring Summary

Complete injection chain at session bootstrap (performed by `ConciergeFactory`):

```
ConciergeFactory.create(session_id, config) -> Concierge:

  # 1. Infrastructure connections (external)
  ws_conn        = await WebSocketPool.accept(session_id)
  sse_conn       = SSEConnectionManager.create(session_id)
  ultrabert      = ModelRegistry.get("ultrabert_v4")
  llm_bus        = LLMRequestBusFactory.create()
  ss_manager     = SessionStateManager.get(session_id)
  writer         = DirectWriterAdapter(ss_manager, authorized_writers={"concierge"})
  fabric         = FabricFactory.get()
  orch_mailbox   = OrchestratorService.mailbox_port
  bus            = BusFactory.get()
  bridge         = BridgeClientFactory.get()
  cold_store     = SQLiteColdStore.open(session_id)

  # 2. Circuit breakers (7 total)
  cb_model       = CircuitBreaker("CB_MODEL_HUB", timeout=...)
  cb_orch        = CircuitBreaker("CB_ORCHESTRATOR", timeout=60_000, reset=60_000)
  cb_fabric      = CircuitBreaker("CB_FABRIC", timeout=30_000, reset=30_000)
  cb_mcp         = CircuitBreaker("CB_MCP", timeout=10_000, reset=30_000)
  cb_session     = CircuitBreaker("CB_SESSIONSTATE", timeout=100, reset=10_000)
  cb_sse         = CircuitBreaker("CB_SSE", timeout=5_000)

  # 3. Adapters (8 production)
  input_adapter    = WebSocketInputAdapter(ws_conn, session_id)
  output_adapter   = SSEOutputAdapter(sse_conn, dlq, cb_sse)
  class_adapter    = UltraBERTAdapter(ultrabert, tokenizer, fallback, cb_model)
  llm_adapter      = ModelGatewayAdapter(llm_bus, "concierge", cb_model, canned)
  state_adapter    = SessionKernelAdapter(ss_manager, writer, session_id, cb_session)
  dispatch_adapter = FabricOrchestratorAdapter(fabric, orch_mailbox, cb_orch, cb_fabric, cb_mcp)
  delta_adapter    = DeltaBusAdapter(bus, "concierge")
  memory_adapter   = BridgeRecallAdapter(bridge, cold_store)

  # 4. Wire ports to adapters (hexagonal injection)
  ports = PortBundle(
      input=input_adapter,
      output=output_adapter,
      classification=class_adapter,
      llm=llm_adapter,
      state=state_adapter,
      dispatch=dispatch_adapter,
      delta=delta_adapter,
      memory=memory_adapter,
  )

  # 5. Construct services with ports (Section 16)
  return Concierge(ports=ports, config=config)
```

16. Internal Services (~585 tests)

16.1 Service Overview

SERVICE	TESTS	RESPONSIBILITY	PORTS USED	KEY INVARIANTS	LATENCY BUDGET
FSMController	~60	State transitions, experience triggers, interrupt handling	IInputPort, IOutputPort, IStatePort	CONC-14 (<=1ms transition), CONC-17 (single FSM)	1ms per transition
TurnProcessor	~90	Per-turn lifecycle: Phase 1 + Phase 2, lock management	IStatePort, IDeltaPort	CONC-04 (single writer lock), CONC-11 (120s watchdog), CONC-15 (1 active turn)	22ms Phase 1 + tier-dependent Phase 2
IntentProcessor	~110	SafetyGate, HypothesisGenerator, GapDetector, ReferenceResolver, ConfidenceScorer	IClassificationPort, IStatePort (read-only)	CONC-05 (safety first), CONC-06 (CRISIS bypass), CONC-13 (safety before routing)	25ms (classification) + 3ms (pipeline)
ComplexityRouter	~45	5-factor weighted complexity scoring, tier assignment	None (pure computation)	–	<0.2ms
ToolDispatcher	~80	Tool allowlist enforcement, schema validation, ReAct loop, budget tracking	ILLMPort, IDispatchPort, IStatePort, IMemoryPort	CONC-09 (<=20 calls/turn), CONC-02 (MutationGuard)	Per-tool variable
OutputManager	~55	9 event types, 3 priority levels, SSE delivery coordination	IOutputPort, IDeltaPort	CONC-02 (all output through channel), CONC-12 (<=50 queue depth)	50ms P95 delivery
DeltaAggregator	~50	500ms batching window, LWW merge, 4 delta types	IDeltaPort	CONC-10 (500ms fixed window)	500ms batch cycle
ClarificationTracker	~40	Max 3 rounds, pending HIL tracking, escalation decisions	None (state only)	CONC-08 (<=3 rounds)	–
ContextAssembler	~55	Token budget management, 6 context profiles, summarization trigger	IStatePort (read), IMemoryPort (prefetch)	CONC-22 (128K window)	<5ms assembly

Total: ~585 estimated tests.

16.2 FSMController (~60 tests)

Top-level session coordinator. Owns the single FSM instance (CONC-17). Delegates all per-turn work to `TurnProcessor`. Runs experience layer on modular turn schedules.

```
class FSMController:
```

```
    """Session-level FSM coordinator.

    Owns:
    - 8-state FSM with transition validation
    - Experience layer trigger schedule
    - Interrupt detection and routing
    - Session lifecycle (init, shutdown, crash_recovery)

    Dependencies (injected):
    - TurnProcessor (per-turn orchestration)
    - PortBundle (8 ports)
    - CircuitBreakerRegistry (7 CBs)
    - ExperienceEngine (4 heuristic components)

    Concurrency:
    - Single event loop per session (CONC-15: 1 active turn)
    - Interrupt detection via IInputPort.has_buffered() polling
    """

    __slots__ = (
        "_state", "_turn_processor", "_ports", "_cbs",
        "_experience", "_turn_count", "_session_id",
        "_transition_table", "_interrupt_task",
    )

    # ------------------------------------------------------------------
    # State Management
    # ------------------------------------------------------------------

    def current_state(self) -> FSMState:
        """Return current FSM state. Pure accessor, no I/O."""
        return self._state

    async def transition(self, event: FSMEvent, ctx: FSMContext) -> TransitionResult:
        """Execute validated state transition.

        Pipeline:
        1. Look up (current_state, event) in _transition_table
        2. If no valid transition: raise InvalidTransitionError
        3. Execute exit_action of current state (if any)
        4. Set new state
        5. Execute entry_action of new state (if any)
        6. Return TransitionResult(from_state, to_state, latency_ms)

        Invariant: CONC-14 -- total transition time <= 1ms (no I/O in transitions).
        All entry/exit actions are pure state mutations, never port calls.
        """
        ...

    def is_interruptible(self) -> bool:
        """True if current state allows interrupt (DISPATCHING, COMPANIONING, PROGRESSING)."""
        return self._state in _INTERRUPTIBLE_STATES

    # ------------------------------------------------------------------
    # Turn Orchestration
    # ------------------------------------------------------------------

    async def handle_message(self, msg: UserMessage) -> None:
        """Main entry point: receive message -> orchestrate turn -> return to LISTENING.

        Flow:
        1. LISTENING -> ACKING (transition)
        2. Delegate to TurnProcessor.process_turn(msg)
        3. TurnProcessor drives: Phase 1 -> Phase 2 -> DELIVERING
        4. turn_end() -> LISTENING

        Interrupt path:
        - If has_buffered() detected during DISPATCHING/COMPANIONING/PROGRESSING:
          -> transition to INTERRUPT_HANDLING
          -> TurnProcessor.cancel_turn()
          -> abbreviated turn_end()
          -> handle_message(buffered_msg) recursively
        """
        ...

    # ------------------------------------------------------------------
    # Experience Layer
    # ------------------------------------------------------------------

    async def tick_experience(self, turn_count: int) -> List[ExperienceAction]:
        """Check modular experience triggers.

        Schedule:
        - turn_count % 20 == 0: NarrativeWeaving -> NarrativeCluster[]
        - turn_count % 25 == 0: EmotionalProcessing -> EmotionalTrajectory
        - turn_count % 30 == 0: AnticipatoryResponse -> Anticipation{prefetched}

        Triggered AFTER turn_end(), BEFORE returning to LISTENING.
        Experience writes go through cognitive tools (update_narrative, refine_affect).
        Max 500ms budget for all experience processing.
        """
        ...

    # ------------------------------------------------------------------
    # Interrupt Handling
    # ------------------------------------------------------------------

    async def on_interrupt(self, new_msg: UserMessage) -> InterruptDecision:
        """Handle user interrupt during interruptible state.

        Decision:
        1. If current state not interruptible: buffer message, continue
        2. If interruptible:
           a. Transition to INTERRUPT_HANDLING
           b. Cancel in-flight dispatches via IDispatchPort.cancel_dispatch()
           c. Flush partial output via OutputManager.flush()
           d. Abbreviated turn_end()
           e. Re-enter ACKING with new message
```

```
            6. Re-enter ACKING with new message

        Returns: INTERRUPTED | BUFFERED | IGNORED
        """
        ...

    # ------------------------------------------------------------------
    # Lifecycle
    # ------------------------------------------------------------------

    async def init(self, session_id: str) -> None:
        """Session initialization (concierge.mmd LC_INIT).

        Steps:
        1. Create 9 services (no I/O)
        2. Connect 8 ports (adapters already injected)
        3. Restore state (Edge-First: LOCAL COLD first)
        4. Load persona -> PersonaEngine + Style
        5. Init 7 circuit breakers -> all CLOSED
        6. Start subscriptions (IDeltaPort) + DeltaAggregator timer
        7. FSM -> LISTENING
        """
        ...

    async def shutdown(self) -> None:
        """Graceful session shutdown (concierge.mmd LC_SHUTDOWN).

        Steps:
        1. IInputPort.close() (reject new input)
        2. Wait active turn (30s max) or force-close
        3. Flush DeltaAggregator + OutputManager
        4. Final checkpoint (LOCAL COLD + K0 5s wait)
        5. Stop background (timer, subscriptions, SSE)
        6. Disconnect ports (LIFO: Memory -> Input)
        """
        ...

    async def crash_recovery(self, session_id: str) -> None:
        """Crash recovery (concierge.mmd LC_CRASH_RECOVERY).

        Steps:
        1. Bootstrap (9 services + 8 ports)
        2. Detect: check turn_lock in control section
        3. Rollback: clear lock, lock_version++, turn_status=ROLLED_BACK
        4. Reconcile Local Outbox (Bridge drain)
        5. Restore from LOCAL COLD (last checkpoint)
        6. Resume: persona, CBs, subscriptions, LISTENING
        7. Emit recovery telemetry
        Max loss: 1 turn (the crashed one)
        """
        ...
```

Test categories (~60 tests):

CATEGORY	COUNT	WHAT IT TESTS
Transition validation	~15	All 14 valid transitions, invalid transition rejection
Interrupt handling	~12	Interrupt from each interruptible state, buffering, cancel
Experience triggers	~8	Modular schedule, correct trigger at turn count boundaries
Lifecycle	~10	init -> LISTENING, shutdown ordering, crash recovery
Edge cases	~15	Double transition, concurrent interrupt, timeout during transition

16.3 TurnProcessor (~90 tests)

Per-turn orchestrator. Manages the two-phase turn model, single writer lock, and watchdog timer.

```
class TurnProcessor:
    """Per-turn lifecycle coordinator.

    Owns:
    - Phase 1 (ACKING): UltraBERT classification -> hypothesis -> complexity routing
    - Phase 2 (DISPATCHING -> DELIVERING): LLM ReAct loop -> tools -> response
    - Single Writer lock (version-based, CONC-04)
    - Watchdog timer (120s max turn duration, CONC-11)
    - Turn boundary writes (history, meta, telemetry)

    Dependencies (injected):
    - IntentProcessor (Phase 1 pipeline)
    - ComplexityRouter (tier assignment)
    - ToolDispatcher (Phase 2 tool execution)
    - OutputManager (response delivery)
    - ContextAssembler (LLM context building)
    - IStatePort (read/write)
    - IDeltaPort (lifecycle events)
    """

    __slots__ = (
        "_intent_proc", "_complexity_router", "_tool_dispatcher",
        "_output_mgr", "_ctx_assembler", "_state_port", "_delta_port",
        "_lock_version", "_turn_sequence", "_watchdog", "_current_turn",
    )

    async def process_turn(self, message: UserMessage) -> TurnResult:
        """Full turn lifecycle.

        Steps:
```

```
        1. turn_start(message) -- acquire lock, read HOT snapshot
        2. phase1_acking(message) -- UltraBERT + hypothesis + routing
        3. FSM: ACKING -> DISPATCHING
        4. phase2_dispatch(phase1_result) -- LLM + tools by tier
        5. FSM: through COMPANIONING -> PROGRESSING -> DELIVERING
        6. turn_end() -- history append, checkpoint, back to LISTENING
        7. Return TurnResult(success, tier, latency_ms, tool_count, error?)
        """
        ...

    async def turn_start(self, message: UserMessage) -> TurnContext:
        """Acquire lock, read snapshot, reset services.

        Steps:
        1. Acquire Single Writer lock (lock_version++)
        2. Increment turn_sequence_number
        3. IStatePort.read_all_hot() -> HOT snapshot (all 8 sections)
        4. Reset: ToolDispatcher.clear_buffer(), ClarificationTracker.reset_turn(),
           ContextAssembler.reset_budget()
        5. Allocate tier budget envelope (timeout based on expected tier)
        6. Start watchdog timer (120s)
        7. Emit k1.concierge.turn.started.v1 via IDeltaPort
        """
        ...

    async def phase1_acking(self, message: UserMessage) -> Phase1Result:
        """Phase 1: deterministic classification + hypothesis + routing.

        Pipeline (22ms + 3ms):
        1. IntentProcessor.check_safety(classification) -> SafetyCheckResult
           - If CRISIS: return Phase1Result(crisis_override=True)
        2. IntentProcessor.generate_hypothesis(classification) -> IntentHypothesis
        3. IntentProcessor.detect_gaps(hypothesis, snapshot) -> gaps[]
        4. IntentProcessor.resolve_references(classification, snapshot) -> resolved[]
        5. ConfidenceScorer.score(hypothesis, gaps, resolved) -> uncertainty
        6. ComplexityRouter.route(HypothesisBundle) -> tier
        7. Write Elision Gate: compute per-section significance, write if needed
        8. Package Phase1Result

        Returns:
            Phase1Result{classification, hypothesis_bundle, tier, safety_band, gaps}
        """
        ...

    async def phase2_dispatch(self, phase1: Phase1Result) -> Phase2Result:
        """Phase 2: LLM-powered tool execution and response generation.

        Tier-specific behavior:
        - LOW: ToolDispatcher runs ReAct loop (max 8 tools, 2s budget)
          -> dispatch_direct() for action tools
          -> IOutputPort for response delivery
        - MEDIUM: Build TaskEnvelope (max 2 capabilities)
          -> dispatch_envelope() to Orchestrator
          -> Companioning while waiting
          -> DeltaAggregator for progress
        - HIGH: Build TaskEnvelope (tier=HIGH)
          -> dispatch_envelope() to Orchestrator -> Planner -> DAG
          -> Extended companioning (Proactive Agent)
          -> Progressive delta streaming

        All tiers end with DELIVERING state:
        1. Aggregate tool results
        2. Final LLM call for polished response
        3. Cognitive tool writes (scoreboard, beliefs, narrative)
        4. OutputManager.enqueue(response_text, INTERACTIVE)
        """
        ...

    async def turn_end(self) -> TurnSummary:
        """Turn boundary writes and cleanup.

        Steps:
        1. Flush OutputManager
        2. Append to history_active via IStatePort.write() (MutationGuard)
        3. Update meta (turn_count++, last_activity_ms)
        4. Update telemetry (tokens, cost, latency)
        5. Release Single Writer lock
        6. Checkpoint LOCAL COLD (SQLite < 1ms)
        7. Checkpoint K0 (fire-and-forget via IMemoryPort.store)
        8. Cancel watchdog timer
        9. Emit k1.concierge.turn.completed.v1 via IDeltaPort
        """
        ...

    async def turn_end_abbreviated(self) -> TurnSummary:
        """Abbreviated turn_end for interrupted turns.

        Steps:
        1. Cancel in-flight dispatches (IDispatchPort.cancel_dispatch)
        2. Flush OutputManager (partial progress)
        3. Record turn_status: INTERRUPTED
        4. Release Single Writer lock
        5. Checkpoint LOCAL COLD
        SKIP: history, telemetry, K0 sync, experience
        """
        ...

    async def cancel_turn(self) -> CancelResult:
        """Force-cancel the current turn (called by FSMController on interrupt)."""
        ...
```

Test categories (~90 tests):

CATEGORY	COUNT	WHAT IT TESTS
Turn lifecycle	~20	start-to-end for each tier (LOW, MEDIUM, HIGH)
Phase 1 pipeline	~15	Classification, hypothesis, gap detection, Write Elision Gate
Phase 2 by tier	~20	LOW direct, MEDIUM envelope, HIGH planner path
Lock management	~10	Acquire/release, concurrent turn rejection, lock timeout
Watchdog	~8	120s timeout, forced termination, partial result delivery
Turn boundary	~10	History append, checkpoint, telemetry update
Cancel/interrupt	~7	Cancel during each phase, abbreviated turn_end

16.4 IntentProcessor (~110 tests)

Executes the Phase 1 pipeline: Safety -> Hypothesis -> Gap Detection -> Reference Resolution -> Confidence Scoring. Detailed in Section 12.

```
class IntentProcessor:
    """Phase 1 processing pipeline (Section 12).

    Pipeline stages (executed in order):
    1. SafetyGate.evaluate() -- ALWAYS first (CONC-05)
    2. HypothesisGenerator.generate() -- form IntentHypothesis
    3. GapDetector.detect() -- find missing slots
    4. ReferenceResolver.resolve() -- pronoun/entity resolution
    5. ConfidenceScorer.score() -- compute uncertainty
    6. Package HypothesisBundle

    Dependencies (injected):
    - IClassificationPort (UltraBERT inference)
    - IStatePort (read-only, for ReferenceResolver)
    - SafetyGate (CRISIS keyword scan + safety_familyos head)
    - HypothesisGenerator (classification -> IntentHypothesis)
    - GapDetector (intent slots -> gaps[])
    - ReferenceResolver (pronouns -> resolved refs)
    - ConfidenceScorer (weighted uncertainty)
    """

    __slots__ = (
        "_classification_port", "_state_port",
        "_safety_gate", "_hypothesis_gen", "_gap_detector",
        "_ref_resolver", "_confidence_scorer",
    )

    async def process(
        self, text: str, snapshot: HotTierSnapshot
    ) -> ProcessedIntent:
        """Full Phase 1 pipeline.

        Returns either:
        - ProcessedIntent(hypothesis_bundle, tier_hint) for normal flow
        - CrisisOverride(safety_band=CRISIS, static_response) for CRISIS bypass

        Timing budget: 25ms (classification) + 3ms (pipeline) = 28ms total.
        """
        ...

    async def check_safety(
        self, classification: ClassificationResult
    ) -> SafetyCheckResult:
        """CRISIS check FIRST (CONC-05, CONC-06).

        Two-layer safety:
        1. safety_familyos head output (from UltraBERT)
        2. Hardcoded CRISIS keyword scan (backup, non-negotiable)

        If CRISIS: return SafetyCheckResult(crisis=True, response=CRISIS_STATIC)
        If RED: flag for special handling but continue pipeline
        If GREEN/AMBER: proceed normally
        """
        ...

    async def generate_hypothesis(
        self, classification: ClassificationResult
    ) -> IntentHypothesis:
        """Convert classification to structured hypothesis (Section 12.4.1)."""
        ...

    async def detect_gaps(
        self, hypothesis: IntentHypothesis,
        classification: ClassificationResult,
    ) -> List[Gap]:
        """Identify missing information slots (Section 12.4.2)."""
        ...

    async def resolve_references(
        self, classification: ClassificationResult,
        state_port: IStatePort,
    ) -> List[ResolvedReference]:
        """Resolve pronouns against SessionState (Section 12.4.3)."""
        ...
```

Test categories (~110 tests):

CATEGORY	COUNT	WHAT IT TESTS
SafetyGate	~25	CRISIS detection (keywords + head), RED flagging, GREEN passthrough
HypothesisGenerator	~20	All 8 intent classes, backchannel detection, continuation detection
GapDetector	~20	5 gap types, required vs optional slots, no false positives
ReferenceResolver	~20	Pronoun resolution, scoreboard antecedents, belief matching
ConfidenceScorer	~10	Weighted formula, edge cases (0.0, 1.0), gap weight accuracy
Full pipeline	~15	End-to-end with scripted ClassificationResult, CRISIS override

16.5 ComplexityRouter (~45 tests)

Pure computation service. No ports, no I/O, no async. Takes a `HypothesisBundle` and returns a `ComplexityTier`. Detailed in Section 12.6.

```python
class ComplexityRouter:
    """5-factor weighted complexity scoring (Section 12.6).

    Factors and weights:
    1. intent_complexity (0.30): base complexity by intent class
    2. entity_count (0.15): normalized evidence token count
    3. gap_count (0.15): normalized gap count
    4. uncertainty (0.25): ConfidenceScorer output
    5. domain_breadth (0.15): normalized domain count

    Thresholds: LOW < 0.3 | MEDIUM 0.3-0.7 | HIGH > 0.7

    Override rules:
    - safety_band == RED -> always MEDIUM minimum
    - safety_band == CRISIS -> CRISIS (bypasses router entirely)
    - is_backchannel == True -> always LOW
    - is_continuation == True -> always LOW
    """

    WEIGHTS: ClassVar[Dict[str, float]] = {
        "intent_complexity": 0.30,
        "entity_count": 0.15,
        "gap_count": 0.15,
        "uncertainty": 0.25,
        "domain_breadth": 0.15,
    }

    THRESHOLDS: ClassVar[Dict[str, float]] = {"LOW": 0.3, "HIGH": 0.7}

    def route(self, bundle: HypothesisBundle) -> ComplexityTier:
        """Compute weighted score and assign tier. Pure, <0.2ms."""
        ...

    def compute_score(self, bundle: HypothesisBundle) -> float:
        """Return raw complexity score [0.0-1.0] for observability."""
        ...
```

Test categories (~45 tests):

CATEGORY	COUNT	WHAT IT TESTS
Factor computation	~15	Each factor independently, normalization, capping
Tier boundaries	~10	Exact boundary values (0.3, 0.7), just-below/above
Override rules	~10	RED -> MEDIUM, CRISIS bypass, backchannel, continuation
Edge cases	~10	Empty hypothesis, max entities, all gaps, no domains

16.6 ToolDispatcher (~80 tests)

Manages the Phase 2 ReAct agentic loop. Enforces tool allowlist, schema validation, and call budget. Detailed in Sections 7 and 10.

```python
class ToolDispatcher:
    """Tool execution engine for Phase 2 (Sections 7, 10).

    7-step dispatch pipeline per tool call:
    1. Allowlist check (state + tier + safety band)
    2. Budget check (CONC-09: <=20 calls/turn)
    3. Schema validation (required params, type checking)
    4. ACK-first enforcement (first call must be acknowledge())
    5. Safety band check (RED+ blocks action tools)
    6. Execute via appropriate port
    7. Record result for LLM feedback

    ReAct loop:
    1. Build LLM prompt (DynamicPromptBuilder)
    2. LLM returns response_text + tool_calls[]
    3. For each tool_call: run 7-step pipeline
    4. Feed results back to LLM
    5. Repeat until LLM emits final response_text (no more tool_calls)
    6. Max iterations: 20 (CONC-09)

    Dependencies (injected):
    - ILLMPort (LLM execution)
    - IStatePort (cognitive tool writes)
    - IDispatchPort (action tool execution)
    - IMemoryPort (recall_memory, promote_belief)
    - IOutputPort (acknowledge() signal)
    - ToolRegistry (13 tools with schemas)
    """

    __slots__ = (
        "_llm_port", "_state_port", "_dispatch_port", "_memory_port",
        "_output_port", "_registry", "_call_count", "_ack_sent",
        "_results_buffer", "_tier", "_safety_band",
    )

    async def run_react_loop(
        self, phase1: Phase1Result, context: LLMContext
    ) -> Phase2Result:
        """Execute ReAct loop until LLM terminates or budget exhausted.

        Returns:
            Phase2Result with response_text, tool_results[], token_usage, latency_ms.
        """
        ...

    async def dispatch_tool(self, tool_call: ToolCall) -> ToolResult:
        """7-step dispatch pipeline for a single tool call.

        Step 1 - Allowlist: check _is_allowed(tool_call.name, _tier, _safety_band)
        Step 2 - Budget: check _call_count < 20
        Step 3 - Schema: validate tool_call.args against ToolSchema
        Step 4 - ACK-first: if _call_count == 0 and tool != "acknowledge" -> reject
        Step 5 - Safety: if _safety_band in (RED, CRISIS) and tool is action -> reject
        Step 6 - Execute: route to appropriate port handler
        Step 7 - Record: append to _results_buffer, increment _call_count

        Returns:
            ToolResult(success, data, error_code?, error_message?)
        """
        ...

    def clear_buffer(self) -> None:
        """Reset per-turn state. Called at turn_start()."""
        ...

    @property
    def active_count(self) -> int: ...
    @property
    def budget_remaining(self) -> int: ...
```

Tool routing table (Step 6 internals):

TOOL CATEGORY	PORT	EXECUTION PATH
Signal: `acknowledge()`	IOutputPort	OutputManager.enqueue(event, REALTIME)
Cognitive (6 tools)	IStatePort	SessionKernelAdapter.write(section, delta) via MutationGuard
Read: `recall_memory()`	IMemoryPort	BridgeRecallAdapter.recall(query)
Read: `discover_capabilities()`	IDispatchPort	DispatchAdapter -> Fabric -> DiscoverCapabilitiesHandler
Read: `summarize_context()`	IStatePort + ILLMPort	ContextAssembler.summarize() (optional LLM call)
Action: `invoke_capability()`	IDispatchPort	DispatchAdapter -> Fabric.execute() (9-step pipeline)
Action: `spawn_via_fabric()`	IDispatchPort	DispatchAdapter -> Fabric -> BuildAgentHandler
Action: `execute_workflow()`	IDispatchPort	DispatchAdapter -> Orchestrator mailbox (async)

Test categories (~80 tests):

CATEGORY	COUNT	WHAT IT TESTS
Allowlist enforcement	~15	Per-tier allowlist, safety band blocking, state-based filtering
Budget enforcement	~10	20-call limit, forced termination, budget remaining
Schema validation	~10	Missing required params, type errors, schema hints
ACK-first rule	~8	First call must be acknowledge(), rejection and recovery
ReAct loop	~15	Multi-iteration, tool chaining, LLM termination conditions
Error recovery	~12	CB open, MutationGuard rejection, timeout, feed errors to LLM
Tool routing	~10	Each of 13 tools dispatched to correct port

16.7 OutputManager (~55 tests)

Manages all user-facing output delivery with priority queuing.

```python
class OutputManager:
    """User-facing output delivery coordinator.

    Owns:
    - Priority queue (REALTIME 60%, INTERACTIVE 30%, BACKGROUND 10%)
    - Queue depth enforcement (CONC-12: <=50 events)
    - Delivery ordering guarantees per priority class
    - ConversationScheduler sub-service (response pacing)

    Dependencies (injected):
    - IOutputPort (SSE delivery)
    - IDeltaPort (publish delivery receipts for observability)
    """

    __slots__ = (
        "_output_port", "_delta_port", "_queue",
        "_depth", "_max_depth", "_conversation_scheduler",
    )

    async def enqueue(self, event: OutputEvent, priority: str = "INTERACTIVE") -> QueuePosition:
        """Add event to priority queue.

        Returns QueuePosition with queue_depth, estimated_delivery_ms.
        Raises QueueFullError if depth > 50 (CONC-12).
        """
        ...

    async def deliver_next(self) -> DeliveryReceipt:
        """Dequeue highest-priority event and deliver via IOutputPort.send().

        WFQ dequeue: REALTIME 60% -> INTERACTIVE 30% -> BACKGROUND 10%.
        """
        ...

    async def flush(self) -> List[DeliveryReceipt]:
        """Deliver all queued events in priority order. Used at turn_end()."""
        ...

    async def stream_progress(self, deltas: AsyncIterator[Delta]) -> StreamReceipt:
        """Convert incoming deltas to OutputEvents and stream to user.

        Used during PROGRESSING state. Each delta -> OutputEvent(progress_update).
        """
        ...

    @property
    def depth(self) -> int: ...
```

Test categories (~55 tests):

CATEGORY	COUNT	WHAT IT TESTS
Priority queueing	~12	WFQ scheduling, REALTIME priority, BACKGROUND demotion
Queue depth	~8	50-event limit, QueueFullError, overflow handling
Delivery	~15	SSE send, stream delivery, retry behavior
Flush	~8	Complete drain, ordering preservation, partial on error
ConversationScheduler	~12	Response pacing, typing indicators, interleaving

16.8 DeltaAggregator (~50 tests)

Batch-collects deltas from K1 Bus (Orchestrator, Planner, sub-agents) in 500ms windows and merges using Last-Writer-Wins (LWW).

```python
class DeltaAggregator:
    """500ms window delta batching with LWW merge (CONC-10).

    Owns:
    - 500ms fixed aggregation window (timer-based)
    - LWW merge for conflicting updates to same section
    - 4 delta types: progress, affective, scoreboard, narrative
    - Flush triggers: timer expiry or explicit flush()

    Dependencies (injected):
    - IDeltaPort (subscribe to incoming deltas)
    - FSMController (signal flush ready for state update)
    """

    __slots__ = (
        "_delta_port", "_fsm", "_window_ms", "_current_batch",
        "_timer", "_merge_strategy",
    )

    async def start_window(self) -> None:
        """Start the 500ms aggregation timer. Called at session init."""
        ...

    async def receive(self, delta: Delta) -> None:
        """Buffer incoming delta. Apply LWW if duplicate section+key.

        LWW merge: for same (section, key), keep the delta with latest timestamp_ms.
        """
        ...

    async def flush(self) -> AggregatedBatch:
        """Merge and return all buffered deltas. Reset window.

        Returns:
            AggregatedBatch with deltas[], merge_conflicts_count, window_duration_ms.
        """
        ...

    def on_error(self, error: BusError) -> None:
        """Handle bus-level error. Log warning, continue (never blocks)."""
        ...
```

Test categories (~50 tests):

CATEGORY	COUNT	WHAT IT TESTS
Window timing	~10	500ms trigger, timer reset, manual flush
LWW merge	~15	Conflicting updates, timestamp ordering, multi-key merge
Delta types	~10	All 4 types processed correctly, unknown types logged
Error handling	~8	Bus errors, malformed deltas, empty windows
Integration	~7	End-to-end: subscribe -> batch -> flush -> OutputManager

16.9 ClarificationTracker (~40 tests)

Tracks clarification rounds per intent and manages pending HIL requests from sub-agents.

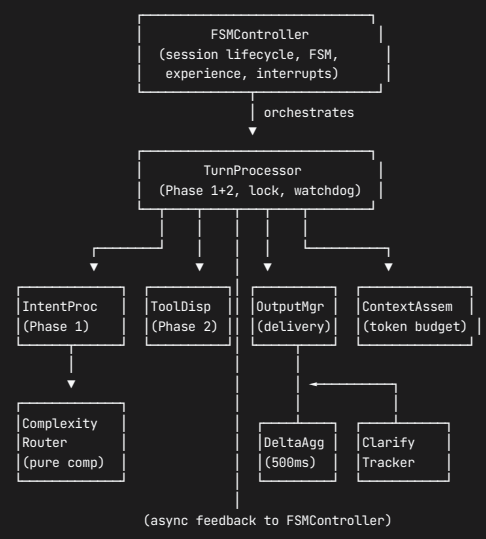
```python
class ClarificationTracker:
    """Clarification round tracking and HIL management (CONC-08).

    Owns:
    - Per-intent round counter (max 3, CONC-08)
    - Pending HIL request queue (from sub-agents via k1.hil.request.v1)
    - Escalation decisions (give up after max rounds)

    No ports: state-only service. Tracks in-memory counters reset per turn.
    """

    __slots__ = ("_round_counts", "_pending_hil", "_max_rounds")

    def track_round(self, intent_id: str) -> int:
        """Increment and return round count for intent. Raises if > max."""
        ...

    def max_reached(self, intent_id: str) -> bool:
        """True if intent has exhausted clarification budget (3 rounds)."""
        ...

    def escalate(self, intent_id: str) -> EscalationDecision:
        """Decide what to do when max rounds reached.

        Options:
        - GIVE_UP: "I'm not sure I understand. Could you rephrase?"
        - BEST_GUESS: proceed with highest-confidence hypothesis
        - DEFER: "Let me know when you're ready to try again"
        """
        ...

    def add_pending(self, request: HILRequest) -> None:
        """Track a pending HIL request from a sub-agent."""
        ...

    def get_pending(self) -> List[HILRequest]:
        """Return all pending HIL requests for SessionState write."""
        ...

    def reset_turn(self) -> None:
        """Clear turn-scoped state. Per-intent counters persist across turns."""
        ...
```

16.10 ContextAssembler (~55 tests)

Builds LLM context according to 6 static profiles, manages token budgets, and triggers summarization.

```python
class ContextAssembler:
    """Token budget management and LLM context building (CONC-22).

    Owns:
    - 6 context profiles (by call_type x tier)
    - Per-turn token tracking (input + output)
    - Summarization trigger at >80% of 128K window
    - Section prioritization (HOT > WARM) for budget allocation

    Dependencies (injected):
    - IStatePort (read SessionState sections)
    - IMemoryPort (prefetch K0 memory)

    Context profiles:
    - (acknowledge, LOW):     150 tokens -- minimal (CONC-19)
    - (response, LOW):        4K tokens  -- beliefs + scoreboard + short history
    - (response, MEDIUM):     8K tokens  -- above + narrative + clarifications
    - (response, HIGH):      16K tokens  -- above + full history + K0 memory
    - (clarification, any):  300 tokens  -- current hypothesis + gaps
    - (preliminary_ack, any): 200 tokens -- intent summary only
    """

    __slots__ = (
        "_state_port", "_memory_port", "_profiles",
        "_token_estimator", "_turn_usage", "_summarization_threshold",
    )

    async def assemble(
        self, call_type: str, tier: str, intent: ProcessedIntent,
        tool_results: Optional[List[ToolResult]] = None,
    ) -> LLMContext:
        """Build complete LLM context within budget.

        Steps:
        1. Select profile by (call_type, tier)
        2. Read required SessionState sections via IStatePort
        3. Estimate tokens: ceil(chars / 3.5) (conservative cross-tokenizer)
        4. Trim lowest-priority sections if over budget
        5. Package as LLMContext with messages[], tools[], constraints

        Returns:
            LLMContext ready for ILLMPort.execute()
        """
        ...

    def track_usage(self, input_tokens: int, output_tokens: int) -> None:
        """Track cumulative token usage for this turn."""
        ...

    def remaining_budget(self) -> int:
        """Return remaining tokens in 128K window."""
        ...

    def needs_summarization(self) -> bool:
        """True if cumulative usage > 80% of 128K (102,400 tokens)."""
        ...

    async def summarize(self, context: LLMContext) -> LLMContext:
        """Compress context via summarization LLM call. Reduces by ~60%."""
        ...

    def reset_budget(self) -> None:
        """Reset per-turn token tracking. Called at turn_start()."""
        ...
```

16.11 Service Dependency Graph (Complete)



No circular dependencies. All arrows point downward or leftward.
DeltaAggregator and ClarificationTracker feed back to FSMController
via async signals (not direct method calls).

17. Concierge State Machine

17.1 State Catalog

STATE	ENTRY ACTION	EXIT ACTION	INTERRUPTIBLE	TYPICAL DURATION	INVARIANT
LISTENING	await_input()	acquire_lock()	No	Indefinite (idle)	CONC-15 (no active turn)
ACKING	run_phase1()	–	No	~28ms	CONC-05 (safety first)
CLARIFYING	emit_question()	–	Yes	User-dependent	CONC-08 (<=3 rounds)
DISPATCHING	dispatch_by_tier()	–	Yes	Tier-dependent	CONC-09 (<=20 tool calls)
COMPANIONING	emit_preliminary_ack()	–	Yes	200ms-2s	–
PROGRESSING	stream_deltas()	–	Yes	500ms-30s	CONC-10 (500ms window)
DELIVERING	emit_final_response()	release_lock()	No	<50ms	CONC-02 (output via channel)
INTERRUPT_HANDLING	cancel_inflight()	–	No	<100ms	CONC-14 (<=1ms transition)

17.2 FSM State Enum and Context

```python
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Optional

class FSMState(Enum):
    """8 states of the Concierge FSM (CONC-17)."""
    LISTENING = auto()
    ACKING = auto()
    CLARIFYING = auto()
    DISPATCHING = auto()
    COMPANIONING = auto()
    PROGRESSING = auto()
    DELIVERING = auto()
    INTERRUPT_HANDLING = auto()


class FSMEvent(Enum):
    """14 events that trigger state transitions."""
    MESSAGE_RECEIVED = auto()      # user input arrives
    PHASE1_COMPLETE = auto()       # UltraBERT + hypothesis done
    GAPS_DETECTED = auto()         # clarification needed
    CLARIFICATION_RECEIVED = auto() # user answered clarification
    MAX_ROUNDS_REACHED = auto()    # 3 rounds exhausted
    DISPATCH_STARTED = auto()      # task dispatched by tier
    PRELIMINARY_ACK_SENT = auto()  # ACK delivered to user
    PROGRESS_RECEIVED = auto()     # delta batch arrived
    DISPATCH_COMPLETE = auto()     # Orchestrator/direct complete
    RESPONSE_DELIVERED = auto()    # final response sent via SSE
    TURN_COMPLETE = auto()         # turn boundary writes done
    INTERRUPT_DETECTED = auto()    # new message during interruptible state
    INTERRUPT_HANDLED = auto()     # cancel done, abbreviated turn_end done
    CRISIS_DETECTED = auto()       # safety override


@dataclass
class FSMContext:
    """Mutable context carried across transitions within a single turn.

    Created fresh at each turn_start(). Accumulated through transitions.
    Released at turn_end() or abbreviated turn_end().
    """
    session_id: str
    turn_sequence: int
    trace_id: str
    lock_version: int
    tier: Optional[str] = None
    safety_band: Optional[str] = None
    phase1_result: Optional["Phase1Result"] = None
    phase2_result: Optional["Phase2Result"] = None
    interrupted: bool = False
    start_time_ms: float = 0.0
    tool_call_count: int = 0
    token_usage: int = 0
    error_count: int = 0
```

17.3 Transition Table (Complete)

All 14 valid transitions. Any (state, event) pair not listed below raises `InvalidTransitionError`.

FROM STATE	EVENT	TO STATE	GUARD CONDITION	ACTION
LISTENING	MESSAGE_RECEIVED	ACKING	turn_lock available	acquire_lock(), start_watchdog(120s), init FSMContext
ACKING	CRISIS_DETECTED	DELIVERING	safety_band == CRISIS	emit_crisis_response(), skip Phase 2 entirely
ACKING	PHASE1_COMPLETE	DISPATCHING	gaps.empty && confidence > threshold	dispatch_by_tier(tier)
ACKING	GAPS_DETECTED	CLARIFYING	gaps.non_empty && round_count < 3	emit_clarification(gaps[0])
CLARIFYING	CLARIFICATION_RECEIVED	ACKING	–	re-run Phase 1 with augmented input
CLARIFYING	MAX_ROUNDS_REACHED	DISPATCHING	rounds >= 3	ClarificationTracker.escalate(), proceed with best_guess
DISPATCHING	PRELIMINARY_ACK_SENT	COMPANIONING	tier in (MEDIUM, HIGH)	emit preliminary via OutputManager(REALTIME)
DISPATCHING	DISPATCH_COMPLETE	DELIVERING	tier == LOW (direct)	package Phase2Result
COMPANIONING	PROGRESS_RECEIVED	PROGRESSING	delta batch available	stream_deltas via DeltaAggregator
COMPANIONING	DISPATCH_COMPLETE	DELIVERING	Orchestrator done	package Phase2Result
PROGRESSING	DISPATCH_COMPLETE	DELIVERING	all dispatches settled	package Phase2Result
DELIVERING	RESPONSE_DELIVERED	LISTENING	–	turn_end() -> release_lock() -> tick_experience()
* (interruptible)	INTERRUPT_DETECTED	INTERRUPT_HANDLING	is_interruptible() == True	cancel_inflight(), abbreviated turn_end()
INTERRUPT_HANDLING	INTERRUPT_HANDLED	ACKING	–	re-enter with new message, new FSMContext

Interruptible states: DISPATCHING, COMPANIONING, PROGRESSING, CLARIFYING. **Non-interruptible states**: LISTENING, ACKING, DELIVERING, INTERRUPT_HANDLING.

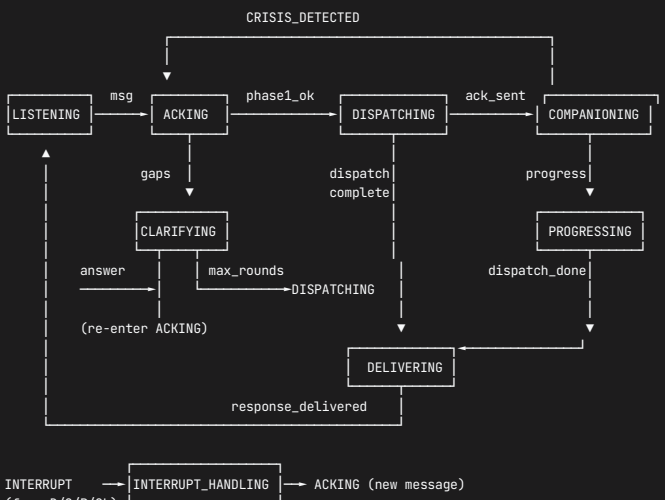
Guard function signatures:

```
def _guard_turn_lock_available(ctx: FSMContext, state: SessionState) -> bool:
    """True if no active turn lock in SessionState control section."""
    return state.control.turn_lock is None

def _guard_gaps_empty(ctx: FSMContext) -> bool:
    """True if Phase 1 found no information gaps."""
    return len(ctx.phase1_result.hypothesis_bundle.gaps) == 0

def _guard_confidence_threshold(ctx: FSMContext) -> bool:
    """True if hypothesis confidence > 0.6 (proceed without clarification)."""
    return ctx.phase1_result.hypothesis_bundle.uncertainty < 0.4

def _guard_is_interruptible(state: FSMState) -> bool:
    """True if state is in the interruptible set."""
    return state in {FSMState.DISPATCHING, FSMState.COMPANIONING,
                     FSMState.PROGRESSING, FSMState.CLARIFYING}

def _guard_max_rounds(tracker: ClarificationTracker, intent_id: str) -> bool:
    """True if 3 clarification rounds exhausted."""
    return tracker.max_reached(intent_id)
```

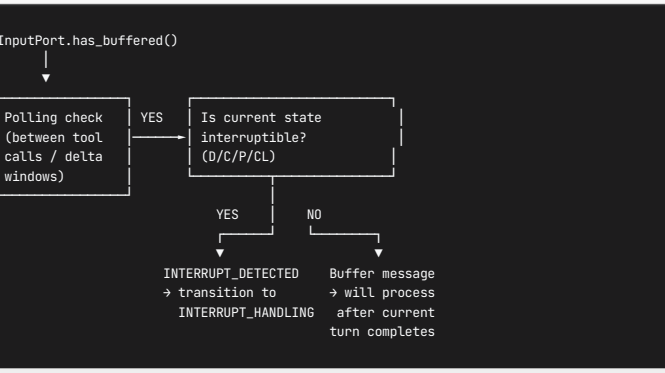
17.4 Transition Validation and Enforcement

```python
# Transition table: Dict[(FSMState, FSMEvent), TransitionSpec]
TransitionSpec = NamedTuple("TransitionSpec", [
    ("target", FSMState),
    ("guard", Optional[Callable[[FSMContext], bool]]),
    ("action", Optional[Callable[["FSMController", FSMContext], Awaitable[None]]]),
])

_TRANSITIONS: Dict[Tuple[FSMState, FSMEvent], TransitionSpec] = {
    (FSMState.LISTENING, FSMEvent.MESSAGE_RECEIVED): TransitionSpec(
        target=FSMState.ACKING,
        guard=_guard_turn_lock_available,
        action=_action_acquire_lock,
    ),
    # ... all 14 entries
}

async def _execute_transition(
    self, event: FSMEvent, ctx: FSMContext
) -> TransitionResult:
    """Core transition engine (CONC-14: <=1ms, no I/O).

    Steps:
    1. Lookup (self._state, event) in _TRANSITIONS
    2. If not found: raise InvalidTransitionError(self._state, event)
    3. If guard and not guard(ctx): raise GuardFailedError
    4. Execute exit_action of current state (pure state mutation)
    5. old_state = self._state; self._state = spec.target
    6. Execute entry_action of new state (pure state mutation)
    7. Emit transition delta via IDeltaPort (fire-and-forget, non-blocking)
    8. Return TransitionResult(old_state, spec.target, elapsed_us)

    INVARIANT: Steps 4-6 are pure. Only step 7 touches I/O (fire-and-forget).
    If step 7 fails, transition still succeeds (delta is observability only).
    """
    ...
```

17.5 FSM Diagram (Full with All Transitions)



17.6 Interrupt Detection Mechanism



Polling frequency: Checked between each tool call in ToolDispatcher.dispatch_tool() and at each DeltaAggregator.flush() boundary. Adds <0.1ms overhead per check.

17.7 Experience Layer Integration

Experience triggers are evaluated AFTER turn_end() completes, BEFORE FSM returns to LISTENING.

```
DELIVERING ──(response_delivered)──► [turn_end()] ──► [tick_experience()] ──► LISTENING

tick_experience(turn_count):
  if turn_count % 20 == 0:
    NarrativeWeaving.weave(history_active)
      -> clusters: NarrativeCluster[]
      -> write via cognitive tool: update_narrative(active)
      -> budget: 200ms

  if turn_count % 25 == 0:
    EmotionalProcessing.process(affect)
      -> trajectory: EmotionalTrajectory
      -> write via cognitive tool: refine_affect()
      -> budget: 150ms

  if turn_count % 30 == 0:
    AnticipatoryResponse.anticipate(beliefs, history)
      -> prefetch: Anticipation{topic, confidence}
      -> prime IMemoryPort.prefetch() for likely recall
      -> budget: 150ms

  Total experience budget: max 500ms combined.
```

17.8 Crash Recovery (Detailed)

Cross-reference: Section 16.2 FSMController.crash_recovery(). Expanded protocol below.

Detection (at session resume):

```
1. Bootstrap: create 9 services + 8 ports (no state needed)
2. Read control section from LOCAL COLD (SQLite)
3. Check: control.turn_lock is not None?
   YES -> crashed during active turn -> proceed to rollback
   NO  -> clean shutdown -> proceed to step 6 (restore only)
```

Rollback (if lock detected):

```
4. Clear turn_lock in control section
5. Increment lock_version (prevents stale writes from ghost processes)
6. Set turn_status = ROLLED_BACK
7. Discard any un-flushed DeltaAggregator batch (from pre-crash state)
8. Drain Local Outbox:
   - Read pending K0 writes from SQLite outbox table
   - Attempt fire-and-forget to Bridge
   - If Bridge offline: leave in outbox for eventual sync
```

Restore:

```
9. Read full HOT tier from LOCAL COLD (all 8 sections)
10. Validate checksums (each section has CRC-32 in meta)
11. If checksum mismatch: rebuild from K0 (full pull, ~500ms)
12. Restore persona (PersonaEngine + style from persona section)
13. Initialize 7 circuit breakers (all CLOSED)
14. Start delta subscriptions + DeltaAggregator timer
```

Resume:

```
15. FSM -> LISTENING
16. Emit k1.concierge.session.recovered.v1 via IDeltaPort
17. Log recovery telemetry: {
      session_id, lock_version, rolled_back_turn, restore_source,
      duration_ms, sections_restored, outbox_drained
    }
```

Crash recovery guarantee: Maximum data loss = 1 turn (the crashed turn). All completed turns are checkpointed in LOCAL COLD. K0 sync may lag by at most 1 turn worth of writes.

18. Error Recovery Paths

18.1 Concierge Error Type System

Modeled on Orchestrator's `ErrorSeverity` / `AdapterError` / `ErrorAction` pattern (see `k1/orchestrator/types.py`), localized for Concierge's 8 ports.

```python
class ConciergeErrorSeverity(Enum):
    """Three severity levels: same semantics as Orchestrator."""
    RECOVERABLE = "recoverable"  # Transient failure. Retry likely fixes it.
    DEGRADED = "degraded"        # Partial failure. Can continue with reduced quality.
    TERMINAL = "terminal"        # Unrecoverable. Turn must abort.


@dataclass(frozen=True)
class ConciergeAdapterError:
    """Immutable error descriptor raised by adapters or services.

    Mirrors k1/orchestrator/types.py AdapterError. Every adapter converts
    port-specific exceptions into this type before propagating upward.
    """
    severity: ConciergeErrorSeverity
    adapter_name: str       # e.g., "UltraBERTAdapter", "ModelGatewayAdapter"
    operation: str          # e.g., "classify", "execute_prompt"
    error_code: str         # e.g., "ERR_CLASSIFICATION_TIMEOUT"
    error_message: str      # human-readable description
    original_exception: Optional[Exception] = None
    fallback_action: Optional[str] = None  # hint for ErrorRouter
    trace_id: Optional[str] = None


class ConciergeErrorAction(Enum):
    """Four recovery actions (same semantics as Orchestrator)."""
    RETRY = "retry"        # Retry same operation (with backoff)
    FALLBACK = "fallback"  # Use fallback value / degraded path
    DEGRADE = "degrade"    # Continue with reduced capabilities
    ABORT = "abort"        # Fail the turn, emit error response


@dataclass(frozen=True)
class ErrorDecision:
    """Output of ConciergeErrorRouter.route_error()."""
    action: ConciergeErrorAction
    retry_count: int = 0
    retry_delay_ms: int = 0
    fallback_value: Optional[Any] = None
    reason: str = ""
    user_message: Optional[str] = None  # user-facing error text (if ABORT)
```

18.2 Error Catalog (Comprehensive)

ERROR ID	SOURCE ADAPTER	PORT	SEVERITY	RECOVERY ACTION	FALLBACK VALUE	USER IMPACT
ERR_CLASSIFICATION_TIMEOUT	UltraBERTAdapter	IClassificationPort	RECOVERABLE	RETRY(1, 50ms)	–	None (retry transparent)
ERR_CLASSIFICATION_FAIL	UltraBERTAdapter	IClassificationPort	DEGRADED	FALLBACK	HeuristicFallbackClassifier result	Minor: heuristic less accurate
ERR_CLASSIFICATION_HEURISTIC_FAIL	UltraBERTAdapter	IClassificationPort	TERMINAL	ABORT	–	Severe: turn fails
ERR_LLM_TIMEOUT	ModelGatewayAdapter	ILLMPort	RECOVERABLE	RETRY(2, 100ms)	–	None (retry transparent)
ERR_LLM_BUDGET_EXCEEDED	ModelGatewayAdapter	ILLMPort	DEGRADED	FALLBACK	Template response by tier	Moderate: canned response
ERR_LLM_CASCADE_EXHAUSTED	ModelGatewayAdapter	ILLMPort	TERMINAL	ABORT	–	Severe: turn fails
ERR_STATE_WRITE_REJECTED	SessionKernelAdapter	IStatePort	RECOVERABLE	RETRY(1, 0ms)	–	None (version bump + retry)
ERR_STATE_WRITE_FAIL	SessionKernelAdapter	IStatePort	DEGRADED	DEGRADE	Stale snapshot (skip write)	Minor: stale state until next turn
ERR_STATE_READ_FAIL	SessionKernelAdapter	IStatePort	DEGRADED	FALLBACK	Empty snapshot	Moderate: no history context
ERR_DISPATCH_TIMEOUT	FabricOrchestratorAdapter	IDispatchPort	RECOVERABLE	RETRY(1, 200ms)	–	Minor: delayed response
ERR_DISPATCH_FAIL	FabricOrchestratorAdapter	IDispatchPort	DEGRADED	DEGRADE	Tier downgrade to LOW	Moderate: reduced capability
ERR_DISPATCH_UNREACHABLE	FabricOrchestratorAdapter	IDispatchPort	TERMINAL	ABORT	–	Severe: turn fails
ERR_DELTA_TIMEOUT	DeltaBusAdapter	IDeltaPort	RECOVERABLE	RETRY(1, 100ms)	–	None
ERR_DELTA_PUBLISH_FAIL	DeltaBusAdapter	IDeltaPort	DEGRADED	DEGRADE	Skip delta (fire-and-forget)	None: delta is observability
ERR_MEMORY_OFFLINE	BridgeRecallAdapter	IMemoryPort	DEGRADED	FALLBACK	Empty RecallResponse	Minor: no K0 memory
ERR_MEMORY_TIMEOUT	BridgeRecallAdapter	IMemoryPort	RECOVERABLE	RETRY(1, 100ms)	–	None
ERR_OUTPUT_SEND_FAIL	SSEOutputAdapter	IOutputPort	RECOVERABLE	RETRY(2, 50ms)	–	Minor: delayed delivery
ERR_OUTPUT_DISCONNECT	SSEOutputAdapter	IOutputPort	TERMINAL	ABORT	–	Severe: client gone
ERR_INPUT_PARSE_FAIL	WebSocketInputAdapter	IInputPort	DEGRADED	FALLBACK	Reject malformed, await next	None: bad frame dropped

18.3 ConciergeErrorRouter

Stateless classification service modeled on `k1/orchestrator/orchestration/error_router.py`. Takes a `ConciergeAdapterError` and returns an `ErrorDecision`.

```python
class ConciergeErrorRouter:
    """Stateless error classification and recovery routing.

    Mirrors Orchestrator ErrorRouter pattern. The router is pure:
    no I/O, no state, no async. Classification is driven by
    a priority-ordered matrix: (adapter_name, severity) -> ErrorDecision.

    Dependencies: None (stateless, no ports).
    Lifecycle: Created once at session init, never changes.
    """

    # Classification matrix: (adapter_name, severity) -> ErrorDecision
    _MATRIX: ClassVar[Dict[Tuple[str, ConciergeErrorSeverity], ErrorDecision]] = {
        # -- IClassificationPort (UltraBERT) --
        ("UltraBERTAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=1, retry_delay_ms=50,
            reason="Transient UltraBERT timeout, single retry"
        ),
        ("UltraBERTAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.FALLBACK,
            fallback_value="HeuristicFallbackClassifier",
            reason="UltraBERT offline, use 5-rule heuristic"
        ),
        ("UltraBERTAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.ABORT,
            reason="Both UltraBERT and heuristic failed",
            user_message="I'm having trouble understanding right now. Could you try again?"
        ),

        # -- ILLMPort (ModelGateway) --
        ("ModelGatewayAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=2, retry_delay_ms=100,
            reason="LLM timeout, retry with same tier"
        ),
        ("ModelGatewayAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.FALLBACK,
```

```
            fallback_value="TemplateResponse",
            reason="Budget exceeded or cascade exhausted, use template"
        ),
        ("ModelGatewayAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.ABORT,
            reason="All LLM tiers exhausted",
            user_message="I can't generate a response right now. Please try again in a moment."
        ),

        # -- IStatePort (SessionKernel) --
        ("SessionKernelAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=1, retry_delay_ms=0,
            reason="MutationGuard version conflict, bump and retry"
        ),
        ("SessionKernelAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.DEGRADE,
            fallback_value="StaleSnapshot",
            reason="State write failed, continue with stale read"
        ),
        ("SessionKernelAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.ABORT,
            reason="State completely unavailable",
            user_message="I'm having a memory issue. Let me restart our conversation."
        ),

        # -- IDispatchPort (FabricOrchestrator) --
        ("FabricOrchestratorAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=1, retry_delay_ms=200,
            reason="Dispatch timeout, single retry"
        ),
        ("FabricOrchestratorAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.DEGRADE,
            fallback_value="TierDowngrade",
            reason="Fabric degraded, downgrade MEDIUM/HIGH to LOW"
        ),
        ("FabricOrchestratorAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.ABORT,
            reason="Fabric unreachable after retry",
            user_message="I can't complete that task right now. Could you try something simpler?"
        ),

        # -- IDeltaPort (DeltaBus) --
        ("DeltaBusAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=1, retry_delay_ms=100,
            reason="Delta bus transient, single retry"
        ),
        ("DeltaBusAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.DEGRADE,
            reason="Delta publish failed, skip (fire-and-forget)"
        ),
        ("DeltaBusAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.DEGRADE,
            reason="Delta bus down, skip all deltas (observability loss only)"
        ),

        # -- IMemoryPort (BridgeRecall) --
        ("BridgeRecallAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=1, retry_delay_ms=100,
            reason="K0 Bridge timeout, single retry"
        ),
        ("BridgeRecallAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.FALLBACK,
            fallback_value="EmptyRecallResponse",
            reason="Bridge offline, proceed without long-term memory"
        ),
        ("BridgeRecallAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.FALLBACK,
            fallback_value="EmptyRecallResponse",
            reason="Bridge completely unreachable, same as DEGRADED for memory"
        ),

        # -- IOutputPort (SSEOutput) --
        ("SSEOutputAdapter", ConciergeErrorSeverity.RECOVERABLE): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=2, retry_delay_ms=50,
            reason="SSE send failed, retry delivery"
        ),
        ("SSEOutputAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.RETRY, retry_count=3, retry_delay_ms=100,
            reason="SSE connection flaky, aggressive retry"
        ),
        ("SSEOutputAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.ABORT,
            reason="Client disconnected, no delivery target",
            user_message=None  # no user to deliver to
        ),

        # -- IInputPort (WebSocketInput) --
        ("WebSocketInputAdapter", ConciergeErrorSeverity.DEGRADED): ErrorDecision(
            action=ConciergeErrorAction.FALLBACK,
            reason="Malformed input frame, drop and continue"
        ),
        ("WebSocketInputAdapter", ConciergeErrorSeverity.TERMINAL): ErrorDecision(
            action=ConciergeErrorAction.ABORT,
            reason="WebSocket connection lost"
        ),
    }

    def classify(self, error: ConciergeAdapterError) -> ErrorDecision:
        """Synchronous classification. No side effects, no delta emission.

        Lookup priority:
        1. (adapter_name, severity) exact match in _MATRIX
        2. Fallback by severity:
           - RECOVERABLE -> RETRY(1)
           - DEGRADED -> DEGRADE
           - TERMINAL -> ABORT
        """
        key = (error.adapter_name, error.severity)
```

```
        if key in self._MATRIX:
            return self._MATRIX[key]
        # Fallback by severity
        return self._severity_fallback(error.severity)

    async def route_error(
        self, error: ConciergeAdapterError, delta_port: IDeltaPort
    ) -> ErrorDecision:
        """Classify AND emit diagnostic delta.

        Emits k1.concierge.error.routed.v1:
        {
            adapter: error.adapter_name,
            operation: error.operation,
            severity: error.severity.value,
            action: decision.action.value,
            trace_id: error.trace_id,
        }
        """
        decision = self.classify(error)
        await delta_port.publish(
            topic="k1.concierge.error.routed.v1",
            payload=_build_error_delta(error, decision),
        )
        return decision

    @staticmethod
    def _severity_fallback(severity: ConciergeErrorSeverity) -> ErrorDecision:
        if severity == ConciergeErrorSeverity.RECOVERABLE:
            return ErrorDecision(action=ConciergeErrorAction.RETRY,
                                 retry_count=1, reason="generic recoverable")
        elif severity == ConciergeErrorSeverity.DEGRADED:
            return ErrorDecision(action=ConciergeErrorAction.DEGRADE,
                                 reason="generic degraded")
        else:
            return ErrorDecision(action=ConciergeErrorAction.ABORT,
                                 reason="generic terminal",
                                 user_message="Something went wrong. Please try again.")
```

18.4 Global Error Decision Tree (Expanded)

```
Error Detected (ConciergeAdapterError)
    |
    ├── 1. Is Safety-Critical? (safety_band == CRISIS)
    |       |
    |       ├── YES → CRISIS Protocol (Section 5.5):
    |       |         - Bypass ErrorRouter entirely
    |       |         - Emit CRISIS static response immediately
    |       |         - Log to safety audit trail
    |       |         - Notify family safety contacts (if configured)
    |       |
    |       └── NO → Continue to step 2
    |
    ├── 2. ErrorRouter.classify(error) -> ErrorDecision
    |       |
    |       ├── RETRY:
    |       |    - Check retry budget (max per-adapter, see matrix)
    |       |    - If retries remaining:
    |       |         Wait retry_delay_ms (exponential backoff on subsequent)
    |       |         Re-execute operation
    |       |         If success: continue turn (transparent to user)
    |       |         If fail again: re-classify with elevated severity
    |       |    - If budget exhausted: escalate to DEGRADED severity
    |       |
    |       ├── FALLBACK:
    |       |    - Use fallback_value from ErrorDecision
    |       |    - Continue turn with degraded input
    |       |    - Log degradation for observability
    |       |    - Update context.error_count++
    |       |
    |       ├── DEGRADE:
    |       |    - Reduce capability level:
    |       |       IDispatchPort DEGRADE -> downgrade tier to LOW
    |       |       IStatePort DEGRADE -> skip write, use stale snapshot
    |       |       IDeltaPort DEGRADE -> skip delta (fire-and-forget)
    |       |    - Continue turn with reduced capability
    |       |    - User MAY see reduced quality (not an error to user)
    |       |
    |       └── ABORT:
    |            - Emit user-facing error message via IOutputPort
    |            - Flush partial results if any (OutputManager.flush)
    |            - Abbreviated turn_end (Section 16.3)
    |            - FSM -> DELIVERING -> LISTENING
    |            - Log turn failure with full error context
    |
    └── 3. Always: emit k1.concierge.error.routed.v1 diagnostic delta
```

18.5 Per-Port Error Recovery Pipelines

18.5.1 IClassificationPort (UltraBERT) Recovery

```
UltraBERT.classify() fails
    |
    ├── TimeoutError (< 40ms)
    |    → RETRY once (50ms delay)
    |    → If still fails: open CB_CLASSIFICATION
    |
    ├── CB_CLASSIFICATION OPEN
    |    → Route to HeuristicFallbackClassifier (5-rule table)
    |    → Heuristic returns ClassificationResult with confidence ~0.5
    |    → Continue Phase 1 (degraded accuracy, user unaware)
    |
    └── Heuristic also fails (extremely rare)
         → ABORT turn
         → "I'm having trouble understanding. Could you rephrase?"
```

18.5.2 ILLMPort (ModelGateway) Recovery

```
LLM.execute_prompt() fails
    |
    ├── Timeout (< tier budget)
    |    → RETRY twice (100ms, 200ms exponential)
    |    → If still fails: cascade to next model tier
    |
    ├── BudgetExceeded
    |    → FALLBACK to template response
    |    → Templates keyed by (intent_class, tier):
    |         TASK+LOW: "I'll help you with that. Let me look into it."
    |         EMOTIONAL+any: "I hear you. That sounds really tough."
    |         QUESTION+any: "That's a great question. Let me find out."
    |
    ├── L1 -> L2 -> L3 cascade exhausted
    |    → ABORT turn
    |    → "I can't generate a response right now. Please try again."
    |
    └── Token limit (128K context exceeded)
         → ContextAssembler.summarize() -> reduce context by 60%
         → Retry with summarized context
         → If still over: trim to essential sections only
```

18.5.3 IStatePort (SessionKernel) Recovery

```
SessionKernel.write(section, delta) fails
    |
    ├── MutationGuard LOCK_CONFLICT
    |    → Bump lock_version, RETRY immediately (0ms delay)
    |    → Single retry only (should always succeed for single writer)
    |
    ├── MutationGuard SIZE_EXCEEDED (section > budget)
    |    → Trigger summarization: compress history_active
    |    → Retry write with summarized content
    |    → If still over: DEGRADE (skip write, stale until next turn)
    |
    ├── MutationGuard INVALID_SECTION
    |    → Bug in Concierge code. Log ERROR, DEGRADE (skip write).
    |    → This should never happen in production.
    |
    └── SessionKernel.read() fails
         → FALLBACK to empty snapshot
         → Phase 1: reduced accuracy (no history context)
         → Phase 2: LLM has no prior conversation (stateless response)
         → User sees coherent but context-free response
```

18.5.4 IDispatchPort (FabricOrchestrator) Recovery – Tier Degradation Cascade

```
Fabric.execute(envelope) fails
    |
    ├── First failure (RECOVERABLE)
    |    → RETRY once (200ms delay)
    |
    ├── Retry fails (escalate to DEGRADED)
    |    → Tier degradation cascade:
    |
    |    HIGH -> MEDIUM:
    |         Re-package envelope: drop DAG planner, keep 2-capability limit
    |         Re-dispatch to Orchestrator (bypass Planner)
    |         If success: continue with MEDIUM quality
    |
    |    MEDIUM -> LOW:
    |         Cancel envelope entirely
    |         Fall back to ToolDispatcher.run_react_loop() (direct tools)
    |         If success: continue with LOW quality
    |
    |    LOW -> ABORT:
    |         All tiers exhausted
    |         → "I can't complete that task right now."
    |
    └── CB_FABRIC OPEN (Fabric completely down)
         → All MEDIUM/HIGH requests -> LOW (direct ToolDispatcher)
         → All LOW requests -> template response
         → CB half-open probe every 30s
```

18.5.5 IDeltaPort (DeltaBus) Recovery

```
DeltaBus.publish() fails
    │
    ├── Any error
    │    → Log warning, continue (fire-and-forget, NEVER blocks)
    │    → Delta loss is acceptable (observability only)
    │    → No retry needed for most delta types
    │
    └── Bus completely down
         → DeltaAggregator.on_error() logs, continues
         → Progress streaming degraded (user gets final response only)
         → All diagnostic deltas lost (acceptable)
```

18.5.6 IMemoryPort (BridgeRecall) Recovery

```
Bridge.recall(query) fails
    │
    ├── Timeout (< 200ms)
    │    → RETRY once (100ms delay)
    │
    ├── Bridge offline (is_available() == False)
    │    → FALLBACK: return empty RecallResponse immediately
    │    → Concierge proceeds without long-term memory
    │    → User gets coherent response but no K0 context
    │
    └── Bridge error (malformed response)
         → FALLBACK: return empty RecallResponse
         → Log ERROR for investigation
```

18.5.7 IOutputPort (SSEOutput) Recovery

```
SSE.send(event) fails
    │
    ├── Transient send failure
    │    → RETRY twice (50ms, 100ms)
    │    → If success: delivery confirmed
    │
    ├── Connection flaky (repeated failures)
    │    → RETRY 3 times (100ms, 200ms, 400ms)
    │    → If success: continue but mark connection degraded
    │
    └── Client disconnected (WebSocket/SSE closed)
         → ABORT: no delivery target
         → Log session termination
         → Turn_end abbreviated (no output to deliver)
         → Trigger session cleanup
```

18.6 Retry Budget and Backoff Strategy

ADAPTER	MAX RETRIES PER TURN	BACKOFF STRATEGY	MAX TOTAL RETRY TIME
UltraBERTAdapter	1	Fixed 50ms	50ms
ModelGatewayAdapter	2	Exponential (100ms, 200ms)	300ms
SessionKernelAdapter	1	Immediate (0ms)	0ms
FabricOrchestratorAdapter	1	Fixed 200ms	200ms
DeltaBusAdapter	0	None (fire-and-forget)	0ms
BridgeRecallAdapter	1	Fixed 100ms	100ms
SSEOutputAdapter	2-3	Exponential (50ms base)	150-700ms
WebSocketInputAdapter	0	None (drop malformed)	0ms

Global retry budget: Maximum 3 retries across ALL adapters per turn. If total retries > 3, subsequent errors escalate directly to DEGRADE or ABORT without retry. This prevents retry storms from consuming the 120s watchdog budget.

18.7 Degradation Severity Levels

LEVEL	TRIGGER	ADAPTERS AFFECTED	USER EXPERIENCE	TELEMETRY
NONE	Retry succeeds transparently	Any (after RETRY)	No visible impact	error_count++, recovered=true
MINOR	Single adapter degraded	UltraBERT (heuristic), Bridge (no memory), Delta (no progress)	Slightly less accurate or no progress streaming	degradation_level=MINOR
MODERATE	Tier downgrade or template response	FabricOrchestrator (tier cascade), ModelGateway (template)	Reduced capability or canned response	degradation_level=MODERATE
SEVERE	Turn aborted	Any TERMINAL error	User sees error message, must rephrase/retry	degradation_level=SEVERE, turn_failed=true

18.8 Turn Failure Recovery

When a turn ends in ABORT:

```
1. ErrorRouter.route_error() returns ABORT
2. OutputManager.enqueue(user_message, REALTIME) -- error message to user
3. OutputManager.flush() -- deliver immediately
4. Abbreviated turn_end():
   a. Release Single Writer lock
   b. Checkpoint LOCAL COLD (includes the error state)
   c. Skip: history append, K0 sync, experience triggers
5. FSM -> DELIVERING -> LISTENING
6. Emit k1.concierge.turn.failed.v1 via IDeltaPort:
   {
     session_id, turn_sequence, error_code, adapter_name,
     severity, action, retries_attempted, degradation_level,
     trace_id, duration_ms
   }
7. Ready for next user message (clean slate)
```

Partial result handling: If ABORT occurs AFTER some tool results have been obtained (mid-ReAct loop):

```
1. ToolDispatcher._results_buffer has partial results
2. If buffer.length > 0 AND any result is useful:
   -> Attempt final LLM call with partial context
   -> Deliver "partial" response: "Here's what I found so far..."
   -> Mark turn as PARTIAL in telemetry
3. If buffer empty or all results are errors:
   -> Deliver error message only
   -> Mark turn as FAILED in telemetry
```

18.9 Safety-Critical Error Handling

Errors during CRISIS handling have a completely separate path that bypasses the ErrorRouter:

```
CRISIS detected (SafetyGate)
    |
    ├── Normal CRISIS path (Section 5.5):
    |    Static response emitted immediately (no LLM, no tools)
    |    If IOutputPort fails during CRISIS:
    |         → Retry aggressively (5 attempts, 50ms each)
    |         → If still fails: escalate to system-level alert
    |         → Log CRITICAL: safety response undeliverable
    |
    └── Error DURING CRISIS handling:
        → Never suppress the safety response
        → If delta fails: ignore (safety > observability)
        → If state write fails: ignore (safety > state persistence)
        → If output completely unavailable: system health alert
        → CRISIS response delivery is the HIGHEST priority operation
```

19. Circuit Breakers & Fault Tolerance

19.1 Circuit Breaker Pattern (from K1 Fabric)

Concierge reuses the `CircuitBreaker` class from `k1/fabric/circuit_breaker/breaker.py`. Each CB instance wraps one external dependency via its production adapter. The pattern:

- **3-state machine**: CLOSED (normal) -> OPEN (failing, reject all) -> HALF_OPEN (single probe) -> CLOSED
- **Sliding window failure tracking**: Failures within `failure_window_ms` are counted. When count >= `failure_threshold`, the breaker opens.
- **Half-open probe**: After `half_open_after_ms`, breaker transitions to HALF_OPEN and allows exactly one request. If it succeeds -> CLOSED. If it fails -> OPEN again.
- **Retry strategy**: Up to `max_retries` per call (attempt 1 + retries). Non-retriable failures bypass retry.
- **Thread safety**: `threading.Lock` protects all mutable state (safe for async event loop + background health checker).
- **State change callback**: `IStateChangeListener.on_state_change()` fires on every transition (wired to telemetry + ErrorRouter).
- **Metrics**: Each state change records `set_circuit_breaker_state()` gauge and `inc_circuit_breaker_trips()` counter.

```python
# From k1/fabric/circuit_breaker/breaker.py (production code)
class CircuitBreaker:
    __slots__ = (
        "_provider_id", "_config", "_on_state_change", "_lock",
        "_state", "_failures", "_opened_at_ms",
        "_half_open_permit", "_consecutive_successes",
    )

    async def call(self, execute_fn, request, context, trace_id) -> CapabilityResult:
        """Execute through CB with retry. Never raises to caller."""
        ...

    def reset(self) -> None:
        """Force CLOSED (from HealthChecker on recovery)."""
        ...

    def trip(self) -> None:
        """Force OPEN (from HealthChecker on failure)."""
        ...

    def allow_probe(self) -> None:
        """OPEN -> HALF_OPEN (single probe permit)."""
        ...
```

19.2 CB State Machine (Detailed)

Transition rules (from `breaker.py`):

FROM	CONDITION	TO	ACTION
CLOSED	`len(failures) >= failure_threshold` (in window)	OPEN	Record `_opened_at_ms`, fire state change callback
OPEN	`elapsed >= half_open_after_ms` (on next request)	HALF_OPEN	Allow request as probe, consume permit
OPEN	`allow_probe()` called by HealthChecker	HALF_OPEN	Set `_half_open_permit = True`
HALF_OPEN	Probe succeeds	CLOSED	Clear failures, reset counters
HALF_OPEN	Probe fails	OPEN	Re-record `_opened_at_ms`
Any	`reset()` called	CLOSED	Clear all state
Any	`trip()` called	OPEN	Force-open

19.3 Concierge Circuit Breaker Inventory (7 CBs)

Each CB is owned by the production adapter that wraps the external dependency. Initialized at `init()` (lifecycle step 5), all start CLOSED.

CB NAME	OWNER ADAPTER	TIMEOUT	FAILURE THRESHOLD	WINDOW	HALF-OPEN AFTER	MAX RETRIES	FALLBACK BEHAVIOR
CB_CLASSIFICATION	UltraBERTAdapter	40ms	3 / 60s	60s	10s	1	HeuristicFallbackClassifier (5-rule table, <1ms)
CB_MODEL	ModelGatewayAdapter	30s (per-provider)	5 / 60s	60s	30s	2	LLM cascade: L1 retry -> L2 half-open probe -> L3 canned response
CB_SESSIONSTATE	SessionKernelAdapter	100ms	5 / 60s	60s	10s	1	Stale read (skip write, use cached snapshot)
CB_ORCHESTRATOR	FabricOrchestratorAdapter	60s	3 / 60s	60s	60s	1	Tier degradation: HIGH->MEDIUM->LOW->canned
CB_PLANNER	FabricOrchestratorAdapter	45s	2 / 60s	60s	45s	1	Skip planning, route MEDIUM->LOW direct
CB_FABRIC	FabricOrchestratorAdapter	30s	3 / 60s	60s	30s	1	Capability unavailable, LOW tier only
CB_SSE	SSEOutputAdapter	5s (reconnect)	5 / 60s	60s	5s	3	Buffer up to 5 messages, flush on reconnect; dead-letter after

CB ownership note: `FabricOrchestratorAdapter` owns 3 CBs (CB_ORCHESTRATOR, CB_PLANNER, CB_FABRIC) because it routes through Orchestrator, which talks to Planner, which talks to Fabric. Each layer has its own failure domain.

19.4 Per-CB Fallback Behavior (Detailed)

19.4.1 CB_CLASSIFICATION OPEN

```
UltraBERT unreachable
    |
    ├── Route to HeuristicFallbackClassifier (Section 15.2.3)
    |    Rule 1: CRISIS keywords scan (hardcoded, NON-NEGOTIABLE)
    |    Rule 2: Question + <20 words -> conversational/LOW
    |    Rule 3: remind/schedule/set -> task/MEDIUM
    |    Rule 4: plan/help me with -> planning/HIGH
    |    Rule 5: Default -> conversational/LOW
    |
    ├── Heuristic confidence: ~0.5 (vs UltraBERT ~0.85+)
    ├── Phase 1 continues normally with degraded classification
    ├── User impact: slightly less accurate routing, unaware of degradation
    |
    └── Half-open probe: every 10s, send one real classification
       If success: CB closes, resume UltraBERT
       If fail: stay OPEN, continue heuristic
```

19.4.2 CB_MODEL OPEN

```
LLM cascade (L1 -> L2 -> L3):
    L1: RETRY (timeout / 5xx, retry once within same provider)
    L2: CB HALF-OPEN probe (30s window, try next capable provider)
    L3: CANNED_RESPONSE (CB fully OPEN)
        |
        ├── Template keyed by (intent_class, tier):
        |    TASK+LOW: "I'll help you with that. Let me look into it."
        |    EMOTIONAL+any: "I hear you. That sounds really tough."
        |    QUESTION+any: "That's a great question. Let me find out."
        |    CONVERSATIONAL+any: "I'm here with you."
        |    Default: "Give me a moment to work on that."
        |
        └── User impact: MODERATE (canned response, no tool execution)
```

19.4.3 CB_ORCHESTRATOR OPEN

```
Orchestrator unreachable
    |
    ├── MEDIUM tier requests:
    |    Cancel envelope, fall back to ToolDispatcher direct (LOW)
    |    User sees: lower quality but functional response
    |
    ├── HIGH tier requests:
    |    Cancel envelope, fall back to MEDIUM first
    |    If MEDIUM also fails: fall back to LOW
    |    User sees: "I'll handle this directly for now."
    |
    ├── LOW tier requests: unaffected (already direct via ToolDispatcher)
    |
    └── Half-open probe: every 60s, route one MEDIUM request normally
       If success: CB closes, resume full tier routing
```

19.4.4 CB_SSE OPEN

```
SSE connection lost
    │
    ├── Buffer up to 5 messages in OutputManager dead-letter queue
    ├── Each buffered message has 60s TTL
    ├── On SSE reconnect: flush dead-letter queue in order
    │
    ├── If > 5 messages while OPEN:
    │    Oldest messages dropped (FIFO eviction)
    │    Lost messages logged to telemetry
    │
    └── If client never reconnects within 120s:
         Session cleanup initiated
         Buffered messages discarded
```

19.5 Circuit Breaker Registry

```python
class CircuitBreakerRegistry:
    """Session-scoped registry of all 7 circuit breakers.

    Created at init(), destroyed at shutdown().
    Provides aggregate health view and admin API.
    """

    __slots__ = ("_breakers", "_state_listener")

    def __init__(self, state_listener: IStateChangeListener) -> None:
        self._breakers: Dict[str, CircuitBreaker] = {}
        self._state_listener = state_listener

    def register(self, name: str, config: CircuitBreakerConfig) -> CircuitBreaker:
        """Create and register a CB. Called 7 times at init()."""
        cb = CircuitBreaker(name, config, on_state_change=self._state_listener)
        self._breakers[name] = cb
        return cb

    def get(self, name: str) -> CircuitBreaker:
        """Retrieve CB by name. Raises KeyError if not registered."""
        return self._breakers[name]

    def health_summary(self) -> Dict[str, CircuitBreakerState]:
        """Return {cb_name: state} for all 7 CBs. Used by admin API."""
        return {name: cb.state for name, cb in self._breakers.items()}

    def all_closed(self) -> bool:
        """True if every CB is CLOSED (healthy system)."""
        return all(cb.state == CircuitBreakerState.CLOSED
                   for cb in self._breakers.values())

    def reset_all(self) -> None:
        """Force all CBs to CLOSED. Admin override only."""
        for cb in self._breakers.values():
            cb.reset()
```

19.6 Interaction with ErrorRouter

The ErrorRouter (Section 18.3) and CircuitBreakerRegistry work in tandem:

```
Adapter error occurs
    │
    ├── Adapter maps to ConciergeAdapterError(severity)
    │
    ├── ErrorRouter.classify(error) -> ErrorDecision
    │    Uses (adapter_name, severity) matrix
    │
    ├── If RETRY and retry succeeds: transparent (CB stays CLOSED)
    │
    ├── If RETRY fails: CB records failure in sliding window
    │    If window threshold reached: CB transitions CLOSED -> OPEN
    │    ErrorRouter now returns FALLBACK or DEGRADE for that adapter
    │
    └── While CB is OPEN:
         All calls to that adapter routed to fallback path
         ErrorRouter skips RETRY (no point retrying an open CB)
         Half-open probe handles eventual recovery
```

20. Performance Targets

20.1 System-Owned Latencies (P99)

These are latencies for code we write and control. Excludes external LLM provider latency (BYOLLM = we own the rails, not the train).

OPERATION	P50	P95	P99	HARD MAX	INVARIANT
UltraBERT classification	18ms	22ms	25ms	40ms (CB timeout)	–
Intent ack SSE delivery	30ms	40ms	50ms	100ms	–
FSM state transition	0.5ms	0.8ms	1ms	1ms	CONC-14
MutationGuard preflight	0.1ms	0.15ms	0.2ms	0.5ms	–
Phase 1 write (deterministic)	0.01ms	0.015ms	0.02ms	0.1ms	–
Phase 2 write (post-LLM)	0.05ms	0.08ms	0.1ms	0.5ms	–
Complexity routing decision	0.1ms	0.15ms	0.2ms	0.5ms	–
Output queue to SSE delivery	2ms	4ms	5ms	10ms	–
Orchestrator envelope dispatch	1ms	1.5ms	2ms	5ms	–
Fabric direct dispatch	1ms	1.5ms	2ms	5ms	–
LOCAL COLD checkpoint (SQLite)	0.3ms	0.7ms	1ms	2ms	–
Delta aggregation window	500ms	500ms	500ms	500ms	CONC-10
Total system overhead/turn	**~25ms**	**~30ms**	**~32ms**	**~45ms**	–

20.2 Budget Envelopes (LLM Provider Rails)

Timeouts imposed on external LLM calls. We do not control provider latency, but we enforce budgets.

ENVELOPE	TIMEOUT	CB TRIPS AT	NOTES
Single LLM call	30s	5 failures / 60s	Per-provider via ModelGateway
LOW tier total turn	2s	–	Phase 1 (28ms) + single LLM call + tools
MEDIUM tier total turn	10s	–	Phase 1 + envelope dispatch + Orchestrator
HIGH tier total turn	45s	–	15s headroom before CB_ORCHESTRATOR 60s
CRISIS tier total	5s	–	Static response, no LLM call
Planner stage timeout	10s / stage	2 failures / 60s	Per DAG stage
Watchdog (absolute max)	120s	–	CONC-11: force-kill entire turn

20.3 Token Rails (max_tokens per Call Type)

From concierge.mmd PERF_TOKENS. Enforced by ContextAssembler (Section 16.10).

CALL TYPE	MAX TOKENS	INVARIANT	ENFORCED BY
LOW response	500	–	ContextAssembler profile
MEDIUM response	2,000	–	ContextAssembler profile
HIGH response	8,000	–	ContextAssembler profile
Intent ack (acknowledge_request)	150	CONC-19	ContextAssembler profile
Preliminary ack (MEDIUM/HIGH)	200	–	ContextAssembler profile
Clarification question	300	–	ContextAssembler profile
Context window per call	128,000	CONC-22	ContextAssembler.needs_summarization()

20.4 Resource Budgets

RESOURCE	BUDGET	INVARIANT	ENFORCED BY
HOT tier memory (8 sections)	48KB	ADR-0017	SessionKernelAdapter.write()
WARM tier memory (4 sections)	48KB	ADR-0017	SessionKernelAdapter.write()
Total SessionState	96KB	ADR-0017	MutationGuard.preflight() SIZE check
Tool calls per turn	20 (hard cap)	CONC-09	ToolDispatcher budget counter
Clarification rounds per intent	3	CONC-08	ClarificationTracker.max_reached()
Output queue depth	50 events	CONC-12	OutputManager.enqueue() QueueFullError
LLM calls per turn	Tier-dependent (LOW: 3, MED: 5, HIGH: 10)	–	ContextAssembler.track_usage()
Concurrent turns per session	1	CONC-15	TurnProcessor.turn_start() lock
Dead-letter buffer per session	10 messages	–	OutputManager dead-letter queue
Experience budget per trigger	500ms combined	–	FSMController.tick_experience()
Delta batch per window	No hard limit (LWW merge reduces)	CONC-10	DeltaAggregator._current_batch

20.5 End-to-End Latency Breakdown by Tier

```
LOW Tier (~800ms P50):
  Phase 1:  28ms  [UltraBERT 22ms + pipeline 3ms + Write Elision Gate 3ms]
  Phase 2: 700ms  [LLM call ~600ms + tool dispatch ~100ms]
  Boundary: 70ms  [history write + checkpoint + delta + output delivery]

MEDIUM Tier (~3s P50):
  Phase 1:    28ms  [same as LOW]
  Dispatch:    5ms  [envelope creation + Orchestrator mailbox]
  ACK:        50ms  [preliminary ack SSE delivery]
  Orchestrator: 2.5s [LLM + tools + sub-agent coordination]
  Delivery:   70ms  [delta flush + final response + boundary writes]
  Companion: 350ms  [interleaved: typing indicators, progress streaming]

HIGH Tier (~12s P50):
  Phase 1:     28ms  [same as LOW]
  Dispatch:     5ms  [envelope to Orchestrator -> Planner]
  ACK:         50ms  [preliminary ack SSE delivery]
  Planning:    2s    [DAG construction + stage planning]
  Execution:   8s    [multi-stage DAG execution via Fabric]
  Delivery:   100ms  [final aggregation + response + boundary writes]
  Companion:  ~1.8s  [progressive delta streaming throughout]
```

21. Observability & Telemetry

21.1 Trace Propagation (OpenTelemetry)

Built on the `k0/obs/tracing.py` OpenTelemetry foundation. Concierge participates in K1's distributed tracing with a `cognitive_trace_id` that flows through the entire stack.

Trace lifecycle:

```
User message arrives (WebSocketInputAdapter)
    |
    ├── Generate trace_id (UUID v4) or extract from existing context
    |   (OpenTelemetry W3C TraceContext propagation via k0.obs.tracing)
    |
    ├── Create root span: "concierge.turn" with attributes:
    |   {session_id, turn_sequence, message_hash, safety_band}
    |
    ├── Propagate cognitive_trace_id via OpenTelemetry baggage:
    |   baggage_key = "cognitive_trace_id" (from k0.obs.tracing.COGNITIVE_TRACE_BAGGAGE_KEY)
    |
    ├── Child spans (auto-created per service boundary):
    |   concierge.turn
    |     ├── concierge.phase1.classification (UltraBERT)
    |     ├── concierge.phase1.hypothesis (IntentProcessor pipeline)
    |     ├── concierge.phase1.routing (ComplexityRouter)
    |     ├── concierge.phase2.dispatch (by tier)
    |     |     ├── concierge.tool.{tool_name} (per tool call)
    |     |     ├── orchestrator.execute (if MEDIUM/HIGH)
    |     |     └── planner.plan (if HIGH)
    |     ├── concierge.phase2.llm_call (per LLM invocation)
    |     ├── concierge.output.delivery (SSE send)
    |     └── concierge.turn_end.checkpoint (state write)
    |
    └── trace_id stamped on:
        - All structured logs (every log entry includes trace_id)
        - All delta emissions (k1.concierge.*.v1 events)
        - All adapter errors (ConciergeAdapterError.trace_id)
        - Orchestrator/Planner/Fabric requests (envelope + context)
        - SessionState writes (telemetry section)
```

Cross-layer propagation:

LAYER	HOW TRACE_ID ARRIVES	HOW TRACE_ID PROPAGATES
Concierge (L1)	Generated at WebSocketInputAdapter	Baggage + span context on all outgoing calls
Orchestrator (L2)	Extracted from TaskEnvelope.trace_id	Forwarded to Planner via context
Fabric (L2.5)	Extracted from ExecutionContext.trace_id	Forwarded to providers via CapabilityRequest
Planner (L3)	Extracted from PlanRequest.trace_id	Forwarded to DAG stage executions
Sub-agents (L4)	Extracted from AgentContext.trace_id	Used in agent-scoped spans
K0 (L6)	Extracted from Bridge request headers	Used in K0 span tree

21.2 Structured Logs

All log entries follow the SessionState structured logging pattern (`k1/sessionstate` test suite). JSON-formatted with mandatory fields.

Mandatory log fields (every entry):

```
{
  "timestamp_ms": 1739712000000,
  "level": "INFO",
  "service": "concierge",
  "session_id": "sess-abc123",
  "trace_id": "trace-def456",
  "turn_sequence": 42,
  "event": "turn.start",
  "data": { }
}
```

Log event catalog:

EVENT	LEVEL	DATA FIELDS	WHEN EMITTED
`session.init`	INFO	`{restore_source, restore_duration_ms, cb_states}`	init() complete
`session.shutdown`	INFO	`{total_turns, duration_s, pending_writes}`	shutdown() complete
`session.recovered`	WARN	`{rolled_back_turn, restore_source, outbox_drained}`	crash_recovery() complete
`turn.start`	INFO	`{message_hash, lock_version}`	turn_start()
`phase1.complete`	INFO	`{latency_ms, tier, safety_band, hypothesis_id, gaps_count, confidence}`	Phase 1 done
`phase2.start`	INFO	`{tier, dispatch_type: direct/envelope}`	Phase 2 begins
`phase2.tool_call`	DEBUG	`{tool_name, tool_args_hash, duration_ms, success}`	Each tool dispatch
`phase2.llm_call`	INFO	`{model, capability, input_tokens, output_tokens, duration_ms, cost}`	Each LLM invocation
`turn.complete`	INFO	`{latency_ms, tier, tool_count, token_usage, degradation_level}`	turn_end() complete
`turn.failed`	ERROR	`{error_code, adapter_name, severity, action, retries, user_message}`	ABORT error path
`turn.interrupted`	WARN	`{interrupted_at_state, new_message_hash}`	Interrupt detected
`dispatch.sent`	INFO	`{tier, envelope_id, target: orchestrator/fabric}`	Envelope dispatched
`dispatch.complete`	INFO	`{envelope_id, duration_ms, result_count}`	Orchestrator response
`error.routed`	WARN	`{adapter_name, operation, severity, action, retry_count}`	ErrorRouter decision
`cb.state_change`	WARN	`{cb_name, old_state, new_state, failure_count}`	Any CB transition
`experience.triggered`	INFO	`{trigger_type: narrative|emotional|anticipatory, duration_ms}`	Experience processing
`clarification.round`	INFO	`{intent_id, round_number, gaps_addressed}`	Clarification sent
`crisis.detected`	CRITICAL	`{keyword_match?, safety_head_score, response_delivered}`	CRISIS override
`checkpoint.local`	DEBUG	`{duration_ms, sections_written, total_bytes}`	LOCAL COLD checkpoint
`checkpoint.k0`	DEBUG	`{bridge_available, fire_and_forget}`	K0 sync attempt

21.3 Prometheus Metrics

Following the `k0/obs/metrics.py` MetricsExporter pattern and `k1/fabric/metrics.py` FabricMetrics pattern. Namespace: `k1_concierge`.

Counters:

METRIC NAME	LABELS	DESCRIPTION
`k1_concierge_turns_total`	`{tier, safety_band, outcome}`	Total turns processed. outcome: success, failed, interrupted
`k1_concierge_tool_calls_total`	`{tool_name, success}`	Tool dispatch count
`k1_concierge_llm_calls_total`	`{model, capability, success}`	LLM invocations via ModelGateway
`k1_concierge_errors_total`	`{adapter, severity, action}`	Errors classified by ErrorRouter
`k1_concierge_clarifications_total`	`{intent_class}`	Clarification rounds
`k1_concierge_cb_trips_total`	`{cb_name}`	Circuit breaker openings
`k1_concierge_retries_total`	`{adapter}`	Retry attempts
`k1_concierge_crisis_total`	–	CRISIS detections
`k1_concierge_checkpoints_total`	`{target: local_cold, k0}`	Checkpoint operations
`k1_concierge_experience_triggers_total`	`{type: narrative, emotional, anticipatory}`	Experience layer triggers

Histograms (buckets in ms):

METRIC NAME	LABELS	BUCKETS	DESCRIPTION
`k1_concierge_phase1_duration_ms`	–	5, 10, 15, 20, 25, 30, 40, 50	Phase 1 (classification + pipeline)
`k1_concierge_turn_duration_ms`	`{tier}`	100, 500, 1000, 2000, 5000, 10000, 30000, 60000	End-to-end turn latency
`k1_concierge_tool_dispatch_duration_ms`	`{tool_name}`	1, 5, 10, 50, 100, 500, 1000	Per-tool dispatch time
`k1_concierge_llm_call_duration_ms`	`{capability}`	100, 500, 1000, 5000, 10000, 30000	Per-LLM-call latency
`k1_concierge_output_delivery_duration_ms`	`{priority}`	1, 5, 10, 25, 50, 100	SSE delivery time
`k1_concierge_checkpoint_duration_ms`	`{target}`	0.1, 0.5, 1, 2, 5, 10	Checkpoint latency

Gauges:

METRIC NAME	LABELS	DESCRIPTION
`k1_concierge_active_sessions`	–	Currently active sessions
`k1_concierge_hot_tier_bytes`	`{session_id}`	HOT tier memory usage
`k1_concierge_output_queue_depth`	`{session_id}`	Current OutputManager queue depth
`k1_concierge_cb_state`	`{cb_name, state}`	Circuit breaker state (1=active, 0=inactive)
`k1_concierge_fsm_state`	`{state}`	Current FSM state (1=active)
`k1_concierge_token_budget_remaining`	`{session_id}`	Remaining 128K budget

21.4 Delta Emissions (via IDeltaPort)

Deltas are fire-and-forget events published to the K1 Bus for downstream consumers (Memory Writer, Learning Loop, admin dashboards).

DELTA TOPIC	TRIGGER	PAYLOAD (KEY FIELDS)	CONSUMERS
`k1.concierge.turn.started.v1`	turn_start()	`{session_id, turn_seq, trace_id}`	Learning Loop
`k1.concierge.turn.completed.v1`	turn_end()	`{session_id, turn_seq, tier, latency_ms, tool_count, tokens}`	Memory Writer, Learning Loop
`k1.concierge.turn.failed.v1`	ABORT path	`{session_id, turn_seq, error_code, adapter, severity}`	Learning Loop, Alerting
`k1.concierge.turn.interrupted.v1`	Interrupt handling	`{session_id, old_turn_seq, new_message_hash}`	Learning Loop
`k1.concierge.phase1.complete.v1`	Phase 1 done	`{tier, safety_band, hypothesis_id, confidence}`	Dashboard
`k1.concierge.error.routed.v1`	ErrorRouter	`{adapter, operation, severity, action, trace_id}`	Alerting, Dashboard
`k1.concierge.cb.state_change.v1`	CB transition	`{cb_name, old_state, new_state, failure_count}`	Alerting, Dashboard
`k1.concierge.session.recovered.v1`	crash_recovery()	`{session_id, rolled_back_turn, restore_source}`	Alerting
`k1.concierge.crisis.detected.v1`	CRISIS override	`{session_id, keyword_match, safety_score}`	Safety Monitor, Alerting
`k1.concierge.affective.update.v1`	Affect change	`{session_id, emotional_state, intensity}`	Dashboard
`k1.concierge.scoreboard.update.v1`	Task status change	`{session_id, task_id, new_status}`	Dashboard

21.5 SessionState Telemetry Section

The telemetry section (WARM tier, 8KB, eviction priority 1) stores per-session metrics within SessionState itself. Written at `turn_end()`.

FIELD	TYPE	UPDATED AT	DESCRIPTION
`tokens.input_total`	int	turn_end	Cumulative input tokens
`tokens.output_total`	int	turn_end	Cumulative output tokens
`cost.total_usd`	float	turn_end	Cumulative LLM cost
`latency.p95_ms`	int	turn_end	Rolling P95 across last 20 turns
`errors.total`	int	on error	Cumulative error count
`errors.by_adapter`	dict	on error	Error count keyed by adapter_name
`summary.turn_count`	int	turn_end	Total turns in session
`summary.tool_call_count`	int	turn_end	Total tool calls in session
`dead_letter.entries`	list (max 10)	on SSE failure	Failed output events for replay on reconnect

21.6 Alerting Thresholds

CONDITION	SEVERITY	ALERT
CB_CLASSIFICATION OPEN > 5 min	WARNING	UltraBERT degraded, heuristic fallback active
CB_MODEL OPEN > 2 min	CRITICAL	LLM unavailable, canned responses only
CB_ORCHESTRATOR OPEN > 5 min	WARNING	Orchestrator down, all requests degraded to LOW
Turn failure rate > 10% (1 min window)	CRITICAL	Systemic failure, investigate
CRISIS detected	CRITICAL	Immediate safety alert (always, regardless of frequency)
Session recovery triggered	WARNING	Crash detected and recovered
HOT tier > 45KB	WARNING	Approaching 48KB budget, summarization may be needed
Output queue depth > 40	WARNING	Approaching 50 limit (CONC-12)

22. Lifecycle (6 Phases)

Six phases cover the complete session and turn lifecycle. Owners: FSMController (session-level: init, shutdown, crash_recovery) and TurnProcessor (turn-level: turn_start, turn_end, turn_end_abbreviated). Cross-reference: Section 16.2 (FSMController), Section 16.3 (TurnProcessor), Section 17.8 (crash recovery).

22.1 INIT (Session Start)

Owner: FSMController. **Trigger**: New session created. **Outcome**: FSM in LISTENING, ready for user messages.

```
async def init(session_id: str) -> None:
    """LC_INIT: Full session initialization (concierge.mmd).

    Steps:
    1. Create 9 services (no I/O):
       FSMController, TurnProcessor, IntentProcessor, ComplexityRouter,
       ToolDispatcher, OutputManager, DeltaAggregator, ClarificationTracker,
       ContextAssembler
       -> Pure construction, wire service dependency graph

    2. Connect 8 ports (adapters already injected by ConciergeFactory):
       IInputPort, IOutputPort, IClassificationPort, ILLMPort,
       IStatePort, IDispatchPort, IDeltaPort, IMemoryPort

    3. Restore state (Edge-First strategy):
       a. Check LOCAL COLD (SQLite) -- always available, P95 < 1ms
       b. If LOCAL COLD exists and fresh: restore from it
       c. If LOCAL COLD stale AND K0 Bridge online: pull from K0 (P95 < 50ms)
       d. If LOCAL COLD missing AND K0 offline: empty session (first time)

    4. Load persona:
       -> PersonaEngine initialized from persona section
       -> Communication style derived from preferences
       -> Affective state initialized (neutral baseline)

    5. Init 7 circuit breakers -> all CLOSED:
       CB_CLASSIFICATION, CB_MODEL, CB_SESSIONSTATE,
       CB_ORCHESTRATOR, CB_PLANNER, CB_FABRIC, CB_SSE
       -> Register in CircuitBreakerRegistry with IStateChangeListener

    6. Start background:
       -> IDeltaPort subscriptions (incoming delta events)
       -> DeltaAggregator 500ms timer (repeating async timer callback)
       -> IInputPort.begin_receive() (WebSocket listener)

    7. FSM -> LISTENING:
       -> Emit k1.concierge.session.init.v1 via IDeltaPort
       -> Log session.init INFO
       -> Ready for first message
    """
    ...
```

Init failure modes:

STEP	FAILURE	RECOVERY
Step 3a (LOCAL COLD)	SQLite corrupt	Rebuild from K0; if K0 offline, empty session
Step 3c (K0 pull)	Bridge timeout	Use LOCAL COLD (stale but available); if none, empty
Step 4 (persona)	Missing persona section	Use default persona configuration
Step 5 (CB init)	Never fails	Pure construction, no I/O
Step 6 (subscriptions)	Bus unavailable	Log warning, DeltaAggregator operates without incoming deltas

22.2 TURN_START

Owner: TurnProcessor. **Trigger**: User message received, FSM transitions LISTENING -> ACKING. **Outcome**: Turn context created, services reset, ready for Phase 1.

```
async def turn_start(message: UserMessage) -> TurnContext:
    """LC_TURN_START: Per-turn initialization.

    Steps:
    1. Acquire Single Writer lock:
       -> lock_version++ (monotonic increment)
       -> If lock already held: reject (CONC-15, impossible if FSM correct)

    2. Increment turn_sequence_number:
       -> Monotonic counter in control section
       -> Used for ordering and crash detection

    3. Read HOT snapshot (all 8 sections):
       -> IStatePort.read_all_hot() -> HotTierSnapshot
       -> Provides: history_active, beliefs, scoreboard, affect, persona,
          control, narrative_active, preferences
       -> Read-only snapshot: no writes during phase 1 (deterministic)

    4. Reset per-turn service state:
       -> ToolDispatcher.clear_buffer() (zero tool results, reset counter)
       -> ClarificationTracker.reset_turn() (clear turn-scoped state)
       -> ContextAssembler.reset_budget() (reset token tracking)

    5. Allocate tier budget envelope:
       -> Pre-allocate timeout based on expected tier
       -> Adjusted after ComplexityRouter.route() in Phase 1
       -> Start 120s watchdog timer (CONC-11 absolute max)

    6. Emit k1.concierge.turn.started.v1 via IDeltaPort
    """
    ...
```

22.3 TURN_END (Normal Path)

Owner: TurnProcessor. **Trigger**: FSM in DELIVERING, response delivered to user. **Outcome**: State persisted, lock released, experience processed, back to LISTENING.

```
async def turn_end() -> TurnSummary:
    """LC_TURN_END: Full turn boundary writes (concierge.mmd).

    Steps:
    1. Flush OutputManager:
       -> Drain all queued events (REALTIME first, then INTERACTIVE, BACKGROUND)
       -> Wait for SSE delivery confirmation
       -> Dead-letter any that fail delivery

    2. Append to history_active (MutationGuard):
       -> Build turn record: {user_message, assistant_response, tool_calls[], tier}
       -> IStatePort.write("history_active", turn_record)
       -> MutationGuard: check_expiry -> validate_writer -> preflight -> mutate

    3. Update meta section:
       -> turn_count++, last_activity_ms = now()

    4. Update telemetry section:
       -> tokens: input_total += turn_input, output_total += turn_output
       -> cost: total_usd += turn_cost
       -> latency: append turn_latency_ms to rolling window
       -> errors: update if any errors occurred

    5. Release Single Writer lock:
       -> Clear turn_lock in control section
       -> turn_status = COMPLETED

    6. Checkpoint LOCAL COLD (SQLite):
       -> Write full HOT snapshot to SQLite (P95 < 1ms)
       -> Atomic transaction (all-or-nothing)

    7. Checkpoint K0 (fire-and-forget):
       -> IMemoryPort.store(session_snapshot) via Bridge
       -> If Bridge offline: queue in Local Outbox (SQLite)
       -> NEVER blocks turn completion

    8. Check experience triggers:
       -> tick_experience(turn_count)
       -> Modular: %20 narrative, %25 emotional, %30 anticipatory
       -> Max 500ms combined budget

    9. Emit k1.concierge.turn.completed.v1 via IDeltaPort:
       -> Payload: {session_id, turn_seq, tier, latency_ms, tool_count, tokens}
       -> Triggers: Memory Writer pipeline, Learning Loop detectors
       -> FSM -> LISTENING
    """
    ...
```

22.4 TURN_END_ABBREVIATED (Interrupt Path)

Owner: TurnProcessor. **Trigger**: Interrupt detected, FSM in INTERRUPT_HANDLING. **Outcome**: Partial state saved, lock released, ready for new message.

```
async def turn_end_abbreviated() -> TurnSummary:
    """LC_TURN_END_ABBREVIATED: Minimal cleanup for interrupted turns.

    Steps:
    1. Cancel in-flight dispatches:
       -> IDispatchPort.cancel_dispatch() (cancel envelope if pending)
       -> ToolDispatcher: stop ReAct loop, collect partial results

    2. Flush OutputManager (partial):
       -> Deliver any completed events
       -> Drop pending events that can't be delivered
       -> Dead-letter REALTIME events only

    3. Record turn_status: INTERRUPTED:
       -> Write to control section: turn_status = INTERRUPTED
       -> No history append (interrupted turn not recorded)

    4. Release Single Writer lock:
       -> Clear turn_lock
       -> lock_version stays incremented (for crash detection)

    5. Checkpoint LOCAL COLD:
       -> Minimal checkpoint (control section only)
       -> Ensures crash recovery can detect the interrupt

    SKIP:
    - History append (incomplete turn)
    - Telemetry update (statistics skewed by interrupt)
    - K0 sync (nothing meaningful to persist)
    - Experience triggers (no completed turn to learn from)
    """
    ...
```

22.5 SHUTDOWN (Graceful)

Owner: FSMController. **Trigger**: Session end requested (user logout, idle timeout, admin command). **Outcome**: All state persisted, all connections closed, session cleaned up.

```
async def shutdown() -> None:
    """LC_SHUTDOWN: Graceful session shutdown (concierge.mmd).

    Steps:
    1. IInputPort.close():
       -> Reject all new input messages
       -> Buffer any in-flight message for processing
       -> WebSocket: send close frame

    2. Wait active turn (30s max) or force-close:
       -> If turn in progress: await turn completion (up to 30s)
       -> If 30s exceeded: force cancel_turn() + turn_end_abbreviated()
       -> If no turn: proceed immediately

    3. Flush DeltaAggregator + OutputManager:
       -> DeltaAggregator.flush() (deliver final batch)
       -> OutputManager.flush() (deliver final events)

    4. Final checkpoint:
       -> LOCAL COLD: full HOT + WARM snapshot (atomic SQLite write)
       -> K0: synchronous Bridge write with 5s timeout
          (this is the ONLY place we wait for K0 -- normal turn_end is fire-and-forget)
       -> If K0 timeout: leave in Local Outbox (eventual sync on next session)

    5. Stop background:
       -> Cancel DeltaAggregator timer
       -> Unsubscribe all delta subscriptions
       -> Close SSE connection (if OutputPort is SSE)

    6. Disconnect ports (LIFO order):
       -> IMemoryPort  (last connected, first disconnected)
       -> IDeltaPort
       -> IDispatchPort
       -> ILLMPort
       -> IClassificationPort
       -> IStatePort
       -> IOutputPort
       -> IInputPort   (first connected, last disconnected)

    7. Emit k1.concierge.session.shutdown.v1 (best-effort, delta port may be closed)
    8. Log session.shutdown INFO with total_turns, duration, final stats
    """
    ...
```

22.6 CRASH_RECOVERY

Owner: FSMController. **Trigger**: Session resume after unexpected termination. **Outcome**: State restored to last clean checkpoint, FSM in LISTENING. Cross-reference: Section 17.8.

```
async def crash_recovery(session_id: str) -> None:
    """LC_CRASH_RECOVERY: Recover from unclean shutdown (concierge.mmd).

    Steps:
    1. Bootstrap (no state needed):
       -> Create 9 services + connect 8 ports
       -> Same as init() steps 1-2

    2. Detect incomplete turn:
       -> Read control section from LOCAL COLD (SQLite)
       -> Check: control.turn_lock is not None?
       -> If None: clean shutdown (skip to step 5)
       -> If set: crashed during active turn

    3. Rollback incomplete turn:
       -> Clear turn_lock in control section
       -> Increment lock_version (prevents ghost writes)
       -> Set turn_status = ROLLED_BACK
       -> Discard any un-flushed DeltaAggregator batch

    4. Reconcile Local Outbox:
       -> Read pending K0 writes from SQLite outbox table
       -> If Bridge online: drain outbox (fire-and-forget)
       -> If Bridge offline: leave in outbox (eventual sync)

    5. Restore from LOCAL COLD:
       -> Read full HOT + WARM snapshot from SQLite
       -> Validate CRC-32 checksums per section
       -> If checksum mismatch: rebuild from K0 (~500ms)
       -> If K0 also unavailable: empty section (data loss bounded to 1 turn)

    6. Resume:
       -> Load persona (PersonaEngine + style)
       -> Init 7 CBs -> all CLOSED (clean slate after crash)
       -> Start subscriptions + DeltaAggregator timer
       -> FSM -> LISTENING

    7. Recovery telemetry:
       -> Emit k1.concierge.session.recovered.v1 via IDeltaPort
       -> Log session.recovered WARN with:
          {session_id, lock_version, rolled_back_turn, restore_source,
           duration_ms, sections_restored, outbox_entries_drained}

    Guarantee: Maximum data loss = 1 turn (the crashed turn).
    All completed turns are checkpointed in LOCAL COLD at turn_end().
    """
    ...
```

22.7 Lifecycle State Diagram

23. Concurrency Model

23.1 Single Writer, Single Event Loop

The Concierge operates on a **single asyncio event loop per session**. No threads are used for service logic. The core concurrency invariant:

- **CONC-15**: Exactly 1 active turn per session at any time
- **turn_lock**: Version-based lock (not a mutex). `lock_version` is a monotonic integer in the `control` section of SessionState.
- **No concurrent Phase 1 + Phase 2**: Phase 1 always completes before Phase 2 begins (CONC-03).

```python
# Turn lock acquisition (at turn_start)
async def _acquire_turn_lock(self) -> int:
    """Acquire Single Writer lock. Returns new lock_version.

    Invariant: CONC-15 -- only one turn at a time.
    Implementation: read control.turn_lock. If None: acquire.
    If already set: BUG in FSM (should never happen if FSM is correct).
    """
    control = await self._state_port.read("control")
    if control.turn_lock is not None:
        raise ConcurrencyViolationError(
            f"Turn lock already held: version={control.lock_version}"
        )
    new_version = control.lock_version + 1
    await self._state_port.write("control", {
        "turn_lock": self._session_id,
        "lock_version": new_version,
        "turn_status": "IN_PROGRESS",
    })
    return new_version
```

23.2 Async Task Topology

Within a single turn, the Concierge uses structured asyncio tasks:

```
Main event loop (session-scoped)
│
├── Turn processing task (one at a time, CONC-15)
│   ├── Phase 1 (sequential, no concurrency)
│   │   └── UltraBERT classify -> hypothesis -> gap detect -> route
│   │
│   ├── Phase 2 (controlled concurrency)
│   │   ├── ReAct loop: sequential LLM calls, TIERED PARALLEL tool dispatches
│   │   │   ├── Ack tools: sequential (user-facing, must be first)
│   │   │   ├── Read + Action tools: asyncio.gather() (no SS writes, safe)
│   │   │   └── Cognitive tools: sequential (SS writes, order-dependent)
│   │   │   (See Section 7.5.1 for full parallelism strategy per ADR-0078)
│   │   │
│   │   ├── Companioning task (concurrent with dispatch):
│   │   │   └── Emit typing indicators + preliminary ack
│   │   │   └── Runs as asyncio.create_task(), canceled on dispatch complete
│   │   │
│   │   └── Progress streaming task (concurrent with dispatch):
│   │       └── DeltaAggregator.flush() -> OutputManager.stream_progress()
│   │       └── Runs on 500ms timer callback
│   │
│   └── Turn boundary (sequential)
│       └── flush -> write -> checkpoint -> experience -> LISTENING
│
├── DeltaAggregator timer (session-scoped background)
│   └── 500ms repeating async timer callback (NOT a thread)
│   └── Calls DeltaAggregator.flush() on each tick
│   └── Safe: flush() acquires no locks, uses fire-and-forget IDeltaPort
│
├── Input listener (session-scoped background)
│   └── IInputPort.receive() awaiting next message
│   └── On receive: check if current state is interruptible
│   └── If interruptible: signal interrupt to FSMController
│   └── If not: buffer message for after turn_end
│
└── Watchdog timer (turn-scoped)
    └── 120s timeout (CONC-11)
    └── Fires: force cancel_turn() + turn_end_abbreviated()
    └── Canceled at turn_end() (normal completion)
```

23.2.1 Foreground/Background Turn Split (Design Proposal)

Status: Design phase. Requires new ADR.

Problem: When the Concierge dispatches a MED/HIGH tier task to the Orchestrator, the user cannot meaningfully interact until the task completes. The current model (CONC-15: 1 active turn) blocks all user input during background execution. The user can only interrupt (which aborts the task) or wait.

Proposal: Split the single-turn model into **foreground** and **background** channels, allowing the user to converse while a background task runs.

New FSM state: `BACKGROUND_WORKING` – between DISPATCHING and DELIVERING.

Concurrency rules for FG/BG split:

RULE	DESCRIPTION
CONC-15 revised	1 BACKGROUND turn + 1 FOREGROUND micro-turn allowed concurrently
Single Writer preserved	Foreground holds write lock during micro-turn; background deltas buffer
Foreground limits	LOW tier only, max 2 tool calls, cognitive tools OK, action tools BLOCKED
Background isolation	Orchestrator DAG runs independently; deltas queue in DeltaAggregator
Lock handoff	FG micro-turn acquires/releases lock within ~200ms; BG never holds write lock
User info injection	User messages during BG can feed INTO the running task (e.g., "she's allergic to shellfish" while party planning)

Foreground micro-turn flow:

1. User sends message while `BACKGROUND_WORKING`.
2. Input listener classifies as **foreground-eligible** (not an interrupt, not a new HIGH task).
3. FSMController spawns a **micro-turn** (LOW tier, 2 tool calls max, 500 token budget).

4. Micro-turn acquires write lock, executes Phase 1 + Phase 2 (fast), releases lock.
5. If micro-turn produces information relevant to BG task, it emits a `k1.concierge.fg_inject.v1` event.
6. BG task's next DAG step picks up injected context.

Why this matters: Without FG/BG split, the user experience during a 30-60s HIGH tier task is silence + progress dots. With it, the user can continue conversing naturally – answering follow-up questions, adding constraints, or asking about unrelated topics – while the background task completes. This is a **transformative UX improvement** for family-OS scenarios where multi-minute tasks (party planning, trip research, health appointments) are common.

> **Needs**: New ADR (proposed ADR-0097), CONC-15 revision, new FSM state, ToolDispatcher foreground-mode flag, DeltaAggregator lock-aware buffering. **Cross-reference**: Section 7.5.1 (Parallel Tools), `concierge.mmd` FSM_CORE_LOOP.

23.3 Cooperative Cancellation

The Concierge uses cooperative cancellation (not pre-emption). Long-running operations check for cancellation at natural boundaries.

```python
class CancellationToken:
    """Cooperative cancellation for long-running turn operations.

    Checked at:
    - Between each tool call in ToolDispatcher.dispatch_tool()
    - Between each LLM call in the ReAct loop
    - At each DeltaAggregator.flush() boundary
    - At each state write in turn_end()

    Set by:
    - FSMController.on_interrupt() (user interrupt detected)
    - Watchdog timer expiry (120s absolute timeout)
    - shutdown() requesting turn completion
    """

    __slots__ = ("_cancelled", "_reason")

    def __init__(self) -> None:
        self._cancelled = False
        self._reason: Optional[str] = None

    def cancel(self, reason: str) -> None:
        self._cancelled = True
        self._reason = reason

    @property
    def is_cancelled(self) -> bool:
        return self._cancelled

    def check(self) -> None:
        """Raise CancellationError if cancelled. Called at checkpoints."""
        if self._cancelled:
            raise TurnCancellationError(self._reason)
```

Cancellation checkpoint locations:

LOCATION	WHAT HAPPENS ON CANCEL
ToolDispatcher: between tool calls	Stop ReAct loop, collect partial results
ToolDispatcher: between LLM calls	Return partial response
DeltaAggregator: at flush()	Flush current batch, stop timer
OutputManager: during delivery	Deliver queued events, stop accepting new
TurnProcessor: at turn boundary	Skip remaining writes, proceed to abbreviated turn_end

23.4 Thread Safety Analysis

COMPONENT	THREAD MODEL	SHARED STATE	PROTECTION
FSMController	Single async task	`_state` (FSMState enum)	Sequential access guaranteed by event loop
TurnProcessor	Single async task	`_lock_version`, `_current_turn`	Sequential: only one turn at a time
IntentProcessor	Called within turn task	None (stateless per call)	No shared state
ComplexityRouter	Called within turn task	None (pure function)	No shared state
ToolDispatcher	Called within turn task	`_call_count`, `_results_buffer`	Reset at turn_start, single writer during turn
OutputManager	Async concurrent-safe	`_queue` (asyncio.Queue)	asyncio.Queue is coroutine-safe
DeltaAggregator	Timer callback + turn task	`_current_batch` (list)	asyncio event loop serialization (single thread)
ClarificationTracker	Called within turn task	`_round_counts` (dict)	Reset at turn_start, single writer
ContextAssembler	Called within turn task	`_turn_usage` (int)	Reset at turn_start, single writer
CircuitBreakerRegistry	Multi-accessor	Per-CB state	`threading.Lock` per CB (from breaker.py)

Why no threading.Lock in services: All 9 services run on the same asyncio event loop. Python's GIL + asyncio's cooperative scheduling means only one coroutine runs at a time. No thread-level races are possible for in-memory state. The only component using `threading.Lock` is `CircuitBreaker` (from Fabric), because it may be accessed by a background health-checker thread.

23.5 Session Isolation

Multiple concurrent sessions are fully isolated:

```
Process / Event Loop
│
├── Session A (user Alice)
│   ├── Own FSMController, TurnProcessor, 9 services
│   ├── Own 8 port adapters (own WebSocket, own SSE)
│   ├── Own 7 circuit breakers (CB state is per-session)
│   ├── Own SessionState (48KB HOT + 48KB WARM)
│   └── Own DeltaAggregator timer
│
├── Session B (user Bob)
│   ├── (same structure, completely independent)
│   └── ...
│
└── Shared (read-only):
    ├── UltraBERT model weights (shared inference, thread-safe)
    ├── Tool registry (static after startup)
    ├── Persona templates (loaded at init, immutable)
    └── Prometheus metrics registry (thread-safe counters/gauges)
```

Cross-session guarantees:

- No session can read another session's SessionState
- No session shares circuit breaker state with another
- CB trips in Session A do not affect Session B
- The only shared resources are thread-safe read-only singletons

23.6 Deadlock Prevention

Three design rules prevent deadlocks:

1. **No nested locks**: Only one lock type exists (turn_lock). Services never acquire additional locks.
2. **No blocking I/O**: All port calls are async. A slow adapter cannot block another adapter.
3. **Watchdog as safety net**: The 120s watchdog timer (CONC-11) force-cancels any turn that appears stuck. Even if a coroutine hangs on an `await`, the watchdog fires and triggers `turn_end_abbreviated()`.

```
Deadlock scenario analysis:

Q: Can two turns deadlock on turn_lock?
A: No. CONC-15 guarantees only 1 turn per session. FSM prevents
   MESSAGE_RECEIVED from ACKING/DISPATCHING/etc. Second message
   is buffered by IInputPort, not processed.

Q: Can DeltaAggregator timer deadlock with turn processing?
A: No. DeltaAggregator.flush() is fire-and-forget (never acquires
   turn_lock). Timer callback runs on same event loop, interleaved
   between awaits in the turn task.

Q: Can Circuit Breaker lock deadlock with asyncio?
A: No. CB threading.Lock is held for microseconds (state check only).
   The _notify_state_change_locked() method explicitly releases lock
   before calling callback, then re-acquires. No async awaits while
   holding CB lock.
```

24. External Touchpoints Summary

The Concierge boundary is the hexagonal shell defined by 8 ports (Section 14). Everything outside these ports is an external touchpoint. This section catalogs every external system the Concierge interacts with, the direction of data flow, the port used, the wire protocol, and the circuit breaker protecting it.

24.1 Inbound (who sends TO Concierge)

SOURCE	PORT	PROTOCOL	EVENTS / DATA	PRIORITY	CB
User (via L0)	IInputPort	WebSocket binary (ADR-0040) / REST JSON (ADR-0041)	`UserMessage` (text, session_id, trace_id, metadata)	REALTIME	None (input is passive listener)
Orchestrator	IDeltaPort (subscribe)	K1 Bus Envelope (FlatBuffers V2)	`k1.orchestration.dag.completed.v1`, `k1.orchestration.delta.v1`, `k1.orchestration.step.*.v1`	INTERACTIVE	None (subscription is passive)
Planner (via Bus)	IDeltaPort (subscribe)	K1 Bus Envelope	`k1.hil.clarification.v1`, `k1.hil.approval_request.v1`	INTERACTIVE	None
Orchestrator (via Bus)	IDeltaPort (subscribe)	K1 Bus Envelope	`k1.hil.fallback.v1`, `k1.hil.progress.v1`	INTERACTIVE	None
K0 Bridge	IDeltaPort (subscribe)	K1 Bus Envelope (relayed from K0 SSE)	`k0.proactive.signal.v1`, `k0.config.hot_reload.v1`	BACKGROUND	None
Sub-agents	IDeltaPort (subscribe)	K1 Bus Envelope (delta lane)	`k1.agent.{id}.delta.v1` (state deltas, clarification requests)	BACKGROUND	None
Learning Loop	IDeltaPort (subscribe)	K1 Bus Envelope	Feedback advisories (optional, non-blocking)	BACKGROUND	None

24.2 Outbound (who Concierge sends TO)

DESTINATION	PORT	PROTOCOL	DATA	PRIORITY	CB
User (via L0)	IOutputPort	SSE (ADR-0016) / WebSocket	9 `OutputEvent` types (Section 14.3)	REALTIME / INTERACTIVE / BACKGROUND	CB_SSE
UltraBERT v4	IClassificationPort	In-process inference (PyTorch)	`classify(text)` -> `ClassificationResult` (12 heads)	N/A (sync, <25ms)	CB_CLASSIFICATION
Model Hub	ILLMPort	K1 LLM Request Bus (capability-tagged `HubRequest`)	`execute()` / `stream_execute()` -> `HubResponse`	N/A (async, budget-gated)	CB_MODEL
SessionState	IStatePort	In-process via SessionKernelAdapter	`read()` / `write()` / `checkpoint()` via MutationGuard	N/A (sync, <1ms)	CB_SESSIONSTATE
Orchestrator	IDispatchPort	K1 Bus Mailbox (`TaskEnvelope` -> `IMailboxPort.enqueue()`)	MEDIUM/HIGH tier task envelopes	INTERACTIVE	CB_ORCHESTRATOR
Fabric	IDispatchPort	In-process via FabricOrchestratorAdapter	LOW tier `dispatch_direct(CapabilityRequest)`	N/A (async)	CB_FABRIC
Planner (indirect)	IDispatchPort	Via Orchestrator's `IPlannerPort`	HIGH tier triggers `PlanRequest` inside Orchestrator	N/A	CB_PLANNER
K1 Bus (delta lane)	IDeltaPort	K1 Bus Envelope (fire-and-forget)	11 delta topics (Section 21.4)	BACKGROUND	None (fire-and-forget)
K0 Bridge	IMemoryPort	Bridge HTTP/2 + FlatBuffers (ADR-0044)	`recall(query)` / `store(snapshot)`	BACKGROUND	None (graceful offline)

24.3 Reads (what Concierge reads FROM)

SOURCE	PORT	WHAT	WHEN	LATENCY
SessionState HOT (48KB)	IStatePort	8 sections: control, beliefs_active, scoreboard, history_active, clarifications, affective_now, narrative_active, meta	Every turn at turn_start (full snapshot)	< 0.2ms
SessionState WARM (48KB)	IStatePort	beliefs_history, history_recent, persona, telemetry	On demand (promote_belief, experience layer)	< 0.5ms
K0 Bridge	IMemoryPort	Long-term memories matching `recall(query, selectors[])`	When LLM calls `recall_memory()` tool	< 200ms (or skip if offline)
Fabric Registry	IDispatchPort	Capability discovery (`discover_capabilities()`)	When LLM calls `discover_capabilities()` tool	< 50ms

24.4 Touchpoint Dependency Matrix

```
                  CRITICAL    DEGRADED      OPTIONAL
                  (turn fails) (reduced)    (skip OK)
                  ----------- ------------ ----------
User (L0)         IOutputPort  --           --
UltraBERT         --           IClassPort   --
                               (heuristic)
Model Hub         ILLMPort*    --           --
SessionState      --           IStatePort   --
                               (stale read)
Orchestrator      --           IDispatchPort --
                               (tier degrade)
Fabric            --           IDispatchPort --
                               (tier degrade)
K0 Bridge         --           --           IMemoryPort
K1 Bus            --           --           IDeltaPort

* ILLMPort is CRITICAL only if template fallback also fails
  (3-tier cascade: retry -> model cascade -> template -> ABORT)
```

25. Delta Lane & Event Lane Integration

The K1 Bus is a single physical bus (`k1/bus/impl/local_bus.py` , `IBus` protocol) with two logical lanes distinguished by topic convention. All messages flow as immutable `Envelope` instances with opaque payloads (bus-is-BLIND pattern, ADR-0048).

25.1 Bus Architecture (from `k1/bus/`)

```
K1 Bus (LocalBus, single in-process instance)
|
+-- Event Lane (k1.* topics, excluding *.delta.*)
|   +-- Delivery: STRICT (ordered, causal wait) or RELAXED
|   +-- Semantics: at-most-once pub/sub
|   +-- Latency: < 2ms publish + trie match
|   +-- Throughput: 1000s events/sec
|
+-- Delta Lane (k1.*.delta.v1 topics)
|   +-- Delivery: BEST_EFFORT (fire-and-forget, drop OK under pressure)
|   +-- Semantics: at-most-once, no ack
|   +-- Aggregation: delegated to Concierge DeltaAggregator (500ms window)
|   +-- Topic convention: k1.agent.{agent_id}.delta.v1
|
+-- Mailbox Router (UUID-to-mailbox, location-transparent)
    +-- Delivery: < 1ms direct
    +-- Used by: Orchestrator (TaskEnvelope), Planner (PlanRequest)
    +-- Backpressure: mailbox size limits per actor
```

Envelope structure (from `k1/bus/envelope/envelope.py`):

FIELD	TYPE	PURPOSE
`topic`	str	Hierarchical routing key (e.g. `k1.concierge.turn.completed.v1`)
`priority`	Priority (0-3)	WFQ scheduling: URGENT(4x), REALTIME(3x), INTERACTIVE(2x), BACKGROUND(1x)
`envelope_id`	int	Global monotonic, bus-assigned
`sequence`	int	Per-topic monotonic, bus-assigned (gap detection)
`cognitive_trace_id`	str	Cross-K0/K1 correlation key (= Section 21.1 trace_id)
`session_id`	str	Session scope
`parent_id`	int	Causal parent (0 = root)
`payload`	bytes	Opaque (JSON or MessagePack, bus never reads)
`ttl_ms`	int	Envelope expiry (0 = no expiry)
`payload_format`	PayloadFormat	0=OPAQUE, 1=JSON, 2=MSGPACK

25.2 Concierge Bus Adapter: DeltaBusAdapter

The `DeltaBusAdapter` implements `IDeltaPort` by wrapping `IBus` . It handles both publish and subscribe.

```
class ConcierDeltaBusAdapter:
    """IDeltaPort implementation for Concierge.

    Publish path:
      IDeltaPort.publish(topic, payload)
        -> Envelope(topic=topic, payload=json.dumps(payload).encode(),
                    priority=BACKGROUND, cognitive_trace_id=current_trace_id,
                    session_id=current_session_id, payload_format=JSON)
        -> IBus.publish(envelope)

    Subscribe path:
      IDeltaPort.subscribe(topics[])
        -> For each topic: IBus.subscribe(topic, handler)
        -> handler: deserialize payload, route to DeltaAggregator or FSMController

    Error handling:
      publish() catches all exceptions -> log + skip (fire-and-forget)
      subscribe() handler exceptions caught by IBus -> logged, never propagate
    """
```

25.3 Delta Bus: What Concierge Emits (Outbound)

All emitted via `IDeltaPort.publish()` at BACKGROUND priority. Complete catalog (also in Section 21.4):

DELTA TOPIC	TRIGGER	PAYLOAD SCHEMA	CONSUMERS
`k1.concierge.turn.started.v1`	turn_start()	`{session_id, turn_seq, trace_id, timestamp_ms}`	Learning Loop
`k1.concierge.turn.completed.v1`	turn_end() step 9	`{session_id, turn_seq, tier, latency_ms, tool_count, tokens, user_message, assistant_response}`	Memory Writer, Learning Loop
`k1.concierge.turn.failed.v1`	ABORT error path	`{session_id, turn_seq, error_code, adapter, severity, trace_id}`	Learning Loop, Alerting
`k1.concierge.turn.interrupted.v1`	Interrupt handling	`{session_id, old_turn_seq, new_message_hash}`	Learning Loop
`k1.concierge.phase1.complete.v1`	Phase 1 done	`{tier, safety_band, hypothesis_id, confidence, latency_ms}`	Dashboard
`k1.concierge.error.routed.v1`	ErrorRouter decision	`{adapter, operation, severity, action, trace_id}`	Alerting, Dashboard
`k1.concierge.cb.state_change.v1`	Any CB transition	`{cb_name, old_state, new_state, failure_count}`	Alerting, Dashboard
`k1.concierge.session.recovered.v1`	crash_recovery()	`{session_id, rolled_back_turn, restore_source}`	Alerting
`k1.concierge.crisis.detected.v1`	CRISIS override	`{session_id, keyword_match, safety_score}`	Safety Monitor
`k1.concierge.affective.update.v1`	Affect state change	`{session_id, emotional_state, intensity, valence}`	Dashboard
`k1.concierge.scoreboard.update.v1`	Task status change	`{session_id, task_id, new_status, progress_pct}`	Dashboard

25.4 Event Bus: What Concierge Publishes (Outbound)

Published via `IDeltaPort.publish()` at higher priority (INTERACTIVE). These are lifecycle and coordination events.

TOPIC	PRIORITY	TRIGGER	PAYLOAD	CONSUMERS
`k1.concierge.session.init.v1`	INTERACTIVE	init() complete	`{session_id, restore_source}`	Kernel telemetry
`k1.concierge.session.shutdown.v1`	INTERACTIVE	shutdown() complete	`{session_id, total_turns, duration_s}`	Kernel telemetry
`k1.concierge.ready.v1`	INTERACTIVE	Bootstrap complete	`{session_id}`	Kernel health check
`k1.concierge.checkpoint.v1`	BACKGROUND	turn_end() step 6	`{session_id, checkpoint_id}`	SessionState
`k1.hil.clarification_response.v1`	INTERACTIVE	User answers clarification	`{request_id, plan_id, user_response}`	Planner
`k1.hil.approval_response.v1`	INTERACTIVE	User approves/rejects plan	`{request_id, plan_id, response_type, modifications}`	Planner
`k1.hil.fallback_response.v1`	INTERACTIVE	User responds to fallback	`{request_id, trace_id, user_response}`	Orchestrator
`k1.hil.override_response.v1`	INTERACTIVE	User overrides execution	`{trace_id, override_action: cancel/modify/continue}`	Orchestrator

25.5 Event Bus: What Concierge Subscribes To (Inbound)

Subscriptions created at init() step 6. Handlers route events to FSMController or DeltaAggregator.

TOPIC PATTERN	SOURCE	PRIORITY	HANDLER	ACTION
`k1.hil.clarification.v1`	Planner	INTERACTIVE	FSMController	Write to PENDING_CLARIFICATIONS, interrupt if interruptible, route to CLARIFYING
`k1.hil.approval_request.v1`	Planner	INTERACTIVE	FSMController	Write to PENDING_CLARIFICATIONS, route to user via OutputManager
`k1.hil.fallback.v1`	Orchestrator	INTERACTIVE	FSMController	Write to PENDING_CLARIFICATIONS, route to user
`k1.hil.progress.v1`	Orchestrator	BACKGROUND	DeltaAggregator	Aggregate into 500ms batch, relay to user as progress
`k1.orchestration.dag.completed.v1`	Orchestrator	INTERACTIVE	TurnProcessor	Match trace_id, extract AggregatedResult, proceed to DELIVERING
`k1.orchestration.delta.v1`	Orchestrator	BACKGROUND	DeltaAggregator	Step-level progress for streaming
`k1.orchestration.step.completed.v1`	Orchestrator	BACKGROUND	DeltaAggregator	Per-step completion for progress narration
`k1.agent.*.delta.v1`	Sub-agents (wildcard)	BACKGROUND	DeltaAggregator	Sub-agent state deltas, LWW merge
`k0.proactive.signal.v1`	K0 Bridge	BACKGROUND	FSMController	Enqueue to Mailbox BACKGROUND priority, inject when LISTENING
`k1.fabric.agent.expired.v1`	Fabric	BACKGROUND	(log only)	TTL-expired agent cleanup notification

25.6 Delta vs Event Lane

ASPECT	DELTA LANE (`*.DELTA.V1`)	EVENT LANE (`K1.*` OTHER)
Delivery mode	BEST_EFFORT	STRICT or RELAXED
Guarantees	At-most-once, drop OK	At-most-once, ordered
Aggregation	500ms DeltaAggregator window	None (immediate handler)
WFQ priority	BACKGROUND (1x)	INTERACTIVE (2x) or REALTIME (3x)
Failure impact	Observability loss only	May affect turn processing
Persistence	None (ephemeral)	None (K1 bus is in-memory only)
Data volume	High frequency (per-step deltas)	Low frequency (lifecycle events)

25.7 HIL Event Flow (Complete)

26. Relationship to Other Components

26.1 Concierge <-> User (L0)

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Inbound	IInputPort	WebSocketInputAdapter	WebSocket binary (ADR-0040)	`UserMessage` (text, session_id, trace_id, metadata)
Outbound	IOutputPort	SSEOutputAdapter	SSE (ADR-0016)	9 `OutputEvent` types, 3 priorities

- **Pattern**: Request-response + streaming progress + proactive interrupts
- **Concierge is the ONLY user-facing component** – Orchestrator, Planner, Fabric, agents never talk to user directly
- **ALL output** goes through `OUTPUT_CHANNEL` (INV-15)
- **CB_SSE**: If output delivery fails, buffer up to 5 messages for reconnect
- **User interrupt**: detected at every `await` boundary in DISPATCHING/COMPANIONING/PROGRESSING
- Cross-reference: Section 14.2 (IInputPort), Section 14.3 (IOutputPort), Section 17.6 (INTERRUPT_HANDLING)

26.2 Concierge <-> Orchestrator (L2)

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Outbound (task)	IDispatchPort	FabricOrchestratorAdapter	K1 Bus Mailbox (Envelope)	`TaskEnvelope` (MEDIUM/HIGH tier)
Inbound (result)	IDeltaPort	DeltaBusAdapter (subscribe)	K1 Bus Envelope	`k1.orchestration.dag.completed.v1` (`AggregatedResult`)
Inbound (progress)	IDeltaPort	DeltaBusAdapter (subscribe)	K1 Bus Envelope	`k1.orchestration.delta.v1` , `k1.orchestration.step.*.v1`
Inbound (HIL)	IDeltaPort	DeltaBusAdapter (subscribe)	K1 Bus Envelope	`k1.hil.fallback.v1` , `k1.hil.progress.v1`
Outbound (HIL)	IDeltaPort	DeltaBusAdapter (publish)	K1 Bus Envelope	`k1.hil.fallback_response.v1` , `k1.hil.override_response.v1`

- **Orchestrator is a Blind DAG Executor** – NO LLM, NO tools. Receives `TaskEnvelope` , executes DAG, returns `AggregatedResult` .
- **CB_ORCHESTRATOR**: 60s timeout, 3 failures/60s -> OPEN. Fallback: degrade all requests to LOW tier.
- **TaskEnvelope contract**: From `k1/orchestrator/types.py` . MEDIUM: 1-2 capabilities required (ORCH-10). HIGH: capabilities may be empty (Planner decides).
- **TaskAck**: Synchronous acknowledgment (ACCEPTED/DUPLICATE/REJECTED_FULL). `REJECTED_FULL` counts as CB failure.
- **Tier degradation is Concierge-owned**: Orchestrator does NOT auto-degrade. Each degradation step creates a new `TaskEnvelope` with same `trace_id` .
- Cross-reference: Section 33 (connection detail), Section 18.5.4 (error recovery), Section 19.3 (CB inventory)

26.3 Concierge <-> Planner (L3)

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Indirect outbound	IDispatchPort	Via Orchestrator	`TaskEnvelope` tier=HIGH -> Orchestrator -> `IPlannerPort.request_plan()`	`PlanRequest` (Orchestrator constructs)
Inbound (HIL)	IDeltaPort	DeltaBusAdapter (subscribe)	K1 Bus Envelope	`k1.hil.clarification.v1` , `k1.hil.approval_request.v1`
Outbound (HIL)	IDeltaPort	DeltaBusAdapter (publish)	K1 Bus Envelope	`k1.hil.clarification_response.v1` , `k1.hil.approval_response.v1`

- **Concierge does NOT call Planner directly.** All interaction mediated via Orchestrator (HIGH tier) and HIL events (K1 Bus).
- **HIL routing**: Planner emits clarification/approval requests -> K1 Bus -> Concierge subscribe handler -> route to user -> user responds -> Concierge publishes response -> K1 Bus -> Planner.
- **PENDING_CLARIFICATIONS**: Concierge tracks active HIL requests keyed by `{request_id, originator, agent_id}` . Matched on user response via `HILResponseDetector` .
- **CB_PLANNER**: 45s timeout, 2 failures/min -> OPEN. Fallback: Orchestrator skips planning, direct execution (effectively MEDIUM degradation).
- Cross-reference: Section 32 (connection detail), Section 25.7 (HIL flow diagram)

26.4 Concierge <-> Fabric (L2.5)

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Outbound (LOW exec)	IDispatchPort	FabricOrchestratorAdapter	In-process `dispatch_direct(CapabilityRequest)`	`CapabilityResult`
Outbound (discovery)	IDispatchPort	FabricOrchestratorAdapter	In-process	`discover_capabilities(query)` -> ranked list

- **Dual role**: (1) LOW tier execution: Concierge calls Fabric directly via `IDispatchPort.dispatch_direct()`. (2) Capability discovery: LLM `discover_capabilities()` tool queries Fabric registry.
- **CB_FABRIC**: 30s timeout, 3 failures/60s -> OPEN. Fallback: mark all capabilities unavailable.
- **Fabric has 3 sub-roles** (from concierge.mmd): Intelligent Retrieval (serves Planner), Resolution + Execution (serves Orchestrator/Concierge), Agent Factory (spawns per-step workers).
- Cross-reference: Section 34 (connection detail), Section 10.12/10.13 (action tools)

26.5 Concierge <-> SessionState (L5)

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Read	IStatePort	SessionKernelAdapter	In-process	`read(sections[])` -> `Snapshot` (lock-free)
Write	IStatePort	SessionKernelAdapter	In-process via `MutationGuard.preflight()`	`write(section, op, data)` -> `WriteResult`
Checkpoint	IStatePort	SessionKernelAdapter	SQLite (LOCAL COLD)	`checkpoint()` at turn_end

- **Concierge is the ONLY writer** (CONC-01, ADR-0017/ADR-0018). All other components read via lock-free snapshots (< 1ms).
- **Sub-agents emit deltas** -> K1 Bus delta lane -> DeltaAggregator (500ms) -> Concierge single writer -> MutationGuard -> SessionState
- **CB_SESSIONSTATE**: 100ms timeout, 5 failures/60s -> OPEN. Fallback: stale cached read.
- **96KB total**: 48KB HOT (8 sections), 48KB WARM (4 sections). See Section 35 for full connection spec.
- Cross-reference: Section 35 (full connection), Section 13 (access pattern), Section 22 (lifecycle checkpoints)

26.6 Concierge <-> Model Hub (L4)

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Outbound	ILLMPort	ModelGatewayAdapter	K1 LLM Request Bus (capability-tagged)	`HubRequest{capability, payload, constraints}` -> `HubResponse`

- **Capability-tagged routing**: Requests tagged with capability: CHAT, TOOL_CALL, STRUCTURED, VISION, REASON. Model Hub routes to appropriate provider based on manifest.
- **CB_MODEL**: Per-provider CB inherited from Model Hub. Concierge-side 30s timeout, 5 failures/default -> OPEN. Fallback: L1 retry -> L2 CB probe -> L3 canned response (Section 18.5.2).
- **Prompt injection boundary**: LLM text is non-operative; side effects only via tool_call. Tool calls are allowlisted + schema-gated by ToolDispatcher.
- **Output Validation Pipeline** (3 tiers): T1 FORMAT (<1ms) -> T2 SAFETY (<5ms) -> T3 BELIEF CONSISTENCY (<50ms).

26.7 Concierge <-> K0 Bridge

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Outbound (recall)	IMemoryPort	BridgeRecallAdapter	Bridge HTTP/2 + FlatBuffers (ADR-0044)	`recall(query, selectors[])` -> `MemoryResult`
Outbound (store)	IMemoryPort	BridgeRecallAdapter	Bridge HTTP/2 + FlatBuffers	`store(session_snapshot)` (fire-and-forget at turn_end)
Inbound (proactive)	IDeltaPort (subscribe)	DeltaBusAdapter	K1 Bus (relayed from K0 SSE, ADR-0042)	`k0.proactive.signal.v1`

- **Edge-First design**: K0 Bridge is OPTIONAL. Concierge operates fully offline with LOCAL COLD (SQLite).
- **Graceful degradation**: If Bridge offline, `recall()` returns empty `MemoryResult` immediately. `store()` queues to Local Outbox (SQLite) for eventual sync.
- **No CB**: Bridge failures are handled via simple timeout + fallback (Section 18.5.6). Not worth CB overhead since it is already fire-and-forget.

26.8 Concierge <-> K1 Bus

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Publish (deltas)	IDeltaPort	DeltaBusAdapter	`IBus.publish(Envelope)`	11 delta topics + 8 event topics (Section 25.3, 25.4)
Subscribe (events)	IDeltaPort	DeltaBusAdapter	`IBus.subscribe(pattern, handler)`	10 subscription patterns (Section 25.5)

- **Single physical bus** (`LocalBus`), two logical lanes (event + delta). See Section 25.1.
- **Fire-and-forget**: Publish never blocks, never raises. Handler exceptions caught by bus.
- **Topic matching**: Exact match or prefix wildcard (e.g., `k1.agent.*.delta.v1`).
- **Tracing**: All envelopes carry `cognitive_trace_id` for cross-layer correlation.

26.9 Concierge <-> Rhythm Controller

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Read	(internal service)	(in-process)	Direct call	Rhythm patterns, conversation beat, dynamic timing

- **ConversationScheduler** (OutputManager sub-component) queries Rhythm Controller for pacing decisions.
- **Timing profiles**: Quick (200-500ms), Thoughtful (1-2s). Applied to inter-event delivery spacing.
- **Not a port boundary**: Rhythm Controller is an internal engine within the Concierge, not an external touchpoint. Listed here for completeness.

26.10 Concierge <-> Proactive Agent System

DIRECTION	PORT	ADAPTER	WIRE PROTOCOL	DATA
Inbound	IDeltaPort (subscribe)	DeltaBusAdapter	K1 Bus Envelope	`k1.proactive.question.v1`, `k1.proactive.message.v1`

- **Proactive signals** arrive via K1 Bus, enqueued to Concierge Mailbox at BACKGROUND priority.
- **Attention Budget Gate**: Token Bucket (3/day) limits proactive interrupts.
- **Injection**: Only when FSM is in LISTENING state. Otherwise buffered until turn completes.

27. Complete Event Catalog

27.1 Events Published by Concierge

#	TOPIC	PAYLOAD TYPE	PRIORITY	TRIGGER	CONSUMERS
1	`k1.concierge.session.init.v1`	`SessionInitEvent`	INTERACTIVE	init() complete	Kernel telemetry
2	`k1.concierge.session.shutdown.v1`	`SessionShutdownEvent`	INTERACTIVE	shutdown() complete	Kernel telemetry
3	`k1.concierge.ready.v1`	`ReadyEvent`	INTERACTIVE	Bootstrap ready	Kernel health
4	`k1.concierge.turn.started.v1`	`TurnStartedEvent`	BACKGROUND	turn_start()	Learning Loop
5	`k1.concierge.turn.completed.v1`	`TurnCompletedEvent`	BACKGROUND	turn_end() step 9	Memory Writer, Learning Loop
6	`k1.concierge.turn.failed.v1`	`TurnFailedEvent`	INTERACTIVE	ABORT error path	Learning Loop, Alerting
7	`k1.concierge.turn.interrupted.v1`	`TurnInterruptedEvent`	BACKGROUND	Interrupt handling	Learning Loop
8	`k1.concierge.phase1.complete.v1`	`Phase1CompleteEvent`	BACKGROUND	Phase 1 done	Dashboard
9	`k1.concierge.error.routed.v1`	`ErrorRoutedEvent`	BACKGROUND	ErrorRouter decision	Alerting, Dashboard
10	`k1.concierge.cb.state_change.v1`	`CBStateChangeEvent`	INTERACTIVE	CB transition	Alerting, Dashboard
11	`k1.concierge.session.recovered.v1`	`SessionRecoveredEvent`	INTERACTIVE	crash_recovery()	Alerting
12	`k1.concierge.crisis.detected.v1`	`CrisisDetectedEvent`	URGENT	CRISIS override	Safety Monitor
13	`k1.concierge.affective.update.v1`	`AffectiveUpdateEvent`	BACKGROUND	Affect state change	Dashboard
14	`k1.concierge.scoreboard.update.v1`	`ScoreboardUpdateEvent`	BACKGROUND	Task status change	Dashboard
15	`k1.concierge.checkpoint.v1`	`CheckpointEvent`	BACKGROUND	turn_end() step 6	SessionState
16	`k1.hil.clarification_response.v1`	`ClarificationResponse`	INTERACTIVE	User answers	Planner
17	`k1.hil.approval_response.v1`	`ApprovalResponse`	INTERACTIVE	User approves/rejects	Planner
18	`k1.hil.fallback_response.v1`	`FallbackResponse`	INTERACTIVE	User fallback reply	Orchestrator
19	`k1.hil.override_response.v1`	`OverrideResponse`	INTERACTIVE	User override	Orchestrator

27.2 Events Subscribed by Concierge

#	TOPIC PATTERN	SOURCE	PRIORITY	HANDLER	ACTION
1	`k1.hil.clarification.v1`	Planner	INTERACTIVE	FSMController	Route to user, write PENDING_CLARIFICATIONS
2	`k1.hil.approval_request.v1`	Planner	INTERACTIVE	FSMController	Route to user, write PENDING_CLARIFICATIONS
3	`k1.hil.fallback.v1`	Orchestrator	INTERACTIVE	FSMController	Route to user, write PENDING_CLARIFICATIONS
4	`k1.hil.progress.v1`	Orchestrator	BACKGROUND	DeltaAggregator	Aggregate, stream progress to user
5	`k1.orchestration.dag.completed.v1`	Orchestrator	INTERACTIVE	TurnProcessor	Match cognitive_trace_id, proceed to DELIVERING
6	`k1.orchestration.delta.v1`	Orchestrator	BACKGROUND	DeltaAggregator	Step-level progress
7	`k1.orchestration.step.completed.v1`	Orchestrator	BACKGROUND	DeltaAggregator	Per-step completion
8	`k1.agent.*.delta.v1`	Sub-agents	BACKGROUND	DeltaAggregator	State deltas, LWW merge
9	`k0.proactive.signal.v1`	K0 Bridge	BACKGROUND	FSMController	Enqueue proactive inject
10	`k1.fabric.agent.expired.v1`	Fabric	BACKGROUND	(log)	Agent TTL cleanup

27.3 Event Payload Schemas

```python
# --- Session lifecycle ---

@dataclass(frozen=True)
class SessionInitEvent:
    session_id: str
    restore_source: Literal["local_cold", "k0", "empty"]
    restore_duration_ms: int
    cb_states: Dict[str, str]            # {cb_name: "CLOSED"}

@dataclass(frozen=True)
class SessionShutdownEvent:
    session_id: str
    total_turns: int
    duration_s: float
    pending_writes: int                   # Outbox entries remaining

@dataclass(frozen=True)
class SessionRecoveredEvent:
    session_id: str
    rolled_back_turn: int
    restore_source: Literal["local_cold", "k0"]
    outbox_drained: int

@dataclass(frozen=True)
class ReadyEvent:
    session_id: str
    version: str                          # Concierge version

# --- Turn lifecycle ---

@dataclass(frozen=True)
class TurnStartedEvent:
    session_id: str
    turn_seq: int
    trace_id: str
    timestamp_ms: int

@dataclass(frozen=True)
class TurnCompletedEvent:
    session_id: str
    turn_seq: int
    trace_id: str
    tier: Literal["LOW", "MEDIUM", "HIGH"]
    latency_ms: int
    tool_count: int
    input_tokens: int
    output_tokens: int
    degradation_level: Literal["NONE", "MINOR", "MODERATE", "SEVERE"]

@dataclass(frozen=True)
class TurnFailedEvent:
    session_id: str
    turn_seq: int
    trace_id: str
    error_code: str
    adapter_name: str
    severity: str
    action: str
    user_message: str

@dataclass(frozen=True)
class TurnInterruptedEvent:
    session_id: str
    old_turn_seq: int
    new_message_hash: str
    trace_id: str

# --- Phase and routing ---
```

```python
@dataclass(frozen=True)
class Phase1CompleteEvent:
    session_id: str
    tier: Literal["LOW", "MEDIUM", "HIGH"]
    safety_band: str
    hypothesis_id: str
    confidence: float
    latency_ms: int

# --- Error and resilience ---

@dataclass(frozen=True)
class ErrorRoutedEvent:
    session_id: str
    adapter_name: str
    operation: str
    severity: str
    action: str
    trace_id: str

@dataclass(frozen=True)
class CBStateChangeEvent:
    session_id: str
    cb_name: str
    old_state: Literal["CLOSED", "OPEN", "HALF_OPEN"]
    new_state: Literal["CLOSED", "OPEN", "HALF_OPEN"]
    failure_count: int

@dataclass(frozen=True)
class CrisisDetectedEvent:
    session_id: str
    keyword_match: str
    safety_score: float
    trace_id: str

# --- State observability ---

@dataclass(frozen=True)
class AffectiveUpdateEvent:
    session_id: str
    emotional_state: str
    intensity: float
    valence: float

@dataclass(frozen=True)
class ScoreboardUpdateEvent:
    session_id: str
    task_id: str
    new_status: str
    progress_pct: float

@dataclass(frozen=True)
class CheckpointEvent:
    session_id: str
    checkpoint_id: str
    turn_seq: int

# --- HIL responses ---

@dataclass(frozen=True)
class ClarificationResponse:
    request_id: str
    plan_id: str
    user_response: str
    session_id: str

@dataclass(frozen=True)
class ApprovalResponse:
    request_id: str
    plan_id: str
    response_type: Literal["approved", "rejected", "modified"]
    modifications: Optional[Dict[str, Any]] = None

@dataclass(frozen=True)
class FallbackResponse:
    request_id: str
    trace_id: str
    user_response: str
    session_id: str

@dataclass(frozen=True)
class OverrideResponse:
    trace_id: str
    override_action: Literal["cancel", "modify", "continue"]
    session_id: str
    modifications: Optional[Dict[str, Any]] = None
```

28. Bootstrap & Kernel Integration

28.1 Kernel Bootstrap Context

Concierge is one of the managed components wired by the K1 Kernel bootstrap (`k1/kernel/kernel.md`). The kernel creates shared infrastructure first, then instantiates components in dependency order.

Kernel-level bootstrap order (from `k1/kernel/kernel.md` Section 3):

```
k1.kernel.bootstrap
  |
  +-- Phase 1: BusFactory.create_local(backend="auto") --> IBus
  +-- Phase 2: BusFactory.create_mailbox_router() --> IMailboxRouter
  +-- Phase 3: FabricFactory.create_with_ports(...) --> Fabric
  +-- Phase 4: SessionStateFactory.create_with_ports(...) --> SessionState
  +-- Phase 5: OrchestratorFactory.create_production(...) --> Orchestrator
  +-- Phase 6: PlannerFactory.create_production(...) --> Planner
  +-- Phase 7: ConciergeFactory.create_production(...) --> Concierge  [NEW]
  +-- Phase 8: Cross-wire (Orchestrator -> Planner mailbox, Concierge -> all)
```

28.2 Concierge Bootstrap Sequence (Phase 7 + 8)

```python
class ConciergeFactory:
    """Factory for creating Concierge instances.

    Called by Kernel bootstrap after all other components are ready.
    Concierge is LAST because it depends on all other components.
    """

    @staticmethod
    async def create_production(
        config: ConciergeConfig,
        bus: IBus,
        fabric: CapabilityFabric,
        session_manager: SessionStateManager,
        orchestrator_mailbox: IMailboxPort,
        model_hub: ModelHub,
        bridge_client: BridgeClient,
    ) -> Concierge:
        """
        Bootstrap steps (maps to Section 22.1 init()):

        Step 1: Create 8 production adapters (inject dependencies):
          a. WebSocketInputAdapter(transport_config)
          b. SSEOutputAdapter(transport_config)
          c. UltraBERTAdapter(model_path, device)
          d. ModelGatewayAdapter(model_hub)
          e. SessionKernelAdapter(session_manager)
          f. FabricOrchestratorAdapter(fabric, orchestrator_mailbox)
          g. DeltaBusAdapter(bus)
          h. BridgeRecallAdapter(bridge_client)

        Step 2: Create 9 internal services (wire dependency graph):
          FSMController -> TurnProcessor -> {IntentProcessor,
            ComplexityRouter, ToolDispatcher, ContextAssembler,
            OutputManager, DeltaAggregator, ClarificationTracker}

        Step 3: Connect 8 ports to adapters (FIFO order)

        Step 4: Restore state (Edge-First: LOCAL COLD first)

        Step 5: Init 7 circuit breakers -> all CLOSED

        Step 6: Start background tasks:
          - IDeltaPort subscriptions (10 topic patterns)
          - DeltaAggregator 500ms timer
          - IInputPort.begin_receive()

        Step 7: FSM -> LISTENING, emit ready event
        """
```

28.3 Adapter Dependency Injection Matrix

ADAPTER	EXTERNAL DEPENDENCY	PROVIDED BY KERNEL
WebSocketInputAdapter	Transport config	`config.transport`
SSEOutputAdapter	Transport config	`config.transport`
UltraBERTAdapter	Model checkpoint path, device	`config.ultrabert`
ModelGatewayAdapter	Model Hub instance	`model_hub` (Phase 3 output)
SessionKernelAdapter	SessionState manager	`session_manager` (Phase 4 output)
FabricOrchestratorAdapter	Fabric + Orchestrator mailbox	`fabric` (Phase 3), `orchestrator_mailbox` (Phase 5)
DeltaBusAdapter	IBus instance	`bus` (Phase 1 output)
BridgeRecallAdapter	Bridge client	`bridge_client` (external connector)

28.4 Shutdown Order (mirrors init in reverse)

```
async def shutdown_concierge(concierge: Concierge) -> None:
    """
    Shutdown sequence (Section 22.5):

    1. IInputPort.close() -- reject new messages
    2. Wait active turn (30s max) or force turn_end_abbreviated()
    3. Flush DeltaAggregator + OutputManager (final batches)
    4. Final checkpoint:
       - LOCAL COLD: synchronous SQLite write
       - K0: synchronous Bridge write (5s timeout, ONLY sync K0 call)
    5. Stop background tasks (cancel timer, unsubscribe, close SSE)
    6. Disconnect ports (LIFO order):
       IMemoryPort -> IDeltaPort -> IDispatchPort -> ILLMPort
       -> IClassificationPort -> IStatePort -> IOutputPort -> IInputPort
    7. Emit k1.concierge.session.shutdown.v1 (best-effort)
    8. Log session.shutdown INFO
    """
```

28.5 ConciergeFactory for Testing

```
class ConciergeFactory:
    @staticmethod
    async def create_for_testing(
        overrides: Optional[Dict[str, Any]] = None,
    ) -> Concierge:
        """In-memory Concierge with all 8 test adapters.

        Test adapters:
          TestInputAdapter       -- inject messages programmatically
          TestOutputAdapter      -- capture + assert on output events
          MockClassificationAdapter -- scripted ClassificationResult
          MockLLMAdapter         -- scripted LLM responses + tool calls
          InMemoryStateAdapter   -- dict-based, tracks all writes
          MockDispatchAdapter    -- captures envelopes + direct requests
          TestDeltaAdapter       -- emit + capture deltas
          MockMemoryAdapter      -- canned recall results

        Usage in tests:
          concierge = await ConciergeFactory.create_for_testing()
          concierge.input_adapter.inject("remind me to call mom")
          outputs = concierge.output_adapter.captured
          assert any(o.event_type == "acknowledge" for o in outputs)
        """
```

28.6 Health Check Endpoint

```
async def health_check() -> HealthStatus:
    """Kernel queries Concierge health as part of system readiness.

    Returns:
      HealthStatus:
        status: "healthy" | "degraded" | "unhealthy"
        details:
          fsm_state: current FSM state
          cb_summary: {cb_name: state} for all 7 CBs
          active_turn: bool
          session_age_s: float
          last_turn_latency_ms: int
          hot_tier_bytes: int

    Rules:
      healthy:   all CBs CLOSED, FSM responsive
      degraded:  any CB OPEN or HOT tier > 45KB
      unhealthy: FSM stuck (no transition in 120s) or SessionState unreadable
    """
```

29. Directory Structure

29.1 Current State (as of 2026-02-16)

```
k1/concierge/
    __init__.py
    README.md
    concierge.md               # This specification document (9000+ lines)
    concierge.mmd              # Authoritative architecture diagram (1256 lines)
    concierge_depricated.md    # Legacy document (to be removed)
    affective/
        __init__.py            # Affective processing (skeleton)
    empathy/
        __init__.py            # Empathy engine (skeleton)
    rhythm/
        __init__.py            # Rhythm controller (skeleton)
    tools/
        __init__.py
        README.md              # Tool registry spec (189 lines)
```

29.2 Implementation Structure (Target)

```
k1/concierge/
    __init__.py                # Module exports: Concierge, ConciergeFactory
    README.md                  # Module overview (13-section format)
    concierge.md               # This specification
    concierge.mmd              # Architecture diagram

    core/
        __init__.py
        concierge.py           # Concierge top-level class (wires services + ports)
        factory.py             # ConciergeFactory (production + testing)
        fsm_controller.py      # FSMController service (~60 tests)
        turn_processor.py      # TurnProcessor service (~90 tests)
        states.py              # FSMState enum (8 states + 4 experience states)
        transitions.py         # Transition table (state, event) -> (target, guard, action)

    processing/
        __init__.py
        intent_processor.py    # IntentProcessor service (~110 tests)
        complexity_router.py   # ComplexityRouter service (~45 tests)
        hypothesis.py          # Hypothesis generation
        gap_detection.py       # Information gap detection
        write_elision.py       # Write Elision Gate (signal significance scoring)
        safety_gate.py         # CRISIS detection (hard stop, first check)

    engines/
        __init__.py
        ultrabert.py           # UltraBERT v4 inference wrapper (12 heads)
        temporal.py            # Temporal resolution engine (NER -> absolute times)
        spatial.py             # Spatial context engine (ADR-0085/0085a)
        persona.py             # PersonaEngine (personality config + style)

    tools/
        __init__.py
        README.md
        dispatcher.py          # ToolDispatcher service (~80 tests)
        cognitive.py           # 6 cognitive tools (update_scoreboard, update_beliefs, etc.)
        read.py                # 3 read tools (recall_memory, discover_capabilities, summarize_context)
        action.py              # 3 action tools (invoke_capability, spawn_via_fabric, execute_workflow)
        signal.py              # 1 signal tool (acknowledge_request)
        allowlist.py           # Per-state/tier/safety tool allowlist

    output/
        __init__.py
        manager.py             # OutputManager service (~55 tests)
        delta_aggregator.py    # DeltaAggregator service (~50 tests)
        events.py              # OutputEvent dataclass (9 types, 3 priorities)
        scheduler.py           # ConversationScheduler (Rhythm integration)

    state/
        __init__.py
        clarification.py       # ClarificationTracker service (~40 tests)
        context_assembler.py   # ContextAssembler service (~55 tests)
        hil_detector.py        # HILResponseDetector (PENDING_CLARIFICATIONS matching)

    experience/
        __init__.py
        emotional.py           # Emotional processing (turn % 25)
        narrative.py           # Narrative weaving (turn % 20)
        anticipation.py        # Anticipatory response (turn % 30)
        affective_mirror.py    # Affective mirroring engine

    ports/
        __init__.py
        input.py               # IInputPort Protocol
        output.py              # IOutputPort Protocol
        classification.py      # IClassificationPort Protocol
        llm.py                 # ILLMPort Protocol
        state.py               # IStatePort Protocol
        dispatch.py            # IDispatchPort Protocol
        delta.py               # IDeltaPort Protocol
        memory.py              # IMemoryPort Protocol

    adapters/
        __init__.py
        production/
            __init__.py
            ws_input.py        # WebSocketInputAdapter (IInputPort)
            sse_output.py      # SSEOutputAdapter (IOutputPort) + CB_SSE
            ultrabert.py       # UltraBERTAdapter (IClassificationPort) + CB_CLASSIFICATION
            model_gateway.py   # ModelGatewayAdapter (ILLMPort) + CB_MODEL
            session_kernel.py  # SessionKernelAdapter (IStatePort) + CB_SESSIONSTATE
            fabric_orchestrator.py  # FabricOrchestratorAdapter (IDispatchPort) + CB_ORCHESTRATOR/PLANNER/FABRIC
            delta_bus.py       # DeltaBusAdapter (IDeltaPort)
            bridge_recall.py   # BridgeRecallAdapter (IMemoryPort)
        test/
            __init__.py
            test_input.py      # TestInputAdapter
            test_output.py     # TestOutputAdapter
            mock_classification.py  # MockClassificationAdapter
            mock_llm.py        # MockLLMAdapter
            memory_state.py    # InMemoryStateAdapter
            mock_dispatch.py   # MockDispatchAdapter
            test_delta.py      # TestDeltaAdapter
            mock_memory.py     # MockMemoryAdapter

    errors/
        __init__.py
        types.py               # ConciergeErrorSeverity, ConciergeAdapterError, ErrorDecision
        router.py              # ConciergeErrorRouter (stateless classification matrix)
        canned_responses.py    # CANNED_RESPONSES static store (7 responses by call_type)

    circuit_breakers/
        __init__.py
        registry.py            # CircuitBreakerRegistry (session-scoped, 7 CBs)
        configs.py             # Per-CB config dataclasses

    types/
```

```
    types/
        __init__.py
        messages.py             # UserMessage, OutputEvent, DeliveryReceipt
        classification.py       # ClassificationResult, Phase1Result, IntentHypothesis, InformationGap
        state.py                # HotTierSnapshot, WarmTierSnapshot, TurnContext, TurnSummary
        errors.py               # ConciergeError, ConciergeAdapterError
        events.py               # All event payload dataclasses (Section 27.3)
        enums.py                # Tier, SafetyBand, FSMState, OutputPriority

    contracts/
        concierge.yaml          # Module contract (k1/contracts/modules/ format)
        events.yaml             # Event schemas (all 19 published topics)
```

29.3 Test File Structure (~585 Tests)

```
tests/k1/concierge/
    __init__.py
    conftest.py                         # Shared fixtures, ConciergeFactory.create_for_testing()

    unit/
        test_fsm_controller.py          # ~60 tests (8 states, transitions, guards, experience)
        test_turn_processor.py          # ~90 tests (turn lifecycle, lock, phase coordination)
        test_intent_processor.py        # ~110 tests (pipeline stages, safety gate, gap detection)
        test_complexity_router.py       # ~45 tests (5-factor scoring, tier thresholds)
        test_tool_dispatcher.py         # ~80 tests (13 tools, allowlist, budget enforcement)
        test_output_manager.py          # ~55 tests (9 events, 3 priorities, queue depth)
        test_delta_aggregator.py        # ~50 tests (500ms window, LWW merge, flush)
        test_clarification_tracker.py   # ~40 tests (3-round limit, escalation, pending HIL)
        test_context_assembler.py       # ~55 tests (budget profiles, summarization trigger, token tracking)

    unit/errors/
        test_error_router.py            # ~30 tests (classification matrix, all 19 error codes)
        test_circuit_breaker_registry.py # ~20 tests (7 CBs, health summary, reset)

    unit/tools/
        test_cognitive_tools.py         # ~40 tests (6 cognitive tools, MutationGuard integration)
        test_read_tools.py              # ~20 tests (3 read tools, memory recall, discovery)
        test_action_tools.py            # ~30 tests (3 action tools, capability invocation)
        test_signal_tool.py             # ~10 tests (acknowledge, skip on trivial)

    unit/experience/
        test_emotional.py               # ~15 tests (trajectory analysis, tone adjustment)
        test_narrative.py               # ~15 tests (thread clustering, arc updates)
        test_anticipation.py            # ~15 tests (predictive completion, prefetch)

    integration/
        test_turn_flow.py               # End-to-end turn: input -> phase1 -> phase2 -> output
        test_tier_routing.py            # LOW/MEDIUM/HIGH routing with mock adapters
        test_error_recovery.py          # CB trips, tier degradation cascade, canned responses
        test_lifecycle.py               # init -> turns -> shutdown -> crash_recovery
        test_hil_flow.py                # Planner clarification -> user -> response routing
        test_interrupt.py               # Mid-turn interrupt detection and handling
```

29.4 Implementation Phases

PHASE	SCOPE	KEY FILES	ESTIMATED TESTS
1	FSM Core	`core/` (fsm_controller, states, transitions)	~60
2	Turn Processing	`core/turn_processor.py`, `processing/intent_processor.py`	~200
3	Routing	`processing/complexity_router.py`, `processing/hypothesis.py`	~45
4	Tool System	`tools/` (dispatcher + 13 tools + allowlist)	~80
5	Output + Delta	`output/` (manager, delta_aggregator, scheduler)	~105
6	State Management	`state/` (clarification, context_assembler, hil_detector)	~95
7	Ports + Adapters	`ports/` (8 protocols) + `adapters/production/` (8 adapters)	~100
8	Error + CB	`errors/` + `circuit_breakers/`	~50
9	Experience	`experience/` (emotional, narrative, anticipation)	~45
10	Integration	`tests/integration/` (6 test files)	~50

30. Open Questions for Design Sessions

30.1 Resolved Questions

ID	QUESTION	RESOLUTION	DECIDED IN
Q1	Single writer vs multi-writer SessionState?	Single writer (CONC-01, ADR-0017)	ADR-0017
Q2	UltraBERT latency target?	22ms P50, 25ms P99	Section 11
Q3	Max clarification rounds?	3 per intent (CONC-08)	Section 8, concierge.mmd
Q4	Delta batch window?	500ms fixed (CONC-10)	Section 16.8
Q5	How does Concierge talk to Planner?	Indirectly via Orchestrator + HIL events. No direct port.	Section 32, concierge.mmd
Q6	Circuit breaker library?	Reuse `k1/fabric/circuit_breaker/breaker.py` (3-state, sliding window)	Section 19
Q7	Persona configuration location?	SessionState WARM tier (`persona` section, 8KB)	Section 35, concierge.mmd
Q8	Turn interrupt model?	Cooperative cancellation with CancellationToken, checked at await boundaries	Section 23.3
Q9	Offline-first SessionState sync?	Edge-First: LOCAL COLD always available, K0 sync fire-and-forget	Section 22, concierge.mmd
Q10	Multi-session concurrency?	One asyncio event loop per session, full isolation (Section 23.5)	Section 23
Q11	How many CBs?	7: CB_CLASSIFICATION, CB_MODEL, CB_SESSIONSTATE, CB_ORCHESTRATOR, CB_PLANNER, CB_FABRIC, CB_SSE	Section 19.3
Q12	Experience layer frequency?	Heuristic: narrative %20, emotional %25, anticipatory %30 turns	Section 9, concierge.mmd

30.2 Open Questions

ID	QUESTION	CONTEXT	OPTIONS	IMPACT
Q13	UltraBERT deployment topology?	Is UltraBERT loaded per-process or shared microservice? In-process gives <1ms overhead. Shared service adds network latency (~5ms) but saves GPU memory for multi-session.	Per-process / Shared service	Phase 7 adapter implementation
Q14	Experience layer cross-session learning?	Should narrative/emotional patterns transfer between sessions for the same user? Requires WARM tier persistence beyond session lifetime.	Per-session only / Cross-session via K0	Phase 9 experience layer
Q15	Proactive agent injection policy?	When exactly should proactive messages interrupt an active conversation? Current: only LISTENING state. Could also inject during natural pauses (>5s no input).	LISTENING only / Natural pause / User-configurable	Proactive system integration
Q16	Sub-agent sandbox model?	Should sub-agents (spawned via Fabric) run in the same asyncio loop or separate process? Same loop is simpler but less isolated. Separate process is safer but adds IPC overhead.	Same loop / Separate process / WASM sandbox	Section 34, Fabric integration
Q17	Multi-modal input handling?	WebSocket binary protocol supports images (ADR-0040). How should IInputPort handle vision + text mixed messages? UltraBERT is text-only; vision requires ILLMPort with VISION capability.	Text-only Phase 1 / Vision-aware Phase 1	UltraBERT + IInputPort design
Q18	Canned response personalization?	Currently 7 static canned responses (Section 18). Should they incorporate persona/style from WARM tier? Adds latency (~2ms for persona read) but feels more natural.	Static / Persona-aware / TBD	Error recovery UX
Q19	Foreground/Background turn split?	Should the Concierge allow micro-turns while a MED/HIGH task runs in background? Requires CONC-15 revision, new BACKGROUND_WORKING FSM state, lock-aware DeltaAggregator buffering.	Single-turn only / FG+BG split / TBD	Section 23.2.1, UX for long tasks
Q20	Parallel tool call safety boundary?	ADR-0078 defines tiered parallelism. Should action tools be parallelized with reads, or only reads? Action tools have external side-effects but no SS writes.	Reads only / Reads+Actions / Full parallel	Section 7.5.1, ToolDispatcher

30.3 Enhancement Roadmap

Enhancements identified through architectural brainstorming (see Sections 1.5.1, 7.5.1, 23.2.1).

ENHANCEMENT	COMPLEXITY	VALUE	TIMELINE	REFERENCE
Parallel read tools (`asyncio.gather` reads)	LOW	20-50% faster read-heavy turns	Now – ADR-0078 already exists	Section 7.5.1
Sub-Agent vs Dynamic Agent editorial clarification	TRIVIAL	Clarity for implementors	Now – editorial change	Section 1.5.1
Parallel read + action tools	MEDIUM	40-60% faster MED/HIGH turns with 3-4 tools	Next sprint	Section 7.5.1, Q20
Foreground/Background turn split	HIGH	Transformative UX for 30-60s tasks	Design phase – needs ADR-0097	Section 23.2.1, Q19
Speculative multi-intent branching	VERY HIGH	Marginal gain (correct intent 85%+ already)	Future – not worth complexity now	N/A

30.4 Design Session Backlog

SESSION	TOPIC	SECTIONS AFFECTED	PRIORITY
DS-01	UltraBERT deployment topology (Q13)	11, 15, 28	HIGH (blocks Phase 7)
DS-02	Sub-agent sandbox model (Q16)	34, 23	MEDIUM
DS-03	Multi-modal input (Q17)	6, 14, 15	LOW (future)
DS-04	Experience cross-session (Q14)	9, 35	LOW (Phase 9)

31. Type Ownership Reference

31.1 Shared Types (owned by other K1 modules)

TYPE	OWNER MODULE	LOCATION	USED BY CONCIERGE IN
`TaskEnvelope`	Orchestrator	`k1/orchestrator/types.py`	FabricOrchestratorAdapter.dispatch_envelope()
`TaskAck`	Orchestrator	`k1/orchestrator/types.py`	FabricOrchestratorAdapter (return from enqueue)
`AggregatedResult`	Orchestrator	`k1/orchestrator/types.py`	TurnProcessor (from dag.completed.v1 event)
`StepResult`	Orchestrator	`k1/orchestrator/types.py`	TurnProcessor (embedded in AggregatedResult)
`CapabilityResult`	Fabric	`k1/fabric/types.py`	ToolDispatcher (from dispatch_direct return)
`CapabilityRequest`	Fabric	`k1/fabric/types.py`	FabricOrchestratorAdapter.dispatch_direct()
`Envelope`	Bus	`k1/bus/envelope/envelope.py`	DeltaBusAdapter (wraps all bus messages)
`Priority`	Bus	`k1/bus/envelope/envelope.py`	DeltaBusAdapter (WFQ scheduling)
`SubscriptionHandle`	Bus	`k1/bus/ports/bus.py`	DeltaBusAdapter (track active subscriptions)
`IBus`	Bus	`k1/bus/ports/bus.py`	DeltaBusAdapter (structural Protocol)
`DeltaPayload`	Fabric	`k1/fabric/ports/delta_bus.py`	DeltaBusAdapter (delta emission)
`CircuitBreaker`	Fabric	`k1/fabric/circuit_breaker/breaker.py`	All production adapters (resilience)
`CircuitBreakerConfig`	Fabric	`k1/fabric/circuit_breaker/breaker_config.py`	CircuitBreakerRegistry (per-CB config)
`HubRequest`	Model Hub	`k1/model_hub/types.py`	ModelGatewayAdapter.execute()
`HubResponse`	Model Hub	`k1/model_hub/types.py`	ModelGatewayAdapter (return)

31.2 Concierge-Owned Types

TYPE	LOCATION	CATEGORY	USED IN
`UserMessage`	`k1/concierge/types/messages.py`	Input	IInputPort, TurnProcessor
`OutputEvent`	`k1/concierge/types/messages.py`	Output	IOutputPort, OutputManager
`DeliveryReceipt`	`k1/concierge/types/messages.py`	Output	IOutputPort return
`StreamReceipt`	`k1/concierge/types/messages.py`	Output	IOutputPort.send_stream return
`ClassificationResult`	`k1/concierge/types/classification.py`	Phase 1	IClassificationPort, IntentProcessor
`ClassificationContext`	`k1/concierge/types/classification.py`	Phase 1	IClassificationPort
`Phase1Result`	`k1/concierge/types/classification.py`	Phase 1	TurnProcessor, ComplexityRouter
`IntentHypothesis`	`k1/concierge/types/classification.py`	Phase 1	IntentProcessor
`InformationGap`	`k1/concierge/types/classification.py`	Phase 1	IntentProcessor, ClarificationTracker
`WriteElisionDecision`	`k1/concierge/types/classification.py`	Phase 1	Write Elision Gate
`ResolvedTime`	`k1/concierge/types/classification.py`	Phase 1	Temporal resolution engine
`ResolvedLocation`	`k1/concierge/types/classification.py`	Phase 1	Spatial context engine
`FSMState`	`k1/concierge/types/enums.py`	Core	FSMController (8 core + 4 experience)
`Tier`	`k1/concierge/types/enums.py`	Core	ComplexityRouter, TurnProcessor
`SafetyBand`	`k1/concierge/types/enums.py`	Core	SafetyGate, IntentProcessor
`OutputPriority`	`k1/concierge/types/enums.py`	Output	OutputManager, OutputEvent
`HotTierSnapshot`	`k1/concierge/types/state.py`	State	TurnProcessor (read at turn_start)
`TurnContext`	`k1/concierge/types/state.py`	State	TurnProcessor (per-turn context)
`TurnSummary`	`k1/concierge/types/state.py`	State	turn_end() return
`ConciergeErrorSeverity`	`k1/concierge/errors/types.py`	Error	ErrorRouter, all adapters
`ConciergeAdapterError`	`k1/concierge/errors/types.py`	Error	All production adapters
`ConciergeErrorAction`	`k1/concierge/errors/types.py`	Error	ErrorRouter
`ErrorDecision`	`k1/concierge/errors/types.py`	Error	ErrorRouter return
`ConciergeErrorRouter`	`k1/concierge/errors/router.py`	Error	TurnProcessor (route_error)
`CancellationToken`	`k1/concierge/core/turn_processor.py`	Concurrency	TurnProcessor, ToolDispatcher
`TransitionResult`	`k1/concierge/core/fsm_controller.py`	Core	FSMController.transition()
`InterruptDecision`	`k1/concierge/core/fsm_controller.py`	Core	FSMController.on_interrupt()

31.3 Event Payload Types (Concierge-Owned)

TYPE	LOCATION	PUBLISHED ON
`SessionInitEvent`	`k1/concierge/types/events.py`	`k1.concierge.session.init.v1`
`SessionShutdownEvent`	`k1/concierge/types/events.py`	`k1.concierge.session.shutdown.v1`
`SessionRecoveredEvent`	`k1/concierge/types/events.py`	`k1.concierge.session.recovered.v1`
`ReadyEvent`	`k1/concierge/types/events.py`	`k1.concierge.ready.v1`
`TurnStartedEvent`	`k1/concierge/types/events.py`	`k1.concierge.turn.started.v1`
`TurnCompletedEvent`	`k1/concierge/types/events.py`	`k1.concierge.turn.completed.v1`
`TurnFailedEvent`	`k1/concierge/types/events.py`	`k1.concierge.turn.failed.v1`
`TurnInterruptedEvent`	`k1/concierge/types/events.py`	`k1.concierge.turn.interrupted.v1`
`Phase1CompleteEvent`	`k1/concierge/types/events.py`	`k1.concierge.phase1.complete.v1`
`ErrorRoutedEvent`	`k1/concierge/types/events.py`	`k1.concierge.error.routed.v1`
`CBStateChangeEvent`	`k1/concierge/types/events.py`	`k1.concierge.cb.state_change.v1`
`CrisisDetectedEvent`	`k1/concierge/types/events.py`	`k1.concierge.crisis.detected.v1`
`AffectiveUpdateEvent`	`k1/concierge/types/events.py`	`k1.concierge.affective.update.v1`
`ScoreboardUpdateEvent`	`k1/concierge/types/events.py`	`k1.concierge.scoreboard.update.v1`
`CheckpointEvent`	`k1/concierge/types/events.py`	`k1.concierge.checkpoint.v1`
`ClarificationResponse`	`k1/concierge/types/events.py`	`k1.hil.clarification_response.v1`
`ApprovalResponse`	`k1/concierge/types/events.py`	`k1.hil.approval_response.v1`
`FallbackResponse`	`k1/concierge/types/events.py`	`k1.hil.fallback_response.v1`
`OverrideResponse`	`k1/concierge/types/events.py`	`k1.hil.override_response.v1`

31.4 Import Dependency Map

```
k1.concierge.types.*          (0 external deps -- pure dataclasses)
    |
k1.concierge.errors.*         (depends on: types.enums)
    |
k1.concierge.ports.*          (depends on: types.*, errors.types)
    |
k1.concierge.core.*           (depends on: ports.*, types.*, errors.*)
    |
k1.concierge.processing.*     (depends on: types.*, ports.state)
    |
k1.concierge.tools.*          (depends on: types.*, ports.*)
    |
k1.concierge.output.*         (depends on: types.*, ports.output, ports.delta)
    |
k1.concierge.state.*          (depends on: types.*, ports.state)
    |
k1.concierge.experience.*     (depends on: types.*, ports.state, ports.dispatch)
    |
k1.concierge.adapters.production.*  (depends on: ports.*, types.*, errors.*)
    |                                (depends on: k1.bus, k1.fabric, k1.orchestrator, k1.model_hub)
    |
k1.concierge.adapters.test.*      (depends on: ports.*, types.* ONLY -- no external K1 deps)
```

Rule: Pure Concierge code (everything except `adapters/production/`) has ZERO imports from other K1 modules. All external dependencies are isolated behind ports. Test code has zero external K1 dependencies (uses test adapters only).

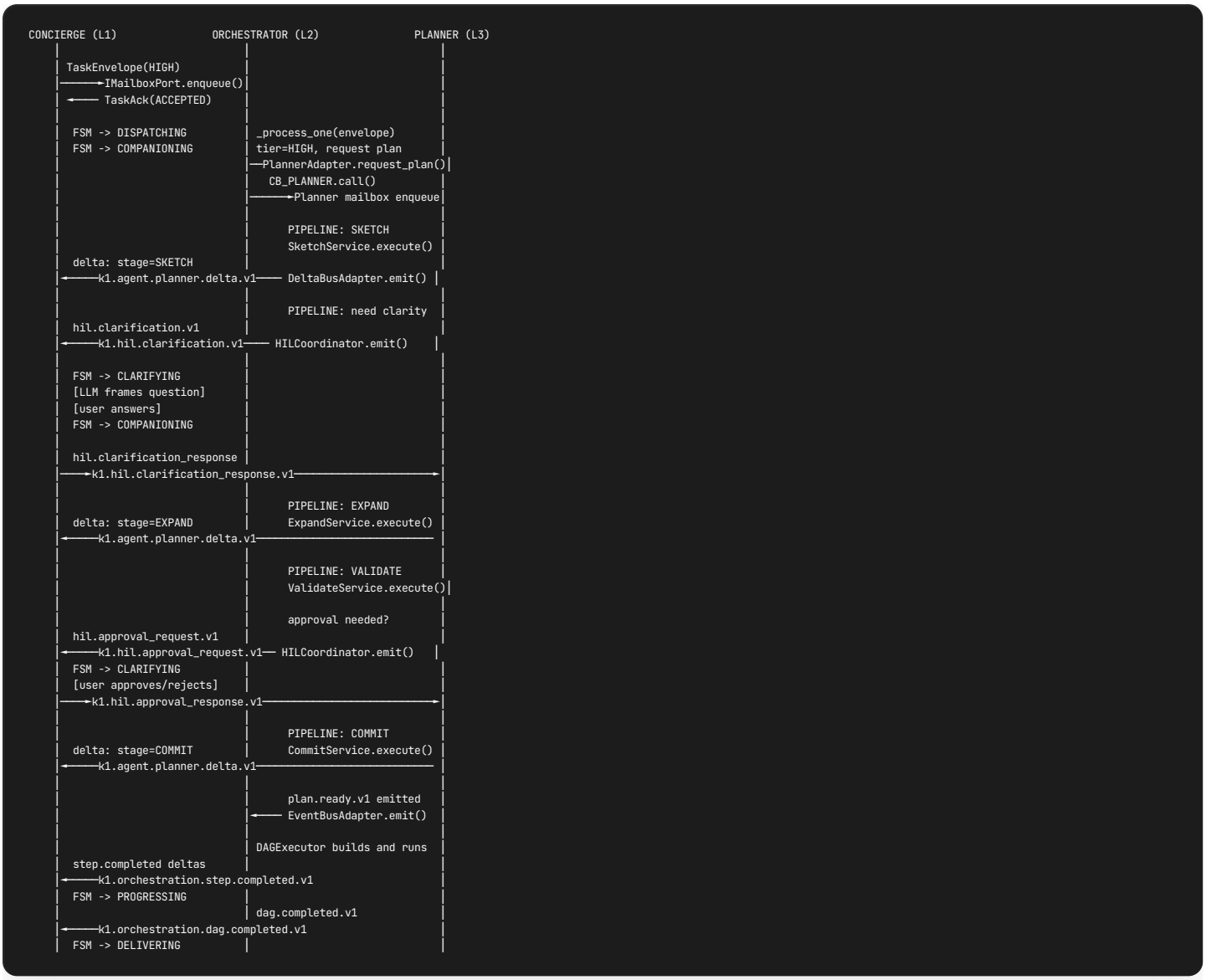
32. Concierge <-> Planner Connection

32.1 Overview

Concierge does NOT call Planner directly. All interaction is mediated via Orchestrator and the K1 Bus event/delta lanes. The Planner is a downstream consumer inside the Orchestrator DAG pipeline; Concierge has zero import-time dependency on `k1.planner.*`. The only data flowing between them is through bus events.

Architectural invariant: Concierge NEVER enqueues into the Planner mailbox. Only Orchestrator's `PlannerAdapter(planner_mailbox, cb_planner)` does that (kernel.md Phase 5b cross-wiring).

32.2 Interaction Pattern



32.3 HIL Routing (Planner-Originated)

EVENT	DIRECTION	BUS TOPIC	CONCIERGE HANDLER	FSM IMPACT
Clarification question	Planner → Concierge	`k1.hil.clarification.v1`	`ClarificationTracker.on_clarification()`	COMPANIONING → CLARIFYING
Clarification response	Concierge → Planner	`k1.hil.clarification_response.v1`	`ClarificationTracker.respond()` → `bus.publish()`	CLARIFYING → COMPANIONING
Approval request	Planner → Concierge	`k1.hil.approval_request.v1`	`ClarificationTracker.on_approval_request()`	COMPANIONING → CLARIFYING
Approval response	Concierge → Planner	`k1.hil.approval_response.v1`	`ClarificationTracker.respond()` → `bus.publish()`	CLARIFYING → COMPANIONING

HIL Timeout: 120s (OrchestratorConfig.hil_timeout_ms). If Concierge does not respond within 120s, the Orchestrator Reaper evicts the pending HIL context and the step fails with `HIL_TIMEOUT`. Concierge must track this timeout locally and prompt the user before it expires.

32.4 Planner Delta Consumption

Concierge subscribes to `k1.agent.planner.delta.v1` to track Planner pipeline stage progress:

```python
# ConcierDeltaBusAdapter subscription (from S25)
async def _on_planner_delta(self, envelope: Envelope) -> None:
    delta = json.loads(envelope.payload)
    stage = delta.get("stage")    # SKETCH, EXPAND, VALIDATE, COMMIT
    status = delta.get("status")  # STARTED, COMPLETED, FAILED

    if status == "STARTED":
        await output_manager.send_companioning(
            f"Planning step: {stage.lower()}..."
        )
    elif status == "FAILED":
        # Planner will emit plan.failed.v1 separately
        await output_manager.send_companioning(
            "Hmm, let me try a different approach..."
        )
```

Delta payload (from PlannerDeltaBusAdapter, agent_id="planner"):

FIELD	TYPE	DESCRIPTION
`stage`	`str`	`SKETCH`, `EXPAND`, `VALIDATE`, `COMMIT`
`status`	`str`	`STARTED`, `COMPLETED`, `FAILED`
`delta_type`	`str`	`"stage_transition"`
`section`	`str`	`"planner"`
`data`	`dict`	Stage-specific metadata (request_id, error info)

32.5 Planner Failure Handling

PLANNER EVENT	BUS TOPIC	CONCIERGE ACTION
`plan.ready.v1`	`k1.planner.plan.ready.v1`	N/A (Orchestrator handles internally)
`plan.failed.v1`	`k1.planner.plan.failed.v1`	N/A (Orchestrator emits `dag.completed.v1` with `success: false`)
`plan.cancelled.v1`	`k1.planner.plan.cancelled.v1`	N/A (Orchestrator emits `dag.completed.v1` with `success: false`)

Concierge does NOT directly subscribe to `plan.ready/failed/cancelled`. Those are consumed by Orchestrator (init step 8 subscriptions). Concierge only sees the final `dag.completed.v1` result. On failure:

1. `dag.completed.v1` arrives with `success: false`
2. Concierge checks error code for `PLANNER_TIMEOUT` or `PLAN_FAILED`
3. **Tier degradation** (Concierge-owned): HIGH → MEDIUM re-submission with pre-resolved capabilities
4. If MEDIUM also fails → LOW → Canned (see S33.4)

32.6 Concierge Timing During Planner Pipeline

PHASE	DURATION	CONCIERGE STATE	USER EXPERIENCE
TaskEnvelope dispatch	<5ms	DISPATCHING	–
Planner queued	0-100ms	COMPANIONING	`acknowledge()` displayed
SKETCH stage	2-10s	COMPANIONING	"Working on a plan..." + optional proactive question
EXPAND stage	2-10s	COMPANIONING	"Putting the details together..."
VALIDATE stage	1-5s	COMPANIONING / CLARIFYING	May prompt for approval
COMMIT stage	<1s	COMPANIONING	–
DAG execution	5-30s	PROGRESSING	Step-by-step deltas
Total	**10-60s**	–	Proactive Agent fills idle time (S9.7)

32.7 Planner K0 Boundary (Transparent to Concierge)

Planner has its own `IBridgePort` – Concierge does NOT mediate this. The Planner's bridge access is entirely internal to the Planner module:

IBRIDGEPORT METHOD	DIRECTION	PURPOSE	CONCIERGE INVOLVEMENT
`recall(query)`	K1 → K0 → K1	Long-term memory for plan grounding (SKETCH stage `recall_for_planning()` tool)	NONE – Planner calls this directly via its own adapter
`persist_plan(frozen_plan)`	K1 → K0 (fire-and-forget)	WAL write to K0 P02 for plan durability (COMMIT stage)	NONE – happens inside CommitService

Planner does NOT do gap detection. When the Planner is confused about intent, it uses its own `HILCoordinator` to ask the user via the K1 Bus (Section 32.3). This is a pure **K1-internal** flow – no K0 involvement:

```
Planner confused about intent (SKETCH stage):
  → HILCoordinator.request_clarification(request_id, context)
  → IEventPort.emit("k1.hil.clarification.v1", payload)
  → K1 Bus (event lane)
  → Concierge ClarificationTracker.on_clarification()
  → Concierge FSM: COMPANIONING -> CLARIFYING
  → LLM frames question naturally for user
  → User responds
  → Concierge emits "k1.hil.clarification_response.v1"
  → K1 Bus (event lane)
  → HILCoordinator._wait_for_response() receives correlated response
  → SketchService resumes with user answer
```

Gap detection and proactive questioning is a SEPARATE system owned by K0 pipelines P03/P06, not by the Planner:

```
K0 P03 Consolidation (background, async, not triggered by Planner):
  → GapDetector detects knowledge gaps during reconciliation (R4/R7)
  → 7 gap types: AMBIGUOUS_ENTITY, LOW_CONFIDENCE_EDGE, MISSING_ATTRIBUTE,
    CONTRADICTION, CONCEPT_DRIFT, STRUCTURAL_HOLE, STALE_ANCHOR
  → GapEmitter persists to st_learning_queue + emits p03.gap.detected.v1
  → K0 P06 Active Learning prioritizes gaps (importance_score)
  → K0 P05 Attention Manager checks token budget (3/day) + context readiness
  → P06 emits curiosity.intent.v1 via K0 SSE Port
  → Bridge SSEReceiver parses event
  → K1 Bus: k1.k0.sse.curiosity.intent.v1 (BEST_EFFORT)
  → Concierge Proactive Agent receives (Section 9.7)
  → Concierge decides if/when to inject question (only in LISTENING state or COMPANIONING during HIGH tier)
```

Key architectural boundary: Planner and K0 gap detection are **completely decoupled**. The Planner's `persist_plan()` writes to K0 P02, which may eventually feed P03 consolidation cycles, but this is a background process with no synchronous connection to planning. Concierge's Proactive Agent consumes K0 gap questions independently of any Planner interaction.

32.8 Concierge K0 Query Service Integration

Concierge has its own K0 boundary separate from Planner. This is the `IMemoryPort` (Section 14.9):

CONCIERGE K0 ACCESS	PORT	ADAPTER	BRIDGE PATH	K0 PIPELINE
Memory recall (read)	`IMemoryPort.recall()`	`BridgeRecallAdapter`	`IKernelQueryPort.query(selectors)` → `/k0/query.recall`	P01 (Recall/Read)
Belief promotion (write)	`IMemoryPort.store()`	`BridgeRecallAdapter`	`IKernelCommandPort.submit(topic="memory.delta")` → `/k0/command.submit`	P02 (Write/Ingest)
Memory prefetch	`IMemoryPort.prefetch()`	`BridgeRecallAdapter`	`IKernelQueryPort.query()` (fire-and-forget, cached 60s)	P01

Proactive signal consumption (K0 → Concierge):

SSE EVENT	K0 SOURCE	BRIDGE TOPIC	K1 BUS TOPIC	CONCIERGE HANDLER
`curiosity.intent.v1`	P06 Active Learning	SSE stream	`k1.k0.sse.curiosity.intent.v1`	ProactiveAgent.on_curiosity_intent()
`k0.proactive.signal.v1`	P05 Attention Manager	SSE stream	`k1.k0.sse.proactive.signal.v1`	ProactiveAgent.on_proactive_signal()
`k0.learning.advisory.v1`	P06 Active Learning	SSE stream	`k1.k0.sse.learning.advisory.v1`	(Learning Loop, not Concierge directly)
`memory.formed.v1`	P03 Consolidation	SSE stream	`k1.k0.sse.memory.formed.v1`	(Informational, no Concierge handler)

Gap resolution answer flow (user answers proactive question → K0):

```
User answers proactive question during COMPANIONING:
  → Concierge captures answer
  → IMemoryPort.store(MemoryItem(
        content=answer,
        correlation={"gap_id": original_gap_id},
        topic="memory.delta"
    ))
  → BridgeRecallAdapter → Bridge IKernelCommandPort
  → K0 /k0/command.submit(topic="memory.delta", body={gap_resolution_id: ...})
  → K0 P02 ingests answer with gap_resolution_id tag
  → K0 P03 next consolidation cycle: resolves gap in st_learning_queue
  → Closed-loop learning complete
```

Offline behavior: When K0 Bridge is unavailable, `BridgeRecallAdapter` falls back to LOCAL COLD tier (K1 SQLite). Proactive questions stop arriving (no SSE), but core Concierge functionality (Phase 1 + Phase 2 + LOW tier execution) is unaffected. Memory writes are queued in Bridge's `LocalOutbox` and drained when K0 reconnects.

33. Concierge <-> Orchestrator Connection

33.1 Overview

Orchestrator (L2) is the **pure deterministic execution coordinator** for MEDIUM and HIGH tier tasks. It has NO LLM, NO tools, and NEVER writes SessionState. Concierge is the Orchestrator's **sole inbound caller** for user-initiated tasks (Workflow Scheduler also feeds it, but not via Concierge). The Orchestrator's mailbox accepts `TaskEnvelope` messages and returns `TaskAck` synchronously; all results arrive asynchronously via K1 Bus events.

Key architectural constraints (from orchestrator.md):

- ORCH-01: Orchestrator NEVER writes SessionState
- ORCH-02: Orchestrator has ZERO LLM calls
- ORCH-03: Orchestrator has ZERO tools – it calls Fabric APIs directly
- ORCH-04: Every step execution goes through Fabric
- ORCH-10: MEDIUM tier: max 2 Fabric calls
- ORCH-11: HIGH tier: CommittedPlan required before execution

33.2 Port & Adapter Mapping

CONCIERGE PORT	ADAPTER	ORCHESTRATOR SURFACE	CB
`IDispatchPort.dispatch_envelope()`	`FabricOrchestratorAdapter`	`IMailboxPort.enqueue(TaskEnvelope, priority)`	`CB_ORCHESTRATOR` (Concierge-owned)
`IDeltaPort.subscribe()`	`DeltaBusAdapter`	K1 Bus subscription to `k1.orchestration.*` topics	None (bus-level)

CB_ORCHESTRATOR configuration:

PARAMETER	VALUE	SOURCE
Failure condition	`enqueue()` raises or returns `REJECTED_FULL`	cross-component-contracts.md
Threshold	3 consecutive failures within 60s	Concierge-owned
Half-open	After 30s, allow 1 probe call	Concierge-owned
Fallback	Degrade to LOW tier or Canned response	Section 33.6

33.3 TaskEnvelope Construction (Per Tier)

Concierge's `TurnProcessor` constructs the `TaskEnvelope` during the DISPATCHING FSM state:

```python
# MEDIUM tier: pre-resolved capabilities from Phase 2 LLM tool calls
envelope_medium = TaskEnvelope(
    intent=phase2_result.classified_intent,
    trace_id=current_turn.cognitive_trace_id,      # MUST be preserved
    caller_id=f"concierge-session-{session_id}",
    tier="MEDIUM",
    capabilities=phase2_result.resolved_capabilities,  # 1-2 items (ORCH-10)
    params=phase2_result.capability_params,             # per-capability params
    context={
        "beliefs_snapshot": hot_tier_snapshot.beliefs_active,
        "entities": hot_tier_snapshot.mentioned_entities,
        "temporal_context": hot_tier_snapshot.mentioned_time,
        "safety_band": current_safety_band,
        "persona_hints": warm_tier.persona if warm_tier else {},
    },
    timeout_ms=10000,   # MEDIUM budget: 10s total
)

# HIGH tier: capabilities left empty (Planner decides)
envelope_high = TaskEnvelope(
    intent=phase2_result.classified_intent,
    trace_id=current_turn.cognitive_trace_id,
    caller_id=f"concierge-session-{session_id}",
    tier="HIGH",
    capabilities=[],                                # Planner resolves
    params={},
    context={
        "beliefs_snapshot": hot_tier_snapshot.beliefs_active,
        "entities": hot_tier_snapshot.mentioned_entities,
        "temporal_context": hot_tier_snapshot.mentioned_time,
        "safety_band": current_safety_band,
        "persona_hints": warm_tier.persona if warm_tier else {},
        "user_preferences": warm_tier.preferences if warm_tier else {},
    },
    timeout_ms=45000,   # HIGH budget: 45s total (4 planner stages + DAG execution)
)
```

TaskEnvelope validation (enforced by `__post_init__`):

- `tier` must be `"MEDIUM"` or `"HIGH"` (LOW never reaches Orchestrator)
- `capabilities` must be non-empty for MEDIUM, 0+ for HIGH
- `len(capabilities) <= 2` for MEDIUM (ORCH-10)
- `intent` and `trace_id` must be non-empty strings

33.4 Tier Routing Matrix

TIER	CONCIERGE ACTION	ORCHESTRATOR BEHAVIOR	PLANNER INVOLVED?
LOW	`dispatch_port.dispatch_direct()` -> Fabric directly	N/A (bypassed)	No
MEDIUM	`dispatch_port.dispatch_envelope(envelope)` -> Orchestrator mailbox	Invoke 1-2 Fabric capabilities directly, aggregate results	No
HIGH	`dispatch_port.dispatch_envelope(envelope)` -> Orchestrator mailbox	`PlannerAdapter.request_plan()` -> 4-stage pipeline -> `DAGExecutor`	Yes
CRISIS	No dispatch – SafetyAgent handles immediately	N/A	No

33.5 Result Consumption (Async Events)

Concierge subscribes to Orchestrator events via `DeltaBusAdapter` at session init:

Primary result event: `k1.orchestration.dag.completed.v1`

```python
# DeltaBusAdapter subscription (registered at session INIT)
async def _on_dag_completed(self, envelope: Envelope) -> None:
    result: AggregatedResult = json.loads(envelope.payload)

    # Match to originating turn
    if result["trace_id"] != current_turn.cognitive_trace_id:
        return  # Not for this session

    if result["success"]:
        # All steps completed -- deliver to user
        await output_manager.deliver_aggregated(result["step_results"])
        fsm_controller.transition(FSMState.DELIVERING)
    else:
        # Partial or total failure -- trigger tier degradation
        await _handle_degradation(result)
```

AggregatedResult fields (Concierge-relevant):

FIELD	TYPE	DESCRIPTION
`result_id`	`str`	Unique result identifier
`total_steps`	`int`	Total steps in DAG
`completed`	`int`	Steps finished successfully
`failed`	`int`	Steps that exhausted retries
`cancelled`	`int`	Steps cancelled (cascade)
`skipped`	`int`	Steps skipped (unmet deps)
`step_results`	`List[StepResult]`	Ordered results with outputs
`success`	`bool`	`True` iff `failed == 0 and cancelled == 0`
`duration_ms`	`int`	Wall-clock execution time
`trace_id`	`str`	Cognitive trace ID (correlates to Concierge turn)
`plan_id`	`Optional[str]`	`None` for MEDIUM tier (no plan)
`compensations`	`List[CompensationRecord]`	Saga rollback actions performed

Progress delta events (streamed during DAG execution):

TOPIC	PURPOSE	CONCIERGE UX
`k1.orchestration.step.started.v1`	Step began	"Working on step 2 of 5..."
`k1.orchestration.step.completed.v1`	Step finished with output	Stream partial result to user
`k1.orchestration.step.failed.v1`	Step failed after retries	"Hmm, that part didn't work..."
`k1.orchestration.delta.v1`	General progress delta	Update progress scoreboard

```python
# Progress streaming during PROGRESSING FSM state
async def _on_step_completed(self, envelope: Envelope) -> None:
    delta = json.loads(envelope.payload)
    if delta.get("trace_id") == current_turn.cognitive_trace_id:
        # Update scoreboard
        scoreboard.update(
            completed=delta["step_index"],
            total=delta["total_steps"],
        )
        # Stream partial result
        await output_manager.stream_progress(
            f"Step {delta['step_index']}/{delta['total_steps']}: "
            f"{delta.get('summary', 'done')}"
        )
        # Collect for aggregation
        delta_aggregator.collect(delta)
```

33.6 Tier Degradation Protocol (Concierge-Owned)

Degradation is entirely Concierge logic – the Orchestrator never auto-degrades. Concierge implements a ladder:

```
HIGH  ->  MEDIUM  ->  LOW  ->  Canned Response
```

FROM	TO	TRIGGER	CONCIERGE ACTION
HIGH	MEDIUM	`dag.completed.v1` with `success: false` + error `PLANNER_TIMEOUT` or `PLAN_FAILED`	Re-submit with `tier: "MEDIUM"`, pre-resolved capabilities from Phase 2
MEDIUM	LOW	All capabilities fail after Orchestrator retries	Route to LOW-tier handler (direct Fabric call, no Orchestrator)
LOW	Canned	LLM call fails or `CB_ORCHESTRATOR` OPEN or `CB_FABRIC` OPEN	Return static canned response from locale store

Degradation rules:

- Each degradation step creates a **new** `TaskEnvelope` (new `envelope_id`, same `trace_id`)
- `trace_id` MUST be preserved across all degradation attempts (end-to-end tracing)
- LOW tier tasks never reach the Orchestrator
- Max 1 degradation per tier per turn (no infinite retry loops)

```python
async def _handle_degradation(self, result: Dict) -> None:
    current_tier = current_turn.complexity_tier
    error_code = result.get("error_code", "UNKNOWN")

    if current_tier == "HIGH" and error_code in ("PLANNER_TIMEOUT", "PLAN_FAILED"):
        # Degrade HIGH -> MEDIUM
        medium_envelope = TaskEnvelope(
            intent=current_turn.intent,
            trace_id=current_turn.cognitive_trace_id,  # preserved
            tier="MEDIUM",
            capabilities=phase2_result.resolved_capabilities,
            params=phase2_result.capability_params,
            timeout_ms=10000,
        )
        ack = await dispatch_port.dispatch_envelope(medium_envelope)
        if ack.status == "ACCEPTED":
            await output_manager.send_companioning(
                "Let me try a simpler approach..."
            )
            return

    if current_tier in ("HIGH", "MEDIUM"):
        # Degrade to LOW
        try:
            result = await dispatch_port.dispatch_direct(
                capability_id=phase2_result.best_capability,
                params=phase2_result.capability_params,
            )
            await output_manager.deliver(result)
            return
        except Exception:
            pass

    # Final fallback: canned response
    await output_manager.deliver_canned(
        intent=current_turn.intent,
        safety_band=current_safety_band,
    )
```

33.7 HIL Override Flow (DAG-Level)

During DAG execution, the Orchestrator may surface HIL requests (distinct from Planner's HIL):

EVENT	DIRECTION	TOPIC	CONCIERGE HANDLER
Override request	Orch → Concierge	`k1.orchestration.delta.v1` (HIL payload)	`ClarificationTracker.on_override_request()`
Override response	Concierge → Orch	`k1.hil.override_response.v1`	`ClarificationTracker.respond()`
Fallback response	Concierge → Orch	`k1.hil.fallback_response.v1`	`ClarificationTracker.respond()`

HIL timeout: 120s (`pending_hil_ttl_s` in OrchestratorConfig). If Concierge does not respond, the Orchestrator Reaper evicts the pending HIL context and the step fails with `HIL_TIMEOUT`.

33.8 Interaction Sequence Diagram

```
CONCIERGE (L1)              ORCHESTRATOR (L2)                  FABRIC (L2.5)
    |                            |                                 |
    | TaskEnvelope(MEDIUM/HIGH)  |                                 |
    |----> IMailboxPort.enqueue()|                                 |
    |<---- TaskAck(ACCEPTED)     |                                 |
    |                            |                                 |
    | FSM: DISPATCHING->         |                                 |
    |   COMPANIONING             | _mailbox_loop() dequeue         |
    |                            |                                 |
    |                            |-- If MEDIUM: FabricGateway.execute()-->|
    |                            |       (1-2 calls, ORCH-10)      |
    |                            |<-- CapabilityResult ------------------|
    |                            |                                 |
    |                            |-- If HIGH: PlannerAdapter--->   |
    |                            |   request_plan() ---------> Planner |
    |                            |   ...4-stage pipeline...        |
    |                            |<-- plan.ready.v1 <--------- Planner |
    |                            |   DAGExecutor.execute(plan)     |
    |                            |------> execute_batch(wave) -------->|
    |                            |<------ wave results <--------------|
    |                            |                                 |
    | step.completed.v1          |                                 |
    |<---- DeltaEmitAdapter.emit() |                               |
    | FSM: -> PROGRESSING        |                                 |
    |                            |                                 |
    | dag.completed.v1           |                                 |
    |<---- DeltaEmitAdapter.emit() |                               |
    | FSM: -> DELIVERING         |                                 |
```

33.9 K0 Bridge Boundary (Orchestrator)

The Orchestrator has its own `IBridgeWritePort` for WAL writes (e.g., workflow state persistence). This is transparent to Concierge. Key point: **Orchestrator's K0 access is entirely its own** – Concierge does not mediate or observe it.

ORCHESTRATOR K0 ACCESS	PORT	PURPOSE	CONCIERGE INVOLVEMENT
WAL write (fire-and-forget)	`IBridgeWritePort`	Workflow state, DAG audit	NONE
WAL read (on recovery)	`IBridgeWritePort.read_wal()`	Crash recovery	NONE

33.10 Error Handling Summary

ERROR CONDITION	DETECTION	CONCIERGE RESPONSE
`CB_ORCHESTRATOR` OPEN	Circuit breaker state check before dispatch	Degrade to LOW tier immediately
`TaskAck.status == "REJECTED_FULL"`	Synchronous return from `enqueue()`	Retry once after 100ms, then degrade
`TaskAck.status == "DUPLICATE"`	Synchronous return	Idempotent – wait for result event
Orchestrator timeout (60s MEDIUM / 120s HIGH)	Concierge-side timer	Cancel turn, apologize, offer retry
`dag.completed.v1` with `success: false`	Async event	Tier degradation ladder (Section 33.6)
Partial failure (some steps completed)	`completed > 0 and failed > 0`	Stream completed results, report partial failure
Saga compensation triggered	`compensations` non-empty in result	Inform user of rollbacks performed

34. Concierge <-> Fabric Connection

34.1 Overview

Capability Fabric (L2.5) is K1's central resolution, retrieval, and execution engine – the **single runtime** through which every tool invocation, agent spawn, workflow execution, and dynamic agent creation flows. Fabric sits between coordination (Orchestrator L2, Planner L3) and execution (Sub-Agents L4).

Concierge interacts with Fabric through two independent mechanisms:

1. **LOW tier direct dispatch**: Concierge calls `IDispatchPort.dispatch_direct()` which routes to `Fabric.execute()`. This is the only path where Concierge talks to Fabric directly. Circuit breaker `CB_FABRIC` (Concierge-owned) protects this path.
2. **MED/HIGH tier indirect dispatch**: Concierge dispatches a `TaskEnvelope` to Orchestrator, which calls `IFabricGatewayPort.execute()` per DAG step. Concierge never sees individual Fabric calls for MED/HIGH – it consumes only the final `AggregatedResult`.

Fabric invariants relevant to Concierge:

- **FAB-01**: Fabric NEVER writes to SessionState. All reads via `ISessionStateReader`.
- **FAB-02**: Every execution traced with `cognitive_trace_id`.
- **FAB-03**: All provider executions go through the same policy engine (safety band enforcement).
- **FAB-04**: Execution overhead < 100ms (P95), < 500ms (max).

34.2 Port & Adapter Mapping

Concierge's `IDispatchPort` is the boundary contract that mediates Fabric access:

```
IDispatchPort (Protocol)
  |
  +-- dispatch_direct(capability_id, params) -> ExecutionResult
  |     LOW tier only. Wraps Fabric.execute(CapabilityRequest).
  |     CB_FABRIC applied at this boundary.
  |
  +-- dispatch_envelope(envelope: TaskEnvelope) -> DispatchReceipt
  |     MED/HIGH tier. Routes to Orchestrator mailbox.
  |     CB_ORCHESTRATOR applied at this boundary.
  |
  +-- cancel_dispatch(envelope_id) -> CancelResult
        Cancel in-flight dispatch.
```

Adapter wiring (from `kernel.md` Section 2 Phase 6):

CONCIERGE PORT	ADAPTER	TARGET	CB
`IDispatchPort.dispatch_direct()`	`DispatchAdapter(fabric)`	`Fabric.execute()`	CB_FABRIC: 3 failures/60s, half-open 30s
`IDispatchPort.dispatch_envelope()`	`DispatchAdapter(orch_mailbox)`	`Orchestrator.mailbox.enqueue()`	CB_ORCHESTRATOR: 3 failures/60s, half-open 30s

Fabric itself has 6 ports (from `kernel.md` Section 4.1). Concierge does NOT wire these directly – the kernel wires them during bootstrap Phase 3:

FABRIC PORT	WIRED ADAPTER	PURPOSE
`IEventPort`	`FabricBusAdapter(bus)`	Emits `capability.invoked`, `capability.completed` to K1 Bus
`IDeltaBusPort`	`FabricBusAdapter(bus)` (same instance)	Sub-agent delta emission on `k1.agent.{id}.delta.v1`
`ISessionStateReader`	`SessionStateReaderAdapter(mgr)`	Read-only SessionState access (FAB-01)
`IBridgePort`	`BridgeConnectionAdapter` (Phase 2)	K0 recall for tool context
`IModelGatewayPort`	Model Hub gateway	LLM access for agent spawning
`IPromptSystemPort`	Prompt registry	Prompt template resolution

34.3 LOW Tier Dispatch Flow (Direct Fabric Execution)

When ComplexityRouter classifies a turn as LOW (complexity < 0.3), Concierge's Phase 2 LLM tool loop invokes capabilities directly via Fabric:

```python
# Phase 2 LLM Tool Loop (ToolDispatcher service)
# LLM selects tool -> ToolDispatcher resolves -> dispatch_direct

async def _execute_tool_call(self, tool_call: ToolCall) -> ExecutionResult:
    """Execute a single tool call via Fabric."""

    # Step 1: Map LLM tool_call to Fabric CapabilityRequest
    capability_id = tool_call.function_name   # e.g., "tool.read.weather_api"
    params = tool_call.arguments              # extracted by LLM

    # Step 2: Execute via IDispatchPort (CB_FABRIC protects)
    try:
        result = await self._dispatch_port.dispatch_direct(
            capability_id=capability_id,
            params=params
        )
    except CircuitBreakerOpenError:
        # CB_FABRIC OPEN: degrade gracefully
        return ExecutionResult(
            success=False,
            error="capability_unavailable",
            message="This feature is temporarily unavailable"
        )

    # Step 3: Feed result back to LLM for next iteration
    return result
```

Inside `dispatch_direct()` the adapter wraps a `CapabilityRequest` and calls Fabric's 9-step execution pipeline:

1. Emit `capability.invoked` event
2. **Resolve**: `Registry.lookup(name)` -> `CapabilityContract`
3. **Select**: `PolicyEngine` -> `ProviderMatcher` -> `ProviderSelector` -> best provider
4. **Build Context**: `ContextBuilder` reads SessionState sections + resolves prompts
5. **Execute**: `Provider.execute()` via circuit breaker
6. **Validate**: 3-tier output validation (structural -> schema -> semantic)
7. Emit `capability.completed` event
8. Update metrics
9. Return `CapabilityResult`

Result mapping back to Concierge:

```python
@dataclass(frozen=True)
class CapabilityResult:
    success: bool
    data: Dict[str, Any]      # Provider output
    error: Optional[str]      # Error code if failed
    duration_ms: int          # Execution time
    provider_type: str        # "MCP" | "WASM" | "Bridge" | "Agent"
    trace_id: str             # cognitive_trace_id for tracing
```

34.4 Capability Discovery (Role 1: Intelligent Retrieval)

During Phase 2 LLM tool loop, the LLM can call `discover_capabilities` to find tools. This is itself a Fabric execution that hits `DiscoverCapabilitiesHandler` internally:

```python
@tool(name="discover_capabilities")
async def discover_capabilities(
    query: str,
    domain: Optional[List[str]] = None,
    intent: Optional[str] = None,
    safety_band: str = "GREEN",
    max_results: int = 5
) -> List[CapabilityDescriptor]:
    """Find available capabilities matching query.

    Internally calls IDispatchPort.dispatch_direct() with:
      CapabilityRequest(name="tool.read.discover_capabilities", params={...})

    Fabric retrieval pipeline:
      1. Embed query (< 5ms)
      2. Hard filter: safety band, availability, input compatibility (< 2ms)
      3. Soft rank: semantic x 0.4 + domain x 0.3 + success x 0.15 + cost/latency x 0.15 (< 10ms)
      4. Top-K selection (< 1ms)
      Total: < 20ms for 10K capabilities, < 50ms for 100K
    """
    return await self._dispatch_port.dispatch_direct(
        capability_id="tool.read.discover_capabilities",
        params={
            "query": query,
            "domain": domain or [],
            "intent": intent or query,
            "safety_band": safety_band,
            "top_k": max_results
        }
    )
```

Discovery results include full contract schemas (inputs, outputs, `tools_granted`, `required_context`) so the Planner (for HIGH tier) or LLM (for LOW tier) can inspect capabilities before composing calls.

34.5 Provider Types & Execution Models

Fabric resolves capabilities to 6 concrete provider types. Concierge encounters all of them through its dispatch path:

PROVIDER TYPE	EXECUTION MODEL	EXAMPLE	CONCIERGE TOUCH POINT
MCP	External tool call via MCP protocol	`tool.execute.send_email`, `tool.read.weather_api`	LOW dispatch_direct, MED/HIGH via Orchestrator
WASM	Sandboxed function execution	`tool.execute.calculate_bmi`	LOW dispatch_direct
Bridge	K0 recall via `IBridgePort`	`tool.read.k0_recall`	All tiers (K0 memory access)
Agent	Sub-agent with scoped LLM + tools	`agent.execute.health_advisor`	MED/HIGH via Orchestrator DAG steps
Workflow	Predefined multi-step orchestration	`workflow.execute.onboarding_flow`	MED/HIGH via Orchestrator
Concierge	Internal Concierge service call	`concierge.clarify`, `concierge.confirm`	LOW tier only (self-referential)

Agent lifecycle (spawned by Fabric Role 3: Agent Factory):

```
PENDING -> WARMING -> ACTIVE -> IDLE (pool, 60s TTL) -> TERMINATED
                        |
                    DRAINING (graceful shutdown)
```

8-step instantiation for agent providers:

1. Load YAML template (`AgentContract` from registry)
2. Create MPSC Mailbox (WFQ INTERACTIVE priority)
3. Grant LLM access via `IModelGatewayPort`
4. Grant SessionState read (declared sections, lock-free)
5. Scope tool access (`tools_granted[]` , strict enforcement)
6. Build initial context via `ContextBuilder`
7. Instantiate Agent object
8. Start lifecycle: PENDING -> WARMING -> ACTIVE

34.6 Meta-Agent Creation (HIGH Tier)

For HIGH tier tasks, Planner discovers capabilities (read-only via `IFabricRetrievalPort`) and commits a DAG that includes `tool.write.build_agent` steps. Orchestrator executes these steps via Fabric, dynamically creating specialist agents:

```
Planner (read-only discovery via IFabricRetrievalPort)
  |
  | commits DAG with build_agent steps
  v
Orchestrator (executes DAG steps via IFabricGatewayPort)
  |
  | step 1: tool.write.build_agent -> AgentSpecValidator -> AgentComposer -> Registry
  | step 2: execute created agent -> AgentFactory.spawn_and_execute()
  v
Fabric (Role 2: execute, Role 3: agent factory)
  |
  | emits k1.fabric.agent.created.v1 event
  v
Concierge (consumes AggregatedResult from Orchestrator)
```

Concierge's involvement: it dispatches the `TaskEnvelope` and receives the final `AggregatedResult` containing `AgentResponsePayload` from specialist agents. Concierge never sees the meta-agent creation internals.

Safety rules for meta-operations (enforced by Fabric's SecurityContext):

- `tool.write.build_agent` requires SafetyBand.AMBER minimum
- Created agents inherit creator's max safety band (no escalation)
- Agents cannot create agents (depth=1 enforcement)
- Violations emit `k1.fabric.meta.operation.blocked.v1`

34.7 Interaction Sequence Diagram (LOW Tier)

```
User: "What's the weather in Amsterdam?"

User ──► Concierge (Phase 1: UltraBERT classify)
  │  intent=weather, complexity=LOW, safety=GREEN
  │
  ├──► Phase 2: LLM Tool Loop
  │     LLM ──► tool_call: discover_capabilities(query="weather amsterdam")
  │              │
  │              └──► IDispatchPort.dispatch_direct("tool.read.discover_capabilities", {...})
  │                    │
  │                    └──► CB_FABRIC check (CLOSED) ──► Fabric.execute()
  │                         │
  │                         └──► CapabilityResult: [tool.read.weather_api]
  │
  │     LLM ──► tool_call: tool.read.weather_api(city="Amsterdam")
  │              │
  │              └──► IDispatchPort.dispatch_direct("tool.read.weather_api", {...})
  │                    │
  │                    └──► CB_FABRIC check (CLOSED) ──► Fabric.execute()
  │                         │
  │                         ├──► Registry.lookup() -> MCP provider
  │                         ├──► PolicyEngine.check() -> GREEN: allowed
  │                         ├──► ContextBuilder.build() -> session context
  │                         ├──► MCPProvider.execute() -> weather data
  │                         └──► CapabilityResult{success=true, data={temp: 12, ...}}
  │
  │     LLM ──► final_response: "It's 12C in Amsterdam with partly cloudy skies."
  │
  └──► Phase 3: OutputManager.deliver()
```

34.8 K0 Bridge Boundary (Transparent to Concierge)

Fabric has its own `IBridgePort` for K0 access (tool registry sync, K0 recall for context building). This is wired during kernel bootstrap Phase 3 and is **completely transparent** to Concierge:

- Fabric's `IBridgePort` uses `BridgeConnectionAdapter` -> K0 `/k0/query.recall` port
- When Fabric's `ContextBuilder` needs K0 memory for an agent's `required_context`, it calls `IBridgePort.recall()` internally
- Concierge does not mediate, intercept, or configure Fabric's K0 access
- If K0 is offline, Fabric degrades to local context only (edge-first design)

Concierge's own K0 access is through `IMemoryPort` (described in Section 32.8), which is an independent path.

34.9 Circuit Breaker: CB_FABRIC

PARAMETER	VALUE
Owner	Concierge
Scope	`IDispatchPort.dispatch_direct()` path only
Failure threshold	3 failures in 60s
Half-open after	30s
Reset on	1 successful execution
Probe	Half-open: single request probe

CB_FABRIC state transitions from Concierge perspective:

CB STATE	CONCIERGE BEHAVIOR
CLOSED	Normal `dispatch_direct()` execution
OPEN	Return "This feature is temporarily unavailable". Degrade: if user asks factual question, attempt K0 recall via `IMemoryPort` instead. Emit `k1.concierge.cb.fabric.open.v1`
HALF-OPEN	Route single probe request. If probe passes -> CLOSED. If fails -> OPEN again. Only probes on user turns (never background)

Note: MED/HIGH tier is NOT affected by CB_FABRIC. Orchestrator calls Fabric via its own `IFabricGatewayPort`, which has no CB (Concierge owns the Orchestrator-level CB_ORCHESTRATOR on the `dispatch_envelope()` path instead).

34.10 Error Handling Summary

ERROR	SOURCE	CONCIERGE RESPONSE	RECOVERY
CB_FABRIC OPEN	Circuit breaker trip	"This feature is temporarily unavailable"	Wait 30s half-open probe
Capability not found	Registry.lookup() miss	"I don't know how to do that yet"	Log capability gap for P06 active learning
Execution timeout (>5s)	Provider.execute()	"This is taking longer than expected..."	Cancel + retry once, then degrade
PolicyEngine rejection	Safety band mismatch	"I can't do that for safety reasons"	Log violation, no retry
Provider error	MCP/WASM/Bridge failure	"Something went wrong with [tool]"	CB_FABRIC increment, retry if transient
Agent spawn failure	AgentFactory error	"I can't complete this task right now"	Degrade tier if MED/HIGH, canned response if LOW
Schema validation failure	3-tier output validation	Treat as provider error	CB_FABRIC increment

35. Concierge <-> SessionState Connection

35.1 Overview

SessionState is K1's tiered, memory-constrained state management system – the **central working memory** for conversational AI. Based on Baddeley's working memory model, it maintains conversation context with strict size limits (96KB: 48KB HOT + 48KB WARM) while preserving human-scale conversations (40+ turns).

Concierge is the **ONLY writer** to SessionState (CONC-01, ADR-0017/ADR-0018). All other components (Orchestrator, Planner, Fabric, sub-agents) read via lock-free snapshots (< 1ms). Sub-agent writes are funneled through the delta aggregation path on K1 Bus.

SessionState invariants relevant to Concierge:

- **SS-01**: 96KB hard cap. Writes exceeding this are hard-rejected.
- **SS-02**: HOT CORE (48KB) is never thrashed. Demotion to WARM only, never eviction.
- **SS-03**: `control` section (8KB) is NEVER evicted. Orchestration survival depends on it.
- **SS-04**: Single writer serialization through Concierge. No concurrent writers.
- **SS-05**: 40-turn human-scale retention. Turns 1-10 full fidelity, 11-30 compressed, 31-40 summarized.

35.2 Port & Adapter Mapping

Concierge's `IStatePort` is the boundary contract for SessionState:

```python
class IStatePort(Protocol):
    """Concierge's bidirectional port for SessionState access."""

    # === READ METHODS (lock-free, < 1ms) ===
    async def read_section(self, section: str) -> SectionData:
        """Read a single HOT/WARM section."""
        ...

    async def read_all_hot(self) -> HotSnapshot:
        """Read all 8 HOT CORE sections as atomic snapshot."""
        ...

    async def read_sections(self, sections: List[str]) -> Dict[str, SectionData]:
        """Read multiple sections atomically."""
        ...

    # === WRITE METHODS (single-writer, MutationGuard protected) ===
    async def write(self, section: str, data: Any, operation: str = "update") -> WriteResult:
        """Write to a section through MutationGuard.

        Flow: IStatePort.write() -> SessionStateAdapter -> MutationGuard.preflight()
              -> SizeTracker.check() -> DirectWriterAdapter.request_mutation()
        """
        ...

    # === LIFECYCLE METHODS ===
    async def checkpoint(self) -> str:
        """Create checkpoint. Returns checkpoint_id for crash recovery."""
        ...

    async def reconstruct(self, session_id: str, mode: str = "partial") -> ReconstructionResult:
        """Reconstruct from COLD. Mode: 'partial' (HOT first) or 'degraded' (HOT only)."""
        ...
```

Adapter wiring (from `kernel.md` Section 4.2):

CONCIERGE PORT	ADAPTER	TARGET
`IStatePort` (read)	`SessionStateAdapter(mgr)`	`SessionStateManager` read path (lock-free)
`IStatePort` (write)	`SessionStateAdapter(mgr)` -> `MutationGuard` -> `DirectWriterAdapter`	Single writer path
`IStatePort` (checkpoint)	`SessionStateAdapter(mgr)` -> `IStoragePort`	`SQLiteStorageAdapter(db_path)` LOCAL COLD
`IStatePort` (reconstruct)	`SessionStateAdapter(mgr)` -> `ReconstructionSLA`	COLD -> HOT hydration

SessionState's own 5 ports (wired by kernel, transparent to Concierge):

SESSIONSTATE PORT	ABC	WIRED ADAPTER	PURPOSE
`IStoragePort`	ABC	`SQLiteStorageAdapter(db_path)`	LOCAL COLD tier persistence
`IEventPort`	ABC	`SessionBusAdapter(bus)`	Emits `session.updated.v1`, `session.eviction.v1` etc. to K1 Bus
`IWriterPort`	ABC	`DirectWriterAdapter()`	Single-process direct mutation
`ILifecyclePort`	ABC	`StandaloneLifecycle(config)`	Self-managed checkpointing
`IK0SyncPort`	ABC (optional)	`NullSyncPort()`	No cloud sync until Bridge ready. Future: `K0SyncAdapter` for async archival

35.3 Single Writer Pattern (CONC-01, ADR-0017)

Concierge is the only component permitted to write to SessionState. This is enforced at two levels:

Level 1: MutationGuard (preflight validation)

```python
class MutationGuard:
    """Three-tier preflight validation for all state mutations."""

    def preflight(
        self,
        section: str,
        operation: str,       # "insert", "update", "delete"
        estimated_kb: int     # Pre-estimated size of mutation
    ) -> Approval:
        """
        Tier 1: Section capacity check (e.g., control max 8KB)
        Tier 2: Tier capacity check (HOT max 48KB, WARM max 48KB)
        Tier 3: Total session capacity check (96KB hard cap)

        Returns: Approved | RejectedWithCapacity | RejectedHard
        """
```

Level 2: Component identity check

```
class SessionStateMutationGuard:
    def validate_writer(self, component_id: str) -> bool:
        return component_id == "concierge"

    def reject_unauthorized(self) -> NoReturn:
        raise UnauthorizedWriteError("Only Concierge may write to SessionState")
```

Delta aggregation path (how sub-agent writes reach Concierge):

```
Sub-Agents emit deltas (fire-and-forget)
  |
  v
K1 Bus Delta Lane (topic: k1.agent.{id}.delta.v1)
  |
  v
AggregationWindow (500ms batching)
  |  Merge: last-write-wins for conflicting keys
  |  Collapse: redundant updates removed
  |  Sort: clarifications first (priority)
  v
Concierge FSM (ACKING state) -> DeltaAggregator service
  |
  v
IStatePort.write() -> MutationGuard -> DirectWriterAdapter
```

Delta types from sub-agents:

- `state_update` : Agent state changes
- `clarification_request` : Need user input (highest priority)
- `task_complete` : Work finished
- `belief_update` : User model changes
- `memory_episodic` : Experience to remember

35.4 Phase 1 Write Connection (UltraBERT -> SessionState)

During Phase 1, UltraBERT head outputs write directly to HOT CORE sections via the **Write Elision Gate** (see Section 6.5). Not every turn causes writes – low-signal messages ("ummm what more?", "ok") are elided to preserve previous state.

ULTRABERT HEAD	TARGET SESSIONSTATE FIELD	WRITE CONDITION
`safety_familyos`	`control.safety_band`	ALWAYS (CONC-05: safety is never elided)
`intent`	`control.intents`	If intent != `other` or confidence > 0.5
`ingress`	`control.domains[]`	If any domain above threshold
`ner_general` + `ner_family`	`beliefs_active.mentioned_entities`	If any spans detected
`temporal`	`beliefs_active.mentioned_time`	If any spans detected
`emotions`	`affective_now.current_emotion`	If non-neutral
`sentiment`	`affective_now.valence`	If changed from previous

Write Elision Gate logic:

```python
async def apply_write_elision(
    classification: ClassificationResult,
    current_state: HotSnapshot
) -> List[MutationRequest]:
    """Determine which UltraBERT outputs actually need writing.

    Low-signal turn ("ummm", "ok", "what else?"):
      - safety_band: ALWAYS written (CONC-05)
      - All other fields: ELIDED (preserve previous state)

    Normal turn (meaningful content):
      - Write only fields that changed from current_state
      - Skip fields where new value == existing value
    """
    mutations = []

    # Safety is NEVER elided
    mutations.append(MutationRequest(
        section="control",
        field="safety_band",
        value=classification.safety_band,
        operation="update"
    ))

    if classification.is_low_signal:
        return mutations  # Only safety written for low-signal

    # Compare each head output to current state, write only changes
    if classification.intent != current_state.control.intents:
        mutations.append(MutationRequest(
            section="control", field="intents",
            value=classification.intent, operation="update"
        ))
    # ... similar for all other heads ...

    return mutations
```

35.5 Phase 2 Write Connection (Cognitive Tools -> SessionState)

During Phase 2 LLM tool loop, cognitive tools write to HOT sections via `IStatePort`. Every write passes through `MutationGuard.preflight()` using the `MutationRequest` / `MutationResponse` protocol:

COGNITIVE TOOL	TARGET SECTION	PURPOSE
`update_scoreboard()`	`scoreboard`	QUD stack, referents, salience
`update_beliefs()`	`beliefs_active`	Turn-specific facts
`update_clarifications()`	`clarifications`	Pending clarification questions
`update_narrative()`	`narrative_active`	Thread tracking, arc position
`refine_affect()`	`affective_now`	LLM-refined emotional state
`promote_belief()`	`beliefs_active` + K0 via `IMemoryPort`	Promote belief to long-term memory
`summarize_context()`	N/A (read-only)	Assemble context for LLM

35.6 Tier Access Matrix

TIER	SIZE	READ	WRITE	SYNC	CONCIERGE TOUCH
HOT CORE	48KB (8 sections)	Every turn (lock-free, < 100us)	Phase 1 (elision-gated) + Phase 2 (MutationGuard) + turn boundary	In-memory always	Direct via `IStatePort`
WARM TIER	48KB (4 sections)	Occasionally (< 200us)	MigrationEngine demotions only	In-memory, evictable	Indirectly (MigrationEngine triggered by SizeTracker pressure)
LOCAL COLD	Unlimited (SQLite)	On recovery/reconstruction (< 100ms)	Checkpoint every turn via `IStoragePort`	K1 SQLite via `SQLiteStorageAdapter`	Checkpoint via `IStatePort.checkpoint()`
K0 Cloud	Unlimited (remote)	Via Bridge (async)	Via Bridge (fire-and-forget at `turn_end`)	Async via `IK0SyncPort` (currently `NullSyncPort`)	Fire-and-forget archival at turn boundary

HOT CORE sections (48KB total):

SECTION	BUDGET	PURPOSE	EVICTION
`control`	8KB	Agent leases, flow state, safety band	NEVER (SS-03)
`beliefs_active`	8KB	Current turn facts, entities	Demote oldest to `beliefs_history`
`scoreboard`	6KB	QUD stack, referents, salience	Demote stale referents
`history_active`	8KB	Last 10 turns (full text)	Demote oldest to `history_recent`
`clarifications`	4KB	Pending clarification requests	Expire answered
`affective_now`	4KB	Current emotional state	Overwrite (single slot)
`narrative_active`	4KB	Active conversation thread	Archive paused threads to K0
`meta`	2KB	Session metadata	NEVER (minimal size)

WARM TIER sections (48KB total):

SECTION	BUDGET	PURPOSE	EVICTION ORDER
`telemetry`	8KB	Performance metrics	1st: aggregate to counters, drop raw
`history_recent`	20KB	Turns 11-30 (compressed)	2nd: summarize oldest, archive to K0
`beliefs_history`	12KB	Previous turn facts	3rd: archive oldest to K0
`persona`	8KB	User personality model	4th: never evict unless EMERGENCY

35.7 Memory Pressure & Emergency Modes

Concierge must handle memory pressure escalation because it is the single writer:

```python
async def handle_write_with_pressure(
    section: str, data: Any, operation: str
) -> WriteResult:
    """Write with memory pressure awareness."""

    # Step 1: Preflight via MutationGuard
    approval = await mutation_guard.preflight(section, operation, estimate_kb(data))

    match approval:
        case Approved():
            return await state_port.write(section, data, operation)

        case RejectedWithCapacity(available_kb=avail):
            # Trigger proactive migration HOT -> WARM
            await migration_engine.demote_to_warm(find_demotable_section())
            # Retry write
            return await state_port.write(section, data, operation)

        case RejectedHard():
            # Total session >= 96KB: EMERGENCY path
            pressure = await size_tracker.get_pressure()

            if pressure == PressureLevel.CRITICAL:  # >= 95KB
                # Stage 1: EMERGENCY_SUMMARIZE
                await eviction_engine.evict_warm(target_kb=85)
                # 50ms GC wait
                await asyncio.sleep(0.05)
                retry_approval = await mutation_guard.preflight(section, operation, estimate_kb(data))
                if isinstance(retry_approval, Approved):
                    return await state_port.write(section, data, operation)

                # Stage 2: EMERGENCY_READONLY -- block writes
                await event_bus.publish("k1.session.emergency.v1", {
                    "session_id": session_id,
                    "mode": "readonly"
                })
                return WriteResult(success=False, error="emergency_readonly")
```

Emergency escalation stages (from SessionState spec Section 14):

STAGE	TRIGGER	ACTION	CONCIERGE BEHAVIOR
EMERGENCY_SUMMARIZE	Total >= 95KB	Aggressive compression: LLM summarize turns 11-40, telemetry -> counters only, target < 90KB	Wait 50ms for GC, retry write
EMERGENCY_READONLY	Still >= 96KB after Stage 1	Block all new writes, allow reads, background cleanup	Skip writes, operate on stale state, warn user
EMERGENCY_SHED	Cleanup failing	Drop entire WARM sections (except persona), preserve HOT at all costs	Operate with HOT only, emit `session.emergency.v1`

35.8 Token Budget Enforcement (ContextAssembler)

Concierge's ContextAssembler reads HOT sections to build LLM context within token budgets:

```python
async def assemble_context(
    budget_tokens: int,
    required_sections: Optional[List[str]] = None
) -> ContextSnapshot:
    """Build LLM context from SessionState within token budget.

    Default required_sections: control, beliefs_active, scoreboard,
    history_active, affective_now, narrative_active (6 of 8 HOT sections).

    Budget allocation:
      - control + meta: always included (low token cost)
      - history_active: up to budget_tokens // 3 (most expensive)
      - beliefs_active + scoreboard: up to budget_tokens // 4
      - affective_now + narrative_active: remaining
    """
    hot = await state_port.read_all_hot()
    total_tokens = estimate_tokens(hot)

    if total_tokens <= budget_tokens:
        return ContextSnapshot(sections=hot, truncated=False)

    # Over budget: summarize history first (largest section)
    if estimate_tokens(hot.history_active) > budget_tokens // 3:
        hot.history_active = await summarize(
            hot.history_active,
            target_tokens=budget_tokens // 3
        )

    # Still over: truncate beliefs to key facts only
    if estimate_tokens(hot) > budget_tokens:
        hot.beliefs_active = extract_key_facts(
            hot.beliefs_active,
            max_facts=5
        )

    return ContextSnapshot(sections=hot, truncated=True)
```

35.9 Checkpoint Protocol

Concierge checkpoints SessionState at the end of every turn for crash recovery:

```python
async def turn_end(self, turn_id: str) -> None:
    """Turn boundary: checkpoint + K0 sync + events."""

    # Step 1: Checkpoint to LOCAL COLD (SQLite)
    checkpoint_id = await self._state_port.checkpoint()

    # Step 2: Emit checkpoint event for observability
    await self._event_bus.publish("k1.concierge.checkpoint.v1", {
        "checkpoint_id": checkpoint_id,
        "turn_id": turn_id,
        "session_id": self._session_id,
        "hot_size_kb": await self._size_tracker.get_tier_size("hot"),
        "warm_size_kb": await self._size_tracker.get_tier_size("warm"),
        "timestamp_ms": monotonic_ms()
    })

    # Step 3: Fire-and-forget K0 sync (archival)
    # IK0SyncPort: currently NullSyncPort (no-op)
    # Future: async delta to K0 via Bridge Command Port (session.checkpoint topic)
    await self._k0_sync_port.sync_delta(checkpoint_id)

    # Step 4: Proactive migration if pressure elevated
    pressure = await self._size_tracker.get_pressure()
    if pressure >= PressureLevel.ELEVATED:
        await self._migration_engine.demote_to_warm(
            find_oldest_demotable_hot_section()
        )
```

Checkpoint is used for crash recovery: On restart, Concierge reconstructs from the last checkpoint via `IStatePort.reconstruct(session_id, mode="partial")`. ReconstructionSLA target: < 100ms. If K0 is slow, degrade to `mode="degraded"` (HOT only, no WARM).

35.10 SessionState Events (Concierge-Relevant)

SessionState emits events via `SessionBusAdapter(bus)`. Concierge subscribes to pressure-related events:

EVENT TOPIC	TRIGGER	CONCIERGE HANDLE
`session.updated.v1`	Any state update	No-op (Concierge is the writer)
`session.eviction.v1`	WARM sections evicted	Log, adjust context if evicted section was in active context
`session.migration.v1`	HOT <-> WARM tier transition	Log, may trigger WARM prefetch
`session.emergency.v1`	Emergency mode activated	Switch to degraded operation, warn user in next response
`session.reconstruction.v1`	COLD -> HOT hydration	Log reconstruction latency, check if within SLA

35.11 K0 Cloud Sync Boundary

SessionState's `IK0SyncPort` mediates K0 cloud archival. Currently wired to `NullSyncPort()` (no-op until Bridge is production-ready):

```
Concierge turn_end()
  |
  | IStatePort.checkpoint() -> SQLiteStorageAdapter (LOCAL COLD)
  |
  | IK0SyncPort.sync_delta(checkpoint_id) -> NullSyncPort() (no-op)
  |
  | Future: K0SyncAdapter -> Bridge Command Port -> K0
  |           topic: session.checkpoint
  |           payload: {session_id, checkpoint_id, delta_sections}
  |           pattern: fire-and-forget (ONE-WAY)
```

K0 archival will use the Bridge Command Port's `session.checkpoint` topic (see Section 32.8). The archived data flows through K0's P02 pipeline into long-term storage, enabling COLD -> HOT reconstruction across sessions.

Concierge does NOT block on K0 sync. If K0 is unavailable, LOCAL COLD (SQLite) provides full crash recovery. K0 cloud adds cross-device continuity but is not required for single-device operation (edge-first design).

35.12 Interaction Sequence Diagram

```
User: "Remember that my daughter is allergic to peanuts"

User ──► Concierge (Phase 1)
  |
  ├──► UltraBERT classify
  |     intent=store_belief, safety=GREEN, entities=[daughter, peanut_allergy]
  |
  ├──► Write Elision Gate (meaningful content -> proceed)
  |     |
  |     ├──► IStatePort.write("control", {safety_band: "GREEN"})
  |     |     └──► MutationGuard.preflight("control", "update", 1) -> Approved
  |     |          └──► DirectWriterAdapter.request_mutation() -> OK
  |     |
  |     ├──► IStatePort.write("control", {intents: "store_belief"})
  |     ├──► IStatePort.write("beliefs_active", {entities: [daughter, peanut_allergy]})
  |     └──► IStatePort.write("affective_now", {emotion: "neutral"}) -> ELIDED (unchanged)
  |
  ├──► Phase 2: LLM Tool Loop (LOW tier)
  |     LLM ──► tool_call: update_beliefs({fact: "daughter allergic to peanuts", confidence: 0.95})
  |              |
  |              └──► IStatePort.write("beliefs_active", {fact: ...})
  |                    └──► MutationGuard.preflight("beliefs_active", "insert", 2) -> Approved
  |
  |     LLM ──► tool_call: promote_belief({fact_id: ..., importance: "high"})
  |              |
  |              ├──► IStatePort.read("beliefs_active") -> fact data
  |              └──► IMemoryPort.store({fact: ...}) -> K0 archival (fire-and-forget)
  |
  |     LLM ──► final_response: "Got it, I'll remember that your daughter is allergic to peanuts."
  |
  ├──► Phase 3: OutputManager.deliver()
  |
  └──► turn_end()
        ├──► IStatePort.checkpoint() -> checkpoint_id
        ├──► event_bus.publish("k1.concierge.checkpoint.v1", {...})
        └──► IK0SyncPort.sync_delta(checkpoint_id) -> NullSyncPort (no-op)
```

35.13 Error Handling Summary

ERROR	SOURCE	CONCIERGE RESPONSE	RECOVERY
CB_SESSION_STATE OPEN	Circuit breaker trip	Stale read from in-memory cache, warn user	Wait for half-open probe
Write rejection (MutationGuard)	Capacity exceeded	RECOVERABLE: skip write, log `k1.concierge.write.rejected.v1`	Trigger proactive migration, retry next turn
Emergency mode (>= 95KB)	SizeTracker.get_pressure() CRITICAL	50ms GC wait + retry, then EMERGENCY_SUMMARIZE	Aggressive eviction, if still full -> EMERGENCY_READONLY
Checkpoint failure	SQLiteStorageAdapter error	Log warning, continue (next turn will retry)	Non-fatal: LOCAL COLD is best-effort
Reconstruction timeout (> 100ms)	ReconstructionSLA breach	Degraded mode: operate without WARM	Serve responses from HOT only, background WARM load
K0 sync failure	NullSyncPort (current) or future Bridge error	Silent (fire-and-forget)	LOCAL COLD provides crash recovery regardless
Corruption detected	Checksum validation per section	Force reconstruction from last good checkpoint	If reconstruction fails -> new session

36. Concierge <-> K0 Kernel: Active Learning & Feedback Integration

36.1 Overview

This section specifies the **complete bidirectional relationship** between Concierge and K0 Kernel through the lens of two continuous learning systems. While Section 32.8 established the basic IMemoryPort bridge (recall/store/prefetch) and Section 9.7 introduced the Proactive Agent, this section provides the comprehensive architecture for how Concierge participates in K0's active learning

loop and feedback pipeline – the two systems that enable the intelligence kernel to **learn continuously from every conversation**.

Two-System Architecture (from bridge_architecture.mmd):

SYSTEM	PURPOSE	DIRECTION	PORT	CONCIERGE ROLE
System 1: Gap Resolution	Fill knowledge gaps via proactive questions	K0 SSE -> K1 Bus -> Concierge -> User -> Concierge -> K0 Command Port	`IMemoryPort` (write) + SSE subscription	Consumer of gap intents, formatter of questions, router of answers
System 2: Model Refinement	Tune K0 pipeline parameters from user feedback	Concierge -> Learning Loop Detectors -> K0 Obs Port	`IDeltaPort` (emit) -> K1 Bus -> Learning Loop	Emitter of turn data, host of ConversationContext tracking

Architectural boundary (zero-tolerance):

```
K0 = detection + receipts + prioritization + storage
K1 = phrasing + action + UX delivery + feedback detection
```

K0 NEVER generates user-facing text. K0 NEVER decides **when** to interrupt the user. K0 detects gaps in the knowledge graph, scores them, and emits structured intent blueprints. Concierge (via its Proactive Agent and Curiosity Agent subsystems) receives these blueprints, decides the right conversational moment, crafts human-friendly phrasing, and routes the user's answer back to K0 for gap closure.

Distinction from Section 12 (Hypothesis & Gap Detection Pipeline): Section 12 describes Concierge's own pre-LLM gap detection for the **current turn** (missing entity slots, ambiguous time references, unresolved pronouns). That is a K1-internal, synchronous, heuristic pipeline running in <3ms. This section describes K0's **background** knowledge graph gap detection across **all past conversations** – an asynchronous, ML-driven system operating across hours and days, surfacing entirely different gap types (stale anchors, concept drift, structural holes).

36.2 System 1: Active Learning Loop (Gap Resolution)

The active learning loop is K0's mechanism for proactively filling knowledge gaps detected during memory consolidation. The full loop spans both kernels:

```
                    K0 (Detection + Prioritization)
                    ================================
User Conversation
    |
    v
K0 P02 Write/Ingest (receives memory.delta from Concierge IMemoryPort.store())
    |
    v
K0 P03 Consolidation (background, runs during reconciliation R4/R7 phases)
    |
    v
GapDetector (detects 7 gap types in knowledge graph)
    |
    v
GapEmitter -> st_learning_queue + p03.gap.detected.v1 (K0 bus)
    |
    v
K0 P06 Active Learning (prioritizes gaps by importance_score)
    |
    v
K0 P05 Attention Manager (token bucket: 3 questions/day, context readiness check)
    |
    v
P06 emits curiosity.intent.v1 via K0 SSE Port
    |
    ==========================================
    |           Bridge (Transport)
    ==========================================
    |
    v
Bridge SSEReceiver parses event
    |
    v
K1 Bus: k1.k0.sse.curiosity.intent.v1 (BEST_EFFORT delivery class)
    |
    ==========================================
    |           K1 (Phrasing + Action)
    ==========================================
    |
    v
Concierge Proactive Agent receives intent
    |
    v
Proactive Decision Engine evaluates: FSM state, user energy, budget remaining
    |
    v
Curiosity Agent formats question (LLM-based, Socratic scaffolding)
    |
    v
OutputManager delivers to user (SSE stream or proactive bubble)
    |
    v
User answers
    |
    v
Concierge captures answer via turn processing
    |
    v
IMemoryPort.store(MemoryItem{content=answer, correlation={gap_id: original_gap_id}})
    |
    ==========================================
    |           Bridge (Transport)
    ==========================================
    |
    v
K0 /k0/command.submit(topic="memory.delta", body={gap_resolution_id: ...})
    |
    v
K0 P02 ingests answer with gap_resolution_id tag
    |
    v
K0 P03 next consolidation cycle: resolves gap in st_learning_queue
    |
    v
gap.status = RESOLVED, closed_at = now()
```

36.2.1 K0 Gap Types (7 Categories)

K0 P03's `GapDetector` identifies these gap types during reconciliation. Each produces a `GapRecord` persisted to `st_learning_queue` and published as `p03.gap.detected.v1`:

GAP TYPE	DETECTION PHASE	WHAT IT MEANS	CONCIERGE RELEVANCE
`AMBIGUOUS_ENTITY`	R4 (entity merge)	Two entities might be the same person	HIGH – user can confirm "Is Dr. Smith your dentist or your GP?"
`LOW_CONFIDENCE_EDGE`	R4 (edge scoring)	Relationship between entities has low certainty	HIGH – user can confirm/deny relationship
`MISSING_ATTRIBUTE`	R7 (schema validation)	Known entity missing expected attributes	MEDIUM – fill in details ("What's Maya's school name?")
`CONTRADICTION`	R4 (conflict detection)	Two facts contradict each other	HIGH – user must resolve ("You said Tuesday but also Thursday")
`CONCEPT_DRIFT`	Entropy Scanner	Anchor belief shifted >20% in 30-day window	MEDIUM – validate shift ("Your bedtime routine seems to have changed?")
`STRUCTURAL_HOLE`	Entropy Scanner	Region of knowledge graph with sparse connections	LOW – exploratory questions to fill sparse areas
`STALE_ANCHOR`	Entropy Scanner	Anchor not validated for >90 days	LOW – periodic confirmation ("Still accurate that...?")

36.2.2 GapRecord Structure (K0 Side, Consumed by Concierge)

```python
@dataclass
class GapRecord:
    """K0 P03 gap record, persisted in st_learning_queue."""
    gap_id: str                  # UUID
    gap_type: str                # One of 7 types above
    entity_id: Optional[str]     # Target entity (if entity-specific)
    description: str             # Machine-readable gap description
    importance_score: float      # 0.0-1.0 (P06 computed)
    priority: int                # Queue position after P06 ranking
    context: Dict[str, Any]      # Supporting evidence from knowledge graph
    created_at: int              # Unix timestamp
    status: str                  # PENDING, EMITTED, ANSWERED, RESOLVED, EXPIRED
    resolution: Optional[str]    # Answer text once resolved
    closed_at: Optional[int]     # When resolved
```

Concierge never reads `st_learning_queue` directly. It receives a processed `CuriosityIntent` (Section 36.7) via SSE after P06 prioritization and P05 attention gating.

36.3 System 2: Feedback Loop (Model Refinement)

The feedback loop is the mechanism by which user behavior during conversations feeds back into K0 to tune pipeline parameters (similarity thresholds, decay rates, salience boosts). Unlike System 1 (which adds NEW facts), System 2 refines HOW K0 processes existing facts.

```
User interacts with Concierge
    |
    v
Concierge turn_end() emits turn.complete.v1 via IDeltaPort
    |
    v
K1 Bus: k1.concierge.turn.completed.v1
    |
    v
Learning Loop Feedback Detectors (5 detectors, background subscribers)
    |
    +--> CorrectionDetector: user rephrased to correct memory (confidence 0.85)
    +--> ValidationDetector: user confirmed/denied recall accuracy (confidence 0.90)
    +--> ReformulationDetector: user rephrased query indicating poor recall (confidence 0.60-0.75)
    +--> AbandonmentDetector: user left without resolution (confidence 0.70)
    +--> HedgingDetector: LLM response was uncertain (confidence 0.50)
    |
    v
Detected signal -> Build FeedbackEnvelope
    |
    v
POST /k0/obs.emit (kind: "feedback") via Bridge Obs Port
    |
    ============================================
    |           K0 (Model Refinement)
    ============================================
    |
    v
observe.py receives FeedbackEnvelope
    |
    v
FeedbackSchemaRegistry validates payload against pipeline-specific schema
    |
    v
st_feedback_signals (persisted with deduplication via payload_hash)
    |
    v
BusDispatcher routes to pipeline-specific handler via feedback.signal.pXX.v1
    |
    v
Pipeline handler applies feedback:
  - P02: adjust similarity_threshold, decay_lambda, salience_boost (Thompson Sampling)
  - P03: adjust consolidation merge thresholds
  - P08: adjust recall ranking weights
  - Results stored in st_learned_weights
```

36.3.1 FeedbackEnvelope Structure

```python
@dataclass
class FeedbackEnvelope:
    """K1 -> K0 feedback signal. Built by Learning Loop detectors."""
    feedback_id: str             # UUID
    pipeline_id: str             # Target pipeline (P02, P03, P08, etc.)
    signal_class: str            # CORRECTION | VALIDATION | IMPLICIT | EXPLICIT | OUTCOME
    signal_subtype: str          # Detector-specific (e.g., "reformulation", "abandonment")
    correlation: CorrelationIds  # Links back to K0 entities
    provenance: Provenance       # Audit trail
    payload: Dict[str, Any]      # Pipeline-specific payload (registered in FeedbackSchemaRegistry)
    payload_hash: str            # SHA-256 for deduplication (same hash within 5s -> ignored)
```

36.3.2 Signal Class Priority

SIGNAL CLASS	CONFIDENCE	SOURCE	PRIORITY	CONCIERGE TOUCH
`CORRECTION`	High (0.85)	User explicitly corrected a recalled fact	P0	Concierge captures correction via `update_beliefs()` tool; Learning Loop detects from turn delta
`VALIDATION`	Very High (0.90)	User confirmed/denied memory accuracy	P0	Direct from user response to `recall_memory()` result
`EXPLICIT`	High	User gave thumbs up/down	P1	UI button → turn metadata
`IMPLICIT`	Medium (0.50-0.75)	Behavioral inference (reformulation, abandonment)	P2	Detectors analyze turn pairs
`OUTCOME`	Medium	Downstream result quality	P2	System-generated after action execution

36.4 Component Placement Matrix (K0 vs K1)

This section provides the definitive placement for all active learning loop and feedback system components. The governing principle is the zero-tolerance boundary: **K0 = detection + receipts, K1 = phrasing + action**.

36.4.1 Active Learning Loop Components

COMPONENT	PLACEMENT	RATIONALE	INTERFACE
P03 GapDetector	K0	Knowledge graph analysis requires direct access to consolidated memory tables	Internal to P03 consolidation pipeline
GapEmitter	K0	Persists to st_learning_queue (K0 storage), publishes K0 bus event	st_learning_queue + p03.gap.detected.v1
st_learning_queue	K0 (storage)	Central gap registry, K0-owned table	SQLite/Postgres in K0 storage layer
P06 Active Learning (Prioritizer)	K0	Scores and ranks gaps using knowledge graph context unavailable in K1	Reads st_learning_queue, computes importance_score
Entropy Scanner	K0	Background process scanning knowledge graph for drift/decay/holes	Scheduled (4AM daily + post-consolidation + manual trigger)
Bayesian Anchors (st_anchors)	K0 (storage)	Beta distribution parameters require consolidation-level updates	K0 storage, updated by P03
P05 Attention Manager	K0	Enforces global token budget (3 questions/day) across all K1 consumers	Token bucket, context readiness scoring
CuriosityIntent emission	K0 (SSE port)	K0's output surface – structured blueprints, never user-facing text	curiosity.intent.*.v1 SSE events
CounterfactualEngine	K0	Causal graph + do-calculus requires K0's structural causal model	curiosity.intent.counterfactual.v1
CausalCuriosityService	K0	Backdoor/instrument analysis on K0's causal graph	curiosity.intent.causal.v1
ScenarioLab	K0	World model operates on K0's longitudinal data	Offline simulation, no SSE emission

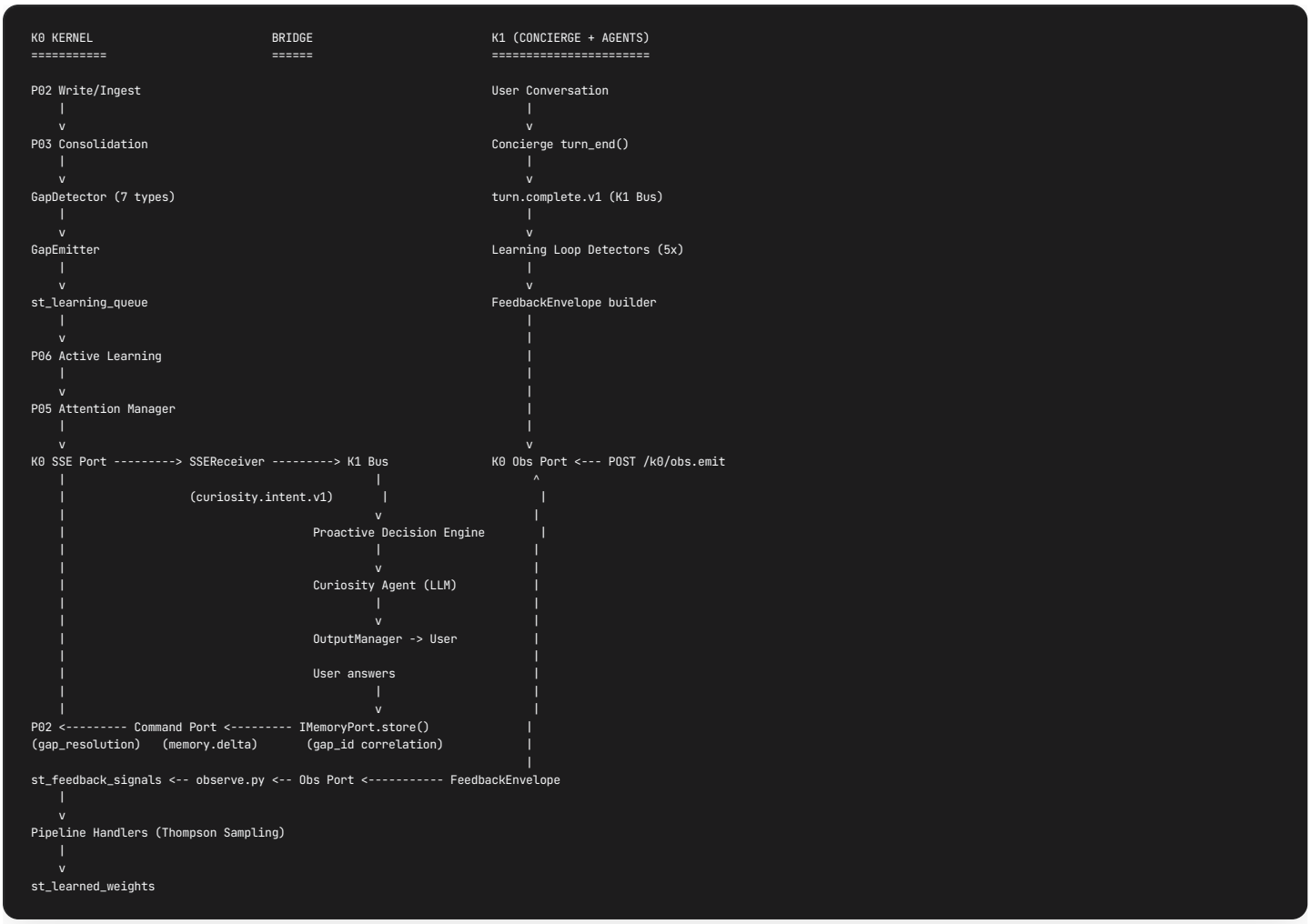
36.4.2 Feedback System Components

COMPONENT	PLACEMENT	RATIONALE	INTERFACE
ConversationContext	K1 (Concierge-adjacent)	Tracks session-local correlation IDs (event_ids, recall_id, grounded_event_ids)	Maintained by Learning Loop, populated from Concierge turn data
CorrectionDetector	K1	Requires conversation context + NLP analysis of user rephrasing patterns	Subscribes to turn.complete.v1
ValidationDetector	K1	Matches user yes/no against preceding recall_memory() results	Subscribes to turn.complete.v1
ReformulationDetector	K1	Semantic similarity comparison between consecutive user messages	Subscribes to turn.complete.v1
AbandonmentDetector	K1	Session-level timeout detection (user left without resolution)	Subscribes to turn.complete.v1 + session events
HedgingDetector	K1	Analyzes LLM response confidence signals	Subscribes to turn.complete.v1
FeedbackEnvelope builder	K1	Constructs envelope with K1-tracked correlation context	Inside each detector
FeedbackSchemaRegistry	K0	Validates payloads server-side against pipeline-specific schemas	K0 obs port validation layer
st_feedback_signals	K0 (storage)	Persists validated feedback for pipeline consumption	K0 storage layer
Pipeline feedback handlers	K0	Apply Thompson Sampling to pipeline parameters	Per-pipeline (P02, P03, P08)
st_learned_weights	K0 (storage)	Stores adapted parameter posteriors	K0 storage layer

36.4.3 Concierge-Facing Components (K1 Proactive System)

COMPONENT	PLACEMENT	RATIONALE	INTERFACE
Proactive Decision Engine	K1	LLM-based context evaluation – “should we ask now?”	Receives SSE intents from K1 Bus
Curiosity Agent	K1	LLM-powered question formatting with Socratic scaffolding	Consumes CuriosityIntent, produces natural language
Attention Budget Gate (K1-side)	K1	Local enforcement mirror of K0’s P05 token bucket	Prevents over-asking if SSE events arrive in bursts
Proactive Agent (Section 9.7)	K1 (Concierge)	Injects gap-filling questions during HIGH tier waits	Inside Concierge FSM COMPANIONING state
SocraticCoach	K1	ZPD estimation + pedagogical move planning for teaching moments	Future: extends Curiosity Agent
AnalogicalReasoner	K1	Cross-domain knowledge transfer requires conversation context	Future: domain mapping agent
QuestionRecommender (CF)	Split: K0 trains model, K1 scores candidates	Training requires cross-family data (K0), scoring needs user context (K1)	Future: federated scoring

36.4.4 Component Placement Diagram



36.5 SSE Event Consumption Catalog (K0 -> Concierge)

K0 emits SSE events via its SSE Port. The Bridge `SSEReceiver` parses these and republishes onto the K1 Bus with topic prefix `k1.k0.sse.`. Concierge subscribes to a subset of these events.

36.5.1 Events Consumed by Concierge

K0 SSE EVENT	K1 BUS TOPIC	CONCIERGE HANDLER	FSM IMPACT	PRIORITY
`curiosity.intent.v1`	`k1.k0.sse.curiosity.intent.v1`	`ProactiveAgent.on_curiosity_intent()`	May inject question in COMPANIONING or at turn boundary	BACKGROUND
`k0.proactive.signal.v1`	`k1.k0.sse.proactive.signal.v1`	`ProactiveAgent.on_proactive_signal()`	Triggers proactive bubble if user idle	BACKGROUND
`curiosity.intent.counterfactual.v1`	`k1.k0.sse.curiosity.intent.counterfactual.v1`	`CuriosityAgent.on_counterfactual_intent()`	Same as curiosity.intent.v1, different question template	BACKGROUND
`curiosity.intent.causal.v1`	`k1.k0.sse.curiosity.intent.causal.v1`	`CuriosityAgent.on_causal_intent()`	Same as curiosity.intent.v1, causal question template	BACKGROUND
`curiosity.intent.visual.v1`	`k1.k0.sse.curiosity.intent.visual.v1`	`CuriosityAgent.on_visual_intent()`	Future: multimodal curiosity	BACKGROUND
`curiosity.intent.simulation.v1`	`k1.k0.sse.curiosity.intent.simulation.v1`	`CuriosityAgent.on_simulation_intent()`	Future: scenario-based questions	BACKGROUND

36.5.2 Events NOT Consumed by Concierge (Informational)

K0 SSE EVENT	K1 BUS TOPIC	ACTUAL CONSUMER	PURPOSE
`k0.learning.advisory.v1`	`k1.k0.sse.learning.advisory.v1`	Learning Loop	Validated feedback advisory (closed-loop confirmation)
`memory.formed.v1`	`k1.k0.sse.memory.formed.v1`	Memory Writer	Confirmation that memory was consolidated
`k0.sync.complete.v1`	`k1.k0.sse.sync.complete.v1`	Sync Manager	Cross-device sync completion
`cognitive.vector.stored.v1`	`k1.k0.sse.cognitive.vector.stored.v1`	Cache Invalidator	Embedding update notification

36.5.3 SSE Subscription Wiring

```python
# Inside Concierge init() -- step 6: Start subscriptions
async def _wire_k0_sse_subscriptions(self) -> None:
    """Subscribe to K0 SSE events via K1 Bus."""

    # System 1: Gap Resolution intents
    await self._event_bus.subscribe(
        topic="k1.k0.sse.curiosity.intent.v1",
        handler=self._proactive_agent.on_curiosity_intent,
        delivery_class="BEST_EFFORT",
        group="concierge-proactive"
    )
    await self._event_bus.subscribe(
        topic="k1.k0.sse.proactive.signal.v1",
        handler=self._proactive_agent.on_proactive_signal,
        delivery_class="BEST_EFFORT",
        group="concierge-proactive"
    )

    # Extended curiosity intent types (same handler pattern)
    for intent_type in ["counterfactual", "causal", "visual", "simulation"]:
        await self._event_bus.subscribe(
            topic=f"k1.k0.sse.curiosity.intent.{intent_type}.v1",
            handler=self._curiosity_agent.on_intent,
            delivery_class="BEST_EFFORT",
            group="concierge-curiosity"
        )
```

36.6 Command & Obs Port Emission Catalog (Concierge -> K0)

Concierge sends data to K0 through two Bridge ports: the **Command Port** (for state-changing operations) and the **Obs Port** (for observability/feedback).

36.6.1 Command Port Emissions (State-Changing)

TOPIC	TRIGGER	PAYLOAD	K0 PIPELINE	PATTERN
`memory.delta`	`IMemoryPort.store()` via `promote_belief()` tool	`{content, metadata, layer, entity_tags}`	P02 Write/Ingest	Fire-and-forget (ONE-WAY)
`memory.delta` (gap resolution)	User answers proactive question	`{content, correlation: {gap_id}, entity_tags}`	P02 Write/Ingest	Fire-and-forget with gap_id tag
`session.checkpoint`	`turn_end()` via `IK0SyncPort.sync_delta()`	`{session_id, checkpoint_id, delta_sections}`	P02 (archival path)	Fire-and-forget (ONE-WAY)
`beliefs.archive`	WARM eviction of `beliefs_history`	`{beliefs[], eviction_reason, session_id}`	P02 Write/Ingest	Fire-and-forget
`history.archive`	WARM eviction of `history_recent`	`{turns[], compression_level, session_id}`	P02 Write/Ingest	Fire-and-forget
`learning.feedback`	Learning Loop emits to P06	`{feedback_type, correlation, signal}`	P06 Active Learning	Fire-and-forget

36.6.2 Obs Port Emissions (Non-State-Changing)

KIND	TRIGGER	PAYLOAD	K0 HANDLER	PURPOSE
`feedback`	Learning Loop detector fires	`FeedbackEnvelope` (Section 36.3.1)	`observe.py` -> `FeedbackSchemaRegistry` -> `st_feedback_signals` -> `BusDispatcher`	Model refinement (System 2)

36.6.3 Emission Path for Gap Resolution

```python
# Inside Concierge turn processing, when user answers a proactive question
async def _handle_gap_resolution_answer(
    self,
    user_answer: str,
    original_intent: CuriosityIntent,
) -> None:
    """Route user's answer to a K0 proactive question back to K0."""

    # Build memory item with gap correlation
    memory_item = MemoryItem(
        content=user_answer,
        metadata={
            "source": "gap_resolution",
            "gap_id": original_intent.gap_id,
            "gap_type": original_intent.gap_type,
            "question_asked": original_intent.rendered_question,
            "intent_id": original_intent.intent_id,
        },
        layer="episodic",           # User's direct statement
        entity_tags=original_intent.entity_tags,
    )

    # Fire-and-forget to K0 via IMemoryPort (Command Port: memory.delta)
    result = await self._memory_port.store(memory_item)

    # Update ConversationContext for correlation tracking
    self._conversation_context.record_gap_resolution(
        gap_id=original_intent.gap_id,
        answer_turn_id=self._current_turn_id,
        wal_id=result.wal_id if result else None,
    )

    # Emit telemetry
    await self._event_bus.publish("k1.concierge.gap.resolved.v1", {
        "gap_id": original_intent.gap_id,
        "gap_type": original_intent.gap_type,
        "answer_length": len(user_answer),
        "session_id": self._session_id,
        "turn_id": self._current_turn_id,
    })
```

36.7 Curiosity Intent Bus Integration

K0 communicates learning opportunities to K1 via structured `CuriosityIntent` objects published on SSE topics. These are NOT questions – they are **blueprints** containing gap metadata and prompt hints. K1 agents transform blueprints into human-facing dialogue.

36.7.1 CuriosityIntent Structure

```python
@dataclass
class CuriosityIntent:
    """K0 -> K1 curiosity blueprint. Consumed by Concierge Proactive/Curiosity agents."""
    intent_id: str                # UUID assigned by K0 P06
    topic: str                    # SSE topic (curiosity.intent.v1, .counterfactual.v1, etc.)
    gap_id: str                   # References st_learning_queue.gap_id
    gap_type: str                 # One of 7 gap types (Section 36.2.1) or extended types
    entity_id: Optional[str]      # Target entity in knowledge graph
    entity_tags: List[str]        # Entity labels for correlation
    priority: float               # 0.0-1.0, computed by P06
    uncertainty: float            # Entropy measure of the gap
    prompt_hints: Dict[str, Any]  # Structured hints for K1 question formatting
    evidence_uri: Optional[str]   # Trace link to supporting evidence
    attention_budget_spent: float  # How much of daily budget this consumes
    created_at: int               # Unix timestamp from K0
```

36.7.2 Prompt Hints by Intent Type

K0 provides structured hints that K1 uses to craft questions. K0 NEVER provides the final user-facing text:

INTENT TOPIC	PROMPT_HINTS KEYS	EXAMPLE HINTS	K1 USES TO...
`curiosity.intent.v1` (standard)	`template_category`, `entity_name`, `relationship`, `last_known_value`	`{"template_category": "CONFIRMATION", "entity_name": "Dr. Smith", "relationship": "dentist", "last_known_value": "twice yearly visits"}`	Craft: "Is Dr. Smith still your dentist? Last I remember, you visited twice a year."
`curiosity.intent.counterfactual.v1`	`template`, `lever`, `target`, `evidence`	`{"template": "If {lever} were {value}, what changes?", "lever": "market_availability", "target": "meal_planning"}`	Craft: "If the farmers' market were closed this weekend, would you still cook veggie lunches?"
`curiosity.intent.causal.v1`	`pattern`, `source`, `target`, `instrument`, `suggested_frame`	`{"pattern": "INSTRUMENT_PROBE", "instrument": "smart_watch_reminder", "target": "bedtime"}`	Craft: "When your smartwatch reminds you earlier, does that actually change bedtime?"
`curiosity.intent.visual.v1`	`media_type`, `scene_entities`, `query`	`{"media_type": "photo", "scene_entities": ["kitchen", "meal_prep"]}`	Future: ask about visual content
`curiosity.intent.simulation.v1`	`scenario`, `constraints`, `target_anchors`	`{"scenario": "sick_day_contingency", "constraints": ["both_children_ill", "business_trip"]}`	Future: "If both kids were sick on the morning you fly, what would you do?"

36.7.3 Intent Processing Pipeline (K1 Side)

```python
class CuriosityIntentProcessor:
    """Processes K0 curiosity intents into deliverable questions."""

    def __init__(
        self,
        proactive_decision: ProactiveDecisionEngine,
        curiosity_agent: CuriosityAgent,
        budget_gate: AttentionBudgetGate,
        output_manager: OutputManager,
    ):
        self.decision = proactive_decision
        self.agent = curiosity_agent
        self.budget = budget_gate
        self.output = output_manager

    async def process(self, intent: CuriosityIntent, fsm_state: str) -> Optional[str]:
        """
        Full processing pipeline for a K0 curiosity intent.

        Returns the delivered question text, or None if filtered out.
        """
        # Gate 1: Budget check (K1-side mirror of K0 P05)
        if not self.budget.can_ask():
            await self._log_filtered(intent, reason="budget_exhausted")
            return None

        # Gate 2: Contextual readiness (FSM state + user energy)
        readiness = await self.decision.evaluate_readiness(fsm_state)
        if readiness.score < 0.3:
            await self._defer(intent, readiness)
            return None

        # Gate 3: Delivery window (only in appropriate FSM states)
        if fsm_state not in ("LISTENING", "COMPANIONING"):
            await self._queue_for_later(intent)
            return None

        # Format question via Curiosity Agent (LLM-based)
        question = await self.agent.format_question(intent)

        # Constitutional AI self-critique check
        if not await self.agent.constitutional_check(question, intent):
            question = await self.agent.rewrite_with_guardrails(question, intent)

        # Deliver
        await self.output.send_proactive(question, source="k0_curiosity")
        self.budget.record_ask(intent.attention_budget_spent)

        return question
```

36.8 Attention Budget Enforcement (Concierge-Side)

K0's P05 Attention Manager enforces a **global** token bucket (3 questions per day, replenish 1 per 8 hours). Concierge maintains a **local mirror** of this budget to prevent over-delivery in case of SSE event bursts or timing gaps.

36.8.1 K1 Attention Budget Gate

```python
class AttentionBudgetGate:
    """
    K1-side mirror of K0 P05 token bucket.
    Prevents over-questioning if SSE events arrive in bursts.

    Invariant: K1 budget <= K0 budget (K1 is more conservative).
    K0 is the source of truth; K1 only prevents accidental over-delivery.
    """

    def __init__(self, max_per_day: int = 3, replenish_interval_hours: int = 8):
        self._max_per_day = max_per_day
        self._replenish_interval = timedelta(hours=replenish_interval_hours)
        self._tokens: float = float(max_per_day)
        self._last_replenish: datetime = datetime.now(timezone.utc)
        self._spent_today: List[BudgetEntry] = []

    def can_ask(self) -> bool:
        """Check if budget allows another question."""
        self._replenish()
        return self._tokens >= 1.0

    def record_ask(self, cost: float = 1.0) -> None:
        """Record a question delivery, consuming budget."""
        self._tokens = max(0.0, self._tokens - cost)
        self._spent_today.append(BudgetEntry(
            timestamp=datetime.now(timezone.utc),
            cost=cost,
        ))

    def _replenish(self) -> None:
        """Replenish tokens based on elapsed time."""
        now = datetime.now(timezone.utc)
        elapsed = now - self._last_replenish
        tokens_to_add = elapsed / self._replenish_interval
        self._tokens = min(float(self._max_per_day), self._tokens + tokens_to_add)
        self._last_replenish = now

    @property
    def remaining(self) -> float:
        """Current remaining budget."""
        self._replenish()
        return self._tokens

    def get_snapshot(self) -> Dict[str, Any]:
        """Budget state for telemetry."""
        return {
            "tokens_remaining": self.remaining,
            "spent_today": len(self._spent_today),
            "max_per_day": self._max_per_day,
        }
```

36.8.2 Budget Coordination (K0 vs K1)

ASPECT	K0 P05 (SOURCE OF TRUTH)	K1 ATTENTIONBUDGETGATE (MIRROR)
Scope	Global (all K1 consumers)	Concierge-local
Token bucket	3/day, replenish 1/8h	3/day, replenish 1/8h (same config)
Context readiness	Full scoring: stress, cognitive load, topic relevance, idle time	Simplified: FSM state + conversation energy
Enforcement	Blocks curiosity.intent.v1 emission at K0 SSE port	Blocks delivery to user at K1 output path
Drift handling	If K0 emits more than expected (bug), K1 gate catches overflow	K1 is the last safety net before user sees question

36.8.3 Budget Telemetry

Concierge reports budget state at `turn_end()` for observability:

```python
# Inside turn_end() -- after step 7 (experience triggers)
if self._attention_budget.spent_today:
    await self._event_bus.publish("k1.concierge.attention_budget.v1", {
        "session_id": self._session_id,
        "budget_snapshot": self._attention_budget.get_snapshot(),
        "last_question_gap_type": self._last_proactive_gap_type,
    })
```

36.9 ConversationContext & Correlation Tracking

The `ConversationContext` is a K1-owned tracking structure that maintains the correlation chain between Concierge operations and K0 entities. It is the foundation of the feedback system – without it, K0 cannot associate feedback signals with the correct entities.

36.9.1 ConversationContext Structure

```python
@dataclass
class ConversationContext:
    """
    Tracks correlation between K1 conversation turns and K0 entities.
    Maintained per-session by the Learning Loop, populated from Concierge turn data.
    """
    session_id: str
    current_turn_id: str

    # K0 entity correlation (populated during recall_memory() and promote_belief())
    event_ids: List[str] = field(default_factory=list)          # K0 event IDs from recall results
    wal_positions: List[str] = field(default_factory=list)      # K0 WAL positions from store results
    recall_id: Optional[str] = None                             # Last recall query ID
    response_id: Optional[str] = None                           # LLM response ID
    grounded_event_ids: List[str] = field(default_factory=list) # Events used in LLM response grounding

    # Gap resolution tracking
    active_gap_ids: List[str] = field(default_factory=list)     # Gaps currently being asked
    resolved_gap_ids: List[str] = field(default_factory=list)   # Gaps resolved this session

    def record_recall(self, recall_result: MemoryResult) -> None:
        """Record K0 recall correlation data."""
        self.recall_id = recall_result.query_id
        for hit in recall_result.hits:
            if hit.event_id and hit.event_id not in self.event_ids:
                self.event_ids.append(hit.event_id)

    def record_store(self, store_result: StoreResult) -> None:
        """Record K0 store correlation data."""
        if store_result.wal_id:
            self.wal_positions.append(store_result.wal_id)

    def record_grounding(self, event_ids: List[str]) -> None:
        """Record which K0 events were used in LLM response."""
        self.grounded_event_ids.extend(event_ids)

    def record_gap_resolution(
        self, gap_id: str, answer_turn_id: str, wal_id: Optional[str]
    ) -> None:
        """Record that a K0 gap was resolved in this session."""
        if gap_id in self.active_gap_ids:
            self.active_gap_ids.remove(gap_id)
        self.resolved_gap_ids.append(gap_id)
        if wal_id:
            self.wal_positions.append(wal_id)

    def to_correlation_ids(self) -> CorrelationIds:
        """Export for FeedbackEnvelope."""
        return CorrelationIds(
            session_id=self.session_id,
            event_ids=self.event_ids.copy(),
            wal_positions=self.wal_positions.copy(),
            recall_id=self.recall_id,
            response_id=self.response_id,
            grounded_event_ids=self.grounded_event_ids.copy(),
        )

    def reset_turn(self, new_turn_id: str) -> None:
        """Reset per-turn fields at turn boundary. Session-level fields persist."""
        self.current_turn_id = new_turn_id
        self.recall_id = None
        self.response_id = None
        self.grounded_event_ids.clear()
```

36.9.2 ConversationContext Lifecycle

```
Session start (init)
    |
    v
ConversationContext created (session_id, empty correlation)
    |
    v
Per turn:
    |
    +--> turn_start(): reset_turn(new_turn_id)
    |
    +--> Phase 2 recall_memory() tool call:
    |    record_recall(recall_result) -> populates event_ids, recall_id
    |
    +--> Phase 2 LLM response grounding:
    |    record_grounding(used_event_ids) -> populates grounded_event_ids
    |
    +--> Phase 2 promote_belief() or gap resolution:
    |    record_store(store_result) -> populates wal_positions
    |
    +--> Proactive question delivered:
    |    active_gap_ids.append(gap_id)
    |
    +--> User answers proactive question:
    |    record_gap_resolution(gap_id, turn_id, wal_id)
    |
    +--> turn_end(): context snapshot included in turn.complete.v1 payload
    |    Learning Loop detectors use context for FeedbackEnvelope correlation
    |
    v
Session end (shutdown)
    |
    v
ConversationContext discarded (all correlation data was already emitted)
```

36.10 Feedback Signal Detection (K1 Detectors)

Five Learning Loop detectors subscribe to `k1.concierge.turn.completed.v1` and analyze turn pairs to detect implicit and explicit feedback signals. These detectors are NOT inside Concierge – they are adjacent K1 services that consume Concierge's turn events.

36.10.1 Detector Specifications

```python
class CorrectionDetector:
    """
    Detects when user explicitly corrects a recalled memory.

    Signal: CORRECTION (confidence 0.85)
    Pattern: recall_memory() returned fact X, user says "No, it's actually Y"
    Regex triggers: "no that's wrong", "actually it's", "that's not right",
                    "I said X not Y", "you're confusing"
    """
    async def analyze(self, turn: TurnCompletedPayload) -> Optional[FeedbackSignal]:
        if not turn.context.recall_id:
            return None  # No recall in this turn, nothing to correct

        correction_patterns = [
            r"no[,.]?\s*(that's|it's)\s*(wrong|incorrect|not right)",
            r"actually\s*(it's|that's|the|my)",
            r"you're\s*(confusing|mixing|wrong)",
            r"i\s*said\s+\w+\s*not\s+\w+",
        ]

        for pattern in correction_patterns:
            if re.search(pattern, turn.user_message, re.IGNORECASE):
                return FeedbackSignal(
                    signal_class="CORRECTION",
                    signal_subtype="explicit_correction",
                    confidence=0.85,
                    target_event_ids=turn.context.grounded_event_ids,
                    evidence={"matched_pattern": pattern, "user_text": turn.user_message},
                )
        return None


class ValidationDetector:
    """
    Detects when user confirms or denies memory accuracy.

    Signal: VALIDATION (confidence 0.90)
    Pattern: recall_memory() returned fact, user says "yes" or "no"
    """
    POSITIVE_PATTERNS = [r"\byes\b", r"\byeah\b", r"\bcorrect\b", r"\bthat's right\b", r"\bexactly\b"]
    NEGATIVE_PATTERNS = [r"\bno\b", r"\bnope\b", r"\bnot quite\b", r"\bnot exactly\b"]

    async def analyze(self, turn: TurnCompletedPayload) -> Optional[FeedbackSignal]:
        if not turn.context.recall_id:
            return None

        for pattern in self.POSITIVE_PATTERNS:
            if re.search(pattern, turn.user_message, re.IGNORECASE):
                return FeedbackSignal(
                    signal_class="VALIDATION",
                    signal_subtype="positive_validation",
                    confidence=0.90,
                    target_event_ids=turn.context.grounded_event_ids,
                    evidence={"validation": "positive"},
                )

        for pattern in self.NEGATIVE_PATTERNS:
            if re.search(pattern, turn.user_message, re.IGNORECASE):
                return FeedbackSignal(
                    signal_class="VALIDATION",
                    signal_subtype="negative_validation",
                    confidence=0.90,
                    target_event_ids=turn.context.grounded_event_ids,
                    evidence={"validation": "negative"},
                )
        return None


class ReformulationDetector:
    """
    Detects when user rephrases the same query, indicating poor recall.

    Signal: IMPLICIT (confidence 0.60-0.75)
    Pattern: Consecutive messages with high semantic similarity but different tokens.
    Uses sentence embedding cosine similarity > 0.8 with token overlap < 0.5.
    """
    SIMILARITY_THRESHOLD = 0.8
    TOKEN_OVERLAP_THRESHOLD = 0.5

    async def analyze(self, turn: TurnCompletedPayload) -> Optional[FeedbackSignal]:
        if not turn.previous_user_message:
            return None

        similarity = await self._compute_similarity(
            turn.previous_user_message, turn.user_message
        )
        token_overlap = self._token_overlap(
            turn.previous_user_message, turn.user_message
        )

        if similarity > self.SIMILARITY_THRESHOLD and token_overlap < self.TOKEN_OVERLAP_THRESHOLD:
            confidence = 0.60 + (similarity - 0.8) * 0.75  # 0.60-0.75 range
            return FeedbackSignal(
                signal_class="IMPLICIT",
                signal_subtype="reformulation",
                confidence=min(0.75, confidence),
                target_event_ids=turn.context.grounded_event_ids,
                evidence={
                    "similarity": similarity,
```

```
                    "token_overlap": token_overlap,
                },
            )
        return None


class AbandonmentDetector:
    """
    Detects when user abandons conversation without resolution.

    Signal: IMPLICIT (confidence 0.70)
    Pattern: Session ends or 5+ minute gap after recall with no follow-up.
    """
    ABANDONMENT_TIMEOUT_MS = 300_000  # 5 minutes

    async def analyze(self, turn: TurnCompletedPayload) -> Optional[FeedbackSignal]:
        if not turn.context.recall_id:
            return None

        if turn.is_session_end and not turn.context.grounded_event_ids:
            return FeedbackSignal(
                signal_class="IMPLICIT",
                signal_subtype="abandonment",
                confidence=0.70,
                target_event_ids=turn.context.event_ids,
                evidence={"reason": "session_end_without_grounding"},
            )
        return None


class HedgingDetector:
    """
    Detects when the LLM response contained uncertainty signals.

    Signal: IMPLICIT (confidence 0.50)
    Pattern: LLM response contains hedging phrases ("I think", "maybe",
             "I'm not sure", "possibly") AND used grounded events.
    """
    HEDGING_PATTERNS = [
        r"\bi think\b", r"\bmaybe\b", r"\bi'?m not sure\b",
        r"\bpossibly\b", r"\bif i remember\b", r"\bit might be\b",
    ]

    async def analyze(self, turn: TurnCompletedPayload) -> Optional[FeedbackSignal]:
        if not turn.context.grounded_event_ids:
            return None

        hedge_count = sum(
            1 for p in self.HEDGING_PATTERNS
            if re.search(p, turn.assistant_response, re.IGNORECASE)
        )

        if hedge_count >= 2:
            return FeedbackSignal(
                signal_class="IMPLICIT",
                signal_subtype="hedging",
                confidence=0.50,
                target_event_ids=turn.context.grounded_event_ids,
                evidence={"hedge_count": hedge_count},
            )
        return None
```

36.10.2 Detector Execution Pipeline

```python
class FeedbackDetectorPipeline:
    """Runs all 5 detectors on every completed turn."""

    def __init__(self, detectors: List[FeedbackDetector], bridge_obs_port: ObsPort):
        self.detectors = detectors
        self.obs_port = bridge_obs_port

    async def on_turn_completed(self, payload: TurnCompletedPayload) -> None:
        """
        Called by K1 Bus subscription to k1.concierge.turn.completed.v1.
        Runs detectors in parallel, emits FeedbackEnvelopes for any signals found.
        """
        results = await asyncio.gather(
            *(d.analyze(payload) for d in self.detectors),
            return_exceptions=True,
        )

        for signal in results:
            if isinstance(signal, FeedbackSignal):
                envelope = self._build_envelope(signal, payload)
                await self._emit(envelope)

    def _build_envelope(
        self, signal: FeedbackSignal, payload: TurnCompletedPayload
    ) -> FeedbackEnvelope:
        """Build FeedbackEnvelope from detected signal + conversation context."""
        return FeedbackEnvelope(
            feedback_id=uuid4().hex,
            pipeline_id=self._infer_pipeline(signal),
            signal_class=signal.signal_class,
            signal_subtype=signal.signal_subtype,
            correlation=payload.context.to_correlation_ids(),
            provenance=Provenance(
                source_message_id=payload.turn_id,
                recall_context_hash=hash(str(payload.context.event_ids)),
                feedback_timestamp=monotonic_ms(),
                detector_name=type(signal).__name__,
                confidence=signal.confidence,
            ),
            payload=signal.evidence,
            payload_hash=sha256(json.dumps(signal.evidence, sort_keys=True).encode()).hexdigest(),
        )

    async def _emit(self, envelope: FeedbackEnvelope) -> None:
        """Emit to K0 via Bridge Obs Port."""
        try:
            await self.obs_port.emit(kind="feedback", payload=asdict(envelope))
        except BridgeUnavailableError:
            # Queue in local outbox for later drain (same as memory writes)
            await self._local_outbox.enqueue(envelope)

    def _infer_pipeline(self, signal: FeedbackSignal) -> str:
        """Determine which K0 pipeline should receive this feedback."""
        if signal.signal_class in ("CORRECTION", "VALIDATION"):
            return "P02"  # Write pipeline (fact accuracy)
        elif signal.signal_subtype == "reformulation":
            return "P08"  # Recall pipeline (retrieval quality)
        elif signal.signal_subtype == "abandonment":
            return "P08"  # Recall pipeline (result relevance)
        elif signal.signal_subtype == "hedging":
            return "P03"  # Consolidation (confidence calibration)
        return "P02"  # Default
```

36.11 Proactive Delivery Strategy (Decision Tree)

The Proactive Decision Engine determines **when** and **how** to deliver K0 curiosity questions to the user. This is the critical UX layer that prevents question fatigue.

36.11.1 Delivery Decision Tree

```
CuriosityIntent arrives from K0 SSE
    |
    v
[Budget Gate] Is K1 attention budget > 0?
    |-- NO --> Queue in deferred_intents (TTL: 24h), emit budget_exhausted telemetry
    |
    v YES
[FSM State Gate] Current FSM state?
    |
    |-- LISTENING (idle) --> [Idle Delivery Path]
    |   |
    |   v
    |   Has user been idle > 30s?
    |       |-- YES --> Deliver as proactive bubble (gentle, non-blocking)
    |       |-- NO  --> Queue for next idle window
    |
    |-- COMPANIONING (HIGH tier in progress) --> [Wait Fill Path]
    |   |
    |   v
    |   Has wait exceeded 5s? (Section 9.7)
    |       |-- YES --> Deliver as gap-fill question ("While I'm working on that...")
    |       |-- NO  --> Hold until 5s threshold
    |
    |-- ACKING/DISPATCHING/PROGRESSING --> Queue (user is actively interacting)
    |
    |-- CLARIFYING --> Queue (user is answering another question)
    |
    |-- DELIVERING --> Queue (final response being assembled)
    |
    v
[Priority Sort] If multiple intents queued:
    Sort by: priority DESC, uncertainty DESC, created_at ASC
    Deliver top-1 only (max 1 proactive question per delivery window)
    |
    v
[Constitutional Check] Does formatted question pass safety/politeness?
    |-- NO --> Rewrite with guardrails, re-check (max 2 rewrites)
    |-- STILL NO --> Drop intent, log constitutional_rejection telemetry
    |
    v
[Delivery] OutputManager.send_proactive(question, source)
    Record: budget consumed, gap_id, delivery timestamp, FSM state at delivery
```

36.11.2 Delivery Modes

MODE	FSM STATE	TRIGGER	USER EXPERIENCE	MAX QUESTIONS
Proactive Bubble	LISTENING (idle > 30s)	Timer fires after idle detection	Gentle notification: “I was thinking about something…”	1 per idle window
Wait Fill	COMPANIONING (wait > 5s)	HIGH tier processing in background	Natural conversation: “While I’m working on that – quick question…”	1 per HIGH tier turn
Session Start	LISTENING (first turn of new session)	Session initialized, last question > 8h ago	Opening: “Welcome back! Before we start, I had a thought…”	1 per session start
Deferred Drain	Any (background)	Deferred intent TTL approaching	Piggyback on next natural turn boundary	1 per drain cycle

36.11.3 Context Readiness Scoring

```python
class ContextReadinessScorer:
    """
    Scores 0.0-1.0 whether now is a good moment to ask a proactive question.
    Uses simplified signals available to Concierge (unlike K0 P05's full model).
    """

    def score(self, fsm_state: str, session_metrics: SessionMetrics) -> ReadinessScore:
        factors = {
            "fsm_state": self._fsm_factor(fsm_state),
            "idle_time": self._idle_factor(session_metrics.idle_ms),
            "turn_cadence": self._cadence_factor(session_metrics.recent_turn_intervals),
            "emotional_state": self._emotion_factor(session_metrics.current_emotion),
            "conversation_energy": self._energy_factor(session_metrics.tokens_last_5_turns),
        }

        # Weighted combination
        weights = {"fsm_state": 0.3, "idle_time": 0.25, "turn_cadence": 0.2,
                   "emotional_state": 0.15, "conversation_energy": 0.1}
        score = sum(factors[k] * weights[k] for k in factors)

        return ReadinessScore(score=score, factors=factors)

    def _fsm_factor(self, state: str) -> float:
        return {"LISTENING": 0.9, "COMPANIONING": 0.7, "DELIVERING": 0.3}.get(state, 0.1)

    def _idle_factor(self, idle_ms: int) -> float:
        if idle_ms > 60_000: return 1.0    # > 1 min idle: great moment
        if idle_ms > 30_000: return 0.7    # > 30s idle: good moment
        if idle_ms > 10_000: return 0.3    # > 10s idle: possible
        return 0.0                          # Actively talking: bad moment

    def _cadence_factor(self, intervals: List[int]) -> float:
        if not intervals: return 0.5
        avg_ms = sum(intervals) / len(intervals)
        if avg_ms > 30_000: return 0.8     # Slow conversation: good
        if avg_ms > 10_000: return 0.5     # Normal pace
        return 0.2                          # Rapid-fire: bad moment

    def _emotion_factor(self, emotion: str) -> float:
        negative = {"anxiety", "sadness", "anger", "frustration", "grief"}
        if emotion in negative: return 0.1  # Never interrupt negative emotions
        if emotion == "neutral": return 0.8
        return 0.6                          # Positive emotions: ok but don't derail

    def _energy_factor(self, tokens: int) -> float:
        if tokens < 50: return 0.9          # Low energy: user receptive to prompts
        if tokens < 200: return 0.5         # Medium energy
        return 0.2                          # High energy: user is driving conversation
```

36.12 Gap Resolution Closed Loop (Detailed Sequence)

This section provides the complete sequence for the most common scenario: K0 detects a gap, Concierge asks, user answers, K0 resolves.

36.12.1 Full Sequence Diagram

```
Timeline: Over hours/days (background learning)

[Day 1, 2:15 PM] User: "My daughter Maya started piano lessons last month"
    |
    v
Concierge Phase 2: promote_belief({fact: "Maya takes piano lessons", confidence: 0.9})
    |
    v
IMemoryPort.store() -> Bridge Command Port -> memory.delta -> K0 P02
    |
    v
K0 P02 ingests: event{entity: Maya, relation: takes, object: piano_lessons, confidence: 0.9}

[Day 1, 4:00 AM next day] K0 P03 Consolidation runs (scheduled)
    |
    v
P03 R4 Reconciliation:
    GapDetector finds: Maya -> piano_lessons has no FREQUENCY attribute
    GapDetector finds: Maya -> piano_teacher entity referenced but missing
    |
    v
GapEmitter persists:
    st_learning_queue:
      - gap_001: {type: MISSING_ATTRIBUTE, entity: Maya, attribute: lesson_frequency}
      - gap_002: {type: AMBIGUOUS_ENTITY, entity: piano_teacher, description: "referenced but unresolved"}
    Publishes: p03.gap.detected.v1 (x2)

[Day 1, 4:01 AM] K0 P06 Active Learning
    |
    v
Prioritizes: gap_001 (importance: 0.6), gap_002 (importance: 0.4)
    gap_001 emitted first (higher importance)

[Day 2, 9:30 AM] K0 P05 Attention Manager
    |
    v
Context readiness check: user active (session open), budget: 3/3 remaining
    Token bucket: spend 1 -> 2 remaining
    |
    v
K0 SSE Port emits:
    curiosity.intent.v1: {
        intent_id: "ci_abc123",
        gap_id: "gap_001",
        gap_type: "MISSING_ATTRIBUTE",
        entity_id: "maya_001",
        entity_tags: ["Maya", "piano_lessons"],
```

```
        priority: 0.6,
        uncertainty: 0.8,
        prompt_hints: {
            "template_category": "ATTRIBUTE_QUERY",
            "entity_name": "Maya",
            "attribute": "lesson_frequency",
            "known_context": "started last month"
        },
        attention_budget_spent: 1.0
    }

[Day 2, 9:30 AM] Bridge SSEReceiver -> K1 Bus
    |
    v
k1.k0.sse.curiosity.intent.v1 delivered to Concierge ProactiveAgent

[Day 2, 9:30 AM] Concierge CuriosityIntentProcessor
    |
    v
Budget Gate: K1 budget 3/3 -> PASS
FSM State: LISTENING (user idle 45s) -> PASS (idle delivery path)
Context Readiness: score 0.82 (idle + neutral emotion + low energy) -> PASS
    |
    v
Curiosity Agent formats question (LLM call):
    Input: template_category=ATTRIBUTE_QUERY, entity=Maya, attribute=lesson_frequency
    LLM: "By the way -- you mentioned Maya started piano lessons. How often does she practice?"
    |
    v
Constitutional Check: PASS (polite, non-invasive, relevant)
    |
    v
OutputManager.send_proactive("By the way -- you mentioned Maya started piano...", source="k0_curiosity")
Budget: record_ask(1.0) -> K1 budget now 2/3

[Day 2, 9:31 AM] User: "She has lessons every Tuesday and Thursday after school"
    |
    v
Concierge processes answer through normal turn pipeline:
    Phase 1: UltraBERT classifies (intent: log_memory, entities: [Maya, Tuesday, Thursday])
    Phase 2: LLM update_beliefs({Maya: {piano_frequency: "Tuesday, Thursday after school"}})
    Phase 2: LLM promote_belief({...})
    |
    v
_handle_gap_resolution_answer(
    user_answer="She has lessons every Tuesday and Thursday after school",
    original_intent=ci_abc123
)
    |
    v
IMemoryPort.store(MemoryItem{
    content="She has lessons every Tuesday and Thursday after school",
    correlation: {gap_id: "gap_001"},
    entity_tags: ["Maya", "piano_lessons"]
})
    |
    v
Bridge Command Port -> K0 P02 ingests with gap_resolution_id = "gap_001"

[Day 3, 4:00 AM] K0 P03 Consolidation
    |
    v
P03 R7 validation: gap_001 matches new event with gap_resolution_id tag
    st_learning_queue: gap_001.status = RESOLVED, gap_001.closed_at = now()
    Maya -> piano_lessons -> frequency: "Tuesday, Thursday after school" (confidence: 0.95)
    |
    v
GAP CLOSED. Active learning loop complete.
```

36.13 Bayesian Anchor Awareness

K0 maintains Bayesian anchors (`st_anchors`) that model family beliefs as Beta distributions. Concierge interacts with anchors through two paths: (1) receiving `CONCEPT_DRIFT` gaps when anchor parameters shift significantly, and (2) indirectly contributing to anchor updates via memory writes.

36.13.1 Anchor Model (K0 Side)

```python
@dataclass
class BayesianAnchor:
    """K0 belief anchor with Beta distribution parameters."""
    anchor_id: str
    entity_id: str
    attribute: str
    alpha: float          # Beta distribution: successes + 1
    beta_param: float     # Beta distribution: failures + 1 (avoiding Python keyword)
    confidence: float     # alpha / (alpha + beta_param) -- posterior mean
    decay_rate: float     # How fast confidence decays without reinforcement
    last_observed: int    # Last observation timestamp
    observation_count: int
```

Concept drift detection (K0 Entropy Scanner):

- Monitors 30-day sliding windows on anchor `confidence` values
- If `|confidence_t - confidence_{t-30d}| > 0.20` -> emits `CONCEPT_DRIFT` gap
- Example: Bedtime anchor confidence drops from 0.85 to 0.60 over 30 days -> “bedtime routine seems unstable”

36.13.2 Concierge Anchor Interactions

INTERACTION	DIRECTION	PATH	EXAMPLE
Anchor reinforcement	Concierge -> K0	`IMemoryPort.store()` with entity_tags matching anchor entity	User confirms "yes, still organic" -> alpha increments
Anchor contradiction	Concierge -> K0	`IMemoryPort.store()` with conflicting fact	User says "we switched to conventional" -> beta increments
Drift notification	K0 -> Concierge	`curiosity.intent.v1` with `gap_type=CONCEPT_DRIFT`	K0 detects bedtime anchor volatility -> asks Concierge to validate
Stale anchor probe	K0 -> Concierge	`curiosity.intent.v1` with `gap_type=STALE_ANCHOR`	Anchor not validated for 90+ days -> periodic confirmation question

Concierge does NOT read `st_anchors` directly. All anchor-related information reaches Concierge via `CuriosityIntent.prompt_hints` which include the anchor's current confidence and drift direction for the Curiosity Agent to use in question framing.

36.14 Offline Behavior & Graceful Degradation

When K0 Bridge is unavailable, all K0-dependent features degrade gracefully. Concierge's core functionality (Phase 1 + Phase 2 + LOW/MEDIUM/HIGH tier execution) is NEVER affected by K0 unavailability.

36.14.1 Degradation Matrix

FEATURE	K0 ONLINE	K0 OFFLINE	RECOVERY
recall_memory()	Full K0 recall via Bridge QueryPort	Falls back to LOCAL COLD (K1 SQLite), tags `source="local_cold"`	Automatic when Bridge reconnects
promote_belief()	Immediate WAL write via Bridge Command Port	Queued in Bridge `LocalOutbox`, drained when reconnected	Outbox drain is automatic, idempotent
Gap resolution answers	Immediate delivery to K0 P02	Queued in Bridge `LocalOutbox` with gap_id correlation preserved	Same outbox drain
Proactive questions	SSE stream delivers curiosity.intent.v1 events	No SSE events arrive -> no proactive questions	Questions resume when SSE reconnects
Feedback emission	FeedbackEnvelopes posted to K0 Obs Port	Queued in Learning Loop's local outbox	Drain on reconnection
Session checkpoint to K0	Fire-and-forget via `IK0SyncPort.sync_delta()`	`NullSyncPort` no-op (or future `K0SyncAdapter` queues)	LOCAL COLD provides crash recovery regardless
Attention budget sync	K0 P05 manages global budget	K1 local budget continues independently (may drift from K0)	Budget reconciliation on reconnection (K0 is source of truth)

36.14.2 Offline User Experience

```
K0 Bridge goes offline:
    |
    v
CB_BRIDGE transitions to OPEN after 3 consecutive failures
    |
    v
Concierge continues normal operation:
  - Phase 1 (UltraBERT): UNAFFECTED
  - Phase 2 (LLM tools): UNAFFECTED (except recall quality)
  - recall_memory(): DEGRADED (LOCAL COLD, stale data, tagged source="local_cold")
  - promote_belief(): QUEUED (LocalOutbox)
  - Proactive questions: STOPPED (no SSE events)
  - Feedback: QUEUED (Learning Loop outbox)
    |
    v
No user-visible error. Response quality may be slightly lower
due to stale memory recall. User is NOT informed of K0 status
unless explicitly asked.
    |
    v
K0 Bridge reconnects:
    |
    v
CB_BRIDGE transitions to HALF_OPEN -> probe -> CLOSED
    |
    v
LocalOutbox drains queued writes (idempotent, ordered)
SSE stream resumes -> proactive questions resume
Learning Loop outbox drains feedback envelopes
K1 attention budget reconciles with K0 P05
```

36.15 Interaction Sequence Diagrams

36.15.1 System 1: Gap-Filling During COMPANIONING (HIGH Tier)

```
User: "Plan Sarah's 10th birthday party"

User ──► Concierge (Phase 1 + Phase 2)
  |
  ├──► UltraBERT: intent=plan_event, safety=GREEN, entities=[Sarah, birthday, 10th]
  ├──► Tier: HIGH (complex planning)
  ├──► LLM: acknowledge("Planning that birthday party for Sarah!")
  ├──► TaskEnvelope(HIGH) -> Orchestrator
  |
  ├──► FSM: DISPATCHING -> COMPANIONING
  |
  |    [t=5000ms] Proactive Agent checks for K0 curiosity intents
  |    Queued intent: curiosity.intent.v1 (gap: Sarah's friend list is sparse)
  |    |
  |    v
  |    Curiosity Agent (LLM): "While I'm working on the party plan --
  |    I don't have a great list of Sarah's closest friends.
  |    Who should definitely be on the invite list?"
  |    |
  |    v
  |    User: "Her best friends are Emma, Lily, and Jake from school"
  |    |
  |    v
  |    Concierge captures: update_beliefs({Sarah: {friends: [Emma, Lily, Jake]}})
  |    _handle_gap_resolution_answer(answer, intent) -> IMemoryPort.store() -> K0
  |
  |    [t=35000ms] Orchestrator returns party plan results
  |    |
  |    v
  |    FSM: COMPANIONING -> PROGRESSING -> DELIVERING
  |    |
  |    v
  |    Final response includes party plan WITH friend names from gap fill:
  |    "Here's the party plan! I've included Emma, Lily, and Jake on the
  |     invite list since you mentioned they're Sarah's closest friends."
```

36.15.2 System 2: Feedback from Correction

```
User: "When did we last take the dog to the vet?"

User ──► Concierge (Phase 2)
  |
  ├──► recall_memory({text: "dog vet visit", layer: "episodic"})
  |     └──► IMemoryPort.recall() -> K0 returns: "March 15, annual checkup"
  |          ConversationContext.record_recall(result)
  |          ConversationContext.record_grounding([event_42])
  |
  ├──► LLM: "Based on my records, you took the dog to the vet on March 15
  |          for an annual checkup."
  |
  ├──► turn_end() -> emit turn.complete.v1 via IDeltaPort

User: "No, that was the cat. The dog went in February."
  |
  ├──► turn.complete.v1 emitted
  |
  ├──► Learning Loop CorrectionDetector:
  |     Regex match: "No, that was the cat"
  |     Signal: CORRECTION, confidence: 0.85
  |     Target: event_42 (from grounded_event_ids)
  |     |
  |     v
  |     FeedbackEnvelope{
  |       pipeline_id: "P02",
  |       signal_class: "CORRECTION",
  |       correlation: {event_ids: [event_42], recall_id: "r_xyz"},
  |       payload: {matched_pattern: "no that was", user_text: "..."}
  |     }
  |     |
  |     v
  |     POST /k0/obs.emit (kind: "feedback") -> K0 observe.py
  |     |
  |     v
  |     FeedbackSchemaRegistry validates -> st_feedback_signals
  |     |
  |     v
  |     BusDispatcher -> feedback.signal.p02.v1
  |     |
  |     v
  |     P02FeedbackHandler:
  |       _handle_correction(event_42, correction_data)
  |       -> Reduce confidence on event_42 (dog vet March 15)
  |       -> Possibly mark for re-consolidation
  |       -> Thompson Sampling: update similarity_threshold posterior
```

36.16 Error Handling Summary

ERROR	SOURCE	CONCIERGE RESPONSE	RECOVERY
SSE stream disconnected	Bridge SSEReceiver	No proactive questions; core functionality unaffected	Auto-reconnect (CB_SSE: 5s reconnect, 3/min)
Curiosity intent malformed	K0 SSE payload parse error	Drop intent, log `k1.concierge.intent.parse_error.v1`	K0 emits corrected intent on next cycle
Budget gate rejects intent	K1 AttentionBudgetGate exhausted	Queue in deferred_intents (TTL 24h)	Automatic: budget replenishes 1 token per 8 hours
Constitutional check rejects question	Curiosity Agent self-critique	Rewrite (max 2 attempts), then drop with telemetry	Next gap may succeed; dropped gap re-emitted by K0 P06
gap_resolution_answer store fails	Bridge Command Port unavailable	Queue in LocalOutbox	Outbox drain on Bridge reconnection
FeedbackEnvelope emission fails	Bridge Obs Port unavailable	Queue in Learning Loop outbox	Outbox drain on Bridge reconnection
ConversationContext corruption	Memory error or session restore	Reset context, lose correlation for current session	New context builds from next turn; feedback accuracy reduced
Deferred intent TTL expires	24h without delivery window	Drop intent, log `k1.concierge.intent.expired.v1`	K0 P06 may re-emit if gap still unresolved
K0 P05 and K1 budget drift	Long offline period	K1 budget may be more conservative than K0 allows	Reconciliation on K0 reconnection (K0 is source of truth)
Proactive question during CRISIS	Safety band escalation mid-delivery	Immediate halt of proactive delivery, CRISIS protocol takes over	Proactive queue paused until safety=GREEN/AMBER

Appendix A: ADR References

ADR	TITLE	RELEVANCE
ADR-0017	SessionState Single Writer	CONC-01 foundation
ADR-0093	ConciergeAgent Pattern	Module design
ADR-0078	Tool Call Batching	Tool execution
ADR-0006f	3-Phase Orchestration	Tier routing
ADR-0007	4-Stage Planning	HIGH tier planning
ADR-0086	Dynamic Agent Creation	Agent spawning

Appendix B: Validation Checklist

ID	CHECK	STATUS
W-01	All 22 invariants documented	[ ]
W-02	All 8 ports defined	[ ]
W-03	All 9 services specified	[ ]
W-04	FSM state diagram complete	[ ]
W-05	Error recovery paths defined	[ ]
W-06	Circuit breakers inventoried	[ ]
W-07	Event catalog complete	[ ]
W-08	Directory structure defined	[ ]
W-09	Test count targets set (~585)	[ ]
W-10	All 4 relationship sections complete	[ ]

End of Document